

APRENDE DART

Moisés Belchín Patricia Juberías

APRENDE DART

No está permitida la reproducción total o parcial de este libro, ni su tratamiento informático, ni la transmisión de ninguna forma o por cualquier medio, ya sea electrónico, mecánico, por fotocopia, por registro u otros métodos, sin el permiso previo y por escrito de los titulares del Copyright.

© Inscrito en el registro de la propiedad Intelectual.
Fecha de Publicación: Junio 2014
1ª edición

Gracias a todos por estar ahí.
Por tantas líneas de código, tantos años,
tantos proyectos, tanto café...
Siempre os llevaremos en nuestro corazón y nuestros backups.

Dedicado a todos los lenguajes de programación con los que
hemos trabajado antes de conocer Dart.

- Los autores.

CONTENIDOS

INTRODUCCIÓN

¿Qué es Dart?
¿Qué ventajas obtenemos al utilizarlo?
¿Cómo surge Dart?
Comparativas con otros lenguajes

PRIMEROS PASOS

Descargar e Instalar el software
Ejecutar Dart Editor
Abrir y ejecutar un ejemplo
Crear nuestras primeras aplicaciones

HERRAMIENTAS

Dart Editor | Dart2Js
Gestor de paquetes de dart PUB
Dartium | DartVM | Dart Fomatter
Generar documentación
Dartdoc | Docgen

DART BÁSICO

Comentarios | Operadores
Expresiones | Variables | Final | Const
Tipos de Datos | Números | Strings
Booleans | Lists | Maps | Sentencias
de Control | If & Else | While | For |
Swich | Funciones

DART AVANZADO

Excepciones | Control de errores
Clases | Instancias | Constructores
Métodos | Clases abstractas
Herencia | Interfaces | Generics
Typedefs | Librerías | Isolates
Programación asíncrona

Dart: HTML | Manipular el DOM | Gestión de eventos Trabajar con CSS Efectos visuales

DART, HTML Y CSS

Dart y los servicios Web Peticiones HTTP asíncronas Ajax | JSON

SERVICIOS WEB

Creación de aplicaciones servidor | Trabajando con procesos | Sockets

EN EL LADO SERVIDOR

Polymer.dart ¿Qué es Shadow DOM? Trabajando con Polymer.dart

POLYMER WEB COMPONENTS

INDICE

Los Autores 10

Presentación 12

INTRODUCCIÓN 14

¿Qué es Dart? 15

¿Como surge Dart? 17

¿Qué ventajas podemos obtener al utilizarlo? 17

¿Comparativa con otros lenguajes de programación? 20

Resumen 27

PRIMEROS PASOS 28

Descargar e instalar el software 29

Ejecutar DartEditor el entorno de desarrollo 31

Abrir y ejecutar un ejemplo 32

Crear nuestra primera aplicación 34

- Crear una aplicación web 34
- Crear una aplicación de línea de comandos 36

HERRAMIENTAS 40

Dart Editor 41

- El editor de Dart en detalle 42
- Abrir algunos ejemplos 43
- Crear una aplicación 47
- Ejecutar aplicaciones 49

¿Cómo funciona este sencillo ejemplo que acabo de crear? 50
Ejemplos avanzados 51
Depurar aplicaciones 56
Compilar a JavaScript 60
Inspeccionar el SDK 62
Configuración y vistas 65
Lanzadores y su configuración 69
Dart2js 74
PUB: El gestor de paquetes de Dart 79
Conoceremos: Dartium y DartVM 86
DartVM 86
Dartium 89
Dart Formatter 90
Generación de documentación 94
Dartdoc 94
Docgen 101

DART BÁSICO 109

Comentarios 110
Introducción 110
Comentarios en línea 111
Comentarios de bloque 111
Comentarios de documentación 112
Operadores y expresiones 115
Introducción a los operadores 115
Tipos de operadores 116

Introducción a las expresiones 119
Tipos de expresiones 119
Crear tus propios operadores 122
Variables y tipos de datos 124
Introducción a las variables y tipos de datos 124
Definición de variables 125
Tipos de variables: Final y Const 126
Tipos de datos 127
Sentencias de control 172
If y Else 172
Bucles For 173
Bucle While 175
Sentencia Switch 177
Funciones 179
Definición 180
Parámetros opcionales por posición 181
Parámetros opcionales por nombre 182
Valores predeterminados 183
Valores de retorno 184
Resumen de sintaxis 186

DART AVANZADO 190

Excepciones y control de errores 191
Tipos de Excepciones 194
Tipos de Errores 194

Definición de Excepciones y errores 195
Clases 196
Variables de instancia 198
Constructores 200
Métodos 205
Herencia 208
Variables y métodos estáticos 211
Interfaces implícitas y clases abstractas 212
Generics y Typedefs 216
Librerías 220
¿Como usar las librerías? 221
Crear librerías con múltiples ficheros 222
Re-exportando librerías 224
Multi-proceso y concurrencia en Dart: Isolates 225
Programación Asíncrona 231
¿Qué es la programación asíncrona? 232
Future 232
Stream 236
Crear funciones asíncronas 238
Resumen de sintaxis 240

DART, HTML Y CSS 243

Trabajando con HTML5 244
Gestión de eventos 251
Trabajando con CSS 254

Efectos visuales 263

SERVICIOS WEB 274

Peticiones HTTP asíncronas con Dart 275

Uso de JSON en Dart 293

DART EN EL LADO SERVIDOR 304

Creando una aplicación servidor 307

Trabajando con procesos 315

Sockets 321

POLYMER WEB COMPONENTS 324

Polymer.dart 326

¿Qué es Shadow DOM? 327

Trabajando con Polymer.dart 330

REFERENCIAS 342

LOS AUTORES

MOISÉS BELCHÍN MARTÍNEZ

Ingeniero Informático y apasionado de las nuevas tecnologías. Comenzó su andadura desarrollando software libre para diferentes distribuciones GNU/Linux, entre ellas RedHat, Debian, Fedora, Knoppix, Gentoo y Suse.
Se fijó en la fuerte corriente proveniente de EEEUU relacionada con el e-learning y comenzó a trabajar para una de las empresas españolas pioneras en este sector.

Dió el salto al desarrollo de aplicaciones para el mundo empresarial donde, durante aproximadamente 7 años, se especializó en el e-commerce y el cloud computing, desarrollando todo tipo de aplicaciones para desktops y dispositivos móviles, en diferentes sectores y con gran variedad de lenguajes, PHP, Java, ASP, .Net, C#, JavaScript, CSS, HTML, Python, PostgreSQL, MySQL, Oracle y Sql Server.

Siguió adelante una nueva etapa como cofundador y director técnico de una startup dedicada en exclusiva al cloud computing. Desarrollando la primera y más completa suite de gestión ERP/CRM para el sector de la mascota con gran éxito y una amplia cartera de clientes de diferentes sectores repartidos por todo el mundo.

Amante de las nuevas tecnologías, está en constante evolución y apuesta por los lenguajes más novedosos y potentes del mercado, por esto, comenzó a trabajar con Google Dart desde su nacimiento, en 2011. Contribuyendo al desarrollo del lenguaje, colaborando con el equipo de desarrolladores de Google, desarrollando parches para la corrección de errores y creador de blogdart.es, el primer punto de referencia sobre este lenguaje en castellano para acercarlo a la comunidad de desarrolladores de habla hispana.

PATRICIA JUBERÍAS GÓMEZ

Apasionada de las nuevas tecnología y la creación gráfica, estudió Diseño y Multimedia, especializándose en el ámbito web, la arquitectura de la información, UX y especificaciones funcionales.

Con grandes dotes para transmitir ideas y conocimientos, comenzó su trayectoria profesional, en empresas de formación y e-learning especializadas en las nuevas tecnologías y los contenidos multimedia.

Emprendedora y consciente de la gran necesidad de la usabilidad web y la adaptabilidad a diferentes tipos de dispositivos, fundó una de las primeras empresas españolas especializadas en la creación de aplicaciones cloud computing y desarrollo web responsive design,

Como desarrolladora web tiene experiencia en los principales lenguajes y plataformas (PHP, HTML, CSS, Python, JavaScript, Dart), pero su visión de la imagen y el diseño, le permiten dotar además a sus proyectos de una agradable apariencia, hacerlos adaptativos y proporcionarles la mejor experiencia de usuario o UX.

Con 10 años de experiencia laboral, ha participado en gran cantidad de proyectos de diversas áreas y ha fundado su propio estudio de comunicación, en el que es directora creativa y donde combina tecnología con diseño y marketing 2.0, ofreciendo soluciones creativas y tecnológicas integrales para empresas y startups.

De esta unión ha surgido un libro muy didáctico con el que pretendemos mostraros de un modo sencillo y sumamente práctico este maravilloso lenguaje de programación, orientado no solo a los programadores, sino también a desarrolladores web, desarrolladores de juegos, diseñadores, etc.

PRESENTACIÓN

Tras muchos años trabajando profesionalmente en el desarrollo y orientados especialmente al desarrollo de grandes sistemas (Moisés) y al desarrollo web (Patricia), nos hemos dado cuenta de la importancia de desarrollar aplicaciones funcionales e intuitivas para garantizar una buena experiencia de usuario. Es por ello que necesitamos lenguajes potentes y versátiles, que nos permitan llevar a cabo estos desarrollos.

En 2011 surgió Dart y comenzamos a trabajar profesionalmente con el, y casi de inmediato, nos dimos cuenta de que teníamos ante nosotros una de las herramientas más potentes de los últimos años.

Hasta ahora, Dart ha ido evolucionando a un ritmo increíble, y estamos seguros, de que Dart, marcará un gran cambio en el desarrollo web, tal y como lo conocemos actualmente.

Fue la falta de información sobre Dart en castellano, lo que nos animó a crear el blog: blogdart.es, y a escribir este libro, para acercarlo a toda la comunidad de desarrolladores de habla hispana.

¿A QUIEN VA DIRIGIDO ESTE LIBRO?

A usuarios como tú que desean aprender el lenguaje Dart en profundidad, de un modo sencillo, práctico y dinámico.
No solo esta orientado a programadores, sino también a desarrolladores y diseñadores web.

¿QUE REQUISITOS SON NECESARIOS?

El libro va orientado a usuarios con unos conocimientos previos en programación. No es necesario que los conocimientos sean muy avanzados, pero si es necesario estar familiarizado con los fundamentos básicos de programación. Si tienes experiencia con el desarrollo web y has trabajado con JavaScript, el libro, simplemente, ¡te encantará!.

Si no tienes conocimientos sobre JavaScript, no te desanimes, verás aspectos de Dart que, sin lugar a dudas te sorprenderán.
Si conoces JavaScript, HTML y CSS verás todo el potencial que ofrece Dart para la web desde el lado cliente.

¿COMO ESTÁ ENFOCADO EL LIBRO?

De forma directa, sencilla, práctica y dinámica. Dividido en capítulos o bloques ordenados e intercalando los contenidos más teóricos con ejercicios, prácticas, ejemplos e interesantes resúmenes de contenidos.

Con un lenguaje claro y sencillo y una maquetación simple y funcional, la intención del libro es que resulte ameno para el lector a la par que interesante y completo por sus contenidos.

El único donde todos los ejemplos tienen la misma apariencia que en el editor de código, con los mismos colores para que puedas identificar rápidamente las partes del código y te habitúes a trabajar con Dart Editor.

¿CUAL ES EL PROPÓSITO DEL LIBRO?

Este libro ha sido creado con la intención de mostrarte las características de este lenguaje, ofrecerte las herramientas para trabajar con él y unas directrices sobre funcionamiento de la programación web estructurada y sus ventajas.

Además, busca suplir la falta de documentación sobre este lenguaje en Castellano (ya que es el primero que se publica en este idioma), lo que permitirá a muchos usuarios de habla no inglesa, poder acercarse a Dart y aprender a trabajar con el.

¿QUÉ VOY A APRENDER CON ESTE LIBRO?

Te daremos una rápida introducción, descargarás las herramientas necesarias para empezar y echaremos un vistazo rápido a unos ejemplos.

Daremos un recorrido completo por todo el lenguaje y las herramientas que lo forman, ahora más en profundidad, y veremos todo el potencial de Dart con las librerías que nos ofrece.

Todo con ejemplos, comentarios, resúmenes y mucho código. Esperamos que lo disfrutes.

1 INTRODUCCIÓN

Lo primero de todo: ¿qué es Dart?

Dart es un lenguaje de programación desarrollado por Google, de código abierto, estructurado y flexible especialmente orientado al desarrollo web, aunque no únicamente. Ha llegado para permitir que los desarrolladores puedan crear aplicaciones web más complejas, con un mantenimiento más estructurado y con un mejor rendimiento.

> Es un modelo de programación que busca mejorar la calidad, aumentar la claridad del código, reduciendo el tiempo de desarrollo, mediante subrutinas y tres estructuras: secuencia, selección (if y switch) e iteración (bucles for y while).
>
> nota: ¿qué es programación estructurada?

Detrás de su creación están Lars Bak y Kasper Lund dos de los creadores del motor V8 de JavaScript para Google Chrome, por lo que a la hora de desarrollar Dart tanto el rendimiento como la eficiencia se han tenido muy en cuenta.

También involucrado en la creación de Dart, está Gilad Bracha creador del lenguaje **Newspeak** y co-autor de las especificaciones de Java y del desarrollo de la segunda edición de la máquina virtual de Java.

Dart es un lenguaje orientado a objetos, basado en clases, con herencia simple. Además Dart soporta interfaces, clases abstractas y tipado opcional de datos.

No te preocupes, veremos todo esto en detalle poco a poco y disfrutarás de todas estas cualidades de manera muy simple.

Hemos comentado que Dart se presenta como una alternativa al desarrollo web especialmente en el lado cliente. Actualmente, la tendencia de desarrollo viene a ser esta, descargar a los servidores de tareas que pueden realizarse perfectamente en el cliente.

En contra de lo que puedas pensar inicialmente, Dart no ha sido creado para “sustituir” a JavaScript, sino para ofrecer una opción alternativa más moderna en el desarrollo web y eficiente, sobre todo, para grandes proyectos.

Su sintaxis es sencilla, similar a C, y por lo tanto, al resto de lenguajes que heredan este. Si conoces cualquiera de estos lenguajes: JavaScript, Java, PHP, C++, Objective C o C# ya conoces Dart.

Dart se destaca como candidato imbatible para grandes proyectos web con gran desarrollo en lado cliente ya que permite organizar el código y mantenerlo de forma mucho más cómoda que, por ejemplo, con JavaScript.

Es muy legible, auto-documentado y compatible con cualquier navegador (es posible compilar el código a JavaScript y ejecutarlo en los navegadores modernos).

Dart corre sobre DartVM, la máquina virtual de Dart, y el código Dart corriendo en su máquina virtual es del orden de dos veces más rápido que Java. Incluso compilado a JavaScript ya es más rápido que código nativo JavaScript corriendo en V8.

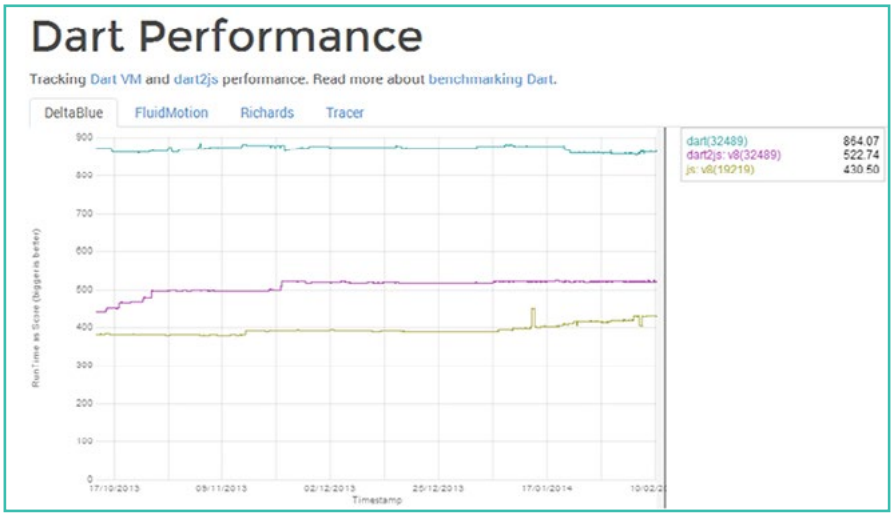


Figura 1. Rendimiento de Dart, compilado a JavaScript y JavaScript en V8.

Puedes ver información sobre el rendimiento de Dart en:
https://www.dartlang.org/performance/

nota

¿Qué ventajas podemos obtener al utilizarlo?

Como principales ventajas encontramos su alto rendimiento, actualmente incluso compilado a JavaScript es superior a código JavaScript nativo en V8, y su sencillez, tanto de uso como para su aprendizaje. Dart nos permite unas posibilidades casi infinitas a la hora de desarrollar y mantener aplicaciones complejas.

Además, los propios desarrolladores de Google y la comunidad ha creado gran cantidad de documentación, tutoriales, ejemplos y contenidos, cosa que siempre es de agradecer a la hora de comenzar a trabajar con un nuevo lenguaje.

Como únicas desventajas que podemos comentarte es que el lenguaje es joven y falta documentación. Hasta ahora el lenguaje aún no se encontraba en una versión estable, recientemente se ha publicado la *release 1.3.3*, y con ella todo será mucho más sencillo. Tanto Google como la comunidad están haciendo grandes esfuerzos para mejorar la documentación de Dart y, nosotros, con este libro esperamos también suplir la falta de documentación en general y en castellano en particular.

¿Como surge Dart?

Actualmente está cambiando el paradigma de programación. Hasta ahora el servidor era quien realizaba todo el trabajo.

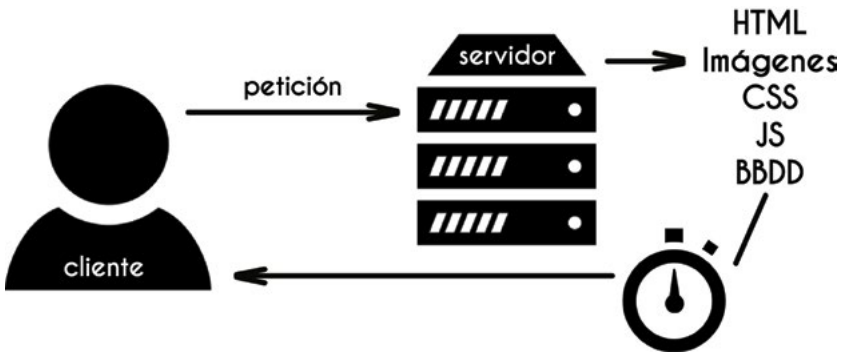


Figura 2. Esquema tradicional de peticiones entre cliente y servidor.

Cuando un visitante accede a una página web, es el servidor el que realiza las consultas a la base de datos, realiza procesos de validación de acceso, compone la estructura HTML del documento y carga vínculos externos CSS, JavaScript, imágenes, documentos, vídeos, etc. Esto es lo que sucede en las aplicaciones desarrolladas en PHP, Python, JSP, Ruby, etc.

Cada enlace que visitas vuelve a repetir todo el proceso para finalmente mostrar un documento HTML que el navegador cliente interpreta, por lo que se van estableciendo peticiones bastante costosas con el servidor. Si pensamos que hoy en día se han masificado los servicios web debido a la proliferación de dispositivos móviles y redes sociales, estamos hablando que miles de personas intentarán acceder a nuestro servidor.

Tenemos un grave problema de atención de solicitudes, nuestro servidor no será capaz de atender toda la demanda que tiene de peticiones porque cada petición consume demasiados recursos. ¿Y si descargamos nuestro servidor de aquellos procesos que puede realizar el cliente? ¿Y si dotamos a los clientes de más capacidad de proceso y mayor inteligencia para que puedan realizar más acciones?

Desde hace unos años esta es la norma habitual. Gracias a HTML, CSS y JavaScript, cada vez que navegas por Internet accedes a un servicio en el que el servidor te da un documento HTML con una serie de scripts, aplicaciones que son capaces de realizar muchos procesos en tu navegador web.

Estas aplicaciones que corren en el navegador realizan peticiones de datos en modo asíncrono (para evitar bloquear el cliente) al servidor y con esos datos van modificando la página web mostrando u ocultando datos. Así conseguimos simplificar las peticiones que realizamos al servidor pudiendo atender más clientes, además de aprovechar la capacidad de proceso de los clientes.

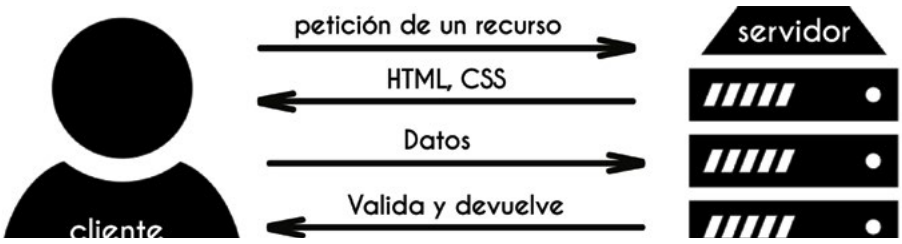


Figura 3. Nuevo modelo de peticiones asíncronas desde el cliente.

Gracias a estas peticiones asíncronas realizadas con JavaScript y a la evolución de HTML podemos realizar muchos procesos en el cliente de forma sencilla, eficiente y estándar, olvidándonos de plugins externos como ActiveX, Java Applets y Adobe Flash Player.

Imagina ahora una aplicación como Gmail o Google Docs . Desarrollar toda la interfaz, gestionar todas las peticiones y todos los eventos que se gestionan en el navegador puede ser una tarea muy costosa. Si pensamos en el mantenimiento de una aplicación de semejante envergadura sin duda es una tarea costosa y compleja, a pesar de los *frameworks JavaScript* que utilicemos.

JavaScript no se diseñó para estos propósitos, no es lo suficientemente robusto, el código se vuelve ilegible e incomprensible cuando empezamos con *callbacks* y padece de grandes problemas de gestión de memoria.

Por este motivo los desarrolladores de Google al crear sus aplicaciones utilizando JavaScript, se preguntaron si realmente era el lenguaje ideal para hacerlo, por lo que intentaron buscar otras alternativas como GWT. Consiguieron con ello un enfoque estructurado, pero muy poco optimizado para la web.

De aquí surge Dart, un lenguaje que permita combinar ambos mundos, el dinamismo de JavaScript con la potencia y estructura de lenguajes como C# o Java.

Dart te permitirá desarrollar tanto aplicaciones complejas, igual que podrías hacer con Java, Python o C#, como crear pequeños scripts para realizar tareas sencillas y concretas en tus páginas web, como puedes hacer con JavaScript.

Actualmente Dart puede correr en todos los navegadores web modernos, pero para ello previamente debe ser compilado a JavaScript. En un futuro cercano Dart correrá de manera nativa en nuestros navegadores web. Google Chrome planea incluir DartVM de forma nativa por lo que podrás correr aplicaciones Dart nativamente en el navegador con todo el rendimiento y la potencia que hemos comentado.

¿Como trabajaré ahora si antes usaba otro lenguaje de programación?

Hemos comentado que Dart se ha desarrollado pensando en las grandes aplicaciones web, en su mantenimiento y en su rendimiento. Conocer el lenguaje a fondo es algo vital y para eso está este libro, pero si ya tienes experiencia, si has utilizado alguna vez JavaScript o el framework jQuery queremos mostrarte rápidamente unos sencillos ejemplos de uso entre estos y Dart.

En Dart los nombres son más simples y estructurados

Los nombres han sido simplificados, están más optimizados. Se ha utilizado una nomenclatura más estándar y se sigue la norma *CamelCase*.

JavaScript	Dart
HTMLElement	Element
ownerDocument	Document
XMLHttpRequest	HttpRequest
CSSStyleSheet	CssStyleSheet

Es un estilo de escritura que se aplica a frases o palabras compuestas. El nombre se debe a que las mayúsculas a lo largo de una palabra en CamelCase se asemejan a las jorobas de un camello. Se podría traducir como Mayúsculas/Minúsculas Camello.

nota: ¿qué es programación CamelCase?

En Dart buscar elementos en el DOM es más sencillo

Los métodos de búsqueda de elementos en el DOM se han simplificado, dejando únicamente dos y compatibles con el nuevo estándar de HTML5 querySelector() y querySelectorAll().

JavaScript	JQuery	Dart
getElementsById('id')	$('#id')	querySelector('#id')
getElementsByTagName('tag')	$('tag')	querySelectorAll('tag')
getElementsByName('name')	$('[name="name"]')	querySelectorAll('[name="-name"]')
getElementsByClassName('class')	$('.class')	querySelectorAll('.class')

Colecciones reales de datos

Dart dispone de diferentes tipos de datos para representar colecciones de datos, *Lists, Maps y Sets*.

En JavaScript, los tipos de colecciones del DOM cuando recuperas una lista de elementos, por ejemplo, utilizando `getElementsByTagName` son diferentes del tipo Array nativo de Javasript, lo que produce errores e inconsistencia de datos.

En Dart, se ha simplificado y los métodos que manejan el DOM como por ejemplo, *children, nodes* o cuando recuperas un conjunto de elementos con *querySelectorAll* se devuelven en colecciones de datos nativas de Dart.

Por ejemplo, utilizando la librería de manejo del DOM de Dart *dart:html* cuando recuperas una lista de elementos por etiqueta HTML obtendremos una lista de elementos *List<Element>*.

```
import dart:html
void main() {
  List<Element> elementos = document.querySelectorAll('div');
}
```

Gracias a que Dart dispone los elementos del DOM en colecciones de datos reales, también simplifica muchos de los métodos nativos de JavaScript. Por ejemplo, en lugar de recuperar un elemento del DOM y tener diferentes métodos para trabajar con sus atributos, Dart trabaja con sus atributos con los métodos generales y comunes a los *Maps* ya que Dart almacena los atributos de los elementos en la estructura de datos *Map*.

JavaScript	JQuery	Dart
elem.hasAttribute('name')	elem.attr('name')	elem.attributes.containsKey('name')
elem.getAttribute('name')	elem.attr('name')	elem.attributes['name']
elem.setAttribute('name', 'value')	elem.attr('name', 'value')	elem.attributes['name'] = 'value'
elem.removeAttribute('name')	elem.removeAttr('name')	elem.attributes.remove('name')

En Dart, tenemos constructores para crear elementos del DOM

Para crear nuevos elementos del DOM, con JavaScript, igual que con jQuery, tenemos `createElement()`. En Dart, como es un un lenguaje orientado a objetos, tenemos constructores para los diferentes tipos de elementos del DOM.

JavaScript	JQuery	Dart
document.createElement('div')	$(document).add('div') $(document).append('div')	new DivElement() new InputElement(type:'checkbox')

Mejora y unifica el sistema de eventos

Los eventos son el cambio más importante y útil que encontrarás en Dart. Se ha modificado el modo en el que están unidos los controladores de eventos.

El DOM tiene dos formas de trabajar con eventos. El modo más antiguo que enlaza un único controlador en un elemento a través de una de las propiedades *on* del elemento. Y el modo más moderno, utilizando las funciones `addEventListener()` y `removeEventListener()` para establecer múltiples oyentes para el mismo evento.

En Dart, se han simplificado las cosas. Se han eliminado todas las propiedades *on* en *Element* y usa el *API Stream* que proporciona un modelo unificado de eventos.
Para cada tipo de evento existente en el DOM cada elemento dispone de una propiedad concreta, por ejemplo, *onClick*, *onMouseOver*, *onBlur*, etc.

JavaScript	JQuery	Dart
elem.addEventListener('click', (event) => print('click'), false);	$('#elem').on('click', function(event) {});	var eventListener = elem.onClick.listen((event) => print('click'));
elem.removeEventListener('click', listener);	$('#elem').off('click');	eventListener.cancel();

Adiós a los prefijos de cada fabricante

Cuando estás trabajando en el lado cliente te interesa que tu aplicación sea lo más estándar posible y que funcione en todos los navegadores, para ello hacemos uso de frameworks que nos facilitan la vida y nos dan todo esto solucionado.

Cuando se trata de trabajar dentro del navegador nos encontramos en muchas ocasiones con que algunas funciones se llaman de diferente modo cuando estás en un navegador o en otro.

Si quieres comprobar si el navegador donde corre tu aplicación soporta IndexedDB deberías preguntar por `window.indexedDB` o `window.mozIndexedDB` o `window.webkitIndexedDB` o `window.msIndexedDB`.

En Dart eso desaparece por completo, solo hay una forma estándar de llamar y comprobar el soporte de indexedDB en el navegador cliente `window.indexedDB`.

Gracias a ello mejoramos nuestro código, su legibilidad y evidentemente problemas, además de que estamos desarrollando de manera nativa y ya con soporte de multi-navegador.

API basadas en Futuros

Cuando en JavaScript queríamos escribir un código asíncrono, debíamos usar funciones *callbacks* como respuesta de retorno de otras funciones o como parámetros para nuevas llamadas a funciones. En aplicaciones grandes el desarrollo con este sistema se vuelve ilegible y muy propenso a errores. El mantenimiento resulta un trabajo muy duro.

Dart ha integrado dentro de su SDK la clase *Future* que permite encapsular en una forma orientada a objetos respuestas asíncronas de manera nativa.

Sin duda algo que hará las delicias de muchos. Un código más claro, limpio y que permite paralelizar tareas de manera muy simple.

Si antes tenías un código JavaScript de este estilo.

```
function showResult(xhr) {
    if (xhr.readyState==4 && xhr.status==200) {
      document.getElementById("result").innerHTML=xhr.responseText;
    }
}
function getResult() {
   var xhr = new XMLHttpRequest();
   xhr.onreadystatechange = showResult(xhr);
   xhr.open('GET', 'http://www.google.es', true);
   xhr.send();
}
```

Ahora con Dart quedaría de esta manera.

```
void getResult() {
    var xhr = HttpRequest.getString('http://www.google.es')
      ..then((resp) => print(resp))
      ..catchError((e) => print('Error!'));
}
```

Sin duda una mejora increíble y notable a la hora de trabajar. Los *Futures* se utilizan constantemente en el SDK de Dart y también los utilizarás con mucha frecuencia en tú código, ya que Dart se ha diseñado desde su origen para que sea totalmente asíncrono.

Librerías

Dart también ha llegado con una gran limpieza y organización en lo que a las librerías del DOM se refiere. Gracias a esta reorganización la librería pura de manejo del DOM y HTML es más ligera y sencilla.

Todo lo relativo a *indexedDB, audio, web_sql* y *svg* se ha extraído del DOM y se han creado librerías independientes para cada bloque, con lo que además de tener librerías concretas para cada función, se han mejorado y simplificado los nombres, eliminando prefijos innecesarios y repetitivos ya que cada librería tiene su propio *namespace*.

> Un *namespace*, se usa para referirnos a una colección de nombres de entidad que hemos definido y en el cual todos los nombres son únicos.
>
> nota: ¿qué es un namespace?

Coherencia cross-browser

Ya lo hemos comentado anteriormente al poder eliminar los prefijos de los diferentes desarrolladores de navegadores y al reorganizar las librerías Dart simplifica nuestro trabajo dándonos soporte multi-navegador y que nuestro código sea compatible en todos los navegadores modernos.

Y todo esto de manera nativa sin necesidad de trabajar con un framework que añada otra capa a tus aplicaciones.

> Cross-Browser se refiere a desarrollos web (webs o aplicaciones) que se ven exactamente igual en cualquier navegador
>
> nota: ¿qué es cross-browser?

Construcción en cascada

Uno de los operadores que se incluyó posteriormente en Dart y que utilizarás mucho es el operador en cascada. Inicialmente Dart no lo contemplaba y se añadió para facilitarnos el trabajo cuando trabajamos con elementos del DOM.

Es muy común estar trabajando con elementos HTML y cambiar o añadir los valores de múltiples propiedades. Esto origina grandes bloques de código, muy repetitivos y que nos llevan a incorporar ilegibilidad a nuestro código.

Si tienes que repetir el nombre del elemento en lugar de darle un nombre descriptivo es muy posible que uses un nombre lo más abreviado posible, lo que lleva a producir un código menos legible.

En JavaScript tendrías un código como este:

```
var ele = document.createElement("input");
ele.setAttribute("id", "identificador");
ele.setAttribute("type", "number");
ele.setAttribute("value", "valor");
ele.setAttribute("name", "nombre");
```

En Dart tendrás un código más compacto y legible.

```
var elementoInputNumeroDeHijos = new InputElement(type: 'number')
    ..id = 'identificador'
    ..valor = '0'
    ..name='numero _ de _ hijos';
```

Resumen

En este bloque hemos aprendido que:

DART ES UN LENGUAJE DE PROGRAMACIÓN:

- Desarrollado por Google.
- Como alternativa a JavaScript en el desarrollo de grandes aplicaciones.
- De código abierto.
- Con alto rendimiento.
- Dispone de su propia máquina virtual.
- Genera un código sencillo y fácilmente mantenible.
- Que sigue el modelo de programación estructurada.
- Es Cross-browser.
- Se puede compilar a JavaScript y ejecutar en cualquier navegador moderno.
- Surge para dar soporte a la nueva tendencia de simplificar el lado servidor e incluir más complejidad en el lado cliente.

2 PRIMEROS PASOS

Ahora que ya hemos realizado una introducción a Dart, y siguiendo con nuestra filosofía de realizar un libro práctico que te permita aprender Dart, vamos a comenzar a trabajar con Dart.

Lo primero que necesitas es instalar el Kit de desarrollo de Dart (Dart SDK) y el editor (Dart Editor). Descargaremos e instalaremos el software y nos familiarizaremos con el entorno de desarrollo. Abriremos una primera aplicación de ejemplo para comenzar a echarle un vistazo en detalle al código.

Descargar e instalar el software

Lo primero que debes hacer es dirigirte a www.dartlang.org, la página oficial del lenguaje Dart y obtener la última y más reciente versión del Kit de desarrollo y del editor de código.

La página principal detectará automáticamente nuestro sistema y versión y nos ofrece la descarga directa de todo lo que necesitamos para comenzar con Dart.

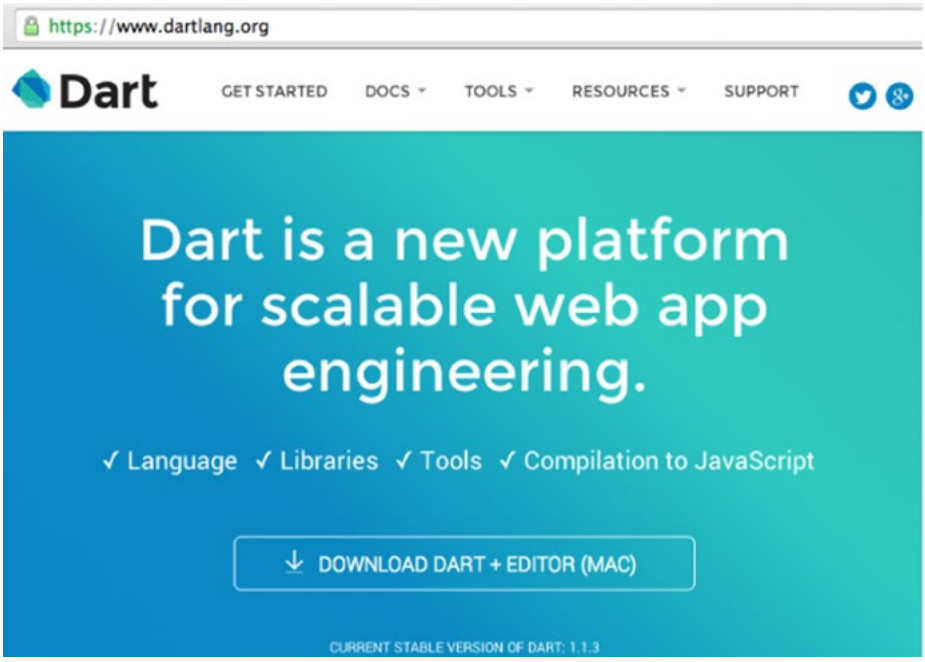


Figura 4. Página principal de dartlang.org para descargar las herramientas.

Haciendo clic en el enlace de descarga comenzaremos a bajar Dart SDK y Dart Editor, un fichero comprimido en formato ZIP.

Tras descargar el fichero lo descomprimimos y veremos una estructura de directorios similar a la que se muestra en la siguiente figura.

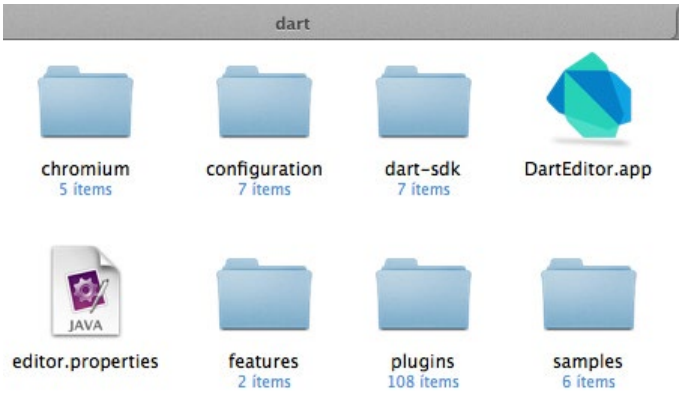


Figura 5. Estructura de directorios tras descomprimir el fichero descargado.

Por el momento no entraremos en detalle, ahora solo queremos empezar a ver y probar Dart lo más deprisa posible, así que haz doble clic sobre el icono *DartEditor* y comenzará a ejecutarse.

Este es el editor de código que utilizarás cuando trabajes con Dart. Está basado en Eclipse, así que si ya conoces o has trabajado con Eclipse te resultará muy familiar.

Así de sencillo, sin necesidad de instaladores, solo descargas un fichero comprimido, lo descomprimes y a trabajar, ¿se puede pedir más simplicidad?

DartEditor es una aplicación Java. Para que funcione correctamente deberás tener instalado en tu sistema el entorno Java. Dirígete a www.java.com para descargar e instalar Java en tu sistema.

nota: el entorno Java

Ejecutar DartEditor el entorno de desarrollo

En el directorio de instalación, haz doble clic en el archivo DartEditor .

Se abrirá la aplicación mostrando la ventana de bienvenida.

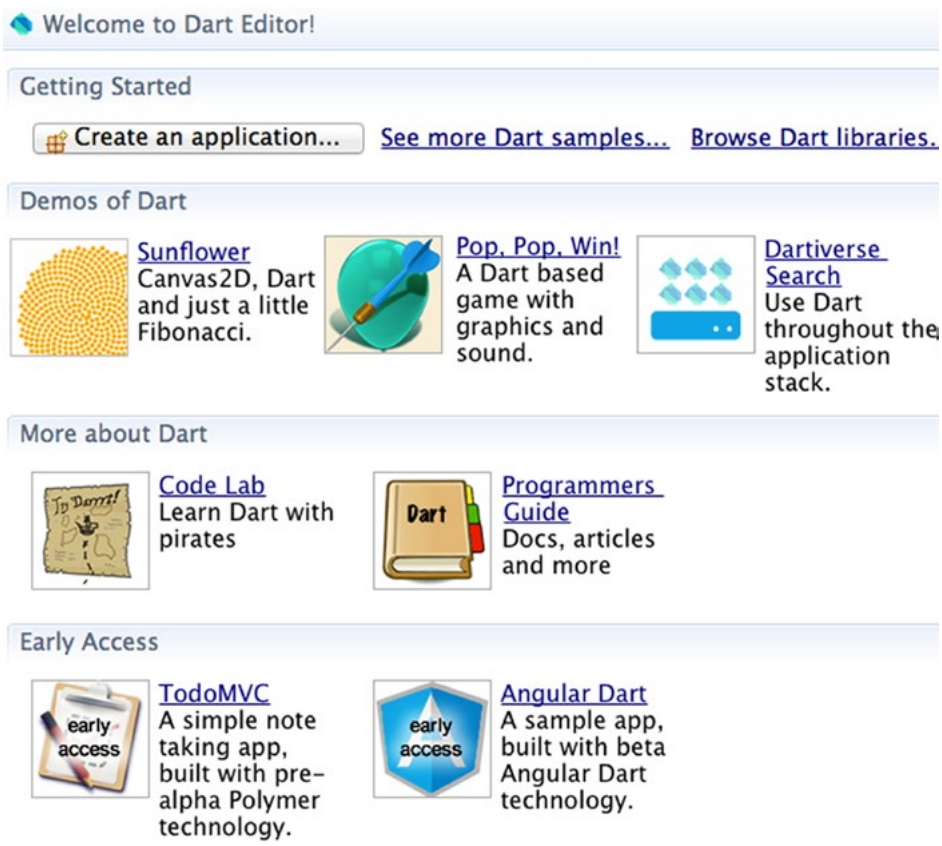


Figura 6. Ventana de Bienvenida de DartEditor.

Abrir y ejecutar un ejemplo

En la página de bienvenida y para que no esperes más, el editor de Dart incorpora varios ejemplos. En el apartado de ejemplos de Dart (Demos of Dart) simplemente selecciona el primero de ellos y DartEditor se encargará del resto.

El editor muestra el contenido de Sunflower.dart, y la vista de archivos (panel lateral izquierdo) muestra la estructura de directorios con los archivos que forman la aplicación girasol (Sunflower).

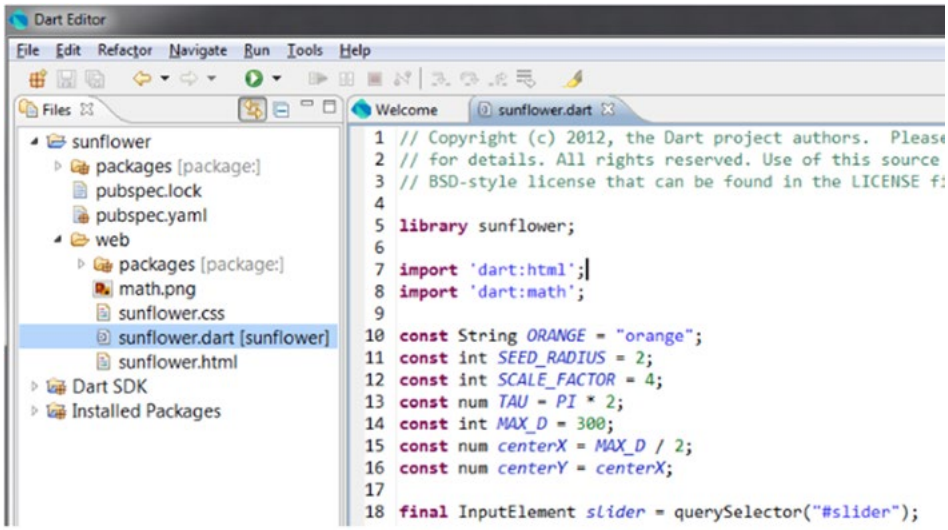


Figura 7. Vista inicial en DartEditor del primer ejemplo de Dart: Sunflower.

Sabemos que lo estás deseando así que vamos a ver este ejemplo en funcionamiento. Fíjate en la barra de herramientas de DartEditor verás un botón redondo verde , es el botón **ejecutar** (Run) que te permite ejecutar aplicaciones Dart desde DartEditor.

La aplicación Sunflower es una aplicación web, esto significa que cuando pulses sobre el botón correr la aplicación se ejecutará en el navegador web, como si de una página web HTML se tratase.

No se abre el navegador web por defecto que tienes instalado en tu sistema, se abre en Dartium, el navegador necesario para ejecutar y probar aplicaciones Dart.

> *Dartium* es una herramienta muy específica de Dart. Aunque te recuerde a Google Chrome no utilices *Dartium* como navegador principal ya que podrías tener problemas de seguridad y estabilidad.
>
> nota: que es Dartium

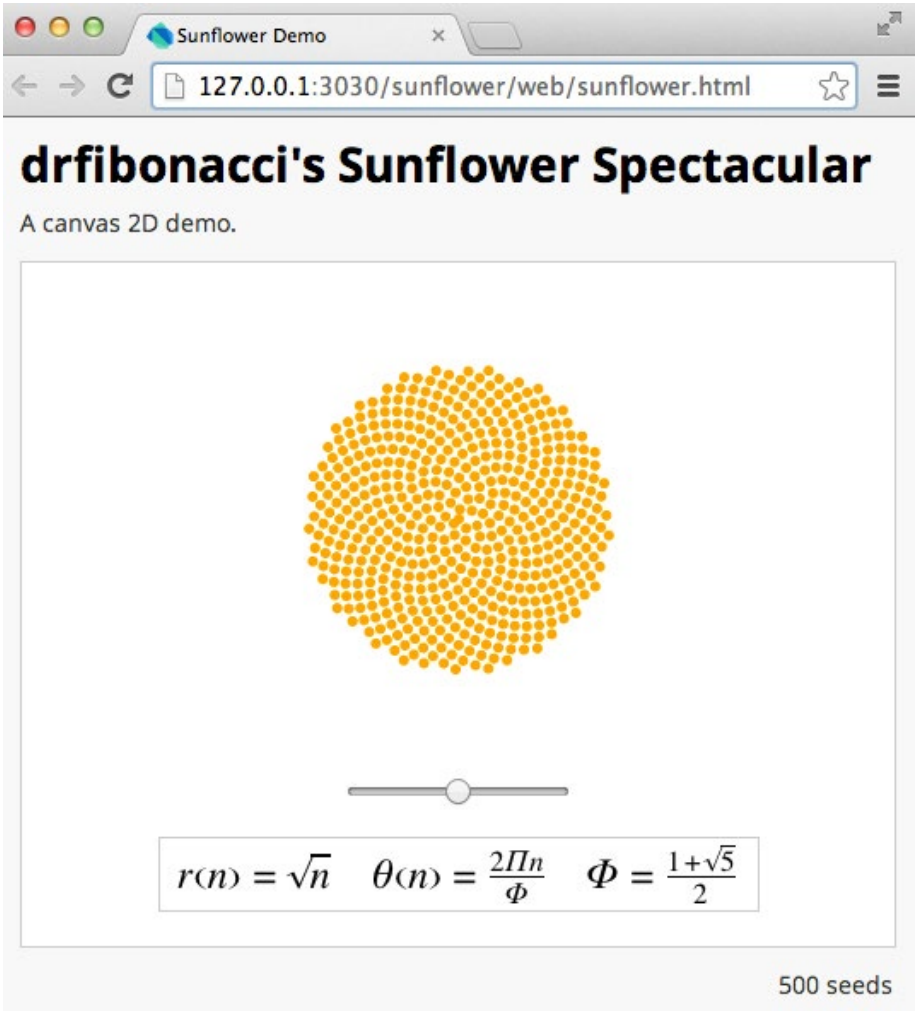


Figura 8. *Dartium* ejecutando la aplicación Dart Sunflower.

Crear nuestra primera aplicación

Ahora que ya has ejecutado *DartEditor* y has probado a abrir y ejecutar un ejemplo básico, vamos a crear una primera aplicación de prueba en *DartEditor*.
Con *DartEditor* resulta realmente sencillo crear una nueva aplicación web o una aplicación de línea de comandos.

Crear una aplicación web

Vamos a crear una sencilla aplicación web y para ello, pulsa en el botón para crear nuevas aplicaciones  de la barra de herramientas. También puedes seleccionar el menú **Archivo > Nueva aplicación** *(File > New Application)*.

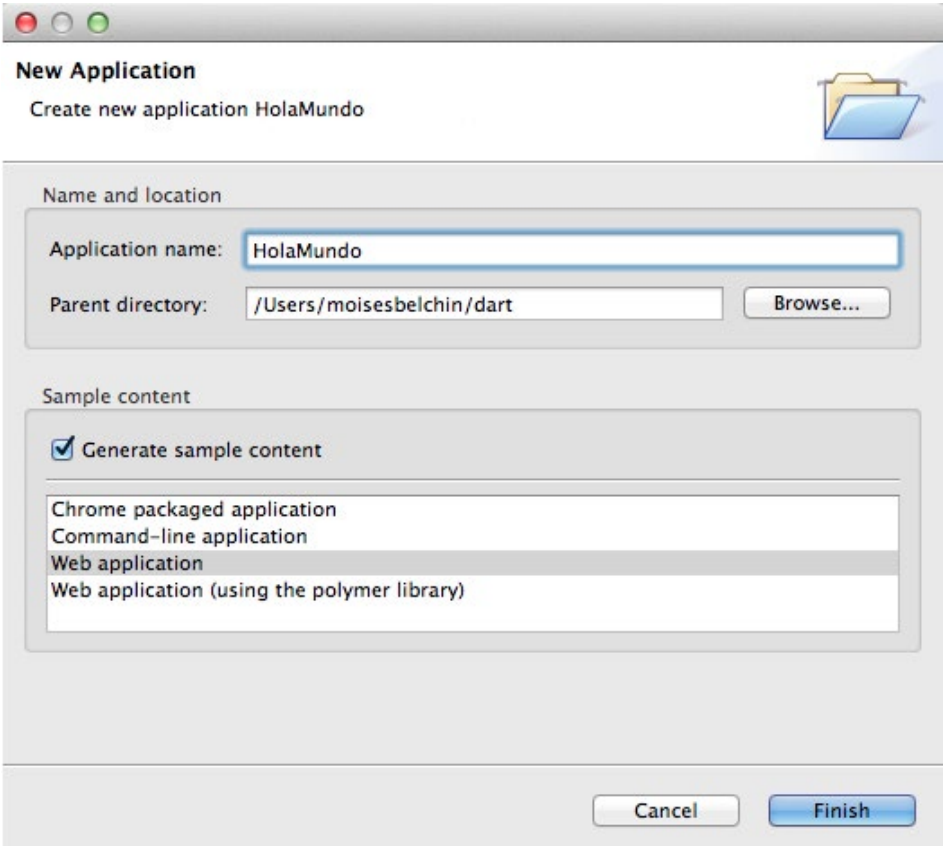


Figura 9. Ventana para establecer nombre y tipo de proyecto a crear.

Aparecerá una nueva ventana donde podrás seleccionar el tipo de aplicación que deseas crear y establecer un nombre para este nuevo proyecto.

Selecciona como tipo de proyecto **Aplicación web** *(Web application)* y establece como nombre *HolaMundo*.

Dejando marcada la casilla **Generar contenido de ejemplo** *(Generate sample content)* DartEditor generará una estructura básica y un pequeño código de ejemplo.

Una vez terminado pulsa **Finalizar** *(Finish)* y podrás ver el nuevo proyecto con su contenido y código de ejemplo.

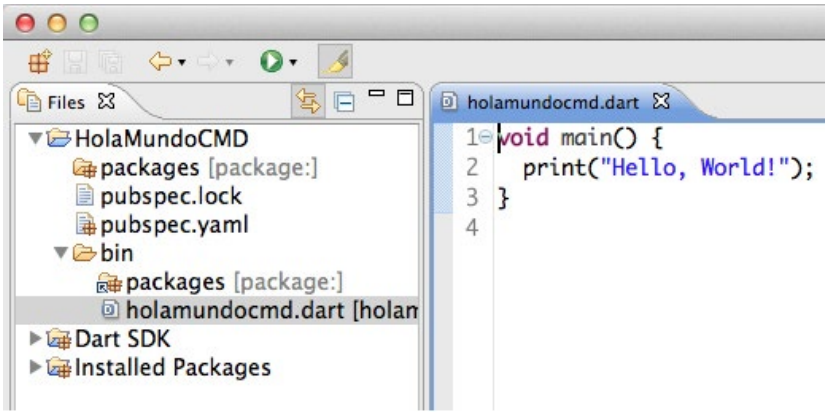


Figura 10. *DartEditor* muestra tu primera aplicación web *HolaMundo*.

DartEditor ha creado una estructura básica de directorios para esta nueva aplicación web y de manera automática ha creado un pequeño código de ejemplo.

Importamos la librería `dart:html` para trabajar con HTML (tiene todo su sentido ya que se trata de una aplicación web) y creamos dos métodos.

El método main, que será el método principal que tendrán todas nuestras aplicaciones y el método `reverseText`. Concretamente el método reverseText recupera un texto de un elemento DIV del DOM del documento HTML `holamundo.html`, le da la vuelta y lo vuelve a mostrar en el elemento DIV `#sample _ text _ id`.

Pulsamos sobre el botón **ejecutar** de la barra de herramientas y veremos como *Dartium* ejecuta esta nueva aplicación. Haz clic en el elemento que dice "Pulsame!" *(Click me!)* y verás como este texto cambia.

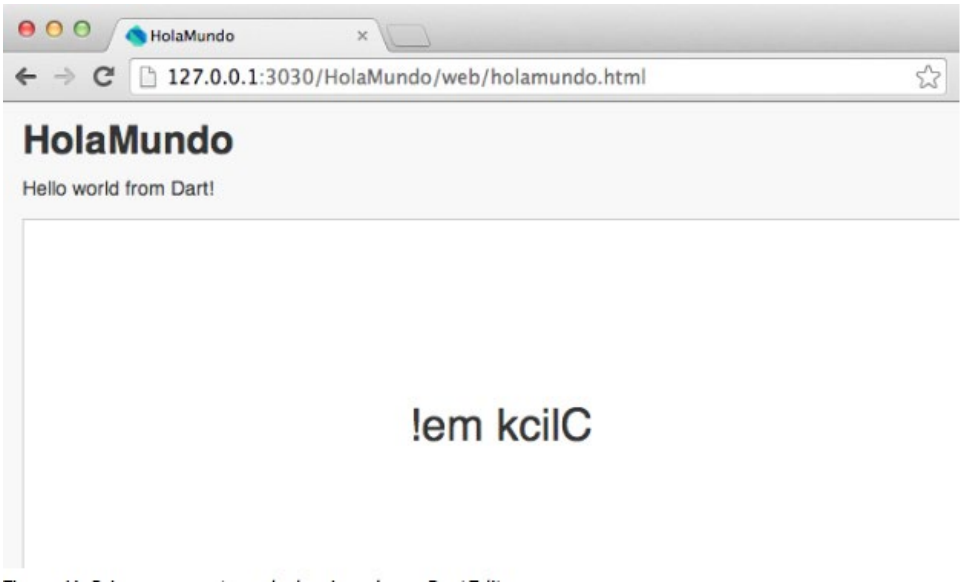


Figura 11. Primer proyecto web de ejemplo en *DartEditor*.

Crear una aplicación de línea de comandos

Ahora que ya hemos visto un ejemplo sencillo de proyecto web en *DartEditor* y hemos analizado brevemente el código de ejemplo, vamos a crear ahora un proyecto diferente.

Vuelve a pulsar en el botón para crear una nueva aplicación y en la ventana que aparece selecciona el tipo de proyecto aplicación de línea de comandos *(Command-line application)*.

Dejamos que *DartEditor* cree el contenido de ejemplo *(Generate sample content)*. Pulsa sobre Finalizar *(Finish)* para crear los archivos.

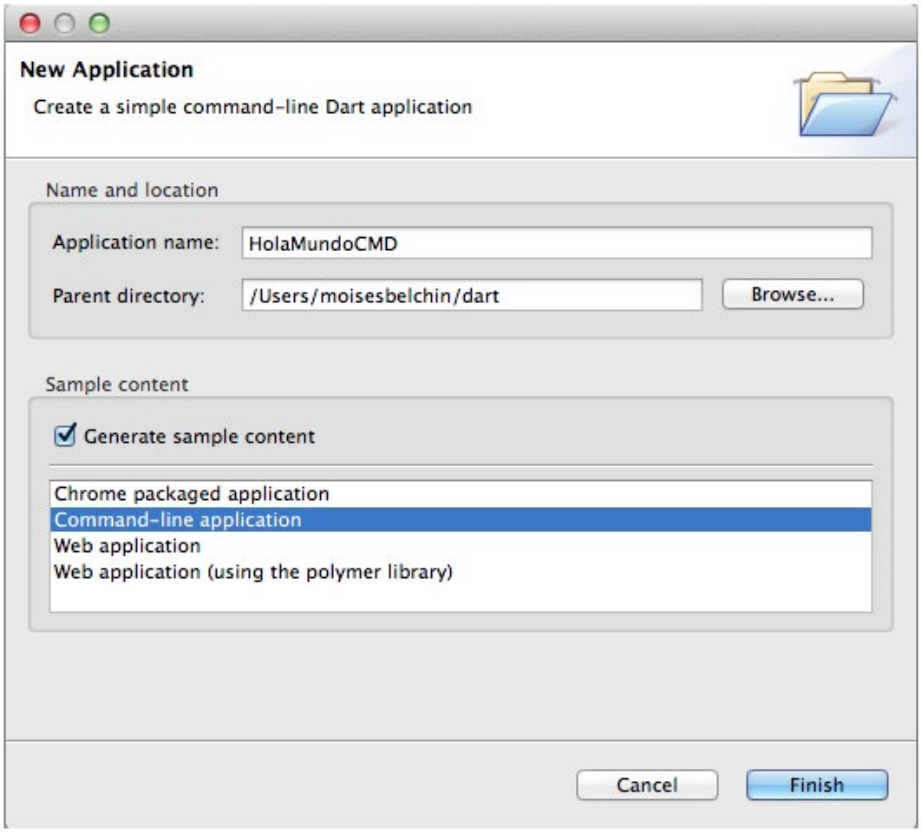


Figura 12. Seleccionamos un proyecto de aplicación de línea de comandos.

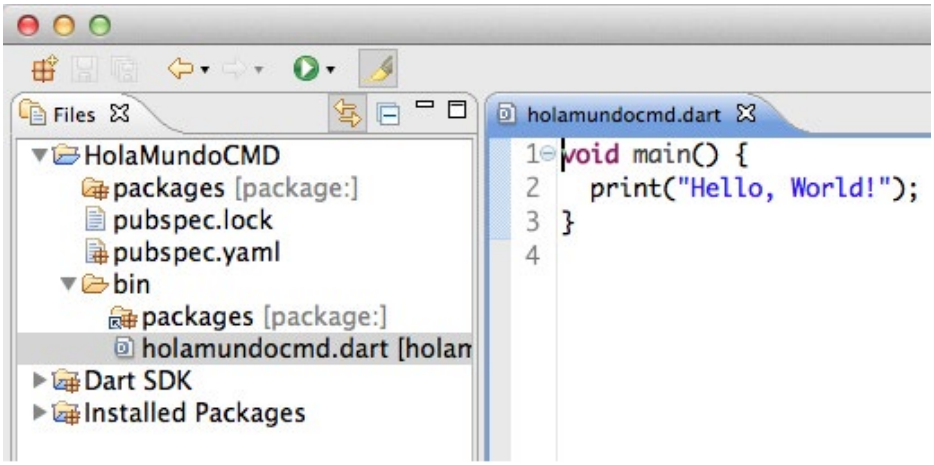


Figura 13. *DartEditor* mostrando nuestro proyecto de aplicación de línea de comandos.

En este caso el ejemplo es mucho más sencillo, no se importa ninguna librería y sólo dispones de un método `main`, ya sabes que es el método principal de cualquier aplicación, con un mensaje que dice "Hola, Mundo !" *(Hello, World!)*.

Pulsa nuevamente en el botón ejecutar (*Run*).

Ahora no se abrirá *Dartium* para mostrarte el resultado de la aplicación de línea de comandos, sino que será el propio *DartEditor* quien muestre la salida del mensaje en una vista especial que tienes a tu disposición que recibe el mismo nombre que el fichero que estés ejecutando.

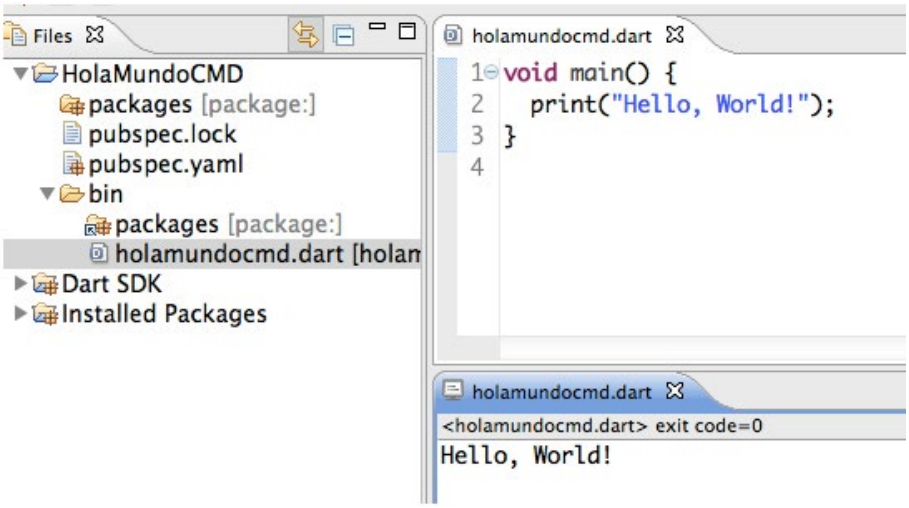


Figura 14. *Resultado de ejecutar nuestro proyecto de línea de comandos en DartEditor.*

3 HERRAMIENTAS

Un buen lenguaje de programación no solo consta de una sintaxis sencilla, cómoda y rápida, o de magníficas librerías que permiten que hagas cualquier cosa que imagines. Un buen lenguaje debe constar de todo eso, pero además debe disponer de buena documentación, de una comunidad amplia que lo respalde y de buenas herramientas.

Cuando te acercas a un lenguaje nuevo te gusta ver algunos ejemplos, leer un poco sobre sus bondades y empezar a trabajar con él de inmediato.

Ya has podido comprobar que Dart es así. Has visto como obtener el editor con todas las herramientas que lo conforman y el SDK de manera sencilla. Has probado algunos ejemplos y has creado alguna pequeña aplicación y todo gracias a estas fantásticas herramientas que hemos comentado.

Ahora profundizaremos más en el manejo de las herramientas y veremos en detalle tanto el editor como el resto de aplicaciones, compilación, gestión de paquetes y generación de documentación.

Gracias a las herramientas que veremos a continuación te darás cuenta que Dart es un magnífico lenguaje de programación, en el que comenzar a trabajar resulta muy sencillo, rápido y cómodo.

Dart Editor

Dart Editor es el editor de código de Dart con el que estarás trabajando constantemente. Si anteriormente has trabajado o conoces el entorno de desarrollo Eclipse estás de enhorabuena, porque Dart Editor es una versión ligera de Eclipse.

El equipo de Dart cogió una versión de Eclipse, eliminaron todo lo que consideraban superfluo y comenzaron a crear Dart Editor con las herramientas absolutamente necesarias para el desarrollo, depuración y ejecución de aplicaciones Dart.

Si no conoces Eclipse no te preocupes vamos a conseguir que te familiarices con Dart Editor en cuestión de minutos y verás lo potente y útil que resulta.

El editor de Dart en detalle

Vamos a dar un paseo por nuestro nuevo entorno de desarrollo para que lo vayas conociendo y te familiarices con él. Verás que es muy sencillo y muy potente.

> Sabemos que estás deseando trabajar con Dart y partimos de la premisa que ya has seguido los pasos del **capítulo "Primeros pasos con Dart"** y dispones de Dart Editor instalado en tu sistema.
>
> nota

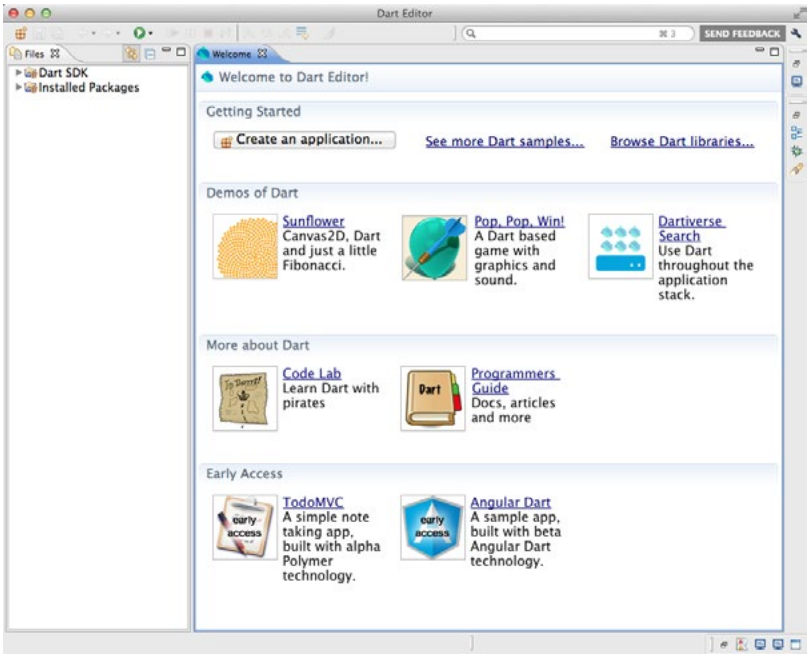


Figura 15. Primer vistazo de *Dart Editor*.

La primera vez que abres Dart Editor puedes observar tres grandes áreas. En la parte superior la barra de menús y opciones del editor. En la parte izquierda la vista de ficheros y aplicaciones (**Files view**). En la parte derecha la página inicial de bienvenida (**Welcome view**).

Más adelante verás más vistas, más panales y cómo configurar en profundidad Dart Editor. Por el momento con este primer vistazo es suficiente.

Abrir algunos ejemplos

Ya tenemos nuestra herramienta de desarrollo. Hemos echado un primer vistazo. Empecemos a jugar con algunos de los ejemplos que el Dart Team ha puesto a tu disposición.

Desde la vista bienvenida ves algunos ejemplos en Dart si haces clic en cualquiera de ellos podrás abrirlo y comenzar a ver todo el potencial de Dart.

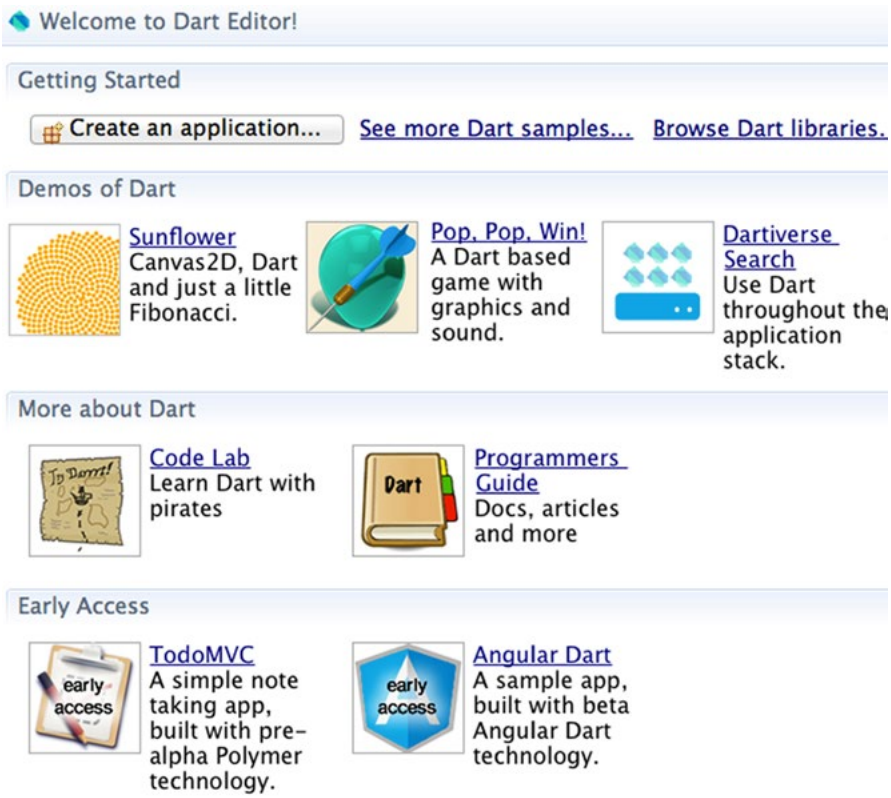


Figura 16. Abrimos el primer ejemplo de Dart.

Vamos a abrir el ejemplo Sunflower un ejemplo que nos permitirá ver como Dart trabaja con el DOM y como utiliza Canvas2D.

Haz clic en el título del ejemplo y verás como aparece toda la aplicación en vista de ficheros y aplicaciones (**Files view**).

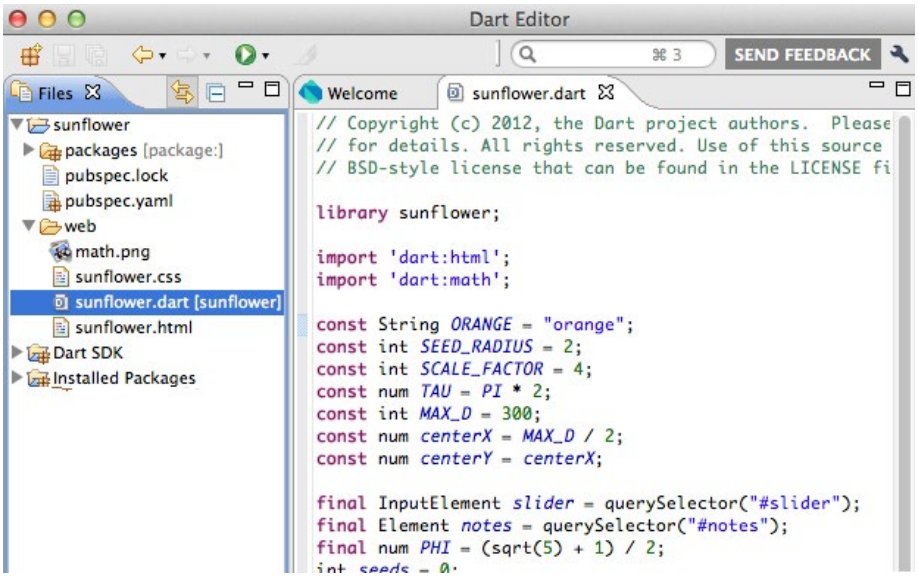


Figura 17. Dart Editor carga *Sunflower* el primer ejemplo Dart.

En el panel de la derecha cargan nuestros ficheros de código. En la parte izquierda vemos todo el contenido de la aplicación.

> En la vista de ficheros además puedes ver los paquetes instalados y el SDK completo de Dart. Este es uno de sus puntos fuertes ya que te permite navegar y estudiar el SDK de Dart sin necesidad de abandonar tu entorno de trabajo.
>
> nota

Puedes observar, viendo este ejemplo, que las aplicaciones de Dart constan de un directorio web donde incluimos nuestro código **.dart** (ficheros de código Dart), **.html** (ficheros HTML para la aplicación), **.css** (ficheros con la definición de estilos).

También en este directorio puedes incluir lo ficheros estáticos que necesiten tus aplicaciones, imágenes, vídeos, música, etc.

Además, las aplicaciones Dart constan de un fichero llamado **pubspec.yaml** donde defines los paquetes que necesitará tu aplicación y Dart Editor automáticamente (esto se puede configurar como veremos más adelante) se encarga de buscar dichos paquetes, descargarlos y dejarlos a tú disposición.

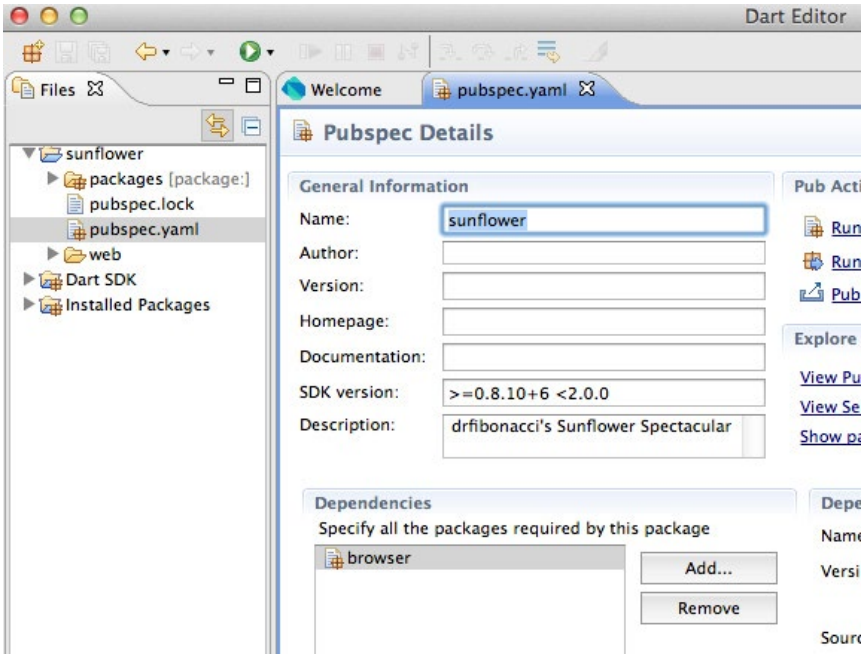


Figura 18. Vista de pubspec.yaml y paquetes utilizados en *Sunflower*.

Haz clic en el icono  y verás como Dart Editor abre una nueva ventana de un navegador (parecido a Google Chrome) llamado **Dartium** donde cargará el ejemplo *Sunflower*.

Si mueves suavemente la barra que se muestra en el ejemplo verás como el gráfico crece o disminuye.

nota

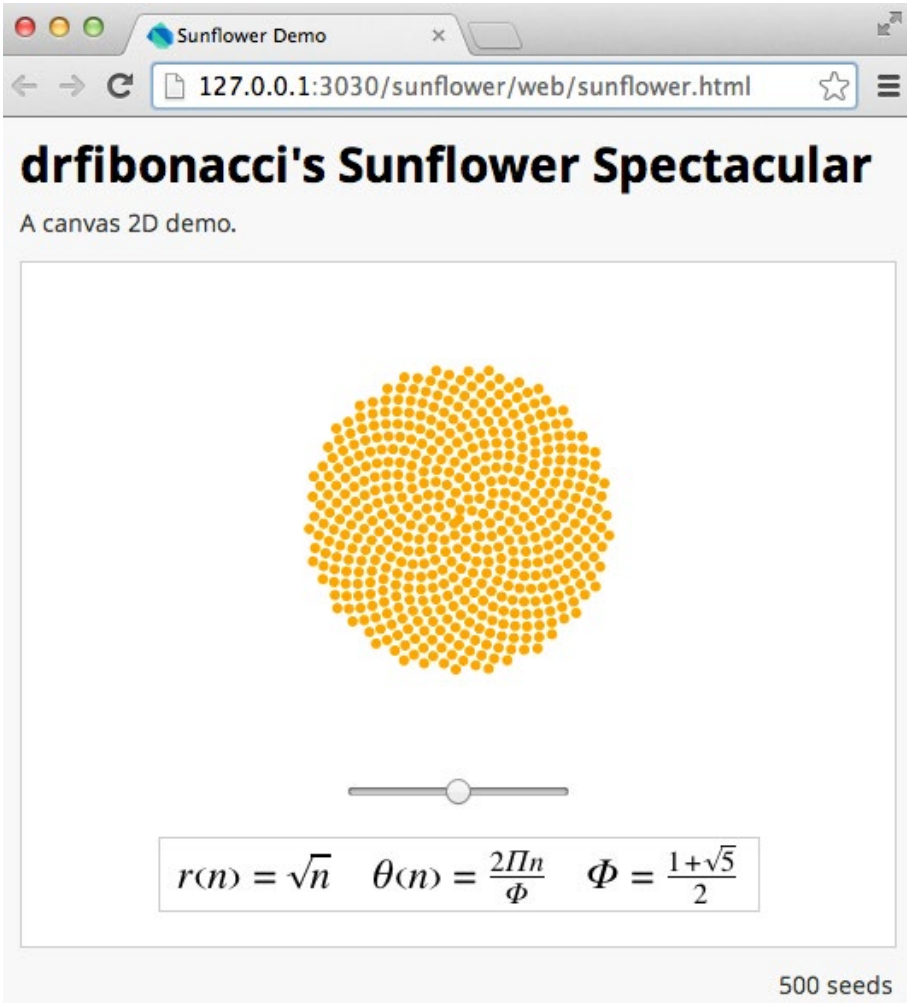


Figura 19. *Sunflower* ejecutándose en *Dartium*.

Crear una aplicación

Bien, ya has abierto una de las aplicaciones de ejemplo que se presentan en la pantalla de bienvenida de Dart Editor. También conoces cómo se deben estructurar las aplicaciones en Dart.

Vamos a crear ahora tu primera aplicación de ejemplo.

Desde la vista de bienvenida, puedes ver la opción de **Crear una aplicación (Create an application**). Haciendo clic en ella crearás una nueva aplicación, puedes hacer lo mismo a través del menú **Fichero > Nueva aplicación** (**File > New application**).

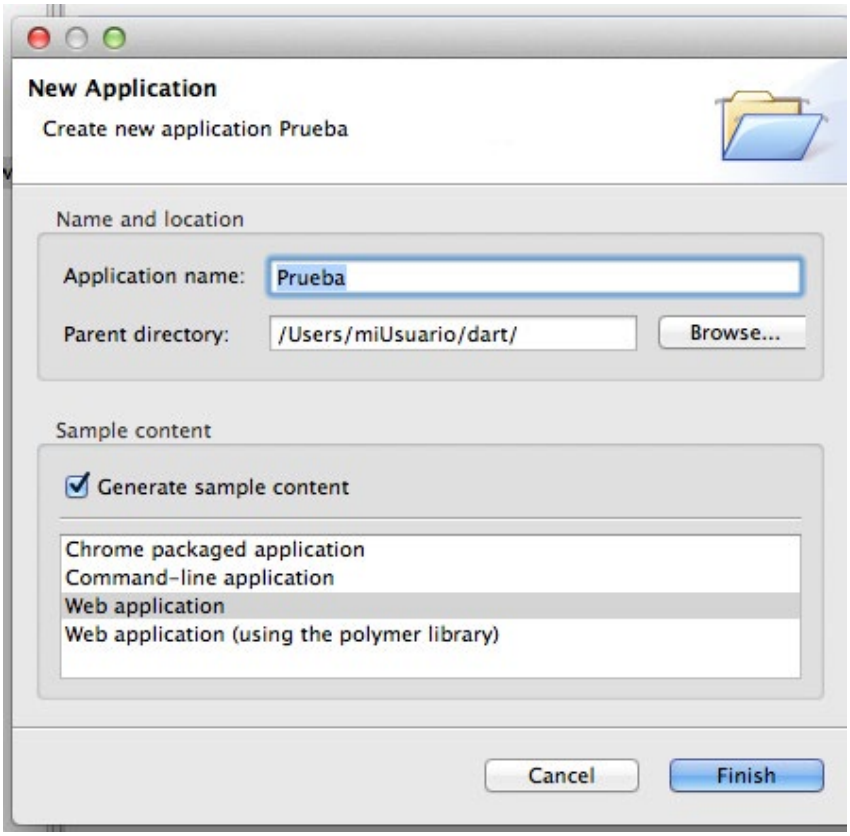


Figura 20. Creamos la primera aplicación de prueba de Dart

Al seleccionar la opción de crear una nueva aplicación te aparecerá una cuadro donde debes indicar el nombre de tu nueva aplicación y seleccionar como tipo de aplicación, en este caso **Aplicación web** (**Web Application**).

> Como ves Dart te permite crear aplicaciones para Chrome, de línea de comandos, aplicaciones web y aplicaciones web utilizando la librería Polymer. Iremos poco a poco todos estos tipos de aplicaciones.
>
> nota

Una vez seleccionado tipo y nombre, Dart Editor ya ha creado tu primera aplicación de ejemplo, siguiendo la estructura de directorios que te hemos comentado antes y el fichero pubspec.yaml más el directorio que contiene los paquetes (**packages**).

Si echas un vistazo rápido al código verás que hace un `import` de la librería **dart:html** necesaria para manejar todo el DOM del documento web. Crea dos métodos, `main` (que como ya sabes es el punto de entrada de las aplicaciones Dart) y `reverseText`, método que le da la vuelta a una cadena de texto.

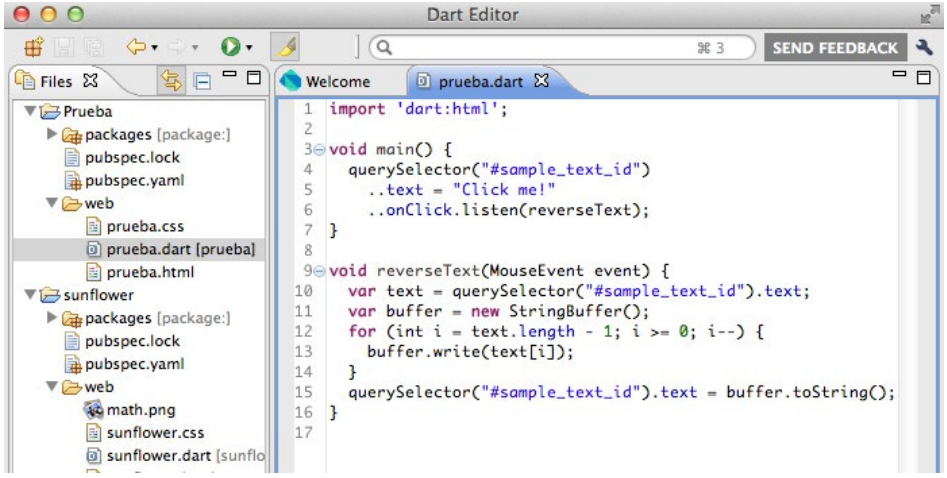


Figura 21. Código de nuestra primera aplicación de prueba.

Ejecutar aplicaciones

Como has visto anteriormente para ejecutar una aplicación en Dart Editor puedes pulsar el botón verde **Ejecutar aplicación**, ejecutar el comando **CMD+R** o pulsar el menú **Ejecutar > Ejecutar** (**Run > Run**).

También te puedes posicionar sobre el fichero .dart en la vista de ficheros y con el botón derecho hacer clic y seleccionar **Run in Dartium**.

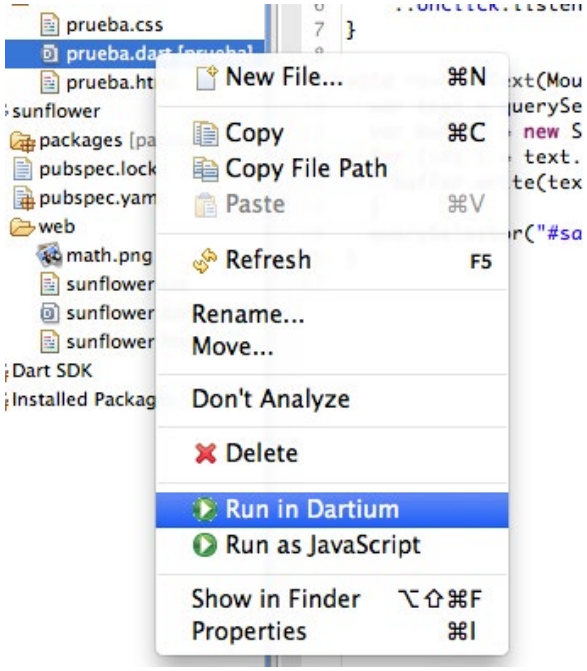


Figura 22. Ejecutando nuestra aplicación de ejemplo.

Con cualquiera de estos métodos la aplicación dart que tengas cargada en Dart Editor se ejecutará en Dartium.

> Las aplicaciones web al ejecutarse lo hacen en Dartium, que puede interpretar y ejecutar aplicaciones Dart nativas ya que dispone de la máquina virtual de Dart integrada.
>
> nota

¿Cómo funciona este sencillo ejemplo que acabo de crear?

Cuando lanzas la aplicación Dart busca el fichero HTML asociado a la aplicación web creada. Carga este fichero con todo su contenido y desde este fichero HTML es desde donde se invoca el fichero .dart, estilos (.css), imágenes, vídeos, etc.



Figura 23. Fichero HTML principal

Fíjate como en este caso también se invoca el paquete **browser**. Después de haber cargado el fichero HTML, Dart Editor carga las referencias a ficheros Dart.

En este ejemplo cargará el fichero **prueba.dart** y comenzará a ejecutar el método `main` presente en dicho fichero.

```
import 'dart:html';
void main() {
  querySelector("#sample_text_id")
    ..text = "Click me!"
..onClick.listen(reverseText);
}
```

El método `main` recupera el elemento párrafo ( ) *sample_text_id* del fichero prueba.html. Establece como texto de dicho elemento "Click me!" y le asocia al evento **click** la ejecución del método `reverseText`.

Cuando hagas clic en el elemento `<p id="sample _ text _ id">Click me !</p>` se ejecutará el método `reverseText`.

```
void reverseText(MouseEvent event) {
  var text = querySelector("#sample _ text _ id").text;
  var buffer = new StringBuffer();
  for (int i = text.length - 1; i >= 0; i--) {
    buffer.write(text[i]);
  }
  querySelector("#sample _ text _ id").text = buffer.toString();
}
```

Este método le da la vuelta al texto del elemento y vuelve a colocarlo en `<p id="sample _ text _ id"></p>`.

Ejemplos avanzados

Nos gustaría mostrarte, ahora que ya has abierto algunos ejemplos y creado los tuyos propios, todo el potencial de Dart.

Si volvemos a la pantalla de bienvenida en la sección de ejemplos de muestra existe uno llamado **Pop Pop Win** y otro llamado **Dartiverse Search**.

A modo de introducción queremos que veas de lo que es capaz de Dart tanto el lenguaje como las herramientas con las que estás trabajando. Empecemos con **Pop Pop Win**.

Abre la ventana de bienvenida en Dart Editor, selecciona el menú **Herramientas > Página de bienvenida** (**Tools > Welcome Page**) y cuando cargue selecciona el ejemplo **Pop Pop Win**, para que se abra este proyecto de ejemplo.



Figura 24. Seleccionando el juego Pop Pop Win

Una vez abierto podrás ver la magnitud de la aplicación que estamos viendo y podrás ver como han organizado y estructurado tanto el código como los elementos adicionales, audio, imágenes, etc.

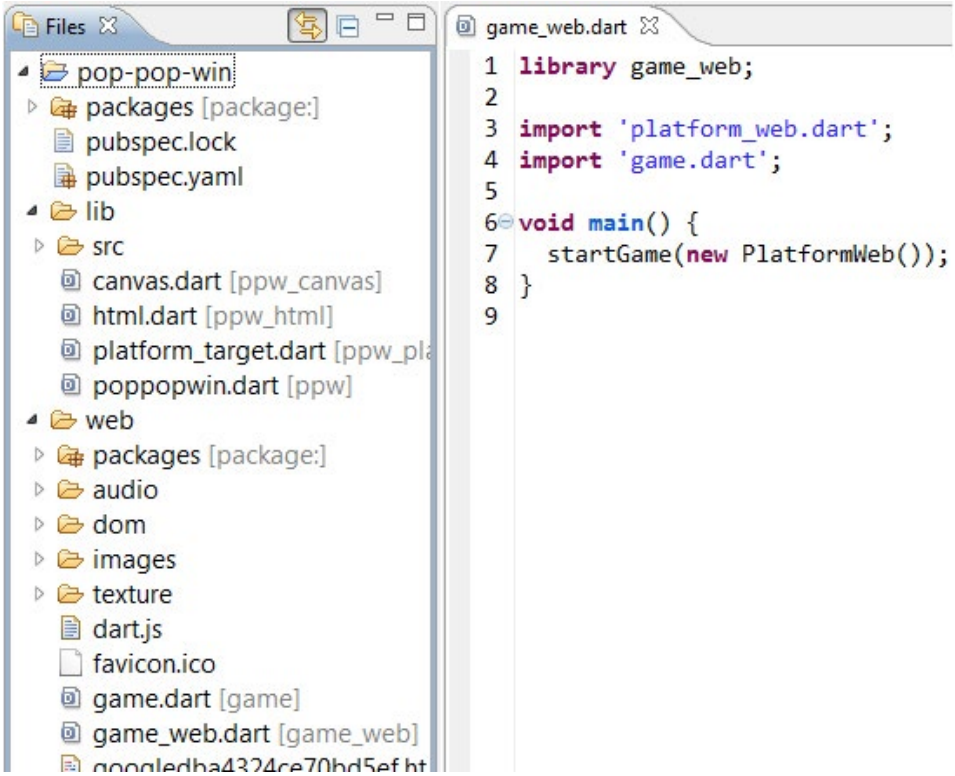


Figura 25. Estructura del ejemplo Pop Pop Win

Como puedes observar tenemos librerías y muchas clases para poder disfrutar de esta magnífica aplicación, pero no te asustes por el momento vamos a disfrutar de este juego.

Pulsa el botón **Ejecutar aplicación** y fíjate como carga el juego. Luego solo tendrás que hacer clic en los globos que desees hacer explotar. El funcionamiento del juego es similar al famoso "busca-minas".



Figura 26. Jugando con Pop Pop Win.

Los efectos del juego son muy buenos, como vuela el dardo hasta su objetivo, como explotan los globos, el audio y la estética general está muy cuidada.

Esto demuestra que es posible, con Dart, desarrollar juegos en una plataforma web nativa gracias a este lenguaje y a las nuevas características tanto de HTML5 como de CSS3.

Si al ejecutar Pop Pop Win no aparece una pantalla que indica que el juego está cargando como esta.



Figura 27. Pantalla inicial de Pop Pop Win.

Necesitas desactivar la opción de parada en excepciones (**Break on exceptions**) a través del menú **Herramientas > Preferencias** (**Tools > Preferences**). En el cuadro que aparece selecciona la opción del panel lateral **Depuración y ejecución** (**Run and Debug**) y selecciona **None** en la opción parada en excepciones (**Break on exceptions**).

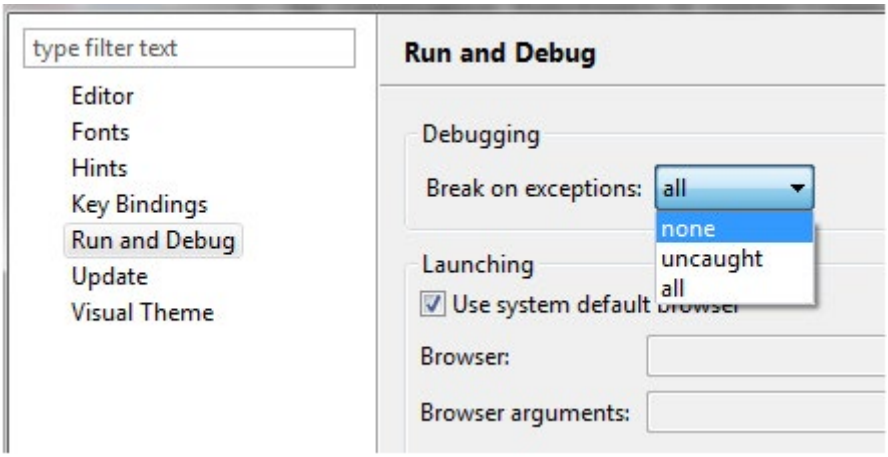


Figura 28. Configuramos Dart Editor para evitar que se pare durante alguna excepción.

Veamos ahora el ejemplo **Dartiverse Search**. Este ejemplo utiliza Dart para ejecutar un servidor http que muestra una página principal con un buscador que te permite realizar búsquedas relacionadas con términos de Dart en StackOverflow y en GitHub.

Parece algo muy sencillo, pero la potencia de este ejemplo está en cómo Dart también nos sirve para desarrollar en el lado servidor.

Para poder ejecutar este ejemplo y verlo en funcionamiento abre el fichero **bin/server.dart**, selecciona la opción **Herramientas > Pub build** y una vez haya finalizado este comando pulsa sobre el botón **Ejecutar aplicación**. Verás este mensaje.

```
INFO: 2014-02-18 13:09:41.635: Search server is running on 'http://127.0.0.1:9223/'
```

Que te indica que el servidor está corriendo en esa dirección `http://127.0.0.1:9223/`. Abre tu navegador y carga esa dirección.



Te aparecerá una pantalla con una caja de búsqueda para localizar recursos relacionados con Dart.

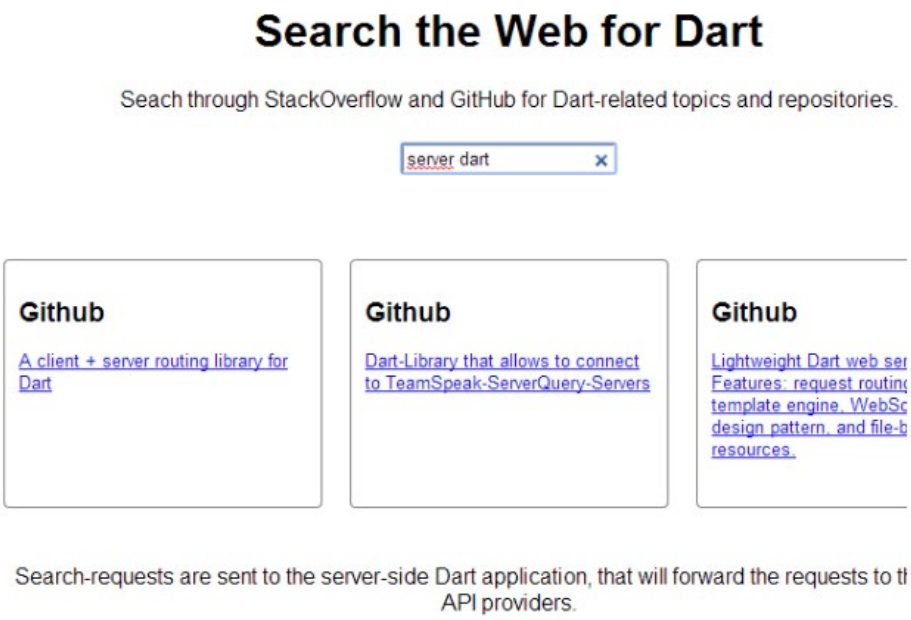


Figura 29. Ejemplo de Dart en el lado servidor.

Hemos buscado el término *server dart* para localizar más información sobre Dart en el lado servidor y estos son los recursos que encontramos.

Depurar aplicaciones

Ya has abierto algunos ejemplos, has creado una simple aplicación y la has podido ejecutar para ver como funciona y los resultados que produce.

Vamos a ir un paso más allá y vamos modificar un poco el código de la aplicación de prueba.

Vas a añadir algún efecto más en la función `reverseText` que, como ya has visto, le da la vuelta al texto del párrafo cuando haces clic sobre él.

¿Qué te parece si le añadimos un poco de color? Cuando hagas clic en el párrafo además de cambiar el texto y darle la vuelta vamos a hacer que cambie el color del contenedor del párrafo.

Dentro del HTML definido existe un elemento `<div id="sample_container_id"></div>` que es el contenedor del párrafo en cuestión.

Vamos a recuperar este elemento con `document.querySelector` y una vez lo tengamos le cambiaremos la propiedad `backgroundColor`.

```
import 'dart:html';
import 'dart:math';
void main() {
  querySelector("#sample_text_id")
    ..text = "Click me!"
    ..onClick.listen(reverseText);
}
void reverseText(MouseEvent event) {
  var list = ['#5a82d7', '#d35a5a', '#d75a5a' , '#5ad7d5', '#5ad773', '#d7c-
f5a'];
  var indice = new Random().nextInt(list.length - 1);
  var color = list[indice];
```

```
    var container = querySelector("#sample _ container _ id");
    container.style.backgroundColor = color;
    var text = querySelector("#sample _ text _ id").text;
    var buffer = new StringBuffer();

    for (int i = text.length - 1; i >= 0; i--) {
      buffer.write(text[i]);
    }
    querySelector("#sample _ text _ id").text = buffer.toString();
  }
```

Defines una lista con una serie de colores, recuperamos un color de manera aleatoria utilizando **dart:math** (hay que hacer el import de esta nueva librería).Recuperamos el DIV contenedor del DOM con querySelector y le cambiamos el color.

Cada vez que hagamos clic irá cambiando nuestro color de manera aleatoria. Imagina que tras hacer esto, hay algo que no funciona correctamente ¿cómo puedes depurar tus aplicaciones en Dart?

Muy sencillo, en la parte izquierda de cada línea de código puedes ver el número de línea, si haces doble clic en la parte izquierda de este número puedes establecer un punto de parada. Dart Editor marcará con un punto donde hayas hecho doble clic.

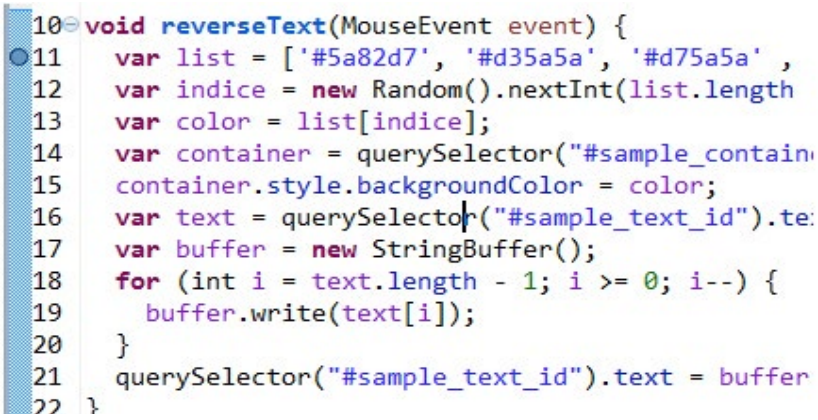


Figura 30. Establecer un punto de parada en Dart Editor.

Si ahora vuelves a ejecutar tu aplicación y haces clic en el párrafo que dice *Click me !* podrás ver que justo en dicho punto Dart Editor se detiene.



Figura 31. Dart Editor detenido en un punto de parada.

Observa que hemos mostrado en la pantalla adjunta la vista del depurador (**Debugger**) para que puedas observar que Dart Editor se detiene en es punto y nos da información ampliada.

Pulsando sobre las teclas de función F5, F6, F7, F8 podemos controlar el depurador.

- F5 entra dentro de la función que se esté ejecutando.
- F6 salta a la línea siguiente.
- F7 salta hasta el siguiente punto de parada.
- F8 ejecuta hasta el final.

También puedes utilizar los botones de la vista del depurador.

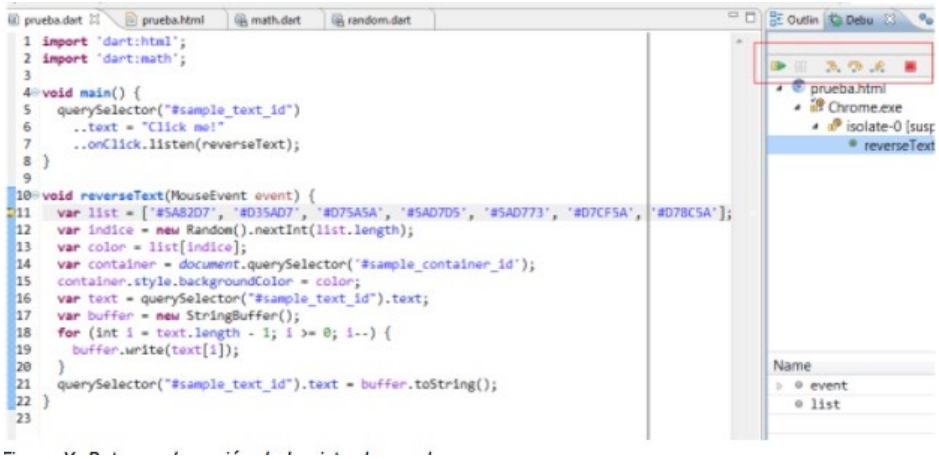


Figura 32. Botones de acción de la vista depurador.

Para hacer visible la vista del depurador haz clic en **Tools > Debugger**.

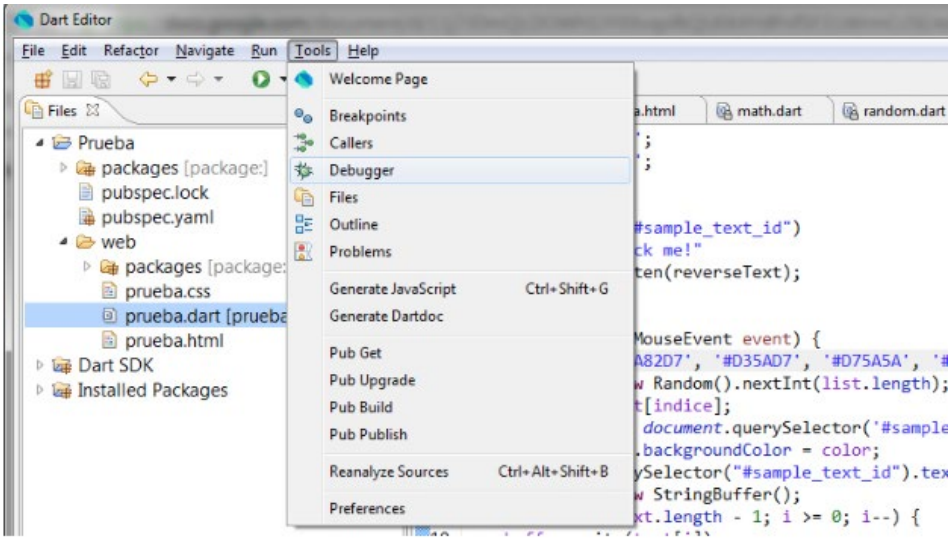


Figura 33. Mostrar la vista del depurador.

Compilar a JavaScript

Ya has creado tu primera aplicación Dart, has aprovechado el ejemplo que te brinda el propio Dart Editor y le has añadido funciones nuevas.

Esta aplicación corre perfectamente en Dartium, pero tú quieres que todo el mundo pueda disfrutar de ella. Llegamos a la compilación a JavaScript.

Actualmente solo Dartium puede correr aplicaciones Dart porque tiene incluída la máquina virtual de Dart (**DartVM**).

Con el tiempo Dart funcionará de manera nativa en todos los navegadores y dentro de poco de manera nativa en Google Chrome porque incluirá la máquina virtual de Dart.

Si ahora mismo quieres que tus aplicaciones funcionen en el resto de navegadores debes compilar tu código Dart a JavaScript, actualmente todos los navegadores disponen de un motor que reconoce JavaScript.

Un momento. ¿Pero no decíais que esto de Dart era estupendo y sencillo?. A mí eso de "compilar" me suena muy mal. No te preocupes es un proceso absolutamente transparente igual de sencillo que ejecutar tu aplicación en Dartium.

Para compilar tu aplicación a JavaScript simplemente haz clic con el botón derecho sobre el fichero **pubspec.yaml** de tu aplicación y selecciona **Pub Build (Generate JS).**

El proceso de compilación lo realiza **dart2js**, transforma la aplicación de código Dart en código JavaScript y puede durar más o menos dependiendo de la cantidad de código de tu aplicación.

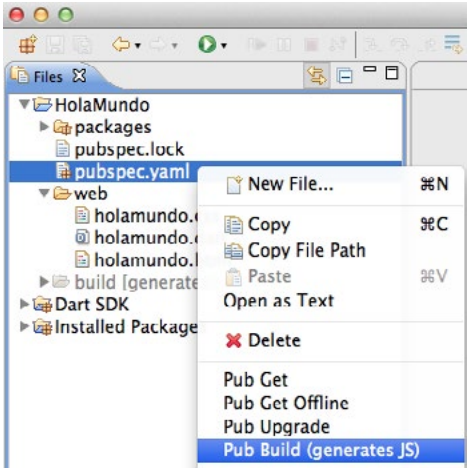


Figura 34. Compilando en JavaScript.

Cuando lances la compilación a JavaScript podrás ver que se abre una nueva ventana llamada **Output** donde te informa de la evolución del proceso de compilación.

Una vez hayas finalizado el proceso de compilación podrás ver que se han generado varios ficheros JavaScript en el panel de ficheros y aplicaciones.

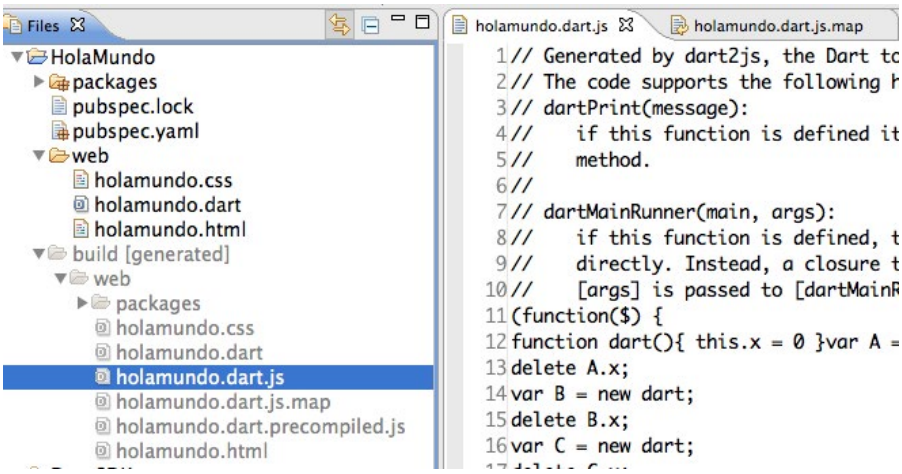


Figura 35. Ficheros JavaScript generados tras la compilación.

Vamos a comentar brevemente los ficheros generados.

- prueba.dart.js. Fichero JavaScript con el código de tu aplicación compilado a JavaScript.
- prueba.dart.js.map. Fichero de mapeo entre ficheros y nombres.
- prueba.dart.precompiled.js. Fichero intermedio generado por dart2js.

Inspeccionar el SDK

Como ya hemos comentado durante esta sección una de las grandes virtudes que tiene Dart Editor es que te permite explorar el SDK mientras estás trabajando.

Por ejemplo durante tu primera aplicación de prueba, has importado **dart:math** para poder utilizar la clase Random y generar números aleatorios para obtener un color al azar de la lista de colores.

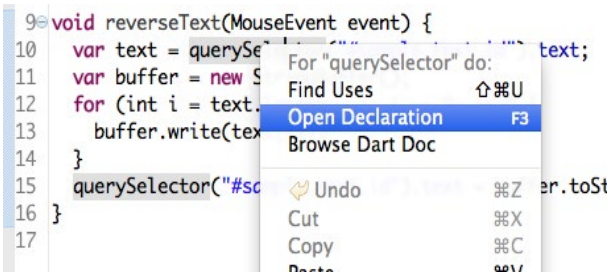


Figura 36. Accedemos a document.querySelector de dart:html

También hemos utilizado algunos métodos propios de **dart:html** para manejar el DOM como `querySelector`.

Si en cualquier momento te surge alguna duda sobre una clase o un método, sitúate sobre esa clase o método y pulsa **F3** o haz clic con el botón derecho y selecciona **Open Declaration**.

Verás como automáticamente Dart Editor te abre **dart:html** y te lleva al método `document.querySelector`.

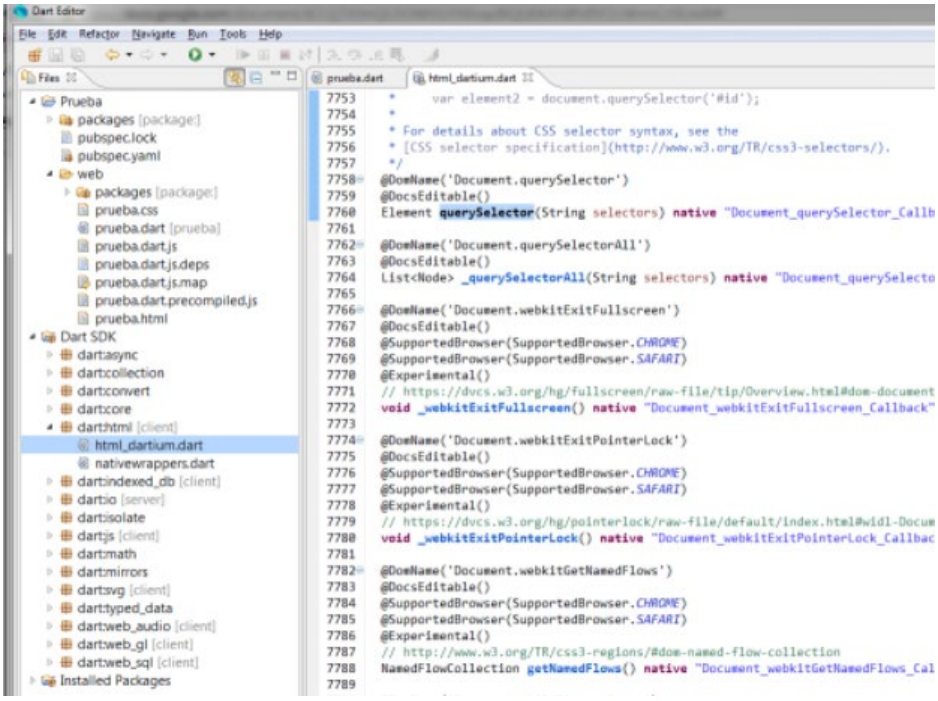


Figura 37. Mostrando el método querySelector de document en dart:html.

Si pulsamos F3 sobre la clase `Random` veremos como Dart Editor nos lleva a la clase `Random` de `dart:math`.

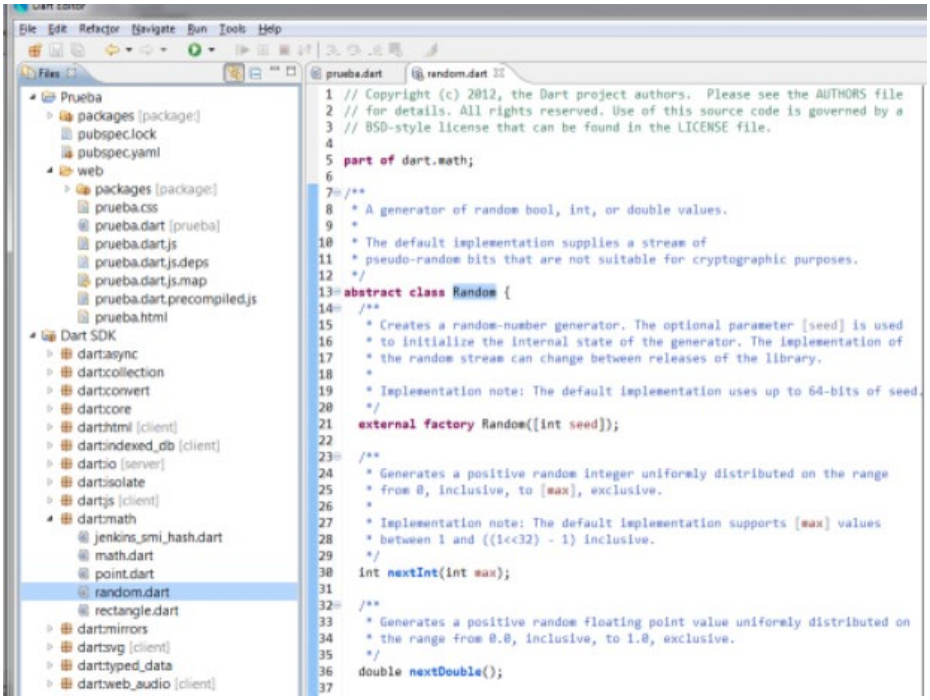


Figura 38. Explorando la clase Random en Dart Editor

Por supuesto, una vez estés explorando el SDK de Dart puedes seguir navegando entre clases y métodos pulsando **F3** o seleccionando la opción **Open declaration** del menú contextual.

Como has podido observar en el SDK todo está perfectamente documentado y explicado y en muchos casos inclusos podrás ver ejemplos de código.

Por poner un simple ejemplo el método `sort()` de la clase `List` de `dart:core`.

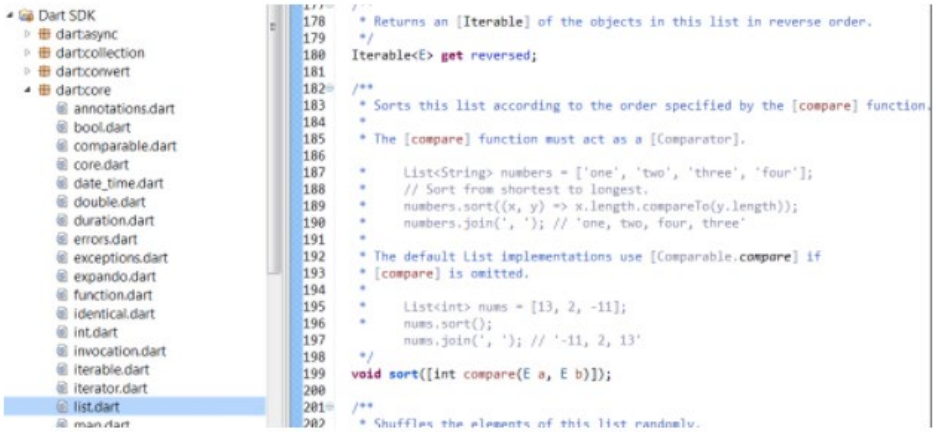


Figura 39. Ejemplo de código en la documentación del SDK

Configuración y vistas

Hemos dado un repaso rápido, pero en profundidad de las principales funciones y herramientas de Dart Editor.

Veamos ahora cómo configurar el editor y las vistas que predefinidas. Si quieres ver todas las vistas que tiene disponibles Dart Editor puedes mostrarlas utilizando el menú **Tools**.

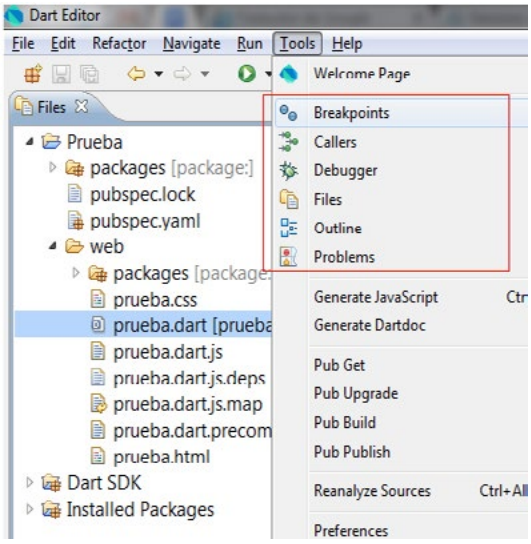


Figura 40. Vistas disponibles en Dart Editor.

- **Breakpoints**: vista que te permite tener acceso rápido a los puntos de parada que hayas establecido en tu aplicación.
- **Callers**: vista que permite ver de un vistazo los métodos que llaman a otros métodos. Puedes utilizar también el botón derecho y seleccionar **Find Uses/Find Callers** para acceder rápidamente a los métodos/clases que llaman al método seleccionado. Encontrarás esta opción muy útil cuando tus aplicaciones crezcan.
- **Debugger**: es la vista del depurador que hemos visto en las secciones anteriores que te permite manejar el debugger de Dart Editor.
- **Files**: Vista de ficheros y aplicaciones que te permitirá gestionar tus proyectos.
- **Outline**: Una de las vistas más utilizadas, te muestra de un vistazo rápido todas las clases y métodos que las componen. Además te permite dirigirte a cualquier método o clase haciendo clic sobre él.
- **Problems**: Esta vista te permite ver cualquier error o problema que exista en tus aplicaciones y acceder a él rápidamente haciendo doble clic.

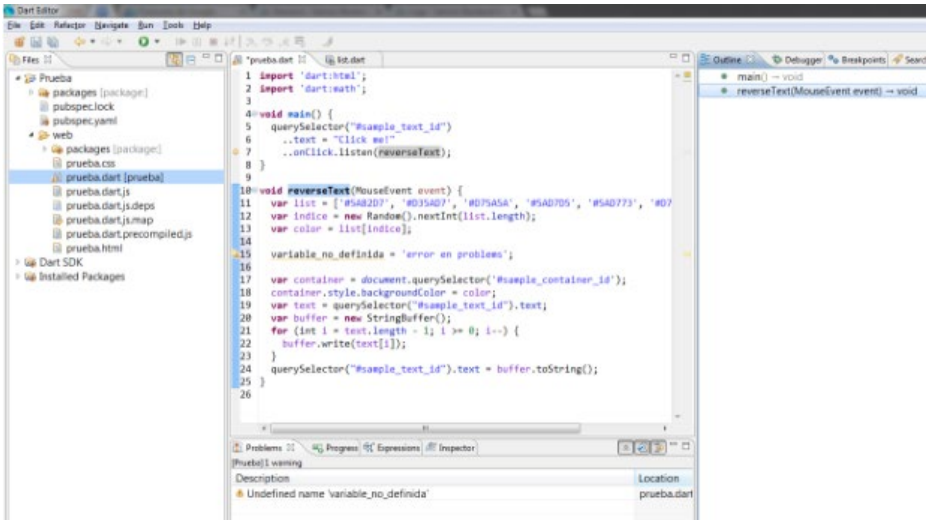


Figura 41. Detalle de la vista Outline y Problems de Dart Editor.

Las principales funciones de Dart Editor se pueden configurar desde el menú **Tools > Preferences**.

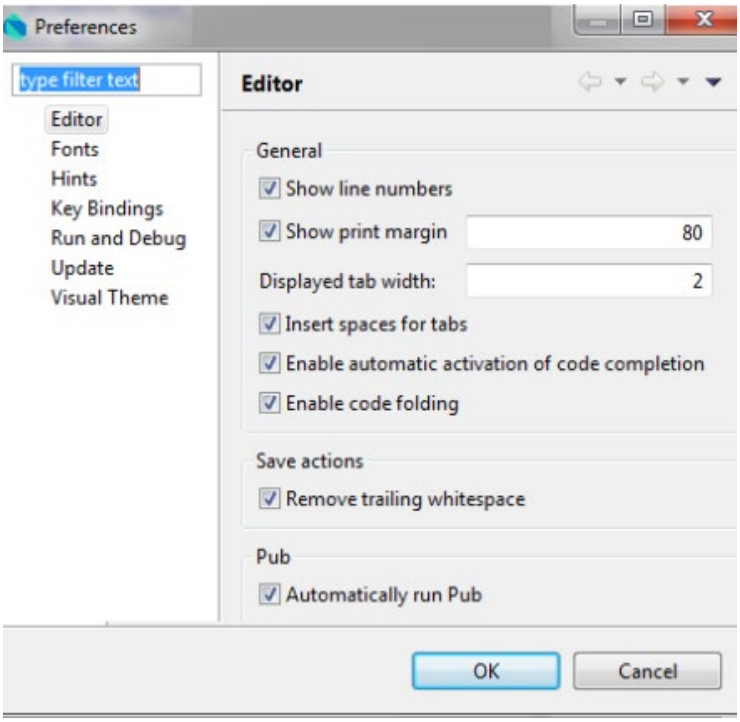


Figura 42. Configuración de preferencias de Dart Editor.

Desde aquí podrás cambiar el tipo y tamaño de letra del editor. Si deseas o no mostrar margen de 80 caracteres por línea, como quieres que se visualicen los tabuladores del editor de texto y si quieres que se reemplacen por espacios.

Puedes activar auto-completado de código y la posibilidad de plegar el código (code folding) para facilitarte el trabajo con grandes ficheros de código.

En el apartado **Hints** puedes habilitar si deseas o no que Dart Editor te de avisos y pistas que te ayudan en tu trabajo diario.

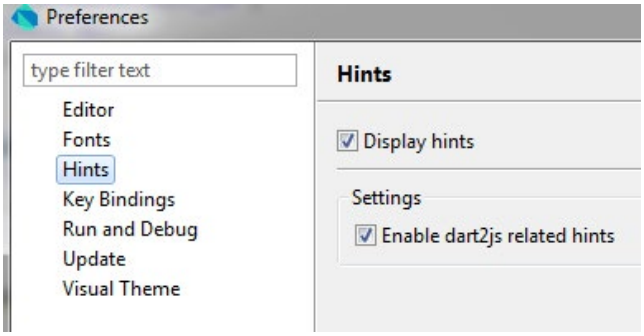


Figura 43. Habilitar Hints en Dart Editor.

También puedes ajustar algunos parámetros de actualización del editor y del tema visual del mismo en el menú **Update** y en **Visual theme**.

Por último, en el apartado **Run and Debug** puedes establecer algunas configuraciones propias de ejecutar y depurar aplicaciones como por ejemplo, si deseas que Dart Editor se detenga en cualquier excepción que se produzca que no haya sido capturada.

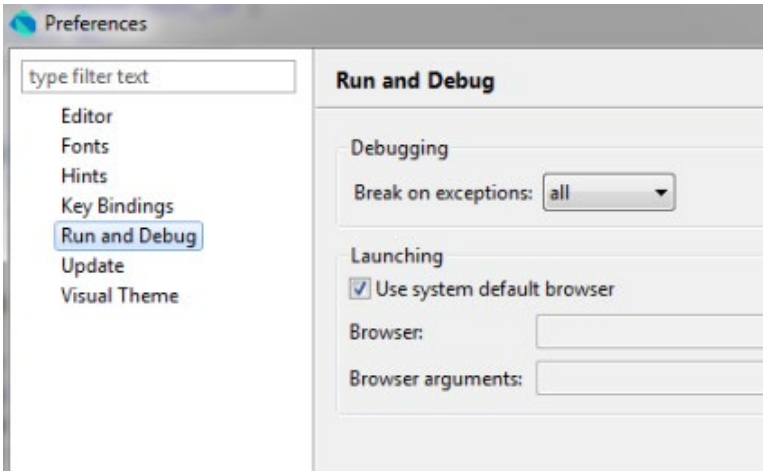


Figura 44. Preferencias sobre ejecución y depuración de código.

Configuración de las vistas

Ahora que ya sabes qué vistas puedes tener a tu disposición con Dart Editor y qué preferencias puedes configurar te interesa saber cómo configurar las vistas para que tengas un buen espacio de trabajo.

Cuando muestras un vista por defecto se puede ir a un lateral o a la parte inferior del editor. Haz clic sobre el nombre de la vista y sin soltar arrastrarlo a cualquier otra parte de editor, podrás ver como puedes anclar diferentes vistas en el panel lateral derecho o en la parte inferior.

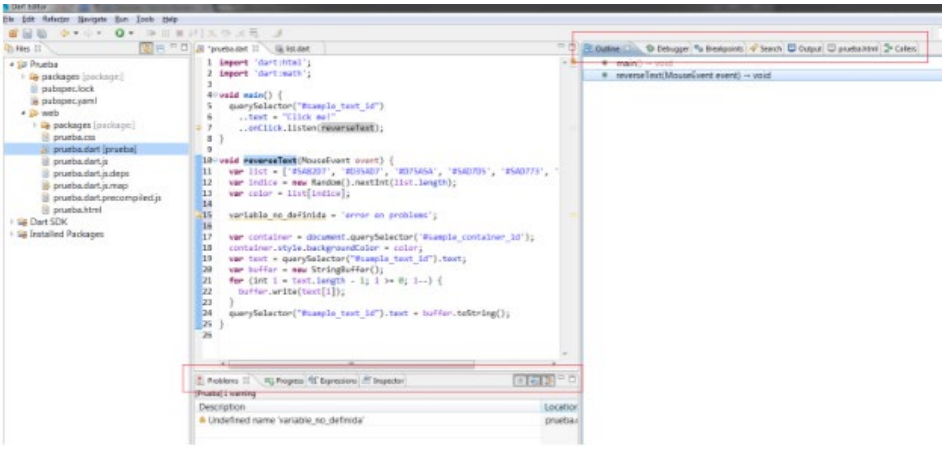


Figura 45. Anclando vistas en la parte inferior y en el panel lateral derecho.

Lanzadores y su configuración

Cada vez que hemos pulsado el comando **CMD+R** o el botón verde Correr hemos visto como nuestra aplicación comenzaba a ejecutarse. Al mismo tiempo se ha creado un lanzador, veamos que son y cómo trabajar con ellos.

Los lanzadores son configuraciones que realizamos sobre nuestras aplicaciones para que se ejecuten de una determinada forma en *Dartium* o en la máquina virtual de Dart (*DartVM*).

Si haces clic en la flecha junto al botón **Correr** verás que se despliega un menú con todos los lanzadores que hayas ejecutado y una opción que dice **Manage Launches**. Puedes acceder a esta misma opción también desde el menú **Run > Manage Launches**.

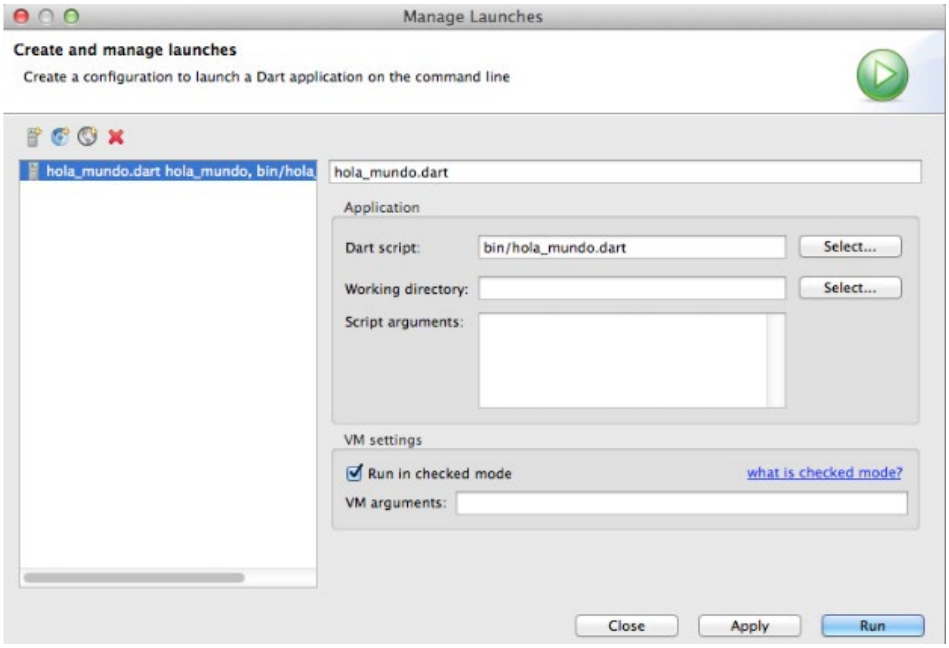


Figura 46. Creando y configurando los lanzadores

Desde esta nueva ventana podemos crear lanzadores para nuestras aplicaciones y además podemos indicar ciertos parámetros de interés para los lanzadores.

Esta ventana y sus opciones cambiarán en función del tipo de lanzador que creemos sea para una aplicación de línea de comandos, una aplicación web que corra en *Dartium* o una aplicación web que se compile a JavaScript.

Lanzador de una aplicación de línea de comandos

En el ejemplo que se muestra estamos creando un lanzador para una aplicación de línea de comandos. Podemos indicar parámetros para el script, establecer el directorio de trabajo, si queremos que se ejecute habilitando el modo de comprobación (*checked mode*) e incluso añadir ciertos argumentos para que los tenga en cuenta la máquina virtual de Dart.

¿Qué es el modo de comprobación o *checked mode*?

Este modo permite realizar comprobaciones cuando se ejecutan las aplicaciones durante la fase de desarrollo. Realiza validaciones de tipo de datos en el paso de parámetros entre funciones, cuando se realizan asignaciones y cuando se retornan resultados de métodos o funciones.

Cuando estás probando tu aplicación y el modo de comprobación está activo, si se produce un error de tipos la aplicación se detiene y te devuelve un mensaje de error con la descripción del suceso.

Así tienes la opción de capturar errores durante el proceso de desarrollo de tus aplicaciones.

> El modo comprobación o *checked mode* es recomendable habilitarlo durante el desarrollo y la fase de pruebas.

Veamos qué opciones tenemos cuando creamos un lanzador para una aplicación web en Dartium o una aplicación web compilada a JavaScript.

Lanzador de una aplicación Web Dartium

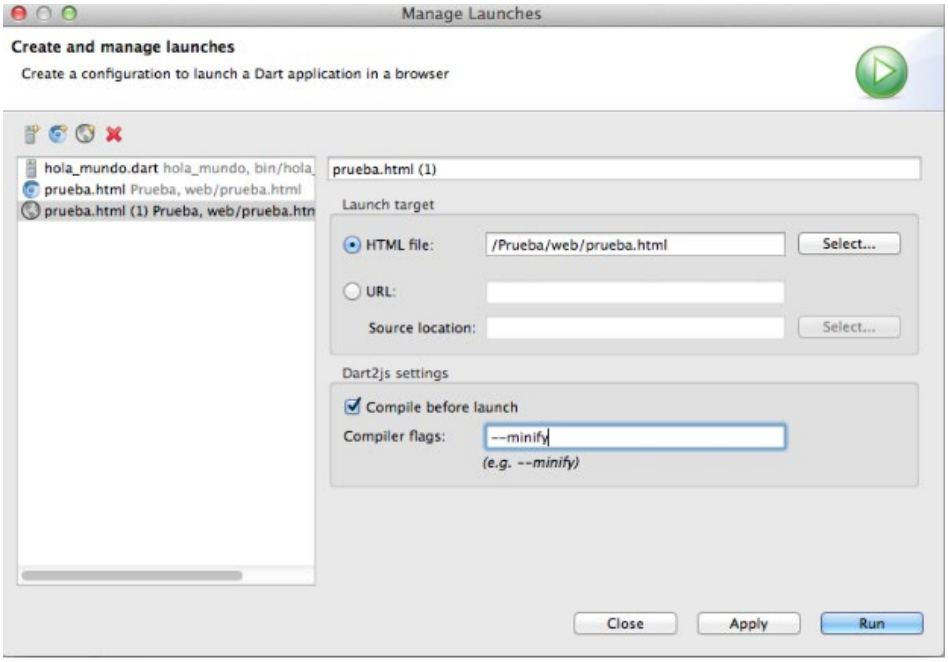


Figura 47. Lanzador de una aplicación web que ejecuta sobre Dartium.

En este caso indicaremos el fichero HTML contenedor de nuestra aplicación, aunque también tenemos opción de cargar este contenedor desde una URL, por ejemplo si lo queremos cargar desde un servidor local, por ejemplo `http://localhost/prueba.html`.

Además tenemos una serie de parámetros extra para *Dartium*, habilitar o no el modo de comprobación, habilitar funciones experimentales como los **Web Components** y mostrar la salida estándar y la salida de errores del navegador en la vista **Output** de Dart Editor.

Igual que con las aplicaciones de línea de comandos tienes opción de pasar argumentos extra al navegador.

Lanzador de aplicación Web compilada a JavaScript

Por último, veamos ahora los lanzadores de las aplicaciones web compiladas a JavaScript.

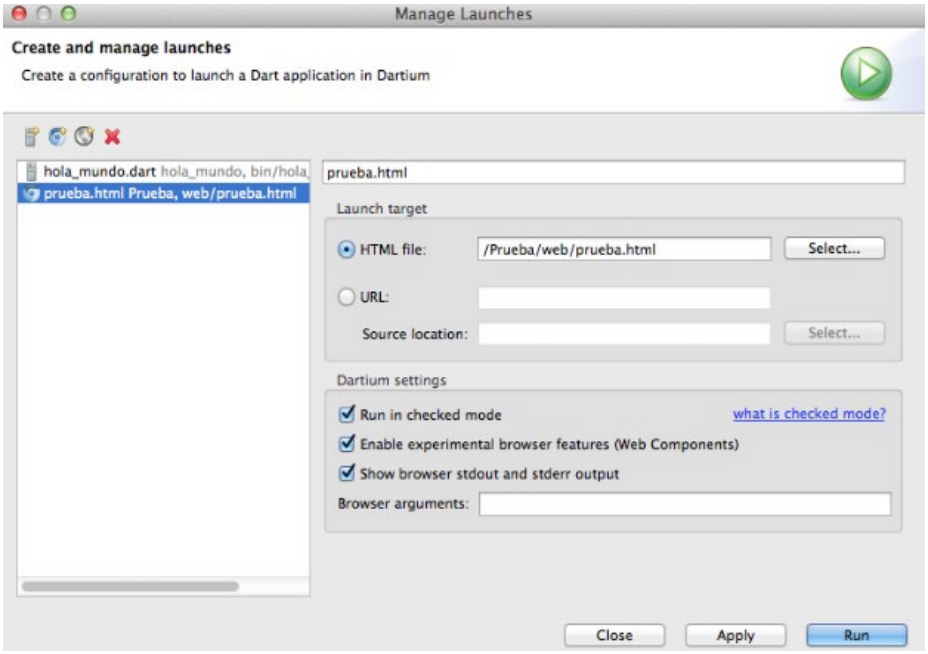


Figura 48. Lanzador de una aplicación web compilada a JavaScript.

Este tipo de lanzadores son los que se ejecutan cuando quieres compilar tu aplicación Dart a código JavaScript.

Igual que con las aplicaciones web en *Dartium* podemos establecer cual será el fichero HTML contenedor de la aplicación o una url que lo cargue dinámicamente y podemos establecer los parámetros y modificadores para la herramienta **dart2js**.

En el ejemplo que se muestra hemos establecido un parámetro que indica a *dart2js* que compile a JavaScript en versión comprimida (*minified*) de este modo el fichero JavaScript resultante reduce su tamaño agilizando la carga de nuestras aplicaciones.

Dart2js

Dart Editor es la herramienta perfecta para comenzar a trabajar con Dart de manera sencilla y rápida. Al hablar de Dart Editor y al realizar algunos ejercicios sencillos ya hemos utilizado **dart2js**, por ejemplo cuando hemos visto los lanzadores y como se compilan las aplicación a código JavaScript.

Esta herramienta te va a permitir compilar tus aplicaciones escritas en Dart a código JavaScript y permitir que funcionen en todos los navegadores web modernos.

Desde Dart Editor ya has utilizado esta poderosa herramienta muy cómodamente, pero ahora le veremos en detalle y te la enseñaremos como aplicación independiente que puedes utilizar.

Como ves Dart no solo es poderoso por el lenguaje sino por todas las herramientas que también componen este fantástico ecosistema de desarrollo.

Veamos un fichero de ejemplo con código Dart. `hola_mundo.dart`

```
void main() => print("Hola, Mundo!");
```

Ejecutando este comando puedes convertir este fichero Dart a código JavaScript.

```
dart-sdk/bin$ dart2js hola_mundo/bin/hola_mundo.dart
```

```
Dart file (dart/hola_mundo/bin/hola_mundo.dart) compiled to JavaScript:
out.js
```

Tras ejecutar este comando *dart2js* te informa que ha generado el fichero *out.js* con un contenido similar a este.

hola_mundo.dart | out.js

```
// Generated by dart2js, the Dart to JavaScript compiler version: 1.1.1.
// The code supports the following hooks:
// dartPrint(message)    - if this function is defined it is called
//                         instead of the Dart [print] method.
// dartMainRunner(main) - if this function is defined, the Dart [main]
//                         method will not be invoked directly.
//                         Instead, a closure that will invoke [main] is
//                         passed to [dartMainRunner].
(function($) {
function dart() {}var A = new dart;
delete A.x;
var B = new dart;
delete B.x;
var C = new dart;
delete C.x;
var D = new dart;
delete D.x;
var E = new dart;
delete E.x;
var F = new dart;
```

Figura 49. out.js fichero Dart *hola_mundo.dart* compilado a JavaScript.

Como has podido ver, tú mismo manualmente puedes realizar las mismas acciones de compilación que realiza Dart Editor. Este tipo de herramientas son muy útiles cuando necesitas crear un script que se ocupe de realizar automáticamente la compilación de todos los ficheros de código.

Veamos ahora algunos parámetros que mejorarán nuestro trabajo con *dart2js*.

Al tratarse de una herramienta en línea de comandos disponemos de algunos parámetros y modificadores que deberás conocer para que le puedas sacar todo el potencial a esta herramienta.

Ejecutando este comando puedes obtener información de las opciones disponibles.

```
    dart-sdk/bin$ dart2js -h
    Usage: dart2js [options] dartfile

    Compiles Dart to JavaScript.

Common options:
  -o <file> Generate the output into <file>.
  -c        Insert runtime type checks and enable assertions (checked mode).
  -m        Generate minified output.
  -h        Display this message (add -v for information about all options).
```

La opción `-o` (`output`) seguida del nombre del fichero destino te permite indicar cual será el fichero JavaScript que almacenará el código JavaScript.

La opción `-c` (`checked`) permite habilitar el modo de comprobación (*checked mode*).

La opción `-m` (`minifiy`) permite generar código JavaScript en versión ofuscada y comprimida. Es muy interesante ya que el código reduce su tamaño y se mejora la velocidad de carga de las aplicaciones.

Por último, la opción `-h` (`help`) muestra la ayuda y documentación de la aplicación dart2js y si añades el parámetro -v puedes obtener información ampliada.

```
    dart-sdk/bin$ dart2js -hv
    Usage: dart2js [options] dartfile

    Compiles Dart to JavaScript.

    Supported options:
      -o <file>, --out=<file>
        Generate the output into <file>.

      -c, --enable-checked-mode, --checked
        Insert runtime type checks and enable assertions (checked mode).

      -m, --minify
        Generate minified output.

      -h, /h, /?, --help
        Display this message (add -v for information about all options).
```

```
-v, --verbose
  Display verbose information.

-D<name>=<value>
  Define an environment variable.

--version
  Display version information.

-p<path>, --package-root=<path>
  Where to find packages, that is, "package:..." imports.

--analyze-all
  Analyze all code.  Without this option, the compiler only analyzes
  code that is reachable from [main].  This option is useful for
  finding errors in libraries, but using it can result in bigger and
  slower output.

--analyze-only
  Analyze but do not generate code.

--analyze-signatures-only
   Skip analysis of method bodies and field initializers. This option
  implies --analyze-only.

--suppress-warnings
  Do not display any warnings.

--suppress-hints
  Do not display any hints.

--enable-diagnostic-colors
  Add colors to diagnostic messages.

--terse
  Emit diagnostics without suggestions for how to get rid of the
  diagnosed problems.
```

The following options are only used for compiler development and may be removed in a future version:

```
--output-type=dart
  Output Dart code instead of JavaScript.

--throw-on-error
```

```
  Throw an exception if a compile-time error is detected.

--library-root=<directory>
  Where to find the Dart platform libraries.


--allow-mock-compilation
  Do not generate a call to main if either of the following
  libraries are used: dart:dom, dart:html dart:io.

--enable-concrete-type-inference
  Enable experimental concrete type inference.

--disable-native-live-type-analysis
  Disable the optimization that removes unused native types from
  dart:html and related libraries.

--categories=<categories>
  A comma separated list of allowed library categories.  The default
  is "Client".  Possible categories can be seen by providing an
  unsupported category, for example, --categories=help.  To enable
  all categories, use --categories=all.

--dump-info
 Generates an out.info.html file with information about generated code.
```

Vamos a realizar algunas pruebas y veamos como cambia el fichero *out.js* que hemos creado al comienzo de este apartado.

Vamos a cambiar el fichero destino y vamos a indicarle a dart2js que queremos el fichero minificado.

```
dart-sdk/bin$ dart2js -o hola_mundo.js -m -c hola_mundo.dart
```

Cuando **dart2js** haya finalizado su trabajo tendremos nuestra aplicación compilada a JavaScript y comprimida en el fichero hola_mundo.js.

Si echas un vistazo a los tamaños te sorprenderá ver como el tamaño del *hola_mundo.js* (33K) generado comprimido es superior a *out.js (16K)* que se generó sin comprimir, ¿a qué es debido esto?.

Si recuerdas hemos habilitado el modo de comprobaciones que como ya sabes realiza comprobaciones de tipado en tiempo de ejecución.

Si vuelves a lanzar el comando eliminando el parámetro `-c`.

```
dart-sdk/bin$ dart2js -o hola_mundo.js -m hola_mundo.dart
```

Fíjate ahora en los tamaños

out.js: 16K

hola_mundo.js: 14K

En una aplicación tan pequeña como esta hemos ahorrado 2K, estamos hablando de un ahorro del 12,5%, imagina el tamaño del ahorro cuando tengamos una aplicación con 15000 líneas de código, por ejemplo.

PUB: El gestor de paquetes de Dart

PUB es el gestor de paquetes de Dart. Es la herramienta que nos permite añadir a nuestras aplicaciones cualquier librería que necesitemos escrita por otro programador de una manera muy cómoda y simple.

PUB se ocupa automáticamente de resolver dependencias y de mantenernos al tanto de las últimas versiones si así se lo indicamos.

Un paquete es una aplicación Dart con un fichero de especificación de dependencias, es decir, un directorio que contiene una serie de ficheros de código y, tal vez, otras librerías.

También puede contener otros recursos como documentación, ejemplos e imágenes, más un fichero de configuración que indica la versión del paquete, información extra de interés y las librerías de las que depende para poder funcionar correctamente.

Gracias a todo esto podemos utilizar PUB para descargar un paquete concreto, comenzar a utilizarlo y PUB automáticamente descarga todo lo que necesite para hacerlo funcionar.

Es una maravilla trabajar de este modo tan simple. Vamos a ver un ejemplo práctico de uso de PUB y los diferentes comandos que tenemos a nuestra disposición para utilizarlo.

Sí, efectivamente, igual que dart2js, PUB es una aplicación independiente (la podrás utilizar también desde línea de comandos) que viene perfectamente integrada dentro de Dart Editor para facilitarte el trabajo.

Imagina que estás escribiendo una aplicación HTML5 para una tienda de comercio electrónico que permite al usuario que la utiliza crear una lista de productos que desea, pero que no quiere comprar ahora mismo.

Sería fantástico poder almacenar estos artículos para que la próxima vez que el usuario venga a nuestra tienda recordarle lo que deseaba comprar.

Pensamos entonces en almacenar estos artículos en el navegador del cliente y pensamos en IndexedDB.

Este fantástico API nos permite crear pequeñas bases de datos en el navegador del cliente y trabajar con ellas.

Dirígete a http://pub.dartlang.org y busca algo relacionado con IndexedDB. Vemos que aparece un paquete que tiene muy buena pinta y que nos permite gestionar no solo *IndexedDB* sino también *local storage* y *websql*.

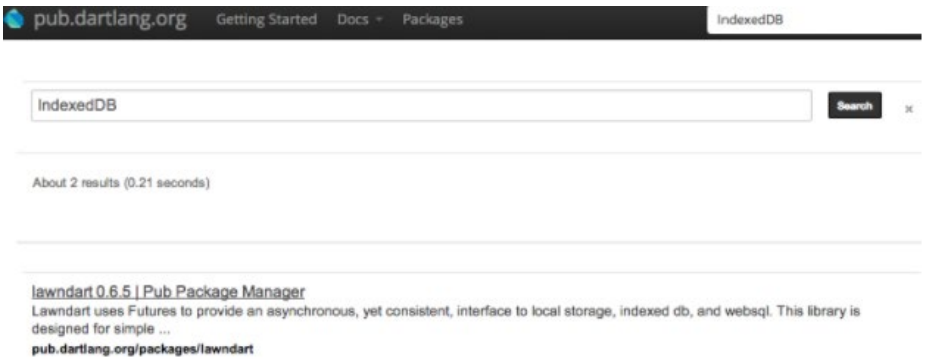


Figura 50. Buscando un paquete para manejar IndexedDB en pub.dartlang.org

Esto pinta bien. Ya tenemos el paquete **lawndart** localizado, ahora ¿cómo lo incluimos en nuestra aplicación?

Vamos a Dart Editor y nos abrimos una nueva aplicación web, le decimos a Dart Editor que nos cree la nueva aplicación web llamada **testPub** y que no nos cree contenido de ejemplo.

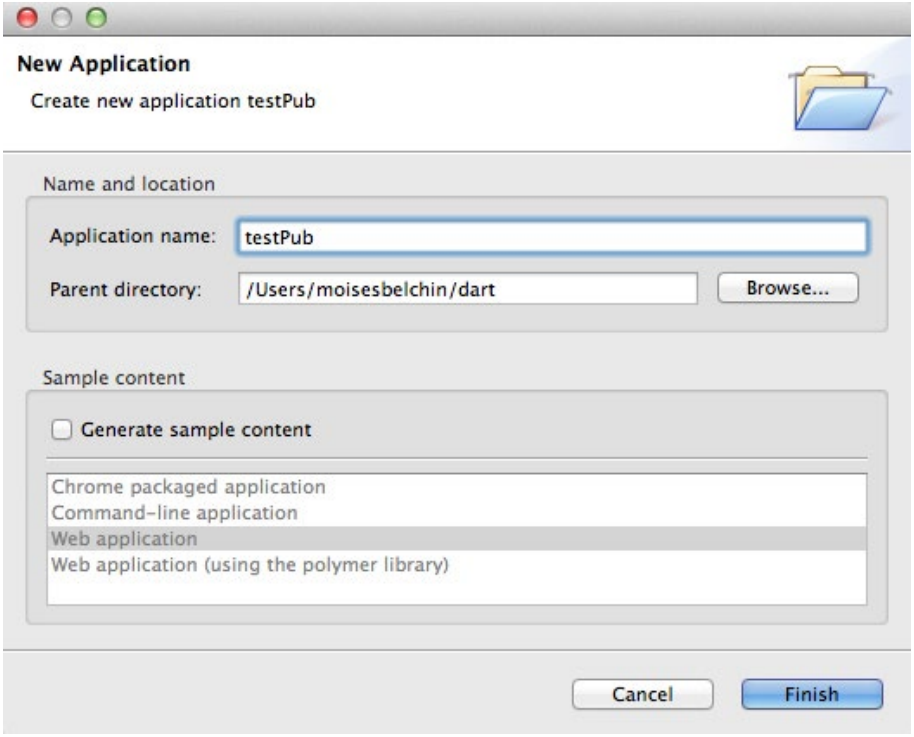


Figura 51. Creando una aplicación de pruebas para utilizar Pub.

Una vez creada la nueva aplicación vamos a crear un fichero llamado **pubspec.yaml**. Este fichero es el que necesitamos para indicar a pub que queremos utilizar una librería en nuestro nuevo proyecto.

Al crear este nuevo fichero automáticamente Dart Editor nos ha creado el fichero **pubspec.lock** y el directorio **packages** donde se almacenarán las librerías que necesitamos para nuestro nuevo proyecto.

El fichero **pubspec.yaml** es un fichero de texto normal, pero Dart Editor nos lo muestra de una manera mucho más cómoda para poder establecer las dependencias que tendrá nuestra aplicación.

En el apartado de dependencias (*Dependencies*) de pubspec.yaml pulsamos el botón **Añadir** (**Add**) para añadir una nueva dependencia. Se abrirá ahora un cuadro de búsqueda que te permitirá buscar, seleccionar y añadir el paquete **lawndart** para gestionar *IndexedDB* en nuestra aplicación.

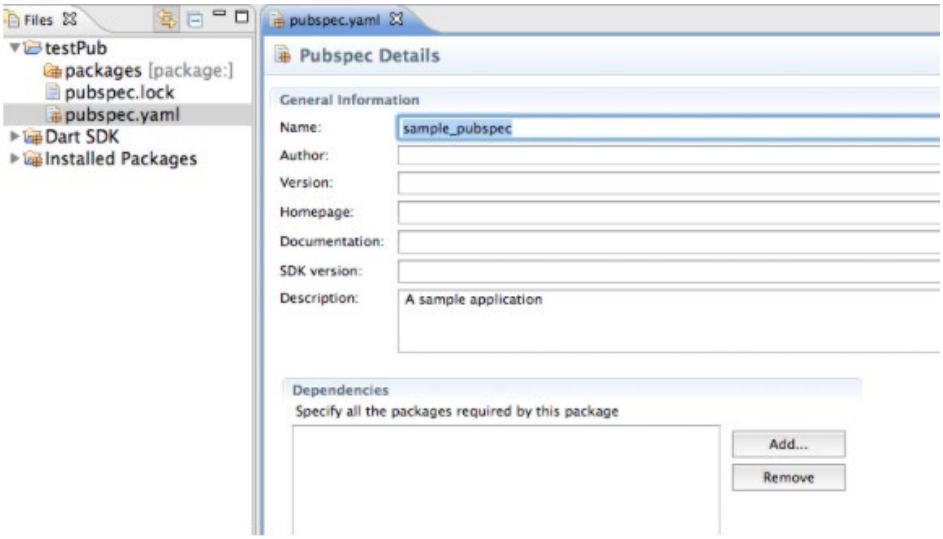


Figura 52. Creamos el fichero pubspec.yaml

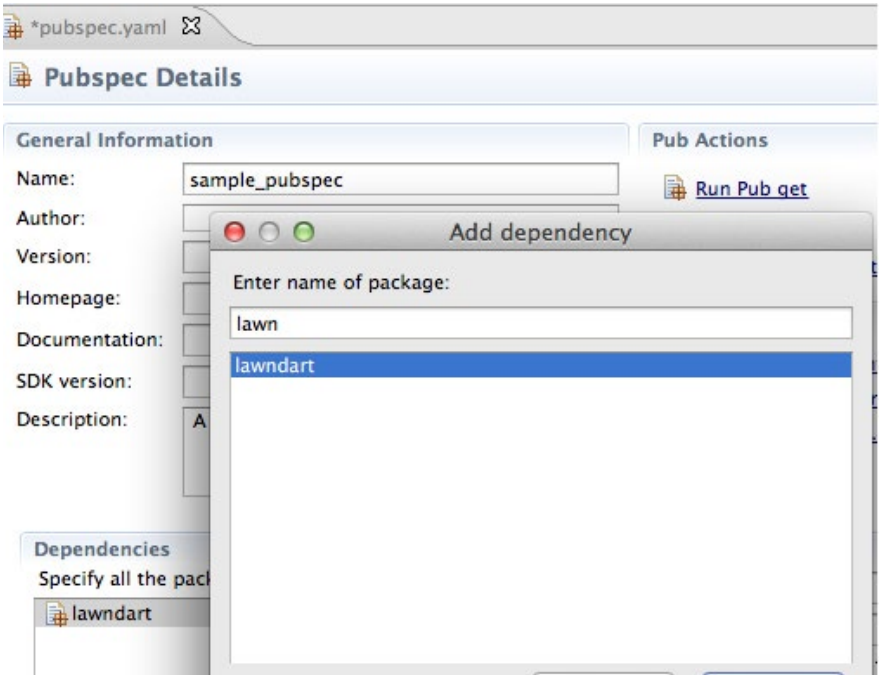


Figura 53. Añadimos el paquete lawndart a nuestra aplicación.

En la nueva ventana que se abre teclea el nombre del paquete, selecciónalo y pulsa OK. Podrás ver como ahora aparece en el apartado de dependencias (*Dependencies*) del fichero pubspec.yaml.

Guarda los cambios realizados en el fichero pubspec.yaml y ya hemos añadido nuestro primer paquete a nuestro proyecto.

Tras guardar el fichero, automáticamente (si así está configurado) Dart Editor comienza a resolver las dependencias, a descargar el nuevo paquete y a instalarlo en el directorio **packages** de tu aplicación.

En cualquier caso podrás utilizar los comandos de *pub* para actualizar y descargar los paquetes.

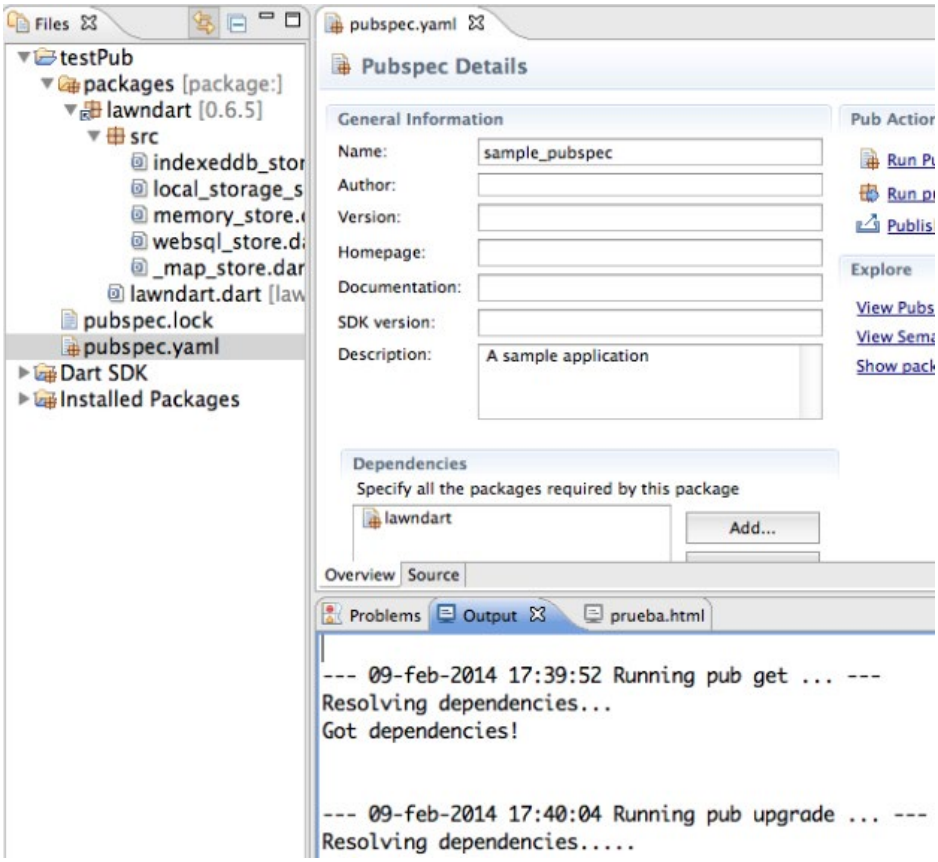


Figura 54. Añadimos las dependencias a **pubspec.yaml** y descargamos **lawndart** en el directorio **packages**

Ejecuta **Herramientas > Pub Get** (**Tools > Pub Get**) para resolver las dependencias y obtener los últimos paquetes. También puedes utilizar **Herramientas > Pub Get** (**Tools > Pub Get**) para actualizar tus paquetes a una nueva versión.

Ahora que ya tienes el paquete disponible en tu nueva aplicación puedes importarlo y comenzar a utilizarlo. El `import` de estos paquetes es similar al `import` de cualquiera de las librerías de Dart solo hay que indicar que se encuentra en el directorio `packages`.

```
import 'package:lawndart/lawndart.dart';
```

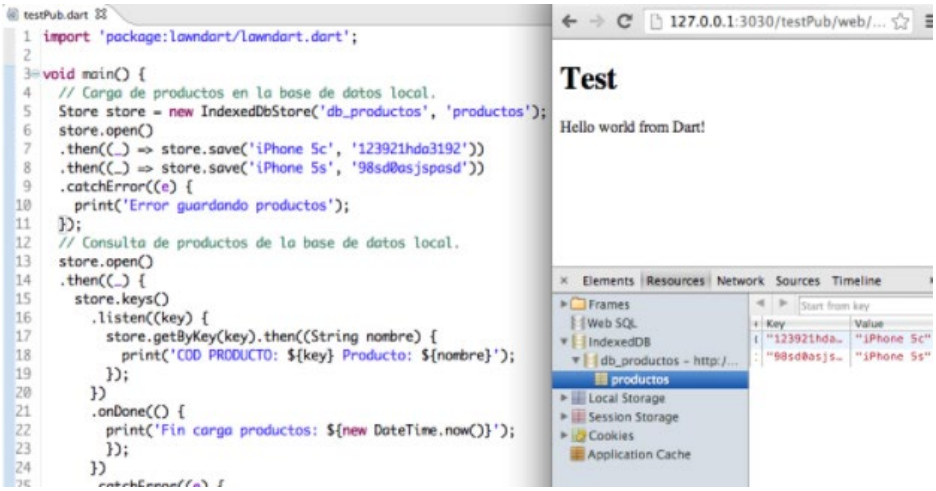


Figura 55. Importando **lawndart** y almacenando productos en la base de datos local.

Como has podido observar, solo por el hecho de crear un fichero pubspec.yaml tu aplicación se ha convertido en un paquete qué podrías publicar utilizando pub para que otras personas se beneficien de ella. ¡Así de sencillo!.

Ahora podrías ejecutar los comandos **Herramientas > Pub Build** (**Tools > Pub Build**) y **Herramientas > Pub publish** (**Tools > Pub Publish**) para generar un paquete de tu aplicación y publicarlo en http://pub.dartlang.org.

Conoceremos: Dartium y DartVM

A lo largo del libro hemos hablado de Dartium y de la máquina virtual de Dart cuando nos hemos referido a cómo ejecutar nuestras aplicaciones.

Veamos más a fondo qué son y cómo utilizarlos.

DartVM

Dart Virtual Machine o DartVM es la máquina virtual de Dart. Al igual que sucede con Java, Dart necesita de un intérprete que sepa reconocer y ejecutar el código Dart.

Por ejemplo, si escribes una aplicación Dart de línea de comandos e intentas ejecutarla en un sistema en el que no tengas instalada la máquina virtual de Dart esa aplicación no funcionará, será un simple fichero de texto con unas instrucciones en su interior que el sistema no sabe reconocer.

Para evitar esto debes instalar DartVM. La máquina virtual de Dart reconocerá y ejecutará aplicaciones de línea de comandos así como scripts del lado del servidor, aplicaciones y servidores.

Igual que *pub* o *dart2js*, dartvm o simplemente dart dispone de una serie de comandos y opciones para su ejecución.

> La máquina virtual de dart se ejecuta desde el directorio donde se encuentre instalado Dart.
>
> nota

Ejecutando este comando podemos ver todas las opciones disponibles en la máquina virtual de Dart.

```
dart-sdk/bin$ dart -h
Usage: dart [<vm-flags>] <dart-script-file> [<dart-options>]

Executes the Dart script passed as <dart-script-file>.

Common options:
--checked or -c
  Insert runtime type checks and enable assertions (checked mode).
--help or -h
  Display this message (add -v or --verbose for information about
  all VM options).
--package-root=<path> or -p<path>
  Where to find packages, that is, "package:..." imports.
--version
  Print the VM version.
```

El parámetro `-c o --checked` permite habilitar el modo de comprobaciones en DartVM. Como ya hemos comentado este modo sólo es recomendable durante la fase de desarrollo y pruebas.

Por defecto, DartVM corre en modo producción.

El parámetro `-h` muestra la ayuda como puedes ver en la ejecución anterior.

El parámetro `--version` muestra la versión que tienes instalada de DartVM.

```
dart-sdk/bin$ dart --version
Dart VM version: 1.1.1 (Wed Jan 15 04:11:21 2014) on "macos _ x64"
```

Por último, `-p o --package-root` seguido de la ruta física donde se encuentran los ficheros permite indicar a DartVM donde encontrará los ficheros que se importan en nuestras aplicaciones.

Cojamos el fichero `hola _ mundo.dart` y vamos a ver como ejecutarlo en DartVM. Podemos hacer que nuestra aplicación `hola _ mundo.dart` funcione en nuestro sistema ejecutando este comando.

```
dart-sdk/bin$ dart hola_mundo.dart
Hola, Mundo!
```

Bueno, no resulta muy espectacular, ¿verdad? Veamos ahora algo más complejo, vamos a crearnos un sencillo servidor web con Dart y lo ejecutaremos con DartVM.

Este es el código para nuestro servidor web sencillo.

```
import 'dart:io';

main() {
  HttpServer.bind('127.0.0.1', 8080).then((server) {
    server.listen((HttpRequest request) {
      request.response.write('Hola, bienvenido a Dart');
      request.response.close();
    });
  });
}
```

El código es realmente simple. Utilizamos la clase `HttpServer` y con el método `bind` comenzamos a escuchar peticiones Http en la dirección y puerto que indiquemos.

Cuando recibamos una nueva petición mostramos un mensaje de bienvenida al fantástico mundo de Dart y cerramos la respuesta con el cliente.

Ejecuta este servidor a través de DartVM con este comando.

```
dart-sdk/bin$ dart server.dart
```

Tras ejecutar el comando puedes ver como DartVM se queda esperando a recibir peticiones Http. Abre ahora un navegador y navega a la dirección http://127.0.0.1:8080

Nuestro pequeño servidor escrito en Dart contestará de esta manera.

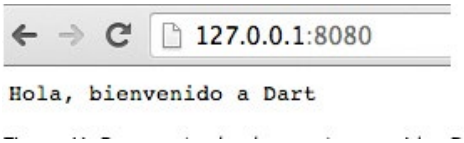


Figura 56. Respuesta desde nuestro servidor Dart.

Dartium

Ahora que conocemos *DartVM* y sabemos que es el corazón necesario para que todo nuestro código funcione, ¿qué es *Dartium*?. Ya hemos hablado de *Dartium* en otras ocasiones, pero ¿por qué viene incluido cuando descargo e instalo Dart en mi sistema?.

Como hemos comentado llegará un momento en que podamos correr nuestras aplicaciones Dart de manera nativa en los principales navegadores y entonces, al instalar nuevas versiones de Dart, Dartium ya no aparecerá en nuestro sistema.

Hasta que ese momento llegue tenemos dos opciones:

- La primera de ellas, ya la conoces y la has utilizado, se trata de compilar nuestras aplicaciones Dart a código JavaScript para que sean entendidas y ejecutadas en cualquier navegador web moderno con soporte de JavaScript.
- La segunda opción es poder correr nuestras aplicaciones Dart en un navegador que tenga ya incluido de manera predeterminada *DartVM*.

Esto es ***Dartium***. *Dartium* es el navegador web de código abierto *Chromium* (en el que se basa Google Chrome) al que se la ha incluído la máquina virtual de Dart, de ahí su nombre *Dart + Chromium = Dartium*.

Gracias a esta mezcla de navegador y máquina virtual podemos ejecutar nuestras aplicaciones Dart directamente en el navegador web sin necesidad de compilarlas previamente a JavaScript.

Dartium igual que *DartVM* viene incluido por defecto en nuestra instalación de Dart Editor junto con el SDK, pub, dart2js, etc. Y cada vez que quieras ejecutar una aplicación web será el navegador que ejecute por defecto tus aplicaciones Dart.

Igual que sucede con el resto de aplicaciones comentadas puedes ejecutar Chromium desde la línea de comandos utilizando este comando si tu sistema es MacOS.

```
$ /Applications/dart/chromium/Chromium.app/Contents/MacOS/Chromium
```

O este otro, para entornos Windows si tienes dart instalado en el directorio `C:\dart`.

```
C:\dart> chromium\chrome.exe
```

Dart Formatter

Recientemente hemos recibido la noticia de una nueva versión de desarrollo de Dart Editor que se terminará convirtiendo en la nueva versión 1.2 de Dart. En esta nueva versión, además de recibir mejoras de rendimiento y estabilidad como suele ser habitual, recibiremos una herramienta nueva que permitirá formatear programas Dart de manera más automática.

Esta nueva herramienta se llama **Dart Formatter** y puedes ejecutarla desde el directorio de aplicaciones del sdk de estos dos modos.

```
dart-sdk/bin/dartfmt [flags] FICHERO.DART
```

```
dart-sdk/bin/dartfmt [flags] DIRECTORIO
```

La forma de funcionar es sencilla, si especificas un fichero con código Dart realiza los cambios correspondientes en dicho fichero, si por el contrario indicas un directorio, Dart Formatter realizará cambios en todos los ficheros que encuentre de manera recursiva en dicho directorio.

Por defecto, los cambios que vaya a realizar dartfmt los muestra en la salida estándar.

Dart Formatter además soporta una serie de opciones (*flags*) que son las siguientes:

`-h, --help.`	Muestra información de uso.
`-w, --write.`	Escribe el resultado del re-formateo directamente sobre los ficheros (sobre-escribe el contenido) y no muestra el resultado en la salida estándar.
`-t, --transform.`	Realiza transformaciones en el código. Un ejemplo de esto, es que añade llaves a las estructuras de control, si no las tiene.
`-l, --max_line_length.`	Ajusta la longitud de las líneas al valor indicado, si no se indica por defecto 80 caracteres. Si no queremos que ajuste las líneas a una longitud concreta puedes indicar el valor "Infinity" o "Inf".
`-m, --machine.`	El re-formateo lo guarda en un formato concreto para un parseo más adecuado, es decir, lo graba como un JSON.

Veamos algunos ejemplos de uso

Ejecutando dartfmt con la opción `-h` nos muestra toda la información de uso de esta aplicación.

```
dart-sdk\bin$ dartfmt -h

dartfmt formats Dart programs.

Without an explicit path, dartfmt processes the standard input.  Given a file, i
t operates on that file; given a directory, it operates on all .dart files in th
at directory, recursively. (Files starting with a period are ignored.) By defaul
t, dartfmt prints the reformatted sources to standard output.

Usage: dartfmt [flags] [path...]
```

```
Supported flags are:
-w, --write                Write reformatted sources to files (overwriting contents).
Do not print reformatted sources to standard output.
-t, --transform            Perform code transformations.
-l, --max_line_length      Wrap lines longer than this length. To never wrap, spe-
cify "Infinity" or "Inf" for short. (defaults to "80")

-m, --machine              Produce output in a format suitable for parsing.
-h, --help                 Print this usage information.
```

Ejecutando dartfmt con la opción `-w` nos escribe los cambios en el propio fichero. Partiendo de un fichero con este código.

```
/**
 * DocString para este fichero.
 */
import 'dart:html';

/// Entrypoint de la aplicacion.
void main() {
  int zero;

  if (0==0)
    zero=0;
  else if (0==1)
    zero=1;
  else if (0==2)
    zero=2;
}
```

Al ejecutar el comando `dart-sdk\bin$ dartfmt -w test.dart` obtenemos el resultado

```
/**
 * DocString para este fichero.
 */
import 'dart:html';

/// Entrypoint de la aplicacion.
void main() {
  int zero;
```

```
  if (0 == 0) zero = 0; else if (0 == 1) zero = 1; else if (0 == 2) zero
= 2;

}
```

Ejecutando dartfmt con la opción -t realiza transformaciones en el código, por ejemplo, añade llaves en las estructuras de control.

Si ejecutamos `dart-sdk\bin$ dartfmt -w -t test.dart` obtenemos este resultado

```
/**
 * DocString para este fichero.
 */
import 'dart:html';

/// Entrypoint de la aplicacion.
void main() {
  int zero;
  if (0 == 0) {
    zero = 0;
  } else if (0 == 1) {
    zero = 1;
  } else if (0 == 2) {
  zero = 2;
  }
}
```

Finalmente, si ejecutas `dart-sdk\bin$ dartfmt -w -t -m test.dart` podrás obtener el resultado en formato JSON

```
{
   "source":"/**\n * DocString para este fichero.\n */\nimport
'dart:html';\n\n/// Entrypoint de la aplicacion.\nvoid main()
{\n int zero;\n if (0 == 0) {\n zero = 0;\n } else if (0 == 1)
{\n zero = 1;\n } else if (0 == 2) {\n zero = 2;\n }\n}\n",
   "selection":{
          "offset":-1,
          "length":-1
   }
}
```

Generación de documentación

En el momento de escribir estas líneas nos encontramos en la versión estable 1.1 de Dart. Ya hemos hablado que el equipo de desarrollo está en constante evolución y haber alcanzado la versión estable 1.0 o 1.1 no significa ni mucho menos que el proyecto esté terminado o que no podamos esperar grandes novedades.

Ya se han publicado algunas nuevas versiones de desarrollo presentando correcciones, mejoras y algunas novedades que conformarán la nueva versión estable 1.2 de Dart.

Por este motivo en esta sección destinada a las herramientas de Dart para generar documentación de tus aplicaciones veremos **dartdoc** y **docgen**.

Dartdoc

Hasta la versión 1.1 estable de Dart, la herramienta dartdoc es la que deberás utilizar cuando desees generar la documentación de tus aplicaciones.

Esta sencilla herramienta escanea el directorio o ficheros que indiques y genera los ficheros estáticos en formato HTML con la documentación. Esta documentación se genera a partir de los comentarios de documentación que hayas incluido en tú código.

Los comentarios de documentación se generan utilizando **///** o **/**/**.

Estos son algunos ejemplos de comentarios de documentación.

```
/**
  * Shift the bits of this integer to the right by [shiftAmount].
  *
  * Shifting to the right makes the number smaller and drops the least
  * significant bits, effectively doing an integer division by
  *`pow(2, shiftIndex)`.
  *
  * It is an error of [shiftAmount] is negative.
  */
int operator >>(int shiftAmount);
```

```
/// Returns true if and only if this integer is even.
bool get isEven;
```

Si ejecutas el comando `dart-sdk\bin$ dartdoc` obtendrás toda la información de uso y opciones que soporta dartdoc

```
Usage dartdoc [options] <entrypoint(s)>
[options] include:

no-code                 Do not include source code in the documentation.

-m, --mode              Define how TML pages are generated.

    [Live nav]          Default) Generated docs do not included baked HTML
                        navigation. Instead a single `nav.json` file is
                        created and the appropriate navigation is generated
                        client-side by parsing that and building HTML.

                        This dramatically reduces the generated size HTML
                        since a large fraction of each static page is just
                        redundant navigation links. Rhis mode, the browser
                        will do a XHR for nav.json which means that preview
                        docs locally, you will need to enable requesting
                        file:// links in your browser or run a little local
                        server like `python -m

      [static]          Generate completely static HTML containing
                        everything you need to browse the docs. The only
                        client side behavior is trivial stuff like syntax
                        highlighting code, and the find-as-you-type search
                        box.

generate-app-cache      Generates the App Cache manifest file, enabling
                        offline doc viewing.

omit-generation-time    Omits generation timestamp from output.

-v, --verbose           Print verbose information during generation.

include-api             Included API libraries in the generated
                        documentation. If the --link-api option is used,
                        this option is ignored.
```

-link-api	Link to the online language API in generated documentation. The option overrides inclusion through --include-api or --include-lib
[no-]show-private	Document private types and members.
--inherit-from-object	Show members inherited from Object.
--enable-diagnostic-colors	
--out	Generate files into directory specified. If omitted the files are generated into ./docs/
include-lib	Use this option to explicitly specify which libraries to include in the documentation. If omitted, all used libraries are included by default. Specify a comma-separated list of library names, or call this opt. multiple times.
exclude-lib	Use this option to explicitly which libraries to exclude from the documentation. If omitted no libraries are excluded. Specify comma-separated list of library names, or call this option multiple times.
package-root	Sets the package directory to the specified directory. If omitted the package directory is the closets packages directory to the entrypoint.
library-root	Sets the library root directory to the specified directory.
--pkg	Deprecated: same as --package-root.

Estas son las opciones en detalle que soporta dartdoc.

Opciones

`--no-code`. No incluirá código fuente en la documentación.

`-m, --mode`. Define cómo se generan las páginas HTML.

`[live-nav]`. Esta es la opción por defecto que genera un fichero llamado nav.json con toda la información necesaria. Es el cliente el que procesa este fichero JSON y va componiendo dinámicamente los ficheros HTML que renderiza y muestra al usuario.

Esta opción reduce muchísimo el tamaño de la documentación ya que hay bloques comunes como la navegación que no son necesarios generar en cada fichero.

Este modo además establece peticiones XHR entre el cliente el fichero `nav.json` por lo que es preciso habilitar este tipo de peticiones o ejecutar un servidor local del tipo "`python -m SimpleHTTPServer`".

`[static]`. Genera los fichero HTML completamente estáticos. Cada fichero contiene todo lo necesario para navegar por la documentación y no es necesario realizar ningún comportamiento concreto en el lado cliente.

`--generate-app-cache`. Genera el fichero de caché de aplicación para poder visualizar la documentación totalmente offline.

`--omit-generation-time`. Omite la marca de tiempo de la generación de la documentación en los ficheros generados.

`-v, --verbose`. Muestra toda la información durante la generación.

`--include-api`. Incluye las librerías del SDK utilizadas cuando genera la documentación.

`--link-api`. Enlace hacia la documentación online del API de Dart. Esta opción anula `--include-api` o `--include-lib`.

`--[no-]show-private`. Indica si se documentan o no tipos y miembros privados.

`--inherit-from-object`. Muestra los miembros heredados de la clase general Object.

`--out`. Indica el directorio donde se genera la documentación, por defecto es "docs".

`--include-lib`. Esta opción permite incluir de forma explícita que librerías se desean incluir en la documentación generada. Si no se especifica esta opción, por defecto se documentan todas las librerías utilizadas. Para indicar diferentes librerías se separan por coma o se especifica esta opción múltiples veces, una por cada librería.

`--exclude-lib`. Permite excluir librerías de la documentación y del mismo modo puedes llamar varias veces a esta opción con las librerías a excluir o separa las librerías por comas.

`--package-root`. Establece el directorio donde se encuentran los paquetes de la librería en cuestión, si no se indica se coge el directorio "packages" más cercano.

`--library-root`. Establece el directorio raíz de la librería.

`--pkg`. Esta opción es similar a la opción `--package-root` y está marcada como obsoleta por lo que no deberías utilizarlo.

Veamos ahora unos ejemplos de uso de esta herramienta. Partiendo de un fichero **test.dart** como el que mostramos a continuación:

```
**
 * An arbitrarily large integer.
 *
 * **Note:** When compiling to JavaScript, integers are
 * implemented as JavaScript numbers. When compiling to JavaScript,
 * integers are therefore restricted to 53 significant bits because
 * all JavaScript numbers are double-precision floating point
 * values. The behavior of the operators and methods in the [int]
 * class therefore sometimes differs between the Dart VM and Dart
      code
 * compiled to JavaScript.
 *
 * It is a compile-time error for a class to attempt to extend or im-
      plement int.
 */
```

```
abstract class int {

  /**
   * Bit-wise and operator.
   *
   * Treating both `this` and [other] as sufficiently large 2 component
   * integers, the result is a number with only the bits set that are
      set in both `this` and [other]
   * Of both operands are negative, the result is negative, otherwise
   * the result is non-negative.
   */
  int operator &(int other);

  /** Returns `this.toDouble()`. */
  double roundToDouble();

  /** Returns `this.toDouble()`. */
  double floorToDouble();

  /** Returns `this.toDouble()`. */
  double ceilToDouble();

  /** Returns `this.toDouble()`. */
  double truncateToDouble();

  /**
   * Returns a String-representation of this integer.
   *
   * The returned string is parsable by [parse].
   * For any `int` [:i:], it is guaranteed that
   * [:i == int.parse(i.toString()):].
   */
  String toString();

  /**
   * Converts [this] to a string representation in the given [radix].
   *
   * In the string representation, lower-case letters are used for di-
      gits above
   *
   * The [radix] argument must be an integer in the range 2 to 36.
   */
  String toRadixString(int radix);
}
```

Vamos a generar su documentación utilizando este comando dart-sdk\bin$ dartdoc **test.dart**

```
Analyzing libraries...
Generating documentation...
Unable to determine package for dart-sdk/bin/test.dart.
Compiling client JavaScript...
Copying static files...
Documentation complete -- documented 1 libraries, 1 types, and 7
members.
```

Como puedes ver se ha generado un directorio llamado docs donde figuran los ficheros generados

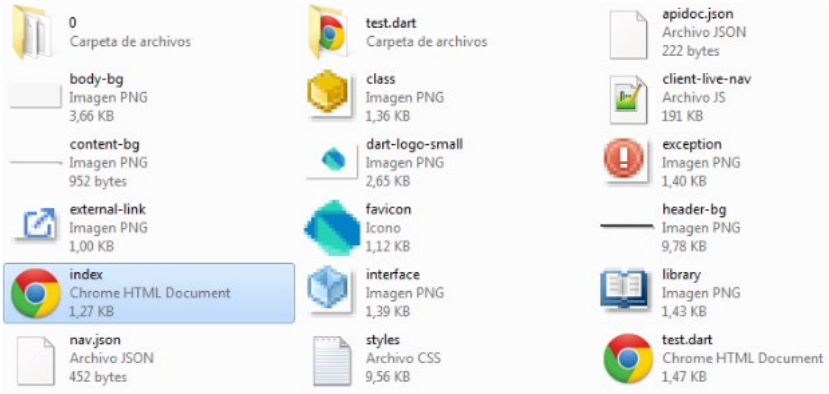


Figura 57. Ficheros generados por DartDoc.

Y si abres el fichero index.html en tu navegador el resultado es el siguiente.

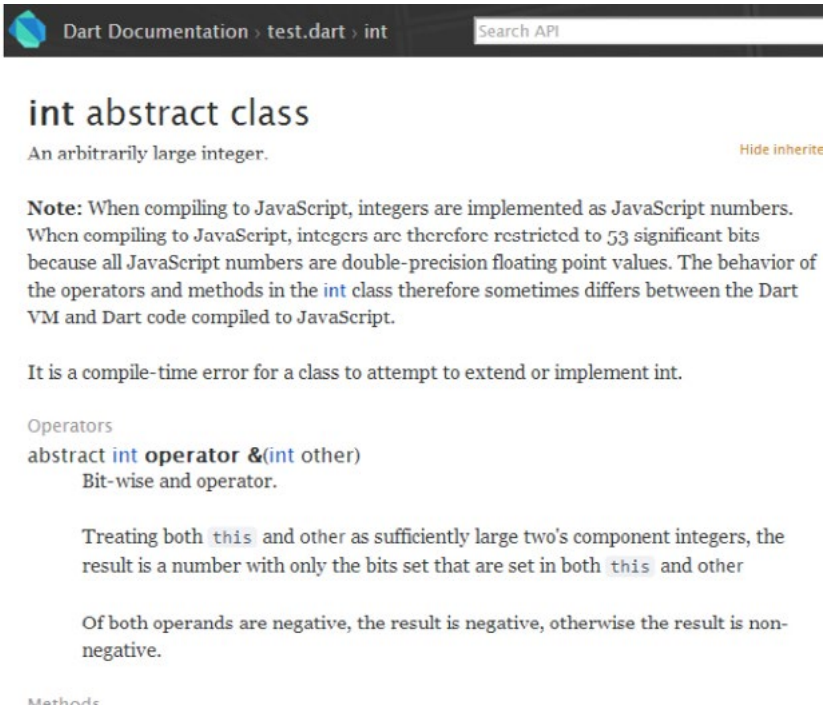


Figura 58. Documentación generada con Dartdoc

Docgen

Ya hemos hablado de la herramienta dartdoc que actualmente (hasta la versión 1.1) es la herramienta por defecto, pero ahora nos gustaría comentarte una nueva herramienta llamada **docgen** que próximamente estará disponible y que viene a reemplazar a **dartdoc**.

El nuevo docgen no solo viene a reemplazar la herramienta dartdoc sino que además cambia complementamente la filosofía de generar la documentación.

Dartdoc genera ficheros **HTML** estáticos con la documentación a partir de los ficheros .dart indicados.

Docgen genera ficheros en formato YAML o JSON que se pueden procesar más fácilmente por herramientas o visores externos.

Docgen además dispone de un sistema que actúa como servidor local para procesar los ficheros json o yaml y cargar la documentación, sí se puede visualizar a través del navegador.

Veamos como funciona y alguno de sus comandos. La forma de utilizarlo es la siguiente

```
dart-sdk\bin$ docgen [OPTIONS] fooDir/barFile
```

Si se indica un directorio actuará de manera recursiva en todos los ficheros encontrados en ese directorio, si se indica un fichero generará la documentación del fichero indicado.

Veamos ahora todas las opciones que soporta docgen. Ejecutando el comando `dart-sdk\bin$ docgen -h` obtendrás toda la descripción de uso de esta nueva herramienta.

```
-h, --help                Prints help and usage information.
-v, --verbose             Output more logging information.
-j, --[no-]json           Outputs to JSON. Files are outputted to YAML by
                          default. If --append is used, it takes the
                          file-format of the previous run stated in
                          library_list.json ignoring flag (defaults to on)

--include-private         Flag to include private declarations.
--[no-]include-sdk        Flag to parse SDK Library files (defaults to on)

--package-sdk             Parses the SDK libraries only.
--package-root            Sets the package root of the library being
                          analyzed.
--append                  Append to the docs folder, library_list.json
                          and index.txt
--compile                 Clone the documentation viewer repo locally
                          (if not already present) and compile with dart2js
```

--serve	Clone the documentation viewer repo locally (if not already present), compile with dart2js, and start a simple server
--no-docs	Do not generate any new documentation
--introduction	Adds the provided markdown text file as the introduction for the generated documentation. (defaults to "")
--out	The name of the output directory. (defaults to "docs")
--exclude-lib	Exclude the library by this name from the documentation
--[no-]include-dependent-packages	Assumes we are documenting a single package and are running in the directory with its pubspec. Includes documentation for all of its dependent packages. (defaults to on)
--sdk	SDK directory
--start-page	By default the viewer will start at the SDK introduction page.To start at some other page, e.g. for a package, provide the name of the package in this argument, e.g. --start-page=intl will make the start page of the viewer be the intl package.

```
Usage: dart docgen.dart [OPTIONS] fooDir/barFile
```

Vamos a comentarte ahora todas las opciones disponibles para docgen.

Opciones

`-h, --help.`	Muestra la ayuda y la información de uso.
`-v, --verbose.`	Muestra más información durante el proceso de generación.
`-j, --[no-]json.`	Genera los ficheros en formato JSON. Por defecto, se generan en formato YAML.
`--include-private.`	Incluye declaraciones privadas.
`--[no-]include-sdk.`	Parsea o no los ficheros del SDK
`--parse-sdk.`	solo parsea las librerías del SDK.
`--package-root.`	Establece el directorio raíz de la librería a analizar.
`--append.`	Añade a la carpeta de destino de la documentación los ficheros `library _ list.json y index.txt.`
`--compile.`	Clona el repositorio del visor de documentación en local y lo compila con dart2js.
`--serve.`	Clona el repositorio del visor de documentación, lo compila con dart2js y arranca un servidor web local para poder acceder a la documentación.
`--no-docs.`	Indica que no queremos generar nueva documentación.
`--introduction.`	Añade el texto que indiquemos como introducción a todos los ficheros de documentación generados.
`--out.`	Para establecer el directorio de salida de la documentación, por defecto, es "docs".
`--exclude-lib.`	Omite la librería que indiquemos de la documentación.
`--[no-]include-dependent-packages.`	Indica si queremos o no incluir información en la documentación de las dependencias. Por defecto, está activo.
`--sdk.`	Indica el directorio del SDK.
`--start-page.`	Por defecto, el visor al arrancar carga la página de introducción del SDK, si deseas indicar otra página de inicio debes utilizar esta opción.

Veamos ahora algunos ejemplo de uso de docgen. Partiremos de un fichero `prueba.dart` con este contenido.

```
/**
 * An arbitrarily large integer.
 *
 * **Note:** When compiling to JavaScript, integers are
 * implemented as JavaScript numbers. When compiling to JavaScript,
 * integers are therefore restricted to 53 significant bits because
 * all JavaScript numbers are double-precision floating point
 * values. The behavior of the operators and methods in the [int]
 * class therefore sometimes differs between the Dart VM and Dart code
 * compiled to JavaScript.
 *
 * It is a compile-time error for a class to attempt to extend or imple-
     ment int.
 */
abstract class int {

  /**
   * Bit-wise and operator.
   *
   * Treating both `this` and [other] as sufficiently large two's component
   * integers, the result is a number with only the bits set that are set
     in
   * both `this` and [other]
   *
   * Of both operands are negative, the result is negative, otherwise
   * the result is non-negative.
   */
  int operator &(int other);

  /** Returns `this.toDouble()`. */
  double roundToDouble();

  /** Returns `this.toDouble()`. */
  double floorToDouble();
```

```
  /** Returns `this.toDouble()`. */
  double ceilToDouble();

  /** Returns `this.toDouble()`. */
  double truncateToDouble();

  /**
   * Returns a String-representation of this integer.
   *
   * The returned string is parsable by [parse].
   * For any `int` [:i:], it is guaranteed that
   * [:i == int.parse(i.toString()):].
   */
  String toString();
}
```

Si ejecutas el comando `dart-sdk\bin$ docgen prueba.dart` verás que docgen ha generado el directorio **docs** (directorio por defecto) con los ficheros en formato JSON que contienen la documentación.

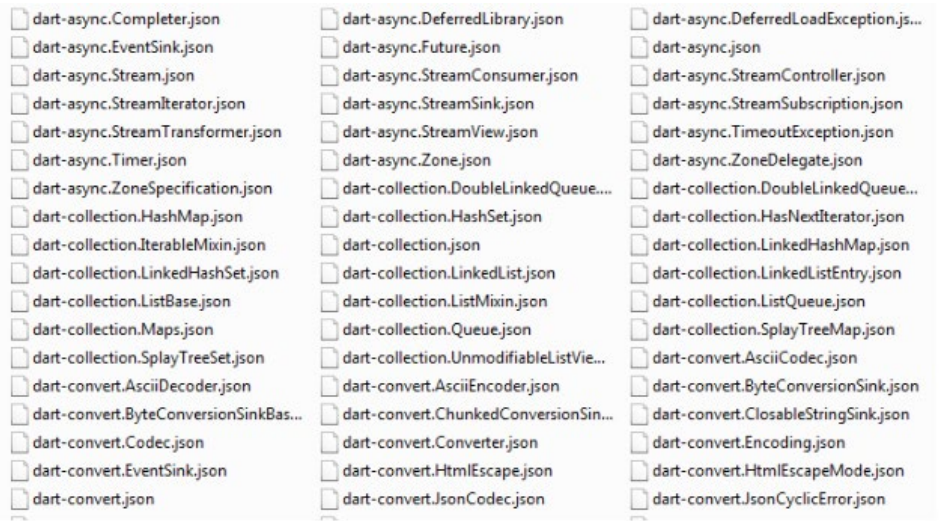


Figura 59. Ficheros JSON generados con docgen.

No está mal, pero no es algo impresionante, porque ¿cómo puedo ver la documentación en un formato algo más humano?.

Probemos ahora algo más interesante, vamos a utilizar la opción `--serve` de docgen, pero antes de ello recuerda que deberás tener instalado GIT en tu sistema.

> GIT es un sistema de control de versiones diseñado por el creador de Linux, Linus Torvalds. Si quieres ampliar información o instrucciones para utilizarlo dirígete a http://git-scm.com/
>
> ¿qué es GIT?

Pero, ¿por qué necesito GIT?. La opción `--serve` descarga el repositorio del visor de documentación, lo compila con dart2js y arranca un servidor web local para poder acceder a la documentación.

Si ejecutas el comando `dart-sdk\bin$ docgen --serve prueba.dart` además de generar los ficheros JSON con la documentación, descargará dartdoc-viewer la aplicación (escrita en Dart) para visualizar en un formato más legible la documentación generada.

Además de descargar el visor, lanza dart2js para compilar dartdoc-viewer y arrancar el servidor web que te permite visualizar la documentación.

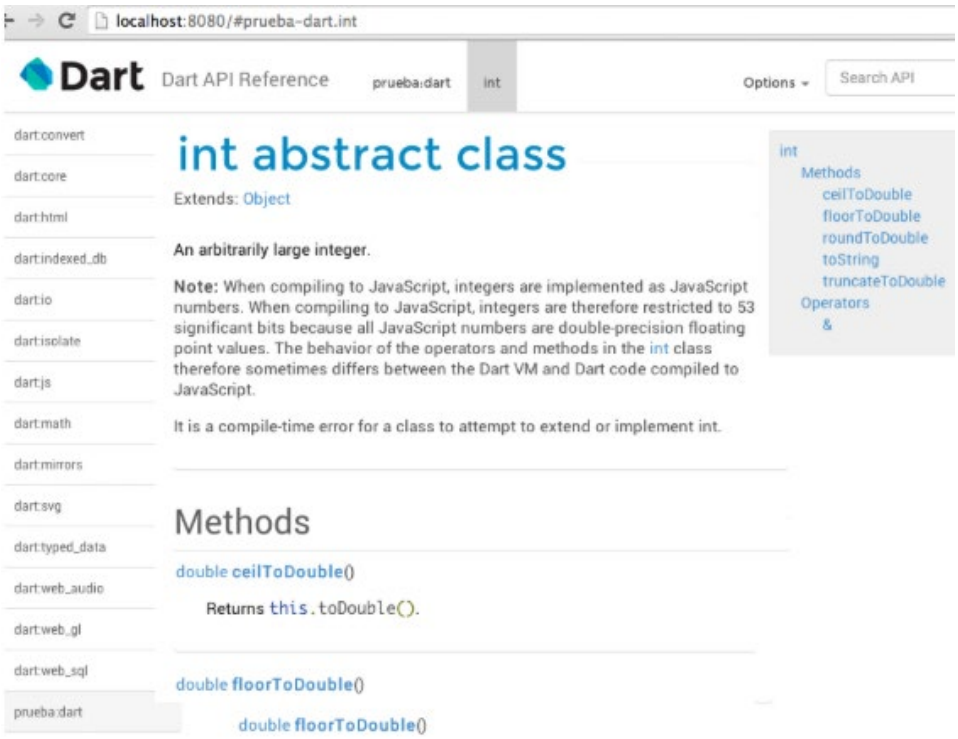


Figura 60. Nuevo formato de documentación generado con docgen.

Como puedes ver solo con abrir tu navegador y apuntar a la dirección http://localhost:8080 ya puedes visualizar la documentación en un nuevo formato realmente bueno.

En nuestro caso podríamos omitir los paquetes del SDK y visualizar tan solo la documentación del fichero prueba.dart ejecutando este comando `dart-sdk\bin$ docgen -no-include-sdk --serve prueba.dart`

4 DART BÁSICO

Ya te hemos contado en profundidad todas las herramientas que vas a necesitar para tu día a día con Dart ahora vamos a comenzar con lo bueno para que empieces a conocer la sintaxis de Dart. Cuáles son sus tipos básicos, las variables, comentarios y todas las sentencias de control condicionales e iterativas.

Veremos por un lado lo más básico y necesario para empezar a escribir programas en Dart. Aunque esto parezca algo insignificante, no te confundas, vas a estar utilizando todo lo que verás en esta primera parte durante el 80% del tiempo que estés trabajando con Dart.

A continuación veremos los fundamentos de programación orientada a objetos.

Comentarios

Introducción

Como ya sabes, los comentarios son parte imprescindible del código fuente de cualquier aplicación. Ayudan a aclarar ciertas partes complicadas del código, sobre todo si tenemos que volver sobre ellas más tarde.

También ayudan al resto de miembros del equipo. En grupos de desarrollo donde hay varias personas involucradas en el mismo proyecto resulta imprescindible que cada uno desarrolle su trabajo, pero también debe facilitar el trabajo al resto dejando claros comentarios sobre la construcción del código.

Los comentarios se utilizan en todos los lenguajes incluso para auto generar la documentación de todo el sistema. Tal es el caso de PHP, Java, JavaScript entre otros, utilizando posteriormente PHPDoc, JavaDoc o JSDoc.

En cualquier caso, resulta evidente que necesitamos documentar y por lo tanto conocer correctamente como crear estos comentarios.

Comentarios en línea

En Dart podemos crear comentarios que ocupen una sola línea estos comienzan con el carácter **//**.

```
String htmlEscape(String text) {
  //Implementación más eficiente.
  return text.replaceAll("&", "&amp;")
             .replaceAll("<", "&lt;")
             .replaceAll(">", "&gt;")
             .replaceAll("'", "&quot;")
             .replaceAll("'", "&apos;");
}
```

En este ejemplo se ha creado una línea de comentario dentro de una función que indica el motivo de por qué se ha utilizado este tipo de implementación.

Comentarios de bloque

Del mismo modo existen comentarios de bloque. Podemos tener la necesidad de ampliar más información sobre nuestro código para ello podemos utilizar comentarios de bloque.

Estos comentarios comienzan con **/*** y se cierran con ***/**.

```
/*
 * Modificamos el método htmlEscape en la versión 1.2
 * Se ha realizado una modificación sobre los métodos de reemplazo
 * para obtener una implementación más eficiente y segura.
 */
String htmlEscape(String text) {
  //Implementación más eficiente.
  return text.replaceAll("&", "&amp;")
             .replaceAll("<", "&lt;")
             .replaceAll(">", "&gt;")
```

```
                .replaceAll('"', "&quot;")
                .replaceAll("'", "&apos;");

  }
```

En este ejemplo puedes ver como se ha creado un comentario que abarca múltiples líneas para añadir más información.

Otro posible formato es el siguiente:

```
/* Modificamos el método htmlEscape en la versión 1.2. Se ha realizado una
modificación sobre los métodos de reemplazo para obtener una implementación
más eficiente y segura. */
String htmlEscape(String text) {
  //Implementación más eficiente.
  return text.replaceAll("&", "&amp;")
             .replaceAll("<", "&lt;")
             .replaceAll(">", "&gt;")
             .replaceAll('"', "&quot;")
             .replaceAll("'", "&apos;");
}
```

Comentarios de documentación

Anteriormente comentamos que existen aplicaciones para generar documentación de nuestro sistema de manera automática a partir de los comentarios de nuestro código. Ya sabes que en Dart también es posible gracias a las herramientas *dartdoc* y *docgen*, para ello debes utilizar los comentarios de documentación cuando escribas tus aplicaciones.

Estos comentarios de documentación son similares a los comentarios de bloque con una pequeña diferencia. Comienzan por **/**** se cierran con ***/** y cada línea va precedida de *****.

También se pueden crear comentarios que generen documentación utilizando el carácter ***///***.

```
/**
 * The [Collection] interface is the public interface of all
 * collections.
 */

 interface Collection <E> extends Iterable <E> {
   /**
    * Applies the function [f] to each element of this collection.
    */
    void forEach(void f(E element));
   /**
    * Returns a new collection with the elements [: f(e) :]
    * for each element [e] of this collection.
    *
    * Note on typing: the return type of f() colud be an arbitrary
    * type and consequently the returned collection's
    * typeis Collection.
    */

    Collection map (f(E element));
```

Figura 61. Comentarios de documentación

```dart
_checkOpen() {
  if (!isOpen) throw new StateError('$runtimeType is not open');
}

/// Returns a Future that completes when the store is opened.
/// You must call this method before using
/// the store.
Future open();

/// Returns all the keys as a stream. No order is guaranteed.
Stream<String> keys() {
  _checkOpen();
  return _keys();
}
Stream<String> _keys();

/// Stores an [obj] accessible by [key].
/// The returned Future completes with the key when the objects
/// is saved in the store.
Future<String> save(V obj, String key) {
  _checkOpen();
  if (key == null) {
    throw new ArgumentError("key must not be null");
  }
```

Figura 62. Comentarios de documentación

Estos comentarios pueden ir o bien al comienzo de nuestras definiciones de clases y métodos o dentro de ellos, por si te puede interesar añadir comentarios de documentación para cualquier propiedad de una clase.

Operadores y expresiones

Introducción a los operadores

Los operadores son parte imprescindible de cualquier lenguaje de programación. Te permitirán efectuar operaciones aritméticas con números, asignar valores entre variables y poder tomar decisiones para que tu programa ejecute una parte u otra.

Ya que Dart es un lenguaje orientado a objetos, los operadores son métodos de instancia con nombres especiales, por ejemplo, +, -, *, %, etc. Básicamente, un método de instancia es una clasificación que se aplica a ciertos métodos que define una clase. Los métodos de instancia operan sobre las variables de instancia de los objetos, aunque también tienen acceso a las variables de la clase.

No te preocupes si no ha entendido nada sobre esto que acabamos de comentar, profundizaremos más cuando estudiemos las Clases y programación orientada a objetos con Dart.

```
class Double implements double {
  factory Double.fromInteger(int value)
      native "Double_doubleFromInteger";
  int hashCode() {
    try {
      return toInt();
    } catch (BadNumberFormatException e) {
      return 0;
    }
  }
  double operator +(num other) {
    return add_(other.toDouble());
  }
  double add_(double other) native "Double_add";

  double operator -(num other) {
    return sub_(other.toDouble());
  }
  double sub_(double other) native "Double_sub";
```

Figura 63. Declaración de una clase con sus operadores.

Los operadores que, por defecto define Dart, se pueden sobrescribir o volver a declarar. Para sobrescribir un operador en una clase debemos usar la palabra reservada **operator**.

En la figura 63. podemos ver como se ha definido la clase **Double** la cual posee una serie de operadores. Operador de suma [+] y operador de resta [-].

Tipos de operadores

Los tipos de operadores que podemos encontrar en Dart se clasifican en:

- Artiméticos
- De igualdad y relacionales
- Test de tipado
- Asigancián
- Condicionales
- De Bit
- Otros

Aquí puedes verlos todos:

Aritméticos	
`+`	Suma
`-`	Resta
`-expr`	Signo negativo
`*`	Multiplicación
`/`	División
`~/`	División entera
`%`	Resto de la división
`++var`	Incrementa valor de la variable antes de realizar otra operación
`var++`	Incrementa valor de la variable después de ejecutar la sentencia
`--var`	Decrementa valor de la variable antes de realizar otra operación
`var--`	Decrementa valor de la variable después de ejecutar la sentencia

De igualdad y relacionales	
==	Igualdad
!=	Distinto
>	Mayor que
<	Menor que
>=	Mayor o igual
<=	Menor o igual

Test de tipado	
as	Casting de tipos para convertir un objeto de un tipo a otro
is	Verdadero si el objeto es del tipo indicado
is!	Falso si el objeto es del tipo indicado

Asignación	
=	Asigna un valor a una variable
*=	Multiplica y asigna
%=	Resto de la división y asigna
&=	Realiza operación de bit AND y asigna
+=	Suma y asigna
/=	Divide y asigna
<<=	Realiza operación de bit de desplazamiento izquierda y asigna
^=	Realiza operación de bit XOR y asigna
-=	Resta y asigna
~/=	Realiza la división entera y asigna
>>=	Realiza operación de bit de desplazamiento derecha y asigna
|=	Realiza operación de bit OR y asigna

Condicionales	
!expr	Invierte el valor de la expresión condicional
||	OR lógico
&&	AND lógico

De Bit	
&	AND
|	OR
^	XOR
~expr	Invierte el valor del Bit
<<	Desplazamiento izquierda
>>	Desplazamiento derecha

Otros	
()	Llamada a una función
[]	Referencia un valor de una lista
cond?exp1:exp2	Si se cumple la condición se asigna el valor de la expresión 1 sino el de la expresión 2
.	Para acceder a los miembros de un objeto
..	Permite realizar múltiples operaciones sobre un objeto
?identificador	Sirve para comprobar argumentos opcionales

Introducción a las expresiones

Las Expresiones en Dart, son fragmentos de código Dart que se evalúan en tiempo de ejecución para obtener un valor, que es siempre un objeto. Cada expresión tiene un tipo estático asociado y cada valor tiene un tipo dinámico asociado.

Una expresión es un conjunto de datos o funciones unidos por operadores aritméticos, lógicos, etc.

Profundizaremos más en los tipos de datos más adelante, por lo que no debes preocuparte demasiado si no entiendes bien la afirmación anterior.

Tipos de expresiones

Condicional

Evalúa una de las dos expresiones en función de un condición booleana.

```
expr1 ? expr2 : expr3
```

Si expr1 es cierto entonces se ejecuta la expr2, en cualquier otro caso, se ejecutará expr3.

Lógica

Combina expresiones lógicas combinan objetos booleanos con operadores de conjunción y disyunción.

```
expr1 || expr2

expr1 ó expr2

expr1 && expr2

expr1 y expr2
```

Bit

```
expr1 & expr2
```

Multiplica lógicamente los bits, por ejemplo 12 & 10 daría como resultado 8.

```
expr1 | expr2
```

Suma lógicamente los bits, por ejemplo 12 | 10 daría como resultado 14.

```
expr1 ^ expr2
```

Suma exclusiva de los bits, por ejemplo 12 ^ 10 daría como resultado 6.

Igualdad

Comprueba en objetos igualdad o identidad.

```
expr1 == expr2
```

La anterior expresión devolverá un objeto booleano true (verdadero) si expr1 es igual que expr2, false (falso) en cualquier otro caso.

```
expr1 != expr2
```

La anterior expresión devolverá un objeto booleano true (verdadero) si expr1 es distinto que expr2.

Relación

Invocan operadores relacionales sobre los objetos de la expresión.

```
expr1 > expr2
```

Devolverá verdadero si expr1 es mayor que expr2, falso en cualquier otro caso.

```
expr1 >= expr2
```

Devolverá verdadero si expr1 es mayor o igual que expr2, falso en cualquier otro caso.

```
expr1 < expr2
```

Devolverá verdadero si expr1 es menor que expr2, falso en cualquier otro caso.

```
expr1 <= expr2
```

Devolverá verdadero si expr1 es menor o igual que expr2, falso en cualquier otro caso.

De desplazamiento

Invocan operadores de desplazamiento sobre los objetos de la expresión.

```
expr1 >> expr2
```

Desplaza un patrón de bits a la derecha descartando los bits por la izquierda.

```
expr1 << expr2
```

Desplaza un patrón de bits a la izquierda rellenando con ceros por la derecha.

Adición

Invocan operadores de adición sobre los objetos de la expresión.

```
expr1 + expr2
expr1 - expr2
```

Multiplicación

Invocan operadores de multiplicación sobre los objetos de la expresión.

```
expr1 * expr2
expr1 / expr2
expr1 ~/ expr2
expr1 % expr2
```

Pre-expresiones y post-expresiones

Tipo concreto de expresiones que permiten realizar operaciones con los objetos de la expresión.

`++expr.` Incrementa en uno expr y devuelve expr.

`expr++.` Devuelve expr y luego incrementa en uno su valor.

`--expr.` Decrementa en uno expr y devuelve expr.

`expr--.` Devuelve expr y luego decrementa en uno su valor.

Asignación

Asignan valor a los objetos de las expresiones. Permiten cambiar el valor de una variable o propiedad siempre que sea mutable, es decir, que se pueda modificar.

El operador básico es [=], pero existen otros, como por ejemplo, [*=], [/=], [~/=], [%=], [+=], [-=], [<<=], [>>=], [&=], [^=], [|=].

expr1 += 5. Toma el valor de expr1 le suma 5 y devuelve el nuevo valor.

```
int NUMBER_SPACING = 19;
double BALL_WIDTH = 19.0;
double BALL_HEIGHT = 19.0;
```

Figura 64. Ejemplo de asignación de valores a diferentes tipos de variables

Crear tus propios operadores

Al comienzo, te contábamos que Dart permite reescribir operadores para cualquier uso que necesitemos. Veamos a continuación ejemplo de lo sencillo que resulta.

Crearemos una clase dirección que almacenará la calle donde vive una persona y utilizaremos el operador [+] para obtener la dirección completa calle, número, piso y ciudad.

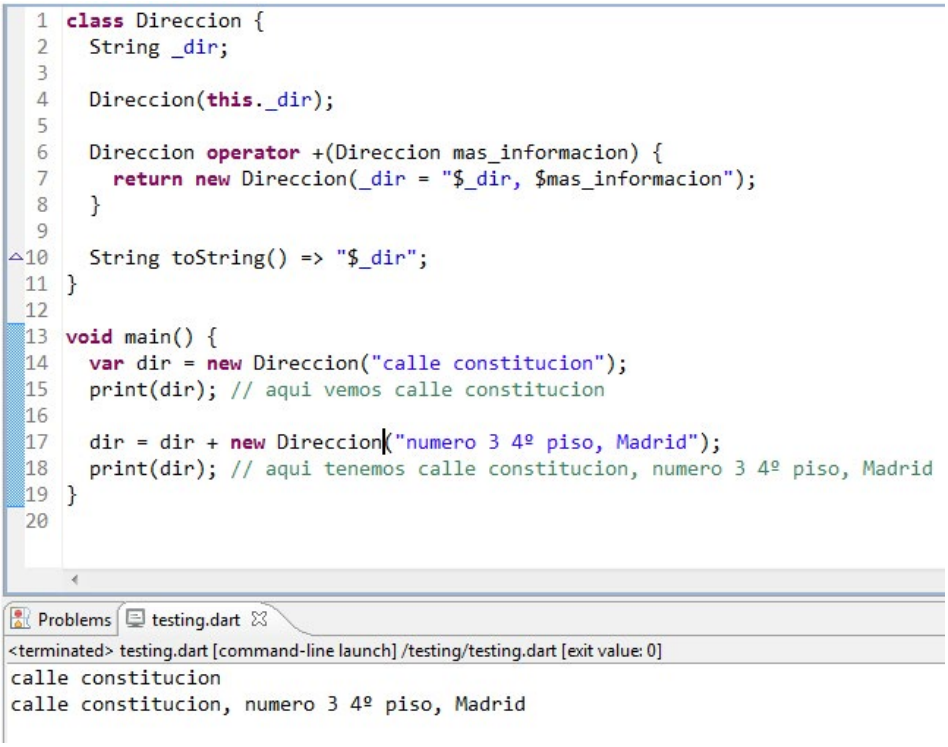


Figura 65. Creación de un operador a medida Variables y tipos de datos

Variables y tipos de datos

Introducción a las variables y tipos de datos

De manera muy simple y a modo de introducción para Dart, como para otros muchos lenguajes, las variables no son más que lugares de almacenamiento, lugares en la memoria que utilizamos para guardar datos.

Ya conoces los operadores, algo necesario y básico en todo lenguaje que se preste, pero sin las variables los operadores no tendrían ningún sentido.

En memoria almacenaremos cadenas, números, resultados de otras funciones, instancias de clases, etc. A todos ellos debes acceder mediante referencias. Estas referencias son las variables. Las variables son nombres que damos a estas partes de la memoria donde hemos almacenado nuestros datos.

Teniendo acceso a nuestros datos podremos operar con ellos. Si en una variable almacenamos el número 5 y en otra variable almacenamos el número 2 podremos utilizar el operador de suma para trabajar con ambas variables y sumar los números contenidos en ellas obteniendo un resultado, este resultado podrá ser almacenado en otra variable para poder utilizarlo más adelante si lo necesitas.

Además Dart soporta diferentes tipos de datos para definir las diferentes variables que vayamos utilizando en nuestros programas.

El tipo de dato es un indicativo que puedes utilizar para saber que el valor que contiene esa variable es un número, una cadena u otro tipo de dato.

Los tipos de datos también ayudan a Dart en el momento de la traducción de código Dart a JavaScript.

Veamos unos sencillos ejemplos de uso de variables.

```
main() {
  var variable1 = 5;
  var variable2 = 2;
  print(variable1+variable2); // El resultado es 7
}
```

Hemos definido dos variables, sin indicar el tipo de dato que van a contener y las hemos sumado mostrando el resultado en pantalla.

En el siguiente ejemplo trabajamos con dos variables diferentes una de tipo String y otra de tipo num. Mostramos un mensaje al usuario concatenando el contenido de ambas variables.

```
main() {
 String variable1 = 'Bienvenido a Dart';
 num variable2 = 1.1;
 print('${variable1} ${variable2}'); // El resultado es "Bienvenido a Dart
 1.1"
}
```

Definición de variables

Como habrás podido observar en los ejemplos anteriores tienes dos formas de definir las variables.

Con la palabra **var** seguido del nombre de la variable.

```
main() {
  var mi_variable;
  var otra_variable = 'Este es el contenido de la variable';
}
```

O especificando el tipo de la variable en el momento de su definición.

```
main() {
  double mi_variable;
  String otra_variable = 'Este es el contenido de la variable';
}
```

Esto es lo que significa que Dart es un lenguaje opcionalmente tipado o "de tipado opcional". Si quieres indicas el tipo de las variables sino quieres no lo indicas.

Indicar el tipo de las variables puede ayudarte a mantener tus aplicaciones. El editor y el compilador utiliza los tipos de datos para ayudarte mientras estás escribiendo código con auto completado de código y proporcionándote alertas o errores.

nota

Fíjate que en el momento de definir una variable puedes indicarle o no valor. En el primer caso la variable no tiene valor asignado por lo que para Dart su valor es `null`, independientemente del tipo de dato. En el segundo caso la variable se inicializa con la cadena "*Este es el contenido de la variable*".

Tipos de variables: Final y Const

Existen dos modificadores que permiten modificar el comportamiento normal de una variable `final` y `const`.

Cuando escribas código debes pensar si una variable va a cambiar o no durante la ejecución, si nunca cambia puedes utilizar `final` o `const` para indicar a Dart que esas variables nunca cambiarán, en lugar de utilizar `var`.

```
main() {
  final app_title = 'Mi primera aplicación Dart';
  final String app_subtitle = 'Dart mola !';
}
```

Puedes utilizar o no el tipo de dato cuando definas la variable con final.

¿Cuál es la diferencia exacta entre *final* y *const*? Una variable declarada como `final` se inicializa una única vez. Una variable declarada como *`const`* se considera una constante en tiempo de ejecución.

Variables de ámbito global o local o variables de clase declaradas como `final` se inicializan la primera vez que se utilicen. Esto se conoce como *lazy initialization* (inicialización perezosa) y ayuda a las aplicaciones a cargar más rápidamente.

Para cualquier valor que quieras que sea constante en tiempo de ejecución utiliza *`const`* seguido del nombre de la variable y del valor que vaya a tener la variable.

```
main() {
  const pi = 3.14159265358979;
}
```

Tipos de datos

Dart dispone de unos tipos de datos que puedes utilizar y a los que se les da un trato especial.

- Numbers
- Strings
- Booleans
- Lists (los conoces también como arrays)
- Maps (los conoces también como diccionarios en Python)

Para Dart cada variable es una referencia a un objeto. Como ya sabes Dart es un lenguaje orientado a objetos y por tanto todo es un objeto, hasta el tipo de dato más básico. Un objeto es la instancia de una clase, hablaremos de esto más adelante.

Cuando creas ese objeto, es decir instancias una clase, utilizas su constructor para inicializar las variables de la clase. Algunos de estos tipos tienen sus propios constructores, por ejemplo, para crear una lista utilizarás el constructor `new List()`.

Pero también puedes inicializar cada uno de estos tipos especiales con un literal.

```
main() {
    var cadena = 'Esto es una cadena'; // Objeto String.

    var numero = 15; // Objeto Number.

    var correcto = true; // Objeto Bool.

    var lista = [1,2,3]; // Objeto List.

    var mapa = {         // Objeto Map.
      'nombre': 'Juan', // propiedad del map de tipo String.
      'edad': 29,       // propiedad del map de tipo Number.
      'enfermo': false  // propiedad del map de tipo Bool.
    };
  }
```

Números

En Dart los números tienen dos tipos `int` o `double` y son un subtipo de `num`.

Cualquier valor numérico entero o decimal que se asigne a una variable definirá esa variable como tipo `num`. El tipo `num` incluye la mayoría de las operaciones que vas a utilizar cuando trabajes con números.

Las operaciones matemáticas que no encuentres en `num` las encontrarás en la librería `dart:math` (que veremos en detalle más adelante), como por ejemplo, `sin`, `cos`, `pow`, `min`, `max`, etc.

`int` son números sin punto decimal.

```
main() {
  var numero = 15;

  var otro_numero = 159763213;

  var numero_hexadecimal = 0x3F;
}
```

`double` son números con punto decimal.

```
main() {
  var double_1 = 15.83;

  var double_2 = 159763213.764398216;

  var double_3 = 1.3e-5;

  var double_4 = -98.72;
}
```

Trabajando con números en Dart

Veamos ahora algunos de los métodos que podemos utilizar cuando trabajemos con números en Dart.

Todos los métodos que verás a continuación están presentes en `dart:core` la librería principal de Dart, que te ofrece un conjunto muy importante, de funciones predefinidas. Esta librería, se importa automáticamente en todos los programas.

Esta librería define tres clases fundamentales para trabajar con tipos de datos `num`, `int`, y `double`. La clase `num` es la super clase de `int` y `double`, es decir que `int` y `double` heredarán todos los métodos que tenga `num`.

Convertir una cadena a número

```
int.parse('189'); // devuelve 189 como objeto int.
double.parse('15.48'); // devuelve 15.48 como objeto double.
```

Los métodos `parse` disponen de su propia gestión de errores en caso de no poder parsear y convertir la cadena a número. Veamos un ejemplo.

```
var numero_entero = int.parse('15');
print(numero_entero); // Muestra 15 como objeto int.
print(numero_entero is int); // Muestra true;

var numero_entero_error = int.parse('15€', onError: (_) {return null;});
print(numero_entero_error); // Muestra null porque no puede parsear la
cadena.
print(numero_entero_error is int); // Muestra false.
var numero_double = double.parse('15.42');

print(numero_double); // Muestra 15.42 como objeto double.
print(numero_double is double); // Muestra true;
print(numero_double is num); // Muestra true;
```

```
var numero_double_error = double.parse('15.42€', (_) {return null;});
print(numero_double_error); // Muestra null porque no puede parsear la
cadena.
print(numero_double_error is double); // Muestra false.
```

Estos métodos y esta gestión de errores resulta muy útil cuando se trata de recoger información por parte del usuario y poder trabajar con ella correctamente.

Si por el contrario quieres convertir un número a `string` entonces puedes utilizar el método `.toString()`.

```
int num = 5;
String cadena = num.toString();
print(cadena); // Muestra objeto String 5.
```

Veamos ahora los métodos comunes de la clase `num` disponibles para trabajar con números en Dart.

Partiendo de un número como este.

```
num numero = -15.84;
```

Con estos métodos puedes saber si un número es finito, infinito, negativo o si no es un número NaN (*not a number*). Devolverán siempre un valor bool `True` o `False`.

```
print(numero.isFinite); // True
print(numero.isInfinite); // False
print(numero.isNegative); // True
print(numero.isNaN); // False
```

Si quieres conocer el signo de un número puedes utilizar este método.

```
print(numero.sign); // -1.0
```

Devolverá -1.0 si es menor que cero, +1.0 si es mayor que cero o el propio valor si es -0.0, 0.0 o NaN.

Si quieres el valor absoluto del número.

```
print(numero.abs()); // 15.84
```

Redondeo hacia el siguiente número entero mayor que el número.

```
print(numero.ceil()); // -15
```

Redondeo hacia el anterior número entero menor que el número.

```
print(numero.floor()); // -16
```

Redondea hacia el número entero más próximo.

```
print(numero.round()); // -16
```

Devuelve la parte entera descartando la parte decimal del número.

```
print(numero.truncate()); // -15
```

Todos los métodos anteriores devolverán un número entero, pero existe la variante `ToDouble` que te devuelve el mismo resultado pero un número double.

```
print(numero.ceilToDouble()); // -15.0
print(numero.floorToDouble()); // -16.0
print(numero.roundToDouble()); // -16.0
print(numero.truncateToDouble()); // -15.0
```

También puedes convertirlo a entero de este modo.

```
print(numero.toInt()); // -15
```

Y puedes obtener una cadena a partir de un número indicando los dígitos que tendrá parte decimal.

```
numero = 3.14159265358979;

print(numero.toStringAsExponential(6)); // 3.141593e+0

print(numero.toStringAsFixed(4)); // 3.1416

print(numero.toStringAsPrecision(2)); // 3.1
```

La clase `int` contiene los métodos vistos hasta ahora más unos cuantos que verás a continuación.

```
int numero = 15;
```

Podrás saber si un número es par o impar.

```
print(numero.isEven); // False

print(numero.isOdd); // True
```

Puedes obtener la longitud de bits de ese número entero en modo binario.

```
print(numero.bitLength); // 4 - número binario 1111
```

También puedes convertir el número a un objeto String en la representación indicada.

```
print(numero.toRadixString(5)); // 30
print(numero.toRadixString(2)); // 1111
print(numero.toRadixString(16)); // f
```

La clase `double` contiene los mismos método comentados hasta ahora y la única diferencia son unas constantes para trabajar con este tipo de números.

```
static const double NAN = 0.0 / 0.0;
static const double INFINITY = 1.0 / 0.0;
```

```
static const double NEGATIVE_INFINITY = -INFINITY;
static const double MIN_POSITIVE = 5e-324;
static const double MAX_FINITE = 1.7976931348623157e+308;
```

El tipo de datos `double` es contagioso lo que significa que cuando realizas operaciones con números de tipo `double` obtienes resultados de tipo `double`.

```
int num1 = 15;
double num2 = 2.0;
print(num1 * num2); // 30.0
```

Librería dart:math

Ahora que ya hemos visto las clases básicas de Dart para trabajar con números vamos también a echar un vistazo a la librería `dart:math` que te permitirá hacer cálculos matemáticos más avanzados con números.

Esta librería proporciona funcionalidades para generación de números aleatorios, funciones de trigonometría y constantes de uso común como pi y e.

Para que puedas utilizar esta librería en tus aplicaciones debes importarla de este modo.

```
import 'dart:math';
```

Además de las funciones comentadas `dart:math` incluye algunas otras clases que te permitirán trabajar con representaciones de posiciones bidimensionales y con rectángulos para diseños gráficos 2D.

Veamos los métodos que pone `dart:math` a tu disposición.

Constantes para utilizar en tus aplicaciones.

```
print(E); // 2.718281828459045

print(PI); // 3.141592653589793
```

Funciones de trigonometría para calcular coseno, seno, tangente, arcocoseno, arcoseno, arcotangente.

```
var grados = 30;
var radianes = grados * (PI / 180); // 0.5235987755982988

print(cos(radianes)); // 0.8660254037844387

print(sin(radianes)); // 0.49999999999999994

print(tan(radianes)); // 0.5773502691896257

print(acos(radianes)); // 1.0197267436954502

print(asin(radianes)); // 0.5510695830994463

print(atan(radianes)); // 0.48234790710102493
```

También para calcular raíz cuadrada, la función exponencial, el logaritmo y la potencia.

```
print(sqrt(25)); // 5.0

print(exp(3)); // 20.085536923187668

print(log(10)); // 2.302585092994046

print(pow(2, 3)); // 8
```

También tienes a tu disposición funciones para calcular el mínimo y el máximo de dos números.

```
print(min(2, 8)); // 2

print(max(15, 10)); // 15
```

Números aleatorios

Además de las operaciones vistas, `dart:math` te permite trabajar con números aleatorios gracias a la clase `Random`.

Instancias un objeto de tipo `Random` llamando a su constructor.

```
var rand = new Random();
```

Puedes generar 10 números `int` aleatorios entre 1 y 100 con un código como este.

```
for(var i=0; i<10; i++) {
  print(rand.nextInt(100));
}
```

Este código te podría devolver una secuencia de números como esta por ejemplo.

```
40, 84, 35, 26, 58, 86, 77, 28, 68, 28
```

Con este código podrás obtener una lista de 10 números `double` aleatorios entre 0 y 1.

```
for(var i=0; i<10; i++) {
  print(rand.nextDouble());
}
```

Y este ejemplo podría devolver una lista de números como esta.

```
0.35400081214332246, 0.2540705633345569, 0.15263890994501117, 0.265262387358984,
0.9386646648924302, 0.8545231577991513, 0.5948728139929099, 0.04713641533447943,
0.6284813334686553, 0.5992417288687604
```

`Random`, además permite generar aleatoriamente valores `bool`.

```
for(var i=0; i<10; i++) {
   print(rand.nextBool());
}
```

Y este pequeño fragmento podría devolver una lista de valores como esta.

```
true, false, false, false, false, true, false, true, true, false
```

Strings

Las cadenas en Dart son secuencias de caracteres UTF-16. Para inicializar una cadena puedes utilizar un literal encerrado entre comillas dobles o comillas simples.

```dart
var cad_1 = "Ejemplo de cadena entre comillas dobles";

var cad_2 = 'Ejemplo de cadena entre comillas simples';

// Puedes incluir un tipo de comillas dentro de otro.
var cad_3 = 'Esto es un "string"';

// Puedes escapar comillas con barra invertida.
var cad_4 = 'Escapando \'comillas simples\' dentro de comillas simples';
```

Dart soporta **interpolación de Strings**, es decir dentro de una cadena puedes incluir el valor de otra variable utilizando este formato `${expresion}`

```dart
var numero = 4000;

var cad_1 = 'Ya utilizamos Dart más de ${numero} personas!';

print(cad_1); // Ya utilizamos Dart más de 4000 personas!
```

También es posible definir cadenas de múltiples líneas utilizando la triple comilla simple o la triple comilla doble, similar a Python.

Dentro de estas cadenas también puedes utilizar interpolación.

```dart
var nombre = 'Juan';

var plantilla = '''
<html>
  <head>
    <tittle> Dart </tittle>
  </head>
  <body>
    <h1> Bienvenido a Dart ${nombre} !! </h1>
  </body>
</html>
''';
```

Más adelante veremos la clase **StringBuffer** y lo eficiente que resulta para concatenar cadenas, pero si necesitas hacer una concatenación sencilla de cadenas puedes utilizar *strings* adyacentes.

```
var direccion = 'Calle Mayor, 12'
 'CP: 28001'
 'Madrid'
 'España';

print(direccion); // Calle Mayor, 12 CP: 28001 Madrid España
```

Trabajo con cadenas

Dentro de `dart:core` además de las clases que has visto para trabajar con número existen otras clases para trabajar con otros tipos de datos básicos de Dart.

En este caso vamos a ver como puedes trabajar con cadenas en Dart y todos los métodos de los que dispone la clase `String` de `dat:core`.

Algo que debes tener en cuenta es que las cadenas en Dart son inmutables, no es posible cambiar una cadena, es decir, puedes realizar operaciones con cadenas y el resultado asignarlo a otra cadena.

```
var cadena = 'Bienvenido a Dart';
var nueva_cadena = cadena.substring(13, 17);
print(nueva_cadena); // Dart
```

Las cadenas internamente funcionan como listas de caracteres por lo tanto puedes acceder a una posición concreta de la cadena y obtener el carácter con el operador **[]**

```
var cadena = 'Bienvenido a Dart';
print(cadena[0]); // B
```

Las cadenas son secuencias de caracteres UTF16 por lo tanto también podrás obtener el código del carácter en dicha posición.

```
print(cadena.codeUnitAt(0)); // 66
```

Puedes obtener todos los códigos que componen una cadena.

```
print(cadena.codeUnits); // [66, 105, 101, 110, 118, 101, 110, 105, 100, 111,
32, 97, 32, 68, 97, 114, 116]
```

Puedes obtener la longitud de la cadena con este método.

```
print(cadena.length); // 17
```

Puedes saber si una cadena finaliza con un determinado carácter, con otra cadena o si termina con una expresión regular. Los métodos `endsWith`, `startsWith`, `split`, `replace`, `contains`, `lastIndexOf`, `indexOf` permiten especificar una expresión regular como argumento.

```
print(cadena.endsWith('t')); // True

print(cadena.endsWith('adios')); // False

print(cadena.startsWith('Bien')); // True
```

Puedes saber en qué posición aparece una cadena o un carácter de este modo.

```
print(cadena.indexOf('Dart')); // 13

print(cadena.indexOf('hola')); // -1

print(cadena.lastIndexOf('a')); // 14
```

También podrás saber si una cadena está o no vacía.

```
print(cadena.isEmpty); // False

print(cadena.isNotEmpty); // True
```

Puedes utilizar este método para quedarte solo con un fragmento de la cadena.

```
print(cadena.substring(13)); // Dart
```

Y puedes eliminar tanto los caracteres en blanco al comienzo y al final con `trim()`.

```
var cadena2 = '       Dart        ';

print(cadena2.trim()); // Dart
```

Con estos métodos puedes saber si una cadena contiene un determinado carácter u otra cadena. Puedes reemplazar solo la primera aparición o todas las que existan de una determinada cadena. Y puedes partir la cadena a partir de un determinado carácter o subcadena.

```
print(cadena.contains('hola')); // False

print(cadena.contains('Dart')); // True

print(cadena.replaceFirst('a', 'to')); // Bienvenido to Dart

print(cadena.replaceAll('e', '3')); // Bi3nv3nido a Dart

print(cadena.split('a')); // [Bienvenido ,  D, rt]
```

Convierte la cadena a minúsculas o mayúsculas así.

```
print(cadena.toLowerCase()); // bienvenido a dart

print(cadena.toUpperCase()); // BIENVENIDO A DART
```

Después de haber echado un vistazo a los métodos que pone a tu disposición la clase String de `dart:core` nos gustaría mostrarte todo el potencial de la clase `StringBuffer` que como te hemos comentado anteriormente es una implementación muy eficiente para la concatenación de strings.

Para crear un nuevo `StringBuffer` debes hacer una llamada al constructor.

```
StringBuffer resultado = new StringBuffer();
```

Una vez creado el objeto puedes comenzar a escribir contenido dentro del StringBuffer con el método `write`.

```
resultado.write('Bienvenido');
```

```
resultado.write(' a ');
resultado.write('Dart');

print(resultado.toString()); // Bienvenido a Dart
```

Puedes añador nuevas líneas al `StringBuffer`.

```
resultado.writeln(); // Escribe una nueva línea.

resultado.write('version 1.1');
print(resultado.toString()); // Bienvenido a Dart
                             // version 1.1
```

Podrás consultar el tamaño del `StringBuffer` y saber si está o no vacío.

```
print(resultado.length); // 29

print(resultado.isEmpty); // False
```

Además puedes escribir un conjunto de cadenas (`List<string>`) en el objeto con el método `writeAll`.

```
resultado.writeAll(['. el ', 'nuevo ', 'lenguaje ', 'de ', 'programación']);
print(resultado.toString()); // Bienvenido a Dart
                             // version 1.1. el nuevo lenguaje de programación
```

Para limpiar y vaciar el objeto `StringBuffer` puedes utilizar el método `clear()`.

```
print(resultado.clear().toString()); // null
```

Expresiones regulares

Hemos visto cómo se trabaja con cadenas y con `StringBuffer` en Dart. Durante estos ejemplos algunos métodos pueden aceptar cadenas, caracteres o expresiones regulares.

Para trabajar en Dart con expresiones regulares `dart:core` pone a tu disposición la clase `RegExp`. Las expresiones regulares tienen la misma sintaxis y semántica que en JavaScript.

Ver las especificaciones de JavaScript:
http://ecma-international.org/ecma-262/5.1/

nota

Corchetes

Se utilizan para realizar búsquedas en un rango de caracteres:

`[abc]`	Busca cualquier carácter indicado entre los corchetes.
`[^abc]`	Busca cualquier carácter NO indicado entre los corchetes.
`[0-9]`	Buscar cualquier dígito indicado entre los corchetes.
`[^0-9]`	Buscar cualquier dígito NO indicado entre los corchetes.
`(x|y)`	Buscar cualquiera de las alternativas indicadas.

Cuantificadores

Permiten establecer números de coincidencias en las búsquedas que realiza la expresión regular:

`n+`	Cualquier cadena que contenga al menos una ocurrencia de *n*
`n*`	Cualquier cadena que contenga ninguna o más ocurrencias de *n*
`n?`	Cualquier cadena que contenga ninguna o una ocurrencia de *n*
`n{X}`	Cualquier cadena que contenga una secuencia de *X n's*
`n{X,Y}`	Cualquier cadena que contenga una secuencia de *X* a *Y n's*
`n{X,}`	Cualquier cadena que contenga una secuencia de al menos *X n's*
`n$`	Cualquier cadena que termine con *n*
`^n`	Cualquier cadena que comience con *n*
`?=n`	Cualquier cadena seguida de la cadena *n*
`?!n`	Cualquier cadena no seguida de *n*

Meta-caracteres

Las expresiones regulares además se sirven de ciertos caracteres especiales para realizar las búsquedas:

`.`	Busca un carácter, a excepción del carácter de terminación de línea o de nueva línea
`\w`	Busca letra
`\W`	Buscar caracteres que NO sean letra
`\d`	Busca dígitos
`\D`	Busca caracteres que NO sean dígitos
`\s`	Busca el carácter espacio en blanco
`\S`	Busca caracteres que NO sean el espacio en blanco
`\b`	Busca coincidencias al comienzo o fin de una palabra
`\B`	Busca coincidencias que NO estén al comienzo o fin de una palabra
`\0`	Busca el carácter NUL
`\n`	Busca el carácter de nueva línea
`\f`	Busca el carácter de alimentación
`\r`	Busca el carácter de retorno de carro
`\t`	Busca el carácter tabulador
`\v`	Busca el carácter tabulador vertical
`\xxx`	Busca el carácter indicado por el número octal xxx
`\xdd`	Busca el carácter indicado por el número hexadecimal dd
`\uxxxx`	Busca el carácter Unicode indicado por el número hexadecimal xxxx

Por ejemplo para localizar números dentro de una cadena puedes utilizar una expresión regular como esta.

```
var texto = 'Bienvenido a Dart';
var re = new RegExp(r'\d+');
print(re.hasMatch(texto)); // False
texto = 'Bienvenido a Dart 1.1';
print(re.hasMatch(texto)); // True
```

El constructor de la clase `RegExp` nos permite crear una nueva expresión regular y definir si es, o no, sensible a mayúsculas y minúsculas y que soporte o no múltiples líneas.

```
var re = new RegExp('en');
```

Partiendo de este texto de ejemplo puedes ver como se maneja el objeto `RegExp`.

```
var texto = 'Bienvenido a Dart';
```

Puedes saber si la expresión regular tiene coincidencias en el texto con el método `hasMatch`.

```
print(re.hasMatch(texto)); // True
```

El método `stringMatch` te devuelve el texto de la primera coincidencia si la hubiera.

```
print(re.stringMatch(texto)); // en
```

Y `allMatches` te permite trabajar con todas las coincidencias obtenidas. En este caso iteramos sobre ellas y mostramos desde qué indice hasta qué indice aparece esa concordancia.

```
re.allMatches(texto).forEach((m) {
   print(m.start);
   print(m.end);
}); // 2, 4
   // 5, 7
```

Estos métodos te devuelven el patrón de la expresión regular crea, si tiene soporte multi línea y si es o no sensible mayúsculas y minúsculas.

```
print(re.pattern);  // 'en'
print(re.isMultiLine); // False
print(re.isCaseSensitive); // True
```

Booleans

Las variables definidas como booleanas con el tipo de dato `bool` suelen representar una variable que varía entre dos estados verdadero o falso. Las variables booleanas se definen en Dart con los literales `true` o `false`.

Dart solo trata como verdadero el valor `true` todo lo demás lo interpreta como falso, al contrario que sucede en JavaScript.

Esto debes tenerlo presente cuando programes en Dart ya que estarás obligado a preguntar y comprobar por los valores que esperas.

Veamos un ejemplo de uso de expresiones condicionales sobre valores no booleanos.

```
var nombre = 'Juan';
if(nombre) {
  print(nombre);
}
```

Si utilizas un código como ese obtendrás un error de Dart indicándote que la variable de tipo `String` no es un subtipo de `Bool` para utilizarlo en una expresión condicional.

En cambio este tipo de condiciones son muy comunes en JavaScript y funcionan perfectamente, pero con Dart la cosa cambia y debes tenerlo en cuenta.

Podrías cambiar el ejemplo anterior y ejecutarlo de este modo.

```
var nombre = 'Juan';
if(nombre != null && nombre.isNotEmpty) {
  print(nombre);
}
```

Trabajando con valores Bool

Dentro de `dart:core` disponemos de la clase `bool` que utilizarás cuando quieras trabajar con valores booleanos. Esta clase no dispone de métodos como sucede con los números o las cadenas, en realidad estos tipos de valores se suelen utilizar en las expresiones condicionales para determinar si se ejecuta o no un determinado bloque de código.

El único método que pone a tu disposición la clase `bool` es el método `.toString()` que te devuelve un objeto cadena con el valor de la variable booleana.

```
bool hecho = false;
if(hecho) {
  print('Buen trabajo has terminado !');
} else {
  print('Lo siento, pero tienes que continuar con ello');
}
print(hecho.toString()); // "false"
```

Lists

Una de las estructuras de datos más comunes y utilizadas en cualquier lenguaje de programación son las listas. En otros lenguajes los conoces como **array**. En Dart los *arrays* son colecciones de objetos y por utilizar un nombre más sencillo y comprensible se llaman listas.

Como has visto anteriormente puedes inicializar una variable de tipo lista con un literal utilizando corchetes y comas para separar los valores.

```
main() {
  var personas = ['Patricia', 'Moises'];
  List mascotas = ['Benchara', 'Erik', 'Lili'];
  List lista_vacia = new List();
```

```
  // Añadimos elementos a la lista vacía.
  lista_vacia.add('Primer elemento');
  lista_vacia.add(28);

  print(personas); // [Patricia, Moises]
  print(mascotas); // [Benchara, Erik, Lili]
  print(lista_vacia); // [Primer elemento, 28]
}
```

En el ejemplo puedes ver como se crea e inicializa una lista a partir del literal o utilizando el constructor `new List()`.

Hemos añadido aquí el método `add()` que permite añadir elementos a una lista, para ponerte un ejemplo de como puedes añadir elementos de cualquier tipo a una lista.

Incluso puedes anidar listas, es decir, dentro de una lista incluir otra lista o cualquier otro tipo de dato.

Como ya sabes Dart posee un sistema de tipado opcional, pero si te interesa definir incluso los valores que irán dentro de tus listas lo puedes hacer utilizando corchetes angulares y el tipo de dato, como ves en este ejemplo:

```
main() {
  List<int> lista_numeros = [1, 2, 3, 4, 5];
  List<String> lista_nombres = ['Pedro', 'Juan', 'Carmen'];
}
```

Trabajando con listas en Dart

Vamos a ver a continuación todos los métodos que pone a tu disposición `dart:core` con la clase `List`.

La clase `List` pone a tu disposición cuatro tipos diferentes de constructores para definir listas en Dart.

El constructor estándar te permite crear una lista sin elementos y sin definir el tamaño de la lista.

```
var lista_vacia = new List();
print(lista_vacia); // []
```

Puedes definir una lista con un tamaño fijo de elementos.

```
var lista_vacia_2_elementos = new List(2);
print(lista_vacia_2_elementos); // [null, null]
```

Puedes crear una lista de tamaño fijo y que al mismo tiempo se inicialicen con un valor, un número, una cadena o un map, por ejemplo.

```
var lista_rellena_num = new List.filled(3, 0);
var lista_rellena_str = new List.filled(3, 'A');
var lista_rellena_map = new List.filled(3, {});
print(lista_rellena_num); // [0, 0, 0]
print(lista_rellena_str); // [A, A, A]
print(lista_rellena_map); // [{}, {}, {}]
```

También puedes utilizar el constructor `List.from` y crear una lista a partir de otra. Este constructor dispone de un parámetro para crear la lista de tamaño fijo o variable, permitiendo que la lista crezca o no.

```
var lista_desde_otra = new List.from(lista_rellena_str,
growable:true);
print(lista_desde_otra); // [A, A, A]
lista_desde_otra.add('B');
print(lista_desde_otra); // [A, A, A, B]
```

Por último tienes el método `List.generate` que te permite crear una lista a partir de otra utilizando un iterator y que se cumplan ciertas condiciones.

Partiendo de una lista de ejemplo como esta.

```
var otra_lista = ['/etc/', '/home/pi/'];
```

Puedes crear una nueva lista a partir de los elementos de la primera y separandolos por la barra invertida.

```
var lista_iterator_1 = new List.generate(otra_lista.length, (e) {
  return otra_lista[e].split('/');
}, growable: true);
print(lista_iterator_1); // [[, etc, ], [, home, pi, ]]
```

O puedes crear una nueva lista separando cada elemento por la barra invertida y descartando elementos vacíos.

```
var lista_iterator_2 = new List.generate(otra_lista.length, (e) {
  return otra_lista[e].split('/').where((e) => e != '').toList();
}, growable: true);
print(lista_iterator_2); // [[etc], [home, pi]]
```

Además de los constructores, tienes a tu disposición todos estos métodos para trabajar con listas.

Vamos a trabajar estos ejemplos de manejo de listas con una lista simple como ésta.

```
var lista = [1, 2, 3];
```

Este método te dirá la longitud de la lista, es decir, cuántos elementos contiene.

```
print(lista.length); // 3
```

Puedes cambiar la longitud de la lista.

```
lista.length = 5;

print(lista.length); // 5
```

Y si quieres acceder a una determinada posición y recuperar su valor puedes utilizar los corchetes indicando el índice.

```
print(lista[1]); // 2
```

Utilizando también los corchetes puedes asignar valores a esas posiciones.

```
lista[3] = 4;

lista[4] = 5;


print(lista); // [1, 2, 3, 4, 5]
```

O puedes utilizar el método add() para añadir items a la lista.

```
lista.add(6);

print(lista); // [1, 2, 3, 4, 5, 6]
```

Existe un método para añadir una lista a otra.

```
lista.addAll([7, 8, 9]);

print(lista); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Puedes darle la vuelta a la lista con el método `reversed()`.

```
print(lista.reversed); // (9, 8, 7, 6, 5, 4, 3, 2, 1)
```

Este método te permite ordenar una lista.

```
lista.sort();
print(lista); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Puedes barajar los elementos de la lista de manera aleatoria con `shuffle()`.

```
lista.shuffle();
print(lista); // [3, 6, 8, 7, 5, 1, 2, 4, 9]
```

Ahora tú lista podría tener estos valores.

```
lista = [3, 6, 8, 7, 5, 1, 2, 4, 9];
```

Este método te permite saber en qué posición se encuentra el elemento 7. Te devolverá -1 si no encuentra dicho elemento.

```
print(lista.indexOf(7)); // 3
```

Puedes obtener la última posición de un elemento con `lastIndexOf()` o -1 si no aparece.

```
print(lista.lastIndexOf('Z')); // -1
```

El método `insert` te permite insertar un determinado valor en una posición. En este ejemplo en la posición 0 insertamos el valor 9.

```
lista.insert(0, 9);
print(lista); // [9, 3, 6, 8, 7, 5, 1, 2, 4, 9];
```

Puedes insertar una lista de valores dentro de otra en una determinada posición con el método `insertAll`.

```
lista.insertAll(5, [3, 3, 3]);
print(lista); // [9, 3, 6, 8, 7, 3, 3, 3, 5, 1, 2, 4, 9]
```

Los dos métodos anteriores permiten insertar contenido dentro de una lista ampliando el tamaño de la misma. Este otro método `setAll` permite reemplazar unos valores de una

lista por otros en una determinada posición.

```
lista.setAll(3, [5, 5, 5]);
print(lista); // [9, 3, 6, 5, 5, 5, 3, 3, 5, 1, 2, 4, 9]
```

Para eliminar un determinado valor de una lista puedes utilizar el método `remove` indicando el elemento a eliminar.

```
lista.remove(6);
print(lista); // [9, 3, 5, 5, 5, 3, 3, 5, 1, 2, 4, 9]
```

También puedes eliminar un elemento en una determinada posición con `removeAt`.

```
lista.removeAt(0);
print(lista); // [3, 5, 5, 5, 3, 3, 5, 1, 2, 4, 9]
```

Puedes eliminar el último elemento de una lista con `removeLast`.

```
lista.removeLast();
print(lista); // [3, 5, 5, 5, 3, 3, 5, 1, 2, 4]
```

Cuando trabajes con listas también eliminar ciertos elementos de una lista con el método `removeWhere` y definir la condición que se debe cumplir para eliminar esos elementos.

```
lista.removeWhere((e) => e==5);
print(lista); // [3, 3, 3, 1, 2, 4]
```

Por el contrario con el método `retainWhere` puedes quedarte solo con ciertos elementos que cumplan la condición que definas.

```
lista.retainWhere((e) => e!=3);
print(lista); // [1, 2, 4]
```

Y puedes quedarte con una parte determinada de una lista indicando el índice desde - hasta.

```
print(lista.sublist(1, 2)); // [2]
```

El método `getRange` te devuelve un `Iterable` que te permite trabajar con los elementos indicados con los índices desde hasta.

En este método igual que en el anterior el índice final se excluye del rango a obtener.

```
print(lista.getRange(0, 2)); // (1, 2)
```

`setRange` te permite copiar los valores de un `Iterable` en el rango desde hasta. Si además especificas el último parámetro (`skipCount`) se descartan esos valores del `Iterable`.

```
print(lista); // [1, 2, 4]
var lista2 = [7, 8, 9, 6];
lista.setRange(1, 2, lista2, 2);
print(lista); // [1, 9, 4]
```

Elimina los elementos en el rango desde hasta.

```
lista.removeRange(1, 2);
print(lista); // [1, 4]
```

Establece los valores del rango desde hasta con el valor indicado.

```
lista.fillRange(1, 2, 2);
print(lista); // [1, 2]
```

Elimina los valores en el rango desde hasta e inserta los valores proporcionados.

```
lista.replaceRange(1, 2, [7, 8, 9]);
print(lista); // [1, 7, 8, 9]
```

También puedes convertir una lista en un diccionario donde las claves serán números consecutivos y los valores del `Map` los elementos que tuviese la lista.

```
var map = lista.asMap();
print(map.keys); // (0, 1, 2, 3)
print(map.values); // (1, 7, 8, 9)
```

Si deseas vaciar una lista puedes utilizar `clear`.

```
lista.clear();
print(lista); // []
```

Iterable

Los métodos vistos hasta aquí son los métodos propios que define Dart en `dart:core` para la clase `List`, pero cuando trabajas con listas o conjuntos, como veremos enseguida, tienes a tu disposición más métodos ya que `List`, igual que Set implementan la interfaz `Iterable`.

Esta interfaz es muy importante ya que también la implementan las clases contenidas `dart:collection`.

Un `Iterable` es un objeto que, utilizando un `Iterator`, permite trabajar con objetos uno cada vez. En otras palabras, te permite iterar utilizando el bucle `for-in` sobre los objetos y hacer algo con ellos.

Estos son algunos ejemplos de iterables utilizando el bucle `for-in`.

```
var lista = [1, 2, 3];
for(var elemento in lista) {
  print(elemento); // 1
                   // 2
                   // 3
```

```
}

var map = {'1': 'uno',
           '2': 'dos',
           '3': 'tres'};
for(var elemento in map.values) {
  print(elemento); // uno
                   // dos
                   // tres
}
```

El método `map` te permite obtener un nuevo iterable donde cada elemento del iterable se reemplaza por el resultado de la función ejecutada en **`map`**.

```
var dicc = {'1': 'uno',
            '2': 'dos',
            '3': 'tres'};
dicc.keys.toList().forEach((k) {
  print('${k} :: String? ${k is String} :: Int? ${k is int}');
});
```

Por cada key del diccionario verás este resultado.

```
// 1 :: String? true :: Int? false
// 2 :: String? true :: Int? false
// 3 :: String? true :: Int? false
```

Ahora vamos a convertir este map donde sus keys son cadenas a un nuevo map donde las keys sean `int`.

```
var keys = dicc.keys.map((key) => int.parse(key));
dicc = new Map.fromIterables(keys, dicc.values);
dicc.keys.toList().forEach((k) {
  print('${k} :: String? ${k is String} :: Int? ${k is int}');
```

```
});
```

Por cada *key* del *map* podrás ver este resultado ahora.

```
// 1 :: String? false :: Int? true
// 2 :: String? false :: Int? true
// 3 :: String? false :: Int? true
```

Puedes obtener un nuevo Iterable con aquellos elementos que cumplan una determinada condición con el método `where`.

```
var dicc = {'1': 'uno',
            '2': 'dos',
            '3': 'tres'};
var odd = dicc.keys.where((e) => int.parse(e).isOdd);
print(odd); // (1, 3)
```

El método `expand` te permite crear un nuevo `Iterable` a partir de los elementos del `Iterable` inicial expandiendo cada elemento en 0 o más elementos a partir del resultado de una función.

```
var queryString = 'arg1=valor1&arg2=valor2&arg3=valor3';
var argumentos = queryString.split('&');
print(argumentos); // [arg1=valor1, arg2=valor2, arg3=valor3]
argumentos = argumentos.expand((String arg) => arg.split('=')).toList();
print(argumentos); // [arg1, valor1, arg2, valor2, arg3, valor3]
```

Si necesitas comprobar si un Iterable contiene un determinado elemento puedes utilizar el método `contains`.

```
var dicc = {'1': 'uno',
            '2': 'dos',
            '3': 'tres'};
```

```
print(dicc.values.contains(4)); // false
print(dicc.values.contains('dos')); // true
```

También puedes iterar sobre cada elemento y aplicar un determinada función con el bucle forEach.

```
dicc.keys.forEach((key) {
  key = int.parse(key);
  print(key); // 1
              // 2
              // 3
});
```

El método reduce te permite reducir una colección de datos a un solo valor iterando sobre los elementos y combinándolos utilizando la función proporcionada.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};

var keys_sum = dicc.keys.reduce((total, elemento) => total + elemento);
print(keys_sum); // 6
```

Tienes a tu disposición otro método similar al anterior que además te permite inicializar la variable que almacenará el resultado con un determinado valor.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};

var keys_sum = dicc.keys.fold(45, (total, elemento) => total + elemento);
print(keys_sum); // 51
```

Y puedes obtener un valor booleano si todos y cada uno de los elementos (`every`) del `Iterable` cumplen o no una determinada condición.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};

var keys = dicc.keys.every((e) => e.isEven);
print(keys); // false

keys = dicc.keys.every((e) => e is int);
print(keys); // true
```

También existe un método que te permite obtener un valor booleano verdadero o falso si sólo alguno de los elementos (`any`) del Iterable cumplen o no una determinada condición.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};

var keys = dicc.keys.any((key) => key.isOdd);
print(keys); // true

keys = dicc.keys.any((key) => key.isEven);
print(keys); // true
```

El método `join` te permite convertir cada elemento a `String` y los termina uniendo con el separador que especifiques.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};
```

```
var keys = dicc.keys.join("--");
print(keys); // 1--2--3
```

Puedes convertir un `Iterable` a `List` o `Set` utilizando los métodos `.toList()` y `.toSet()` respectivamente.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};

print(dicc.keys.toList()); // [1, 2, 3]
print(dicc.keys.toSet()); // {1, 2, 3}
```

Puedes conocer la longitud y saber si está o no vacío un `Iterable`.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};
print(dicc.keys.length); // 3
print(dicc.keys.isEmpty); // false
print(dicc.keys.isNotEmpty); // true
```

Puedes obtener el primer y último elemento, o el único que contenga el Iterable. Si el Iterable está vacío o tiene más de un único elemento el método single lanzará una excepción.

```
print(dicc.keys.first); // 1
print(dicc.keys.last); // 3
print(dicc.keys.single); // Bad state: More than one element
```

Si quieres acceder al `Iterable` y obtener un determinado elemento en una determinada posición utiliza este método.

```
print(dicc.values.elementAt(1)); // dos
```

Puedes recuperar un número determinado de elementos con el método `take`.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};

print(dicc.values.take(2)); // (uno, dos)
```

Y puedes recuperar una serie de elementos con `takeWhile` y una condición. Este método se detendrá cuando la condición ya no se cumpla más.

```
print(dicc.keys.takeWhile((k) => k.isEven)); // ()
print(dicc.keys.takeWhile((k) => k.isOdd)); // (1)
```

Igual que puedes quedarte con algunos elementos del objeto también puedes descartar algunos elementos del `Iterable`.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres',
            4: 'cuatro'};

print(dicc.values.skip(2)); // (tres, cuatro)
print(dicc.keys.skipWhile((k) => k.isEven)); // (1, 2, 3, 4)
print(dicc.keys.skipWhile((k) => k.isOdd)); // (2, 3, 4)
```

Por último también puedes quedarte con el primer, el último y un único elemento que cumpla una determinada condición con estos métodos.

```
var dicc = {1: 'uno',
            2: 'dos',
            3: 'tres'};

print(dicc.keys.firstWhere((k) => k.isOdd)); // 1
print(dicc.keys.lastWhere((k) => k.isOdd)); // 3
print(dicc.keys.singleWhere((k) => k.isEven)); // 2
```

Sets

Vamos a ver ahora el tipo de dato conjunto (*Set*) que te permitirá trabajar con conjuntos de datos y operar con ellos como si fuesen listas. La principal característica de los conjuntos es que no permiten almacenar elementos duplicados y son desordenados.

Como hemos visto los conjuntos heredan de `Iterable` y comparten todos los métodos vistos en la sección anterior.

Veamos ahora algunos ejemplos de conjuntos y cómo podemos operar con ellos.

Para definir un `Set` podemos utilizar cualquiera de sus constructores. Fijate como al crear el conjunto los elementos duplicados desaparecen.

```
var con = new Set();
con.addAll([1, 1, 2, 3]);
print(con); // {1, 2, 3}

con = new Set.from([1, 1, 2, 2, 3, 3, 4]);
print(con); // {1, 2, 3, 4}
```

Al trabajar con conjuntos puedes comprobar si contienen un determinado elemento con el método `contains` o si contiene un conjunto de elementos con `containsAll`.

```
print(con.contains(1)); // true
print(con.contains(5)); // false
print(con.containsAll([2, 3])); // true
```

También puedes buscar un determinado valor y si el valor se encuentra dentro del conjunto, la clase `Set` te devolverá el elemento buscado o `null` en caso de no encontrarlo.

```
print(con.lookup(9)); // null
print(con.lookup(4)); // 4
```

Para añadir elementos al conjunto puedes utilizar tanto `add` como `addAll` que funcionan del mismo modo que con `List`.

```
con.add(5);
print(con); // {1, 2, 3, 4, 5}

con.addAll([6, 7, 8, 9]);
print(con); // {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

Y para eliminar elementos de un conjunto puedes utilizar estos métodos para eliminar un único elemento, eliminar varios o eliminar aquellos que cumplan una determinada condición.

```
con.remove(9);
print(con); // {1, 2, 3, 4, 5, 6, 7, 8}

con.removeAll([8, 7, 6]);
print(con); // {1, 2, 3, 4, 5}

con.removeWhere((e) => e==5);
print(con); // {1, 2, 3, 4}
```

Puedes quedarte con ciertos elementos de un conjunto utilizando retainAll seguido de un *Iterable* o *retainWhere* y seguido de una condición para obtener un nuevo conjunto solo con esos elementos.

```
con.retainAll([1,2,3]);
print(con); // {1, 2, 3}

con.retainWhere((e) => e.isOdd);
print(con); // {1, 3}
```

Por último, esta clase pone a tu disposición los métodos de intersección, unión y diferencia necesarios para trabajar con conjuntos de datos.

```
var con2 = new Set.from([3, 4]);
print(con.intersection(con2)); // {3}
print(con.union(con2)); // {1, 3, 4}
print(con.difference(con2)); // {1}
```

Maps

Un `map` es un objeto que asocia claves (keys) y valores (values), si conoces Python te resultará muy familiar ya que es similar a un diccionario en Python.

Se pueden definir igualmente mediante un `string` literal o con su constructor `new Map()`. Si utilizas una cadena para inicializarlo las claves (`keys`) deben ser literales y los valores (`values`) pueden ser de cualquier tipo de dato incluso listas u otros mapas.

En cambio, si utilizas el constructor `new Map` las claves (`keys`) pueden ser de cualquier tipo.

```
var persona1 = {
  'nombre': 'Juan',
  'edad': 29
};
```

```
// Puedes añadir Keys a un map en otro momento.
persona1['apellidos'] = 'Alguien';

Map persona2 = new Map();
persona2['nombre'] = 'Pedro';
persona2['edad'] = 32;

Map resultados = new Map();
resultados[1] = 'Primer puesto 1500 puntos';
resultados[2] = 'Segundo puesto 990 puntos';
resultados[3] = 'Tercer puesto 786 puntos';

// Acceso a las keys de un map.
print(persona1['nombre']); // Juan
print(persona1['apellidos']); // Alguien

print(persona2['edad']); // 32
print(persona2['apellidos']); // null
```

Puedes añadir más `keys` para un `Map` en cualquier otro momento en tu aplicación y si intentas acceder a una propiedad no definida de un `Map` Dart te devolverá `null`.

Del mismo modo que con las listas, también puedes indicar el tipo de dato de los valores de las `keys` y de los `values` de un `map`.

```
Map<String, double> = campeones = {
  'Sebastian Vettel': 397.0,
  'Fernando Alonso': 242.0,
  'Mark Webber': 199.0
};
```

```
Map<int, List> posiciones = {
  1: [1, 3, 4, 1, 7, 8],
  2: [4, 13, 1, 2, 4, 5],
  3: [7, 8, 9, 10, 2, 4]
}
```

La clase `Map` tiene una serie de métodos para poder trabajar con ellos que vamos a ver a continuación.

Además de los constructores que acabamos de comentar la clase `Map` dispone de otros constructores para crear nuevos mapas desde otros `Maps` o incluso desde `Iterables`.

```
var persona1 = {
  'nombre': 'Juan',
  'edad': 29
};

var persona2 = new Map.from(persona1);
persona2['edad'] = 32;

print(persona1); // {nombre: Juan, edad: 29}
print(persona2); // {nombre: Juan, edad: 32}

var lista = ['dos', 'tres', 'cuatro'];
var mapa = new Map.fromIterable(lista,
    key: (item) => item.toString().length,
    value: (item) => item);

print(mapa); // {3: dos, 4: tres, 6: cuatro}

var keys = [1, 2, 3];
var values = ['uno', 'dos', 'tres'];
```

```
var mapa2 = new Map.fromIterables(keys, values);
print(mapa2); // {1: uno, 2: dos, 3: tres}
```

Puedes conocer si contiene una determinada clave o valor con estos métodos.

```
var mapa = {1: 'uno', 2: 'dos', 3: 'tres'};
print(mapa.containsKey('7')); // false
print(mapa.containsKey(2)); // true
print(mapa.containsValue('7')); // false
print(mapa.containsValue('tres')); // true
```

La clase `Map` dispone de un método para añadir una clave si esta no existe ya en el objeto y al mismo tiempo asociar un valor.

```
mapa.putIfAbsent(4, () => 'cuatro');
print(mapa); // {1: uno, 2: dos, 3: tres, 4: cuatro}
```

También te permite añadir de forma masiva otros pares clave-valor con `addAll`.

```
mapa.addAll({5: 'cinco', 6: 'seis'});
print(mapa); // {1: uno, 2: dos, 3: tres, 4: cuatro, 5: cinco, 6: seis}
```

Puedes eliminar un clave del objeto `map` con `remove`. Se elimina la clave y el valor asociado.

```
mapa.remove(5);
print(mapa); // {1: uno, 2: dos, 3: tres, 4: cuatro, 6: seis}
```

Y también puedes aplicar una función a cada par clave-valor con `forEach`.

```
mapa.forEach((k, v) {
  print('Key ${k} = ${v}'); // Key 1 = uno
                            // Key 2 = dos
                            // Key 3 = tres
                            // Key 4 = cuatro
                            // Key 6 = seis
});
```

La clase `Map` pone a tu disposición una serie de métodos para conocer las claves y valores del objeto, la longitud de pares clave-valor y si está o no vacío.

```
print(mapa.keys); // (1, 2, 3, 4, 6)
print(mapa.values); // (uno, dos, tres, cuatro, seis)
print(mapa.length); // 5
print(mapa.isEmpty); // false
print(mapa.isNotEmpty); // true
```

Por último, puedes vaciar un `map` completamente con el método `clear`.

```
mapa.clear();
print(mapa); // {}
```

Fechas y horas

Además de los tipos de datos básicos que ofrece Dart, hemos visto algunas de las clases más interesantes de dart:core, *Lists, Sets, Maps, RegExp*, etc. que te permiten trabajar con otras estructuras de datos muy útiles.

Ahora vamos a ver la clase ***DateTime*** que también utilizarás con mucha frecuencia cuando necesites trabajar con fechas y horas.

Esta clase dispone de un conjunto de constructores para definir objetos *`DateTime`*.

Puedes definir un objeto fecha hora con instante actual con este constructor.

```
var fecha = new DateTime.now();
print(fecha); // 2014-03-08 18:52:33.906
```

También puedes definirlo indicando explícitamente el año, mes, día, hora, minuto, segundo y milisegundo. Si no se indica algún valor se inicializa por defecto con el valor 0.

```
fecha = new DateTime(2014, 3, 14, 17, 30, 0);
print(fecha); // 2014-03-14 17:30:00.000
```

Esta clase también permite definir objetos a partir de cadenas que cumplan el formato ISO 8601.

```
fecha = DateTime.parse('2014-07-09 09:00:00');
print(fecha); // 2014-07-09 09:00:00.000
```

Puedes definir también objetos `DateTime` directamente en hora universal UTC.

```
fecha =  new DateTime.utc(2014, 3, 14, 17, 30, 0);
print(fecha); // 2014-03-14 17:30:00.000Z
print(fecha.isUtc); // true
```

O definirlos a partir de un Timestamp indicando o no los milisegundos.

```
fecha = new DateTime.fromMillisecondsSinceEpoch(1394280000000);
print(fecha); // 2014-03-08 13:00:00.000
```

Además de estos constructores `DateTime` pone a tu disposición una serie de *getters* y métodos para trabajar con fechas.

Para obtener los valores del objeto `DateTime` puedes utilizar estos *getters*. Como puedes ver los días de la semana en Dart comienzan en 1 para el Lunes y terminan en 7 Domingo.

```
var hoy = new DateTime.now(); // 2014-03-08 18:52:33.906
print(hoy.year); // 2014
print(hoy.month); // 3
print(hoy.day); // 8
print(hoy.hour); // 18
print(hoy.minute); // 52
print(hoy.second); // 33
print(hoy.millisecond); // 906
print(hoy.weekday); // 6 - Sabado
```

Puedes obtener la hora local o la hora universal de un objeto `DateTime` en cualquier momento de este modo.

```
print(hoy.toLocal()); // 2014-03-08 18:52:33.906
print(hoy.toUtc()); // 2014-03-08 17:52:33.906Z
```

Y convertirla en un objeto *`String`* o `String` en formato ISO 8601.

```
print(hoy.toString()); // 2014-03-08 18:52:33.906
print(hoy.toIso8601String()); // 2014-03-08T18:52:33.906
```

Vamos a definir ahora otro objeto `DateTime` y veamos que operaciones podemos hacer con ellos.

```
var ayer = new DateTime(2014, 3, 7, 18, 23, 0);
```

Podemos comparar ambos objetos fácilmente de este modo. Obtendrás un valor *booleano* indicando si una fecha-hora es igual, menor o mayor que otra.

```
print(hoy==ayer); // false
print(hoy.isBefore(ayer)); // false
print(hoy.isAfter(ayer)); // true
```

DateTime implementa otro tipo de método que permite realizar comparaciones entre objetos fecha-hora obteniendo como resultado valores *int*.

```
print(hoy.compareTo(ayer)); // 1
print(ayer.compareTo(hoy)); // -1
print(hoy.compareTo(hoy)); // 0
```

Cuando se trabaja con objetos *DateTime* lo más habitual es que necesites añadir, restar fechas-horas o calcular diferencias entre objetos de este tipo.

Todas estas operaciones se pueden realizar de manera muy sencilla en Dart, pero para poder realizar tenemos que hablar ahora de otra clase disponible en *dart:core* llamada *Duration* que es la que se utiliza como parámetro o la que se devuelve como resultado indicando estas diferencias entre fechas.

La clase `Duration` además dispone de sus propios métodos y operadores para comparar, convertir a `String` y realizar operaciones con objetos de este tipo como sumar, restar, multiplicar o dividir.

Lo más interesante de la clase `Duration` es su constructor que define un objeto `Duration` a partir del parámetro indicado, si no se indica ninguno por defecto será 0.

```
var dur = new Duration(days: 20, hours: 4, minutes: 2, seconds: 17,
milliseconds: 300, microseconds: 155897);
print(dur); // 484:02:17.455897
```

Puedes obtener su valor en días, horas, minutos, segundos, milisegundos o microsegundos con estos *getters*.

```
print(dur.inDays); // 20
print(dur.inHours); // 484
print(dur.inMinutes); // 29042
print(dur.inSeconds); //  1742537
```

```
print(dur.inMilliseconds); // 1742537455
print(dur.inMicroseconds); // 1742537455897
```

Si quieres añadir un día a una fecha debes utilizar el método `.add()` y como parámetro un objeto `Duration` indicando el número de días.

```
print(hoy.add(new Duration(days:1))); // 2014-03-09 18:52:33.906
```

También puedes añadir horas indicando como parámetro un objeto `Duration` expresado en horas.

```
print(hoy.add(new Duration(hours:10))); // 2014-03-09 04:52:33.906
```

Puedes restar un cierto valor de una fecha con el método `subtract()` y un objeto `Duration` expresado en el valor que te interese, por ejemplo, días u horas.

```
print(hoy.subtract(new Duration(days:1))); // 2014-03-07 18:52:33.906
print(hoy.subtract(new Duration(hours:10))); // 2014-03-08 08:52:33.906
```

Si deseas saber qué diferencia existe entre dos fechas puedes utilizar este método y obtendrás como resultado un objeto `Duration` el cual podrás mostrar en el valor que te interese.

```
print(hoy.difference(ayer).inDays); // 1
print(hoy.difference(ayer).inHours); //  24
print(hoy.difference(ayer).inSeconds); // 88173
```

Para terminar te mostramos estos dos métodos de la clase `DateTime` que te permiten obtener el nombre o el *offset* (será un objeto `Duration`) de la zona horaria del cliente.

```
print(hoy.timeZoneName); // CET
print(hoy.timeZoneOffset.toString()); // 1:00:00.000000
```

Sentencias de control

Como ya sabes, un programa es una sucesión de sentencias ejecutadas secuencialmente. Están las sentencias simples (que se ejecutan una a continuación de otra) y las de control (que permiten modificar el flujo de ejecución del programa introduciendo ciclos y condiciones que deben cumplirse para que se ejecuten ciertos bloques de código). De este tipo de sentencias vamos a hablar a continuación.

If y Else

La sentencia `if` permite elegir si se ejecuta o no un bloque de instrucciones. Si esta sentencia incluye un bloque `else`, ejecutará uno si se cumple la condición y otro diferente si esta no se cumple. Dart permite sentencias `if` con sentencias `else` opcionales.

La particularidad es que en Dart, hay que ser más explícito, ya que considera que todos los valores distintos de `True` (verdadero) son `False` (falso), este comportamiento difiere del comportamiento de JavaScript y, como ya te comentamos en la sección de *Definición de Variables y tipos de datos*, si utilizas este tipo de comprobaciones obtendrás un error.

```
var nombre = 'Juan';
if(nombre) {
  print(nombre);
}
```

Veamos cómo escribir sentencias condicionales `if-else` en Dart y como norma de estilo se recomienda utilizar llaves explícitas.

```
var clima = 'riesgo';
if (clima.isNotEmpty && clima == 'riesgo') {
  precaucion();
} else {
  sinPrecaucion();
}
```

Las expresiones `if-else-if` pueden anidarse de este modo.

```
var color = '#0000ff';
if(color == '#ff0000') {
  print('Rojo');
} else if(color == '#00ff00') {
  print('Verde');
} else if(color == '#0000ff') {
  print('Azul');
} else {
  print('No conozco este color!');
}
```

Este tipo de estructuras se desaconsejan ya que para poder realizar esto disponemos de la sentencia `switch` que es más rápida que encadenar `if-else` y genera un código más legible.

Bucles For

Un bucle sirve para repetir una determinada sentencia un número determinado de veces. En Dart dispones de varios tipos de bucles diferentes, pero ahora nos centraremos en los bucle `for`.

Este bucle tiene un formato que te resultará muy familiar ya que hereda la sintaxis de JavaScript, C, etc.

```
for (var i=0; i < 3; i++) {
  print(i);  // 0
             // 1
             // 2
}
```

Los bucles `for` también te permiten iterar sobre objetos `List`, `Map` o `Set`.

```
var lista = ['uno', 'dos', 'tres', 'cuatro'];
for (var i=0; i < lista.length; i++) {
  print('${i} :: ${lista[i]}'); // 0 :: uno
                                // 1 :: dos
                                // 2 :: tres
                                // 3 :: cuatro
}
```

Si el objeto que estás recorriendo es una colección, puedes utilizar el método `forEach()` que es una opción perfecta si no necesitas saber el contador de iteración actual.

```
var lista = ['uno', 'dos', 'tres', 'cuatro'];
lista.forEach((item) {
  print(item); // uno
               // dos
               // tres
               // cuatro
});
```

Para iterar sobre una colección, o cualquier `Iterable` sin necesidad de conocer el contador de iteración, puedes usar un bucle `for-in`.

```
var documentos = ['pedidos', 'albaranes', 'facturas', 'abonos'];
for (var doc in documentos) {
  print(doc); // pedidos
              // albaranes
              // facturas
              // abonos
}
```

Sobre cualquier objeto `Iterable` se puede utilizar el bucle `for-in`.

Bucle While

El bucle `While`, repite un bloque de código mientras una condición se mantenga `True` (verdadera). Evalúa la condición antes de entrar en el bucle y lo repite mientras la condición sea verdadera.

```
while(llamadasPerdidas()) {
  parpadearLuz();
}
```

Si quieres evaluar la condición después de ejecutar el ciclo del bucle (es decir, ejecutaremos el bucle al menos una vez), puedes utilizar `do-while`.

```
do {
  parpadearLuz();
} while (llamadasPerdidas());
```

Al trabajar con bucles también tienes a tu disposición estas dos sentencias: `break` y `continue`. Estas sentencias te servirán para controlar la ejecución del bucle.

`Break` sirve para detener la ejecución del bucle.

```
var i = 100;
while(true) {
  i -= 5;
  print(i);
  if(i < 75) {
    break;
  }
}
```

Este ejemplo devolverá este resultado:

```
95
90
```

```
85
80
75
70
```

En cambio `continue` te permite saltar a la siguiente iteración del bucle.

```
for (var i = 100; i > 0; i--) {
  if (i.isEven) {
    continue;
  }
  print(i);
  if (i < 75) {
  break;
  }
}
```

La ejecución te mostrará un resultado como este:

```
99
97
95
93
91
89
87
85
83
81
79
77
75
73
```

Sentencia Switch

Como ya hemos comentado la sentencia `switch` permite realizar una selección entre múltiples alternativas evitando anidar sentencias `if-else`. Esto mejora la legibilidad del código y además es una estructura que se ejecuta mucho más rápido que utilizar `if-else` anidados.

```
var accion = 'Abrir';
switch (accion) {

  case 'Pagada':
    AnotarPago();
    break;

  case 'Impagada':
   dejarPendiente();
   break;

  case 'Rechazada':
   GenerarNuevamente();
   break;

  default:
   Comprobar();
}
```

Como puedes ver la estructura de esta sentencia define una serie de casos de coincidencia (`case`) y de acciones a realizar en caso de que una de ellas se cumpla. Tras cumplirse esa coincidencia y ejecutarse la acción siempre debe figurar la sentencia `break` para salir de `switch`.

De forma opcional se puede indicar el caso por defecto (`default`) que será lo que se ejecute si ninguna de las demás opciones coinciden, en este caso no es necesario escribir la sentencia `break` porque después de ella no existen más casos.

La sentencia `switch` puede contener casos vacíos como puedes ver a continuación en el caso de Impagada.

```
switch (estadoFactura) {
  case 'Pagada':
    AnotarPago();
    break;

  case 'Impagada':
  case 'Rechazada':
    dejarPendiente();
    break;

  default:
    throw('Estado de Factura erróneo !');
}
```

Lo que significa que se ejecutará la función `dejarPendiente()` si el estado de la factura es Impagada o Rechazada.

Un caso de coincidencia puede contener llamadas a otras funciones como has podido ver en los ejemplos anteriores, pero también cualquier bloque de código que desees ejecutar, incluso definir variables locales que necesites. Estas variables locales son sólo visibles dentro del ámbito de ese caso.

```
switch (estadoFactura) {
  case 'Pagada':
    AnotarPago();
    break;

  case 'Impagada':
  case 'Rechazada':
    dejarPendiente();
    break;
```

```
    default:
      var error = 'Estado de factura erróneo !';
      enviaEmail(asunto:error, para:'errores@empresa.com');
      throw(error);
}
```

Funciones

Hasta ahora has visto sentencias simples, sentencias de control y sentencias iterativas. Con esto ya puedes empezar a construir aplicaciones en Dart, pero a medida que la aplicación crece resulta necesario estructurar mejor el código y aprovechar código que has escrito en una parte del sistema en otra. Gracias a las funciones puedes realizar esta reutilización y conseguir un código más limpio, eficiente y mantenible.

Una función es un grupo de instrucciones que realizan una tarea específica. Se ejecuta llamándola desde otra parte del sistema, un método de clase u otra función. Pueden llamarse tantas veces como lo necesites e incluso pueden llamarse a sí mismas (recursiva).

Las funciones puede recibir parámetros con los que trabajar y pueden devolver un resultado. Los parámetros de las funciones en Dart pueden ser obligatorios u opcionales. En caso de ser opcionales pueden ser opcionales por posición u opcionales por nombre.

Utilizamos funciones para dividir una tarea grande en otras más simples o para implementar operaciones que vamos a utilizar varias veces en nuestra aplicación.

Definición

Veamos cómo definir funciones en Dart.

```
saluda(nombre) {
  print('Bienvenido a Dart ${nombre}');
}
```

Aunque el código anterior es perfectamente válido es recomendable seguir la guía de es-

tilo de Dart. Por lo tanto, deberías especificar siempre el tipo de dato de retorno de la función y los tipos de datos de los parámetros que recibe la función.

```
void saluda(String nombre) {
  print('Bienvenido a Dart ${nombre}');
}
```

Para declarar correctamente una función en Dart y cumplir con la guía de estilo deberás indicar el tipo de dato de retorno de la función, **void** en caso de no retornar ningún valor, seguido del nombre de la función y a continuación entre paréntesis los tipos de datos y nombres de los parámetros que reciba. Todo lo que se encuentra entre la llave de apertura y cierre es el cuerpo de la función.

En caso de funciones sencillas que solamente tengan una única sentencia se puede utilizar la forma abreviada en su definición. Este es un ejemplo.

```
void bienvenido() => print('Bienvenido a Dart!');
bienvenido(); // Bienvenido a Dart!
```

Hemos comentado que los parámetros que puede recibir una función pueden ser obligatorios u opcionales.

Los parámetros obligatorios son los que se indican entre paréntesis. Como puedes ver a continuación.

```
void saluda(String parametro_obligatorio_1,
            num parametro_obligatorio_2) {
  // Cuerpo de la función
}
```

Los parámetros opcionales también se indican entre los paréntesis de la función, pero en esta ocasión hay que indicarlos con una nomenclatura concreta.

Si se trata de parámetros opcionales por posición, utilizarás corchetes `[String nombre]`. En cambio para los parámetros opcionales por nombre utilizarás llaves `{String nombre}`.

Parámetros opcionales por posición

Los parámetros opciones por posición se indican entre corchetes [], y siguen la misma descripción que el resto de los parámetros, tipo de dato y nombre del parámetro.

```
void saluda(String nombre, [String apellidos, num edad]) {
  if(apellidos != null && edad != null) {
    print('Bienvenido a Dart ${nombre} ${apellidos} tienes ${edad} años.');
  } else {
    print('Bienvenido a Dart ${nombre}.');
  }
}
```

Si realizas unas llamadas a esta función con estos tipos de parámetros diferentes podrás obtener estos resultados:

No escribir la sentencia break tras el caso de coincidencia puede hacer que obtengas un comportamiento extraño o inadecuado

Error común

```
saluda('Pedro'); // Bienvenido a Dart Pedro.
saluda('Pedro', 'García'); // Bienvenido a Dart Pedro.
saluda('Pedro', 'García', 28); // Bienvenido a Dart Pedro García
                               // tienes 28 años.
```

Parámetros opcionales por nombre

Además de los parámetros opcionales por posición dispones de parámetros opcionales por nombre. La verdad es que son muy útiles porque no tienes que llamar a la función del mismo modo siempre, y mejora la documentación de la propia función cuando se realiza la llamada.

```
void saluda({String nombre, String apellidos, num edad}) {
  var saludo = new StringBuffer('Bienvenido a Dart');
  if (nombre != null) {
    saludo.write(' ${nombre}');
  }
```

```
    if (apellidos != null) {
      saludo.write(' ${apellidos}');
    }

    if (edad != null) {
      saludo.write(' tienes ${edad} años.');
    }
    print(saludo.toString());
  }

saluda(nombre:'Pedro'); // Bienvenido a Dart Pedro
saluda(apellidos:'García', nombre:'Pedro'); // Bienvenido a Dart Pedro García
saluda(apellidos:'García', edad: 29, nombre:'Pedro'); // Bienvenido a Dart Pedro Gar
                                                      // cía tienes 29 años.
saluda(); // Bienvenido a Dart
```

Observa que no importa el orden en el que pases los parámetros a la función.

Esto es algo estupendo porque si pensamos en funciones que puedan aceptar muchos parámetros opcionales, recordar el orden exacto del paso de parámetros es algo pesado y muy propenso a errores.

Al hacerlo por nombre nos olvidamos del orden y además dejamos la llamada a la función mejor documentada para poder mantener mejor el código.

Valores predeterminados

Las funciones con parámetros opcionales además pueden tener valores predeterminados. Un valor predeterminado se asigna al parámetro opcional en caso de que dicho parámetro no se especifique en el momento de la llamada a la función.

Para indicar el valor predeterminado de un parámetro opcional usamos el signo igual `(=)` o los dos puntos `(:)` dependiendo de si se trata de un parámetro opcional por posición o por nombre.

Este es un ejemplo de valor predeterminado para parámetros opcionales por posición.

```
void saluda([String nombre='ANONIMO']) {
  var saludo = new StringBuffer('Bienvenido a Dart');
  if (nombre != null) {
    saludo.write(' ${nombre}');
  }
  print(saludo.toString());
}
saluda(); // Bienvenido a Dart ANONIMO
saluda('Juan'); // Bienvenido a Dart Juan
```

Y este otro es un ejemplo de valor predeterminado para parámetros opciones por nombre.

```
void saluda({String nombre:'ANONIMO'}) {
  var saludo = new StringBuffer('Bienvenido a Dart');
  if (nombre != null) {
    saludo.write(' ${nombre}');
  }
  print(saludo.toString());
}
saluda(); // Bienvenido a Dart ANONIMO
saluda(nombre:'Juan'); // Bienvenido a Dart Juan
```

Las funciones pueden pasarse como parámetro a otras funciones.

```
void bienvenido(elemento) => print('Bienvenido a Dart elemento
${elemento}');

var lista = [1, 2, 3];
lista.forEach(bienvenido); // Bienvenido a Dart elemento 1
                           // Bienvenido a Dart elemento 2
                           // Bienvenido a Dart elemento 3
```

También puedes asignar una función a una variable.

```
var hola = (elemento) => print('Bienvenido a Dart elemento ${elemento}');
hola(1); // Bienvenido a Dart elemento 1
```

Valores de retorno

Hemos comentado al comienzo de esta sección que cuando una función no retorne ningún valor como norma de sintaxis y estilo se debe indicar la palabra `void`.

Todas las funciones retornan un valor, pero cuando tú función no indica de manera explícita ningún valor la función devuelve `null`.

```
num suma(num a, num b) {
  return a+b;
}
var resultado = suma(1,5);
print(resultado); // 6
```

En este primer ejemplo la función suma te devuelve un número.

```
void saluda() {
  print('Hola!');
}
resultado = saluda(); // Ejecuta print Hola!
print(resultado); // null
```

En este otro caso, la función ejecuta la sentencia `print` mostrando el mensaje 'Hola!', pero no devuelve valor alguno, por lo tanto Dart devuelve `null`.

Resumen de sintaxis

Sobrescribir un operador		`operator`
OPERADORES	Negación	`!, ~`
	Multiplicación	`*`
	División	`/`
	División entera	`~/`
	Resto de la división	`%`
	Suma	`+`
	Resta	`-`
	Desplazamiento a izquierda	`<<`
	Desplazamiento a derecha	`>>`
	Mayor que	`>`
	Mayor o igual que	`>=`
	Menor que	`<`
	Menor o igual que	`<=`
	Igual que	`==`
	Distinto que	`!=`
	Estrictamente igual	`===`
	Estrictamente distinto	`!==`

Comentarios en una línea	`//comentario`
Comentarios de bloque	`/* comentario */`

Comentarios de documentación	`/** comentario */ y cada línea va precedida de *`
	
	`///comentario`

Incrementa en uno expr y devuelve expr	`++expr`
Devuelve expr y luego incrementa en uno su valor	`expr++`
Decrementa en uno expr y devuelve expr.	`--expr.`
Devuelve expr y luego decrementa en uno su valor.	`expr--.`
Asignar valor a los objetos de las expresiones	`[=]`
Otros operadores para asignar valor	`[*=], [/=], [~/=], [%=], [+=], [-=], [<<=], [>>=], [&=], [^=], [|=]`
Definir una variable	`var nombreDeLaVariable`
Definir una constante	`const nombreDeLaVariable valorDeLaVariable`
Definir una cadena en una línea	`cadena _ 1 = "Ejemplo"` `cadena _ 1= "Ejemplo ${valor de otra variable} de cadena"`

Cadena de múltiples líneas	`cadena _ 1 = '''`
	`contenido`
	`contenido`
	`''';`

Tipos de números	`num, int, double`
Valores booleanos	`true, false`
Crear nueva lista	`new List()`

Definir un mapa	`new Map()`
Definir un conjunto	`new Set()`

Sentencia If-else	`if (condición) {`
	`  //instrucciones`
	`} else {`
	`  // instrucciones`
	`}`

Sentencia Switch	`switch(variable) {`
	`  case valor1:`
	`  // instrucciones`
	`  break;`
	`  case valor2:`
	`  // instrucciones`
	`  break;`
	`  default:`
	`  // instrucciones`
	`}`

Bucle While	`While(condición) {`
	`  // instrucciones`
	`}`
	
	`do {`
	`  // instrucciones`
	`} while(condición);`

Bucle For	`for(inicialización; condición; inc./dec.) {`
	`  // instrucciones`
	`}`

Bucle For-in	`for(elemento in Iterable) {`
	`  // instrucciones`
	`}`

Método forEach	`Iterable.forEach((e) {`
	`  // instrucciones`
	`});`

Definición de funciones	`valor _ retorno nombre _ funcion(parámetros) {`
	`  // cuerpo de función`
	`}`

Funciones simples	`retorno nombre(parámetros) => cuerpo;`

Parámetros opcionales por posición	`retorno funcion([tipo param1, tipo param2, ...]) {`
	`  // cuerpo función`
	`}`

Parámetros opcionales por nombre	`retorno funcion({tipo nom _ param1, tipo nom _ param2, ...}) {`

Valores predeterminados	`retorno funcion([param1=valor, param2=valor, …]) {`
	`  // cuerpo función`
	`}`
	
	`retorno funcion({nom _ param1:-valor, nom _ param2: valor, ...}) {`
	`  // cuerpo función`
	`}`

5 DART *AVANZADO*

Ya has visto una parte importante de Dart. Ahora profundizaremos un poco más y veremos el control de errores, las excepciones y su manejo. Todo lo relacionado con la creación y utilización clases, constructores y herencia.

También hablaremos de *Generics, Isolates, Typedefs* y de cómo funcionan las librerías en Dart.

Excepciones y control de errores

Todo lenguaje moderno y orientado a objetos que se precie posee excepciones y control de errores. Dart no podía ser menos.

Cuando estás programando una aplicación, por mucho control que quieras tener, siempre habrá un conjunto de datos que no hayas probado o que el comportamiento que tiene el usuario en una pantalla no sea para el que se definió. En estos casos se producirán errores y nuestra aplicación fallará o incluso se bloqueará.

Dart incorpora un mecanismo de excepciones que permite capturar casos de error excepcionales no contemplados y actuar en consecuencia. De este modo podrás mostrar un mensaje de error más adecuado para el usuario y que el sistema siga funcionando.

En otros casos te puede interesar lanzar excepciones en determinadas partes de tu aplicación. Dart también incorpora mecanismos para hacer esto.

Cuando algo inusual sucede en tu aplicación Dart lanzará una excepción, si esta excepción no es capturada el **isolate** que generó la excepción se interrumpirá y el programa terminará.

Dart proporciona las clases `Exception` y `Error` para el manejo de errores y excepciones. Y también te permite crear tus propias excepciones.

Los mecanismos que presenta Dart para manejar errores y excepciones son `throw`, `catch` y `finally`.

`Throw` permite lanzar excepciones. Esta expresión permite lanzar una excepción en cualquier parte de nuestro desarrollo.

```
if(usuario == null) {
  throw new Exception('Usuario no reconocido en el sistema !!');
}

if(password == null) {
  throw new Exception('La constraseña no es correcta !!');
}

if(datos == null) {
  throw new NullThrownError();
}

if((datos is Map) == false) {
  throw new TypeError();
}
```

`Catch` es el mecanismo que tiene Dart para capturar las excepciones producidas, de este modo se detiene la propagación de la excepción y puedes manejarla.

Para ser más exactos, Dart captura las excepciones mediante los bloques `try {} on {} catch {}`. Estos bloques pueden anidarse y capturar más de un tipo de excepciones.

```
try {
  login();
} on PasswordVaciaException { // Captura de una excepción concreta.
  reLogin();
} on Exception catch(e) { // Captura cualquier Exception.
  print('Pero que ha pasado aquí ?!?!');
} catch(e) { // Captura cualquier otra cosa que suceda.
  print('Algo ha ido realmente mal. No tengo ni idea de que ha pasado !');
}
```

Como puedes ver en el ejemplo puedes ir encadenando bloques dependiendo de los diferentes tipos de excepciones que te interese capturar.

Durante la captura de una excepción puedes utilizar `on`, `catch` o ambas.

Por último, Dart permite ejecutar un fragmento de código se produzca, o no, una excepción en un determinado código.

`finally` te va a permitir ejecutar un fragmento de código se produzca, o no, excepción.

```
var resultado;
try {
  // Intento acceder al sistema.
  resultado = login();
} on Exception catch(e) {
  // Se produce un error en el sistema.
  resultado = 'No has dicho la palabra mágica !';
} finally {
  // Muestro la respuesta al usuario.
  print(resultado);
}
```

En `dart:core` tienes a tu disposición la clase `Exception` y la clase `Error` para gestionar excepciones o errores en el sistema, estas clases además te permiten heredar de ellas para definir tus propias excepciones o errores.

Tipos de Excepciones

Dart presenta tres tipos de excepciones dentro del paquete *exceptions* de *dart:core*.

- **Exception**. La clase abstracta que representa un fallo del sistema y de la cual deberás heredar para crear tus propias excepciones.
- **FormatException**. Define un error de formato cuando una cadena o cualquier otro valor no representa el formato correcto.
- **IntegerDivisionByZeroException**. Representa una excepción de división de un valor por cero.

Tipos de Errores

El paquete `errors` de `dart:core` pone a tu disposición una serie de clases que definen diferentes tipos de errores comunes:

- **AssertionError**, se lanza cuando una sentencia assert falla.
- **TypeError**, se lanza cuando una sentencia de tipo assert falla.
- **CastError**, se lanza cuando se trata de convertir un valor de un tipo a otro y falla.
- **NullThrownError**, se lanza este tipo de error cuando se realiza un throw de null.
- **ArgumentError**, se lanza cuando se produce algún error de argumentos en funciones.
- **RangeError,** se lanza cuando se intenta acceder a un índice que está fuera del rango de índices del objeto.
- **NoSuchMethodError**, se lanza cuando intentas acceder a un método inexistente en un objeto.
- **UnsupportedError,** operación no permitida por un objeto.

- **UnimplementedError**, se lanza cuando el método no está implementado en el objeto.
- **OutOfMemoryError**, sin memoria suficiente.
- **StackOverflowError**, desbordamiento de pila.

Definición de Excepciones y errores

La creación de excepciones y errores es realmente sencilla, solo necesitas crear la clase de excepción o error que desees definir y extender de la clases básicas que Dart pone a tu disposición, veamos unos sencillos ejemplos de definición de excepciones y errores.

Para definir tu propia clase de excepción debes implementar la clase básica *Exception*.

```
class MiExcepcion implements Exception {
  final msg;
  const MiExcepcion([this.msg = ""]);
  toString() => 'Excepción grave en el sistema.$msg';
}
```

Después puedes utilizar tu excepción en tu sistema de este modo:

```
throw new MiExcepcion(); // Exception: Excepción grave en el sistema.
throw new MiExcepcion('Error de Acceso.'); // Exception: Excepción grave
                                  // en el sistema.Error de Acceso.
```

En el caso de definir tus propias clases de error debes extender de la clase base Error.

```
class MiError extends Error {
  final msg;
  MiError([this.msg = "DESCONOCIDO"]);
  toString() => 'Error: $msg';
}
```

Y utilizarlos de este modo:

```
throw new MiError(); // Exception: Error: DESCONOCIDO
throw new MiError('Acceso prohibido.'); // Exception: Error: Acceso prohibido.
```

Ahora, vamos a comenzar a hablar de clases y te explicaremos la diferencia entre `extends` e `implements`.

Clases

Como ya vimos, Dart es un lenguaje orientado a objetos, con clases y herencia simple. En Dart todo es un objeto, incluso los tipos de datos predefinidos son objetos.

Un objeto es la instancia de una clase y para crear un objeto lo hacemos utilizando la palabra reservada `new` más el constructor de la clase.

```
class Persona {
  Persona() {}
}

void main() {
 Persona juan =  new Persona(); // Juan es un objeto, la instancia de Persona.
}
```

Los objetos poseen miembros, funciones y datos. Los métodos de una clase son las funciones y las variables de instancia los datos.

```
class Persona {
  // Variables de instancia.
  String nombre;
  num edad;
```

```
  // Constructor.
  Persona() {}

  // Método que lanza un saludo.
  void saluda() => print('Hola me llamo $nombre');
}

void main() {
 Persona juan =  new Persona(); // Juan es un objeto,
                                // la instancia de Persona.
 juan.nombre = 'Juan Alguien';
 juan.edad = 29;
 juan.saluda(); // Hola me llamo Juan Alguien
}
```

En este ejemplo hemos creado una sencilla clase con unas variables de instancia y un método. Cuando definas el objeto, puedes acceder a sus variables de instancia o a sus métodos utilizando la variable `objeto` seguido de un punto y el método o la variable de instancia que quieras consultar.

Del mismo modo que podemos definir las variables como `const` o `final`, podemos hacerlo con las clases y sus variables de instancia o métodos.

Es posible definir una clase que disponga de un constructor constante y se creará un objeto constante en tiempo de ejecución. Para ello, crearemos el objeto, usando la palabra `const` en lugar de `new` cuando llamemos al constructor.

```
class DaVinci {
  final nombre = 'Leonardo da Vinci';
  const DaVinci(); // Constructor constante.
}
```

```
void main() {
 var leo = const DaVinci();
 print(leo.nombre); // Leonardo da Vinci
}
```

Variables de instancia

Hemos comentado que las variables de instancia son los "datos" del objeto. Estas variables de instancia se definen en la clase y se declaran como hemos visto anteriormente, en la declaración de variables.

Indicando de manera opcional su tipo, el nombre de la variable y asignándole un valor.

> Recuerda, que si no indicas ningún valor, por defecto será *null*
>
> nota

Todas las variables de instancia que se definan de manera implícita, generan un método ***getter*** (para obtener el valor de la variable de instancia), por lo que no es necesario, que lo declares.

Las variables de instancia que se definan con tipo de dato o con `var`, generan de manera implícita un método ***setter*** (para establecer el valor de la variable de instancia).

Hemos visto con el ejemplo anterior, en la clase `Persona` cómo podíamos establecer el valor del nombre o su edad y consultarlos sin necesidad de crear estos métodos ***getter/setter***. Cuando veamos como crear métodos para tus clases también te enseñaremos como crear getters y setters.

```
class Persona {
  // Variables de instancia.
  String nombre; // variable inicializa a null.
  num edad; // variable inicializa a null.
  num hijos = 0; // variable inicializa a 0.
```

```
  // Constructor.
  Persona() {}

  // Método que lanza un saludo.
  void saluda() => print('Hola me llamo $nombre');
}

void main() {
 Persona juan =  new Persona(); // Juan es un objeto, la instancia de Persona.
 juan.nombre = 'Juan Alguien';
 juan.edad = 29;
 print('Nombre: ${juan.nombre}'); // Nombre: Juan Alguien
 print('Edad: ${juan.edad}'); // Edad: 29
}
```

Las variables de instancia pueden ser visibles o no visibles desde otras librerías, es decir, puedes acceder a ellas desde la instancia creada, o no.

Con tan solo comenzar el nombre de la variable con un guión bajo (_) ya estás indicando si esa variable de instancia es, o no privada.

Puedes identificar perfectamente las variables visibles y privadas en la vista *outline* por el icono que se les asigna.

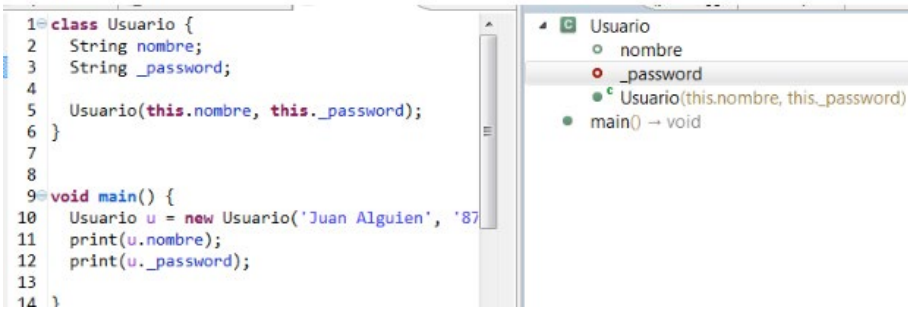


Figura 66. *Outline* de Dart Editor indicando las variables privadas/públicas de la clase.

Constructores

Ya hemos visto algunos aspectos de los constructores de clases, pero vamos a profundizar sobre ellos. El constructor de la clase es un método de la clase que se ejecuta en primer momento cuando se define el objeto y se instancia la clase.

El constructor es un método especial que se crea con el mismo nombre que la clase.

Veamos un ejemplo.

```
class Persona {
  // Variables de instancia.
  String nombre; // variable inicializa a null.
  num edad; // variable inicializa a null.
  num hijos = 0; // variable inicializa a 0.

  // Constructor.
  Persona(String nombre, num edad) {
    this.nombre = nombre;
    this.edad = edad;
  }

  // Método que lanza un saludo.
  void saluda() => print('Hola me llamo $nombre y tengo $edad años.');
}

void main() {
 Persona p =  new Persona('Juan Alguien', 26);
 p.saluda(); // Hola me llamo Juan Alguien y tengo 26 años.
}
```

Dentro del constructor podemos referirnos a las variables de instancia utilizando la palabra reservada `this`. Si has trabajado con Java, C# o JavaScript anteriormente esto te resultará familiar.

Así quedaría el ejemplo anterior, sin utilizar `this`.

La guía de estilo de Dart desaconseja utilizarlo de manera habitual y solo emplearlo cuando tengas un conflicto de nombres

nota

```
class Persona {
  // Variables de instancia.
  String nombre; // variable inicializa a null.
  num edad; // variable inicializa a null.
  num hijos = 0; // variable inicializa a 0.

  // Constructor.
  Persona(String n, num e) {
    nombre = n;
    edad = e;
  }

  // Método que lanza un saludo.
  void saluda() => print('Hola me llamo $nombre y tengo $edad años.');
}
```

Del mismo modo que si defines variables de instancia con `tipo` o con `var`, si no especificas un constructor para la clase, Dart creará de manera implícita uno sin argumentos y será el primer método que se ejecutará al crear el objeto.

Dentro de los constructores debes prestar atención a los constructores con nombre (*named constructors*) que permiten definir múltiples constructores para una clase.

Ya hemos visto esto cuando aprendimos la clase `DateTime`, y recuerda que teníamos diferentes formas de definir un objeto `DateTime`.

Los constructores con nombre se pueden utilizar por diferentes motivos, porque existen diferentes formas de inicializar las variables de instancia o por clarificar las funciones de la clase.

```
class Persona {
  // Variables de instancia.
  String nombre; // variable inicializa a null.
  num edad; // variable inicializa a null.
  num hijos = 0; // variable inicializa a 0.

  // Constructor.
  Persona(String n, num e) {
    nombre = n;
    edad = e;
  }

  // Constructor con nombre.
   Persona.json(Map datos) {
    nombre = datos['nombre'];
    edad = datos['edad'];
    hijos = datos['hijos'];
  }

  // Método que lanza un saludo.
  void saluda() => print('Hola me llamo $nombre y tengo $edad años.');
}

void main() {
 Persona p =  new Persona('Juan Alguien', 26);
 p.saluda(); // Hola me llamo Juan Alguien y tengo 26 años.

 var p2 = new Persona.json({'nombre': 'Manuel', 'edad': 45, 'hijos': 2});
 p2.saluda(); // Hola me llamo Manuel y tengo 45 años.
}
```

Los constructores se pueden re-dirigir. Esto es útil:

- Cuando hay herencia y desde el constructor de la clase hija re-dirigimos al constructor de la clase padre.
- Dentro de la propia clase, cuando con esto se clarifica la utilidad.

```
class Persona {
  // Variables de instancia.
  String nombre; // variable inicializa a null.
  num edad; // variable inicializa a null.
  num hijos = 0; // variable inicializa a 0.

  // Constructor.
  Persona(String n, num e) {
    nombre = n;
    edad = e;
  }

  Persona.sinHijos(String n, num e):this(n, e);

  // Método que lanza un saludo.
  void saluda() => print('Hola me llamo $nombre y tengo $edad años.');
}

void main() {
var p =  new Persona('Raquel', 19);
p.saluda(); // Hola me llamo Raquel y tengo 19 años.
}
```

Como comentamos anteriormente, también existen los constructores constantes. Los debes utilizar si sabes que un objeto no cambiará a lo largo de la ejecución.

Para ello, creamos la clase con su constructor constante, utilizando `const` y debemos asegurarnos que todas las variables de instancia son, al mismo tiempo, constantes con `final` o `const`.

```
class Origen {
  final num x;
  final num y;
  const Origen(this.x, this.y);
}
```

Por último, vamos a hablar de los *Factory constructors*. Cuando los utilicemos al instanciar una clase, no se creará una nueva instancia, sino que se recupera desde una caché, así puedes ahorrar mucha memoria en tus aplicaciones.

Los *Factory constructors* se suelen utilizar cuando nos interesa separar la creación de un objeto de su implementación. Son muy útiles cuando necesitamos elegir entre un cierto número de clases intercambiables en tiempo de ejecución.

En otros lenguajes ya se utiliza el *Factory pattern*, pero debes utilizar una combinación de métodos estáticos, utilidades de la clase, etc.

En Dart ya se ha incluido este patrón de manera nativa y lo tienes a tu disposición. Veamos un ejemplo.

```
class Jefe {

  // nombre del jefe.
  String name;

  // cache donde almacenar la instancia creada.
  static final Map<String, Jefe> _cache = <String, Jefe>{};

  Jefe._internal(this.name);

  factory Jefe(String name) {
    if(_cache.containsKey(name)) {
      return _cache[name];
```

```
    } else {
      final jefe = new Jefe._internal(name);
      _cache[name] = jefe;
      return jefe;
    }
  }

  void saluda() => print('Hola soy $name, el Boss!!');
}

void main() {
 var j1 =  new Jefe('Bruce Springsteen');
 var j2 =  new Jefe('Bruce Springsteen');

 j1.saluda(); // Hola soy Bruce Springsteen, el Boss!!
 j2.saluda(); // Hola soy Bruce Springsteen, el Boss!!
 print(j1==j2); // true.
}
```

Métodos

A lo largo de los ejemplos anteriores, ya hemos visto algún método. Los métodos son lo que dota de funcionalidad a tus objetos.

Un método es una función definida dentro de una clase y es el que se ocupa de que el objeto tenga funciones que lo hagan útil.

```
class Persona {
  // Variables de instancia.
  String nombre; // variable inicializa a null.
  num edad; // variable inicializa a null.
  num hijos = 0; // variable inicializa a 0.
```

```
  // Constructor.
  Persona(String n, num e) {
    nombre = n;
    edad = e;
  }

  void saluda() => print('Hola me llamo $nombre y tengo $edad años.');
  void trabaja() => print('Trabajando...');
  void come() => print('Ummm delicioso !');
  void duerme() => print('Buenas noches.. ZZZZZZZZ');
}

void main() {
  var p1 = new Persona('Raquel', 19);
  p1.saluda(); // Hola me llamo Raquel y tengo 19 años.
  p1.come(); // Ummm delicioso !
  p1.trabaja(); // Trabajando...
  p1.come(); // Ummm delicioso !
  p1.duerme(); // Buenas noches.. ZZZZZZZZ
}
```

Además de los métodos de instancia que pueden tener tus objetos, también tenemos los **getters/setters** que, como hemos comentado anteriormente, Dart se ocupa de crear por nosotros.

Si necesitases crear un *getter/setter* para alguna de tus clases lo puedes hacer así.

```
class Persona {
  // Variables de instancia.
  String nombre; // variable inicializa a null.
  num edad; // variable inicializa a null.
  num hijos = 0; // variable inicializa a 0.
```

```
  // Constructor.
  Persona(String n, num e) {
    nombre = n;
    edad = e;
  }

   bool get isPadre => (hijos > 0);
        set padre(bool p) => (p==true)?hijos=1:hijos=0;
}

void main() {
  var p1 = new Persona('Raquel', 19);
  print(p1.isPadre); // false

  p1.padre = true;
  print(p1.isPadre); // true
  print(p1.hijos); // 1
}
```

Al definir una clase, es posible sobre-escribir los operadores, por ejemplo: +, –, etc.

Imagina que tienes una clase que define una posición GPS, en la que te podría interesar definir el operador + y – si quieres sumar o restar dos posiciones GPS y obtener una tercera. Veamos un ejemplo.

```
class GPS {

  num latitud;
  num longitud;

  GPS(this.latitud, this.longitud);
```

```
  GPS operator +(GPS g) {
    return new GPS(latitud + g.latitud, longitud + g.longitud);
  }

  GPS operator -(GPS g) {
    return new GPS(latitud - g.latitud, longitud - g.longitud);
  }
}

void main() {
  var g1 = new GPS(40.311235, 32.87578786);
  var g2 = new GPS(38.642331, 41.79786401);

  var g3 = g1 + g2;
}
```

Los operadores que puedes sobre-escribir son:

Operadores			
<	+	|	[]
>	/	^	[]=
<=	~/	&	~
>=	*	<<	==
-	%	>>	

Herencia

La herencia te permite crear nuevas clases a partir de otras existentes, añadiendo nuevo comportamiento, datos o funciones sobre lo que heredan de las clases padre y evitando el re-diseño, modificación y verificación de la parte ya implementada.

En Dart puedes crear subclases utilizando la palabra `extends` y puedes referirte a la clase padre con la palabra `super`.

```
class Animal {
  String especie;
  String variedad;

  Animal.datos(this.especie, this.variedad);

  void respira() {
    print('Configurando sistema respiratorio...');
    print('Sistema respiratorio configurado !');
  }

  void come() {
    print('Configurando sistema digestivo...');
    print('Sistema digestivo configurado !');
  }
}

class Perro extends Animal {
  String color;
  num patas;

 Perro(String variedad, String color, num patas) : super.datos('mamifero',
 variedad) {
    this.color = color;
    this.patas = patas;
  }

  void respira() {
    super.respira();
    print('respiro por la nariz');
  }
}
```

```
void main() {
  var p = new Perro('Labrador Retriever', 'Negro', 4);
  p.respira(); // Configurando sistema respiratorio...
               // Sistema respiratorio configurado !
               // respiro por la nariz
  p.come(); // Configurando sistema digestivo...
            // Sistema digestivo configurado !
}
```

En la clase hija puedes crear nuevas variables de instancia o nuevos métodos y también puedes re-escribir los métodos del padre. Siempre que necesites referirte un método o variable de instancia de la clase padre, utiliza la palabra `super`.

Anteriormente vimos los constructores y desde el punto de vista de la herencia, debes saber que los constructores no se heredan (tampoco los constructores con nombre).

Una subclase no heredará los constructores de la clase superior. Si no se define constructor para la subclase, dispondrá del constructor implícito que crea Dart sin parámetros.

```
class Animal {
  String nombre;
  String tipo;
  String raza;
  num edad;
  num patas;

  Animal() {}

  Animal.json(Map datos) {
    nombre = datos['nombre'];
    tipo = datos['tipo'];
    raza = datos['raza'];
    edad = datos['edad'];
    patas = datos['patas'];
  }
}
```

```
class Perro extends Animal {
  Perro.json(Map datos) : super.json(datos);
}
```

Fíjate como para ejecutar un constructor de la clase padre en la clase hija, debes hacerlo de manera explícita. Deberás definir el constructor en la clase hija y dentro del constructor, utilizando la palabra `super` al llamar al constructor padre.

Variables y métodos estáticos

En Dart tenemos la posibilidad de crear variables y métodos estáticos al definir las clases. Los métodos y variables estáticos no requieren de la creación de un objeto para poder utilizarlos y son de ámbito local a la clase donde se definan, pero su existencia es permanente.

Para crear métodos o variables estáticas tan sencillo como utilizar la palabra `static`.

```
class Coche {
  static num velocidad_maxima = 120;
}

void main() {
  print(Coche.velocidad_maxima); // 120
}
```

Los métodos estáticos o métodos de clase no operan sobre las instancias y, por lo tanto, no pueden utilizar la palabra `this`.

```
class _CryptoUtils {
  static String bytesToHex(List<int> bytes) {
    var result = new StringBuffer();
    for (var part in bytes) {
      result.write('${part < 16 ? '0' : ''}${part.toRadixString(16)}');
    }
```

```
    return result.toString();
  }
}

void main() {
  print(_CryptoUtils.bytesToHex([6,4,7,8,1,0])); // 060407080100
}
```

La guía de estilo Dart recomienda utilizar funciones de ámbito global en lugar de utilizar métodos estáticos para las tareas más comunes o utilidades de uso habitual en tus aplicaciones.

Interfaces implícitas y clases abstractas

Cuando definimos una clase, implícitamente se define una interfaz que contiene todos los miembros de instancia, datos y funciones de la clase y de las interfaces que implementa.

En pocas palabras, si queremos escribir una clase A que soporte el API de una clase B sin heredar la implementación de B, la clase A deberá implementar la interfaz de B.

Para poder hacer esto, cuando definas la clase puedes añadir la palabra `implements` seguida de las clases que implementa y definir la implementación de las interfaces.

```
// En la interfaz implícita de la clase Animal está el método habla()
class Animal {
  String _nombre;       // En la interfaz, pero visible solo en la clase Animal.
  Animal(this._nombre); // El constructor no está en la interfaz.
  void habla() => print('Hola soy un Animal y me llamo $_nombre');
}
```

```
// Implementación de la interfaz Animal.
class Perro implements Animal {
String _nombre; // Se define porque solo es visible en la clase Animal.

void habla() => print('Hola soy un Perro y no tengo nombre');
}

// Función que recibe un objeto y ejecuta su método habla().
saluda(Animal animal) => animal.habla();

void main() {
  saluda(new Animal('Benchara')); // Hola soy un Animal y me llamo Benchara
  saluda(new Perro()); // Hola soy un Perro y no tengo nombre
}
```

En este sencillo ejemplo implementamos en la clase *Perro* la interfaz de la clase Animal. Debemos implementar todo lo que esté en la interfaz implícita de la clase *Animal*, métodos y variables de instancia.

Se ha definido el método `habla()` y la variable de instancia `_nombre`, aunque luego no se utilice.

Si echas un vistazo al SDK de Dart, puedes ver como esta mecánica es muy común. Veamos un ejemplo de implementación de interfaces, en este caso múltiples, disponible en *dart:html*.

```
9662 /**
9663  * An abstract class, which all HTML elements extend.
9664  */
9665 @DomName('Element')
9666 abstract class Element extends Node implements ParentNode, ChildNode {
9667
9668 /**
9669  * Creates an HTML element from a valid fragment of HTML.
9670  *
9671  *     var element = new Element.html('<div class="foo">content</div>');
9672  *
9673  * The HTML fragment should contain only one single root element, any
9674  * leading or trailing text nodes will be removed.
9675  *
9676  * The HTML fragment is parsed as if it occurred within the context of a
9677  * `<body>` tag, this means that special elements such as `<caption>` whi
9678  * must be parsed within the scope of a `<table>` element will be dropped.
9679  * [createFragment] to parse contextual HTML fragments.
9680  *
9681  * Unless a validator is provided this will perform the default validatio
9682  * and remove all scriptable elements and attributes.
9683  *
9684  * See also:
9685  *
9686  * * [NodeValidator]
9687  *
9688  */
9689 factory Element.html(String html,
9690     {NodeValidator validator, NodeTreeSanitizer treeSanitizer}) {
9691   var fragment = document.body.createFragment(html, validator: validator,
9692       treeSanitizer: treeSanitizer);
9693
9694   return fragment.nodes.where((e) => e is Element).single;
9695 }
9696
```

Figura 67. *La clase Element hereda de Node e implementa las interfaces de ParentNode y ChildNode.*

Si hablamos de interfaces, debes conocer lo que son las clases abstractas. Una clase abstracta se define igual que una clase normal, pero utilizando la palabra `abstract` antes de la definición de la clase.

¿Qué significa clase abstracta? Hay ocasiones, cuando se desarrolla una jerarquía de clases en que algún comportamiento está presente en todas ellas pero se materializa de forma distinta para cada una, esto se desarrolla utilizando clases abstractas.

nota

Las clases abstractas definen un comportamiento común a todas las clase que heredarán o implementarán esta clase. Las clases abstractas son útiles para definir interfaces.

Además de las clases, también los métodos pueden ser abstractos, y la filosofía es la misma. Un método abstracto será una función común presente en todas las clases, pero que funcionará de manera diferente en cada una.

Definimos la clase abstracta y los métodos abstractos, generalmente sin implementación y las clases que implementen esta interfaz definirán el código.

```
// Define interfaz FiguraGeometrica. Nunca se instanciará.
abstract class FiguraGeometrica {
  // Define constructores, métodos y variables de instancia.
  void dibujar(); // Metodo abstracto.
}


class Circulo extends FiguraGeometrica {
  void dibujar() {
    // Codigo para dibujar cirulo.
  }
}
```

Generics y Typedefs

Como hemos comentado en varias ocasiones, Dart es un lenguaje opcionalmente tipado, es decir, no tienes por qué indicar los tipos de datos de las variables de tus aplicaciones.

No obstante el tipado de datos puede ser muy útil para auto-documentar tú código y facilitarte el mantenerlo.

Si indicamos tipos de datos en las aplicaciones, sucede algo alucinante, las herramientas comienzan a trabajar por tí y te avisan mientras estás trabajando. Indican errores de tipado entre los parámetros de una función o el resultado de una función y la asignación a una variable.

En definitiva, el tipado de datos en un lenguaje como Dart, que es opcional es como indicar una anotación, *"Ey, recuerdame que esto es un número !"*

Los **Generics** son anotaciones para las colecciones de datos. Ya hemos visto como indicar el tipo de dato que contendrá un `Map` o un `List`, pues eso son los *Generics*.

Utilizando los corchetes angulares seguidos del tipo de dato (**<..>**) le indicamos a Dart que lo que contendrá nuestra lista o nuestro mapa será de ese tipo.

```
List<int> l_num_1 = [1, 2, 3, 4, 5];

var l_num_2 = <int>[1, 2, 3, 4, 5];
```

Los *Generics* también te ayudan a reducir código. Imagina que creas una clase que manipula una estructura de datos. Decides definir el tipo de datos de esta estructura para valerte de la ayuda del chequeo de datos durante el desarrollo.

Decides establecer tipos y utilizas `String`. Más adelante, te das cuenta que esta clase que has escrito es fantástica y que la puedes utilizar también para manipular otra estructura de datos con `num`, te tocaría duplicar la clase, ¿correcto?.

Si utilizamos *Generics* no es necesario duplicar esta clase que has creado. Podemos utilizar un **generic** como parámetro a nuestra clase. Podrás ver muchos ejemplos de ello en el SDK de Dart.

```
abstract class Cache<T> {
  T getByKey(String key);
  setByKey(String key, T value);

  factory Cache() {}
}

void main() {
  var cadenas = new Cache<String>();
  var numeros = new Cache<num>();
}
```

Indicando este *Generic* como parámetro ahorramos duplicar código y seguimos obteniendo los beneficios del tipado de datos.

Typedefs

Los tipos de datos son útiles y puedes valerte de ellos para depurar tus aplicaciones incluso antes de que se ejecuten, en el mismo momento de su diseño y programación. Algo increíblemente poderoso.

Vamos a ver ahora un ejemplo de **callback** que muchos programadores de JavaScript están muy acostumbrados a utilizar.

Un **callback** es una función que se utiliza como parámetro de otra función y en muchos casos se asigna a una variable y es esta variable la que se pasa como parámetro. Veamos un ejemplo sencillo en JavaScript.

```
var out = console.log;
function login(usuario, password, out) {
  if (usuario == '' or password == '') {
   out('Error de acceso al sistema. Indique usuario y contraseña');
  }
}
```

Veamos ahora un caso similar en Dart y lo que sucede si no utilizamos `typedef`.

```
String formatLog(String msg, {int pos: 2}) {
  String cad = '';
  for(var i=0; i<pos; i++) {
    cad += '-';
  }
  return '${cad}${msg}';
}

void main() {
  var log = formatLog;
  var cad = 2 + log('Mensaje');
}
```

Si observamos el ejemplo anterior, vemos que existe un error claro. Dart Editor debería avisarte que no puedes concatenar un `num` con un `String` (el valor devuelto por la función formatLog).

¿Por qué Dart Editor no me avisa de esto? Porque al asignar una función a una variable Dart Editor pierde el tipado de la función original. Y ¿cómo podemos evitar esto?

La respuesta es sencilla, utilizando `typedef`.

Es un alias de tipado para indicar que esa función, aunque se asigne a una variable o se pase como parámetro debe devolver un resultado de tipo String.

¿ Qué es Typedef ?

```
String formatLog(String msg, {int pos: 2}) {
  String cad = '';
  for(var i=0; i<pos; i++) {
    cad += '-';
  }
  return '${cad}${msg}';
}

typedef String LoggerOutputFunction(String msg);

void main() {
  LoggerOutputFunction log = formatLog;
  var cad = 2 + log('Mensaje');
}
```

Gracias a `typedef` puedes conseguir alertas por parte de Dart Editor en el momento de diseñar y programar tus aplicaciones.

Ahora Dart Editor te da avisa indicándote que no podrás concatenar un `String` con un `num`.

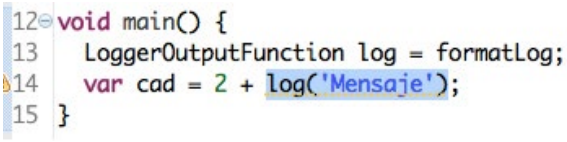


Figura 67. Dart Editor muestra la alerta de tipado gracias al nuevo *typedef*.

Librerías

Ya hemos visto cómo definir variables, funciones, utilizar las sentencias de control e iteración y crear clases en Dart. Hemos visto una gran parte de lo que el lenguaje nos ofrece.

Tratándose de un lenguaje que pretende convertirse en un estándar para el desarrollo de aplicaciones web de nueva generación, debe presentarnos un mecanismo para organizar nuestro código, sobre todo si pensamos en grandes proyectos.

En Dart, generalmente cuando escribas una aplicación tendrás múltiples clases, algunas heredarán de otras, tendrás en ficheros separados diferentes partes del sistema, etc. Dart pone a tu disposición el concepto de librería.

Actualmente en JavaScript tenemos esta gran limitación, tenemos que trabajar muy duro para poder organizar nuestro proyecto en ficheros de código.

Las librerías, permiten crear código modular, que pueda compartirse y reutilizarse fácilmente. Una librería no es más que un paquete que creamos a partir de interfaces, clases y funciones globales que empaquetamos para reutilizar en otro proyecto bajo la palabra reservada `library` seguido de un nombre.

Este es un ejemplo sencillo de creación de librería, en el que hemos omitido la implementación de las clases y de las funciones globales.

```
library animales;

num NUMERO_MAXIMO = 12;

void crearAnimal() {}

class Animal {}

class Perro extends Animal {}

class Gato extends Animal {}
```

¿Como usar las librerías?

Ahora que sabemos lo que son las librerías y como se crean, vamos a ver como trabajar con ellas.

Una vez definida una librería que engloba una serie de interfaces, clases, funciones, constantes, etc. Puedes utilizarla en cualquier parte de tu sistema simplemente importando la librería con la palabra reservada `import` seguido de la ruta donde se encuentra almacenada.

Un ejemplo de esto, ya lo vimos cuando hablamos de la librería *dart:math* cuando hablábamos de números y te contamos cómo puedes trabajar con números aleatorios por ejemplo.

```
import 'dart:math';
```

La única curiosidad es que las librerías nativas que incorpora Dart se importan utilizando el nombre `dart:` seguido del nombre de la librería.

Lo mismo sucede cuando quieres trabajar con algún paquete descargado con *pub*.

```
import 'package:lawndart/lawndart.dart';
```

Utilizas el nombre especial `package:` seguido del nombre o ruta de la librería que deseas utilizar.

Cuando trabajes en tu sistema, lo normal es que utilices bastantes librerías y algunas de ellas pueden entrar en conflicto con otras porque tienen nombres similares. Esto puedes solucionarlo de manera sencilla cuando realices la importación indicando un prefijo para una de ellas.

Imagina que trabajas en una aplicación científica donde, además de utilizar dart:math dispones de otra librería con funciones matemáticas y que, casualmente también se llama math.

Podrías resolver el conflicto de nombres de este modo.

```
import 'dart:math' as Math;
import 'math.dart' as MathLib;
```

Luego podrás referirte a cualquiera de las funciones, clases o constantes de cada una de las librerías utilizando el prefijo que has definido en el momento de la importación.

```
Math.PI;
MathLib.ALPHA;
```

En otras ocasiones puedes querer importar solo una parte de una librería. Para poder realizar esto debes utilizar la palabra reservada `show` que indica a Dart qué parte debe mostrar de esa librería.

```
import 'dart:convert' show JSON;

void main() {
  print(JSON.encode({'msg': 'Hola'})); // String: {"msg":"Hola"}
  print(JSON.decode('{"msg":"Hola"}')); // Map: {msg: Hola}
}
```

Crear librerías con múltiples ficheros

Al comienzo de este apartado veíamos como crear de manera muy simple una librería. Habíamos creado un único fichero llamado *animales.dart* con la directiva `library` seguida de su nombre.

Dentro de este fichero teníamos definidas todas las clases que componen nuestra librería.

En Dart también podemos definir librerías con múltiples ficheros. En el fichero principal de la librería debemos indicar qué ficheros componen la librería con la directiva `part` seguido del nombre del fichero.

En cada fichero también debemos indicar la directiva `part of` y su nombre, para indicar que ese fichero forma parte de esa librería.

Veamos un ejemplo sencillo siguiendo con la definición de nuestra librería de animales.

Creamos un fichero base que será nuestra librería `animales.dart` con este contenido.

```
library animales;

part 'perros.dart';
part 'aves.dart';

class Animal {}
```

Ahora vamos a crear los ficheros `perros.dart` y `aves.dart` con las clases que definene nuestros perros y nuestras aves y que forman parte de nuestra librería de animales.

Fichero `perros.dart`.

```
part of animales;

class LabradorRetriever {}
class Beagle {}
```

Fichero `aves.dart`

```
part of animales;

class Canario {}
class Jilguero {}
```

Ahora desde cualquier parte de tu sistema o desde otra aplicación puedes utilizar esta librería con esta sentencia.

```
import 'animales.dart';

void main() {

  var obj = new Animal();
  var p1 = new LabradorRetriever();
  var a1 = new Jilguero();
}
```

Re-exportando librerías

Se puede dar un caso especial en el que crees una nueva librería que incorpore funciones de otra y quieras poder utilizarla en otra aplicación. O crear un pequeña librería a partir de un subconjunto de funciones de otra.

Para poder hacer esta clase de cosas, Dart incorpora los mecanismos de exportación que vamos a ver a continuación con un sencillo ejemplo.

A partir de nuestra librería animales vamos a crear una nueva cogiendo lo que más no interese de ella, lo vamos a empaquetar y vamos a crear una nueva librería.

Vamos a crear un nuevo fichero llamado `mamiferos _ caninos.dart` con este contenido.

```
library mamiferos _ caninos.dart;

import 'animales.dart';
export 'animales.dart' show Animal, LabradorRetriever, Beagle;

class MamiferoCanino extends Animal {}
```

Hemos creado una nueva librería partiendo de Animales y re-exportando la clase base Animal y las dos clases de Perros que queremos.

Ahora podremos utilizar esta nueva librería en cualquier aplicación de este modo tan sencillo.

```
import 'mamiferos_caninos.dart';

void main() {
  var obj = new Animal();
  var p1 = new LabradorRetriever();
  var p2 = new Beagle();
}
```

En esta aplicación no podremos definir objetos de tipo Jilguero ni de tipo Canario ya que no están visibles en nuestra librería `mamiferos_caninos`.

Multi-proceso y concurrencia en Dart: Isolates

Dart también nos permite trabajar concurrentemente y utilizar todas las capacidades del multi-procesamiento gracias a los multi-core de los que disponemos en nuestros equipos.

Conocemos esto como hilos (**thread**) en otros lenguajes. La diferencia entre **threadings** en otros lenguaje y los **isolates** de Dart es que estos últimos no comparten memoria, cada uno es independiente del otro y se comunican entre ellos mediante paso de mensajes.

El que los hilos compartan memoria es algo muy propenso a errores y que complica mucho el código.

Cada `isolate` de Dart tiene su propia memoria y corre de manera *aislada* del resto, de ahí su nombre.

Los isolates son posibles en Dart y en su traducción a JavaScript precisamente porque HTML5 dispone de los **Web Workers**. Los web workers son tareas JavaScript que se ejecutan en background evitando que la página web se quede bloqueada hasta que el proceso termine.

Cuando Dart compila a JavaScript utiliza Web Workers para su funcionamiento, aunque lo ideal, y esperamos que en un futuro así sea, utilice `Futures` y `Streams`.

Veamos un simple ejemplo de `isolates` en Dart.

```
import 'dart:async';
import 'dart:isolate';

void procesoCostoso(SendPort responderA) {
  var port = new ReceivePort();
  responderA.send(port.sendPort);
  port.listen((msg) {
    var data = msg[0];
    responderA = msg[1];
    if (data == "COMIENZO") {
      responderA.send("Ejecutando proceso costoso....");
      responderA.send("TERMINADO");
      port.close();
    }
  });
}

main() {

  print('Comienzo ejecución aplicación');

  // Puerto de respuesta para el isolate.
  var respuesta = new ReceivePort();

  Future<Isolate> iso = Isolate.spawn(procesoCostoso, respuesta.sen-
   dPort);
  iso.then((_) => respuesta.first).then((port) {
```

```
    // Una vez creado el Isolate envio el mensaje de comienzo de
    // proceso y me quedo esperando a recibir respuestas del gran
    // proceso a ejecutar.
    respuesta = new ReceivePort();
    port.send(["COMIENZO", respuesta.sendPort]);
    respuesta.listen((msg) {
      print('Mensaje recibido desde isolate: $msg');
      if (msg == "TERMINADO") {
        print('Proceso costoso terminado con éxito !!!');
      }
    });

  });

  // Podemos seguir ejecutando más código mientras esperamos la
  // respuesta de la ejecución de nuestro proceso en background.
  print('Sigo ejecutando más instrucciones');
}
```

Si ejecutas el proceso anterior obtendrás este resultado.

```
Comienzo ejecución aplicación
Sigo ejecutando más instrucciones
Mensaje recibido desde isolate: Ejecutando proceso costoso....
Mensaje recibido desde isolate: TERMINADO
Proceso costoso terminado con éxito !!!
```

En este caso te hemos mostrado cómo ejecutar un proceso en paralelo disponible en el mismo fichero de código donde tenemos la función `main()` de la aplicación.

Dart también permite ejecutar concurrentemente mediante `isolates` un proceso que esté en otro fichero de código. Veamos a un ejemplo sencillo partiendo de una estructura de ficheros como la siguiente.

El fichero **test.html** tendrá este contenido. Es el ejemplo básico que carga Dart Editor cuando le pedimos que nos cree una aplicación de ejemplo con código. <!DOCTYPE html>

```
<html>
  <head>
    <meta charset="utf-8">
    <title>Test</title>
    <link rel="stylesheet" href="test.css">
  </head>
  <body>
    <h1>Test</h1>

    <p>Hello world from Dart!</p>

    <div id="sample _ container _ id">
      <p id="sample _ text _ id"></p>
    </div>

    <script type="application/dart" src="test.dart"></script>
    <script src="packages/browser/dart.js"></script>
  </body>
</html>
```

Luego tendremos el fichero **test.dart** con este código.

```
import 'dart:isolate';
import 'dart:html';
import 'dart:async';

main() {
  Element resultado = querySelector('#sample _ text _ id');

  // Defino puertos de envio/respuesta.
  SendPort sendPort;
  ReceivePort receivePort = new ReceivePort();
  receivePort.listen((msg) {
    if (sendPort == null) {
      sendPort = msg;

    } else {
```

```
    switch(msg) {
      case "DESDE PROCESO: COMIENZO":
        resultado.innerHtml += 'Ejecutando proceso costoso....<br/><br/>';
        break;

      case "DESDE PROCESO: FINALIZADO":
        resultado.innerHtml += 'Finalizado con éxito el proceso costoso
        <br/><br/>';
        break;

      default:
       resultado.innerHtml += 'Recibido desde el isolate: $msg<br/><br/>';
    }
  }
});

  // Este es el proceso costoso que queremos ejecutar.
  String procesoCostoso = 'proceso _ costoso.dart';
  int contador = 0;
  var timer;
  Isolate.spawnUri(Uri.parse(procesoCostoso), [], receivePort.sendPort).
  then((isolate) {
    print('Isolate lanzado !!');
       timer = new Timer.periodic(const Duration(seconds: 1), (t) {

      switch (contador) {
        case 0:
          sendPort.send("COMIENZO");
          break;

        case 5:
          sendPort.send("FINALIZADO");
        timer.cancel();
        break;

      default:
        // Cada segundo le enviamos un mensaje al proceso costoso.
        sendPort.send('Desde la aplicación: ${contador}');
    }
    contador += 1;
  });
});
```

```
  // Podemos seguir ejecutando más código mientras esperamos la
  // respuesta de la ejecución de nuestro proceso en background.
  print('Sigo ejecutando más instrucciones');
  print('.....');
  print('Finalizado el programa completo');
}
```

Por último, tenemos otro fichero llamado **proceso_costoso.dart** con el código de nuestro proceso costoso y que será el que llamemos desde el nuevo `isolate` creado en **test.dart**.

```
import 'dart:isolate';

main(List<String> args, SendPort sendPort) {
  ReceivePort receivePort = new ReceivePort();
  sendPort.send(receivePort.sendPort);
  receivePort.listen((msg) {
    sendPort.send('DESDE PROCESO: $msg');
  });
}
```

Si ejecutas este código en la vista *Output* de Dart Editor verás el texto de las sentencias `print` que hemos puesto en el código.

```
Sigo ejecutando más instrucciones
.....
Finalizado el programa completo
Isolate lanzado !!
```

Y además en la ventana de Dartium verás este resultado ejecutado segundo a segundo.

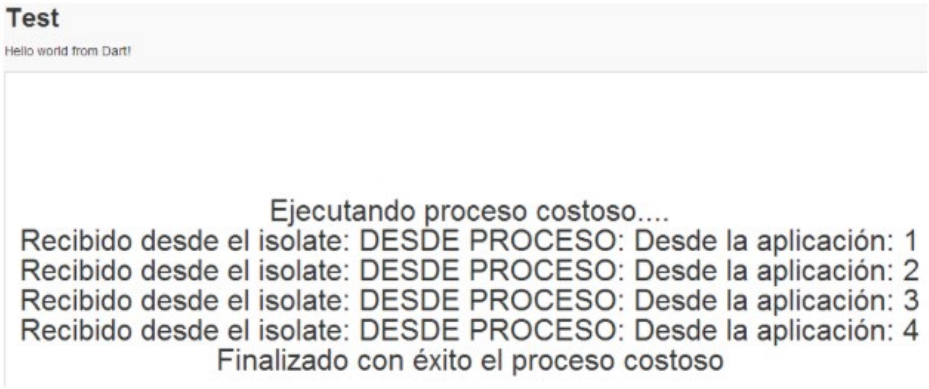


Figura 68. El resultado de la comunicación entre un fichero y otro.

Programación Asíncrona

Ya te habrás dado cuenta del gran potencial de Dart, no sólo por las grandes herramientas que nos facilita sino por cómo unifica toda la amalgama de herramientas, lenguajes y navegadores que tenemos a nuestra disposición para el desarrollo web.

Además nos ofrece APIs que ya podemos utilizar y que aún no están disponibles en JavaScript, por ejemplo, hasta que se publique la nueva versión del lenguaje.

Gran parte de las funciones y APIs que incluirá la nueva versión 6 de JavaScript (esperada para finales de 2014) ya están disponibles en Dart, como por ejemplo:

- Variables de ámbito de bloque.
- Valores por defecto en funciones.
- Parámetros opcionales por nombre.
- Interpolación de Strings.
- Arrow functions.
- Promises API.

¿Qué es la programación asíncrona?

La programación asíncrona, tal como la presenta Dart y como lo hará JavaScript 6 con su API `Promises`, es una forma de poder trabajar, ejecutar y crear procedimientos asíncronos de una manera natural, sencilla y estructurada.

Dart pone a tu disposición el API `dart:async` con las clases `Future` y `Stream` para poder realizar programación asíncrona. En realidad, es básico que entiendas qué es y cómo funciona `Future` y `Stream` antes de empezar a trabajar a fondo con Dart ya que ésta es una forma habitual de trabajo en el SDK.

Una vez lo veas en funcionamiento y lo utilices te darás cuenta de su potencial, ya que propone una manera muy estructurada para trabajar de manera asíncrona y evitar el infierno de los *callbacks* que se utilizan en JavaScript.

Future

Un objeto `Future` representa un proceso, función o valor de retorno que se recuperará en un futuro en modo diferido. Ejecutarémos una función, por ejemplo, y el valor de retorno de la función no lo obtendremos de manera inmediata sino en otro momento.

Piensa un momento en como funciona una página web con enlaces a otros documentos. El navegador renderiza el documento y se lo muestra al usuario. El usuario puede comenzar a leer un texto durante un tiempo y luego hacer clic en un enlace.

Si quieres saber cuándo el usuario hace clic en uno de esos enlaces, debes capturar el evento clic del usuario. El evento es el resultado de quedarte escuchando sobre ese enlace (`Future`) hasta que el usuario lo pulsa.

Cuando el usuario haga clic en el enlace obtendrás un resultado en un tiempo futuro, que pueden ser 5 segundos o 45 minutos desde la carga de la web, depende de lo que el usuario esté haciendo.

Salvando las diferencias así es como funciona un objeto de tipo `Future` en Dart. Lanzas un proceso que realiza una serie de acciones y te quedas esperando a obtener el resultados.

Lo mejor de todo es, que cuando te quedas esperando a obtener el resultado no te bloqueas, tu aplicación continua su curso y sigue ejecutando más acciones.

Con la clase `Future` podrás manejar estos objetos.

Trabajando con Future

Veamos algunos ejemplos de `Future` y cómo podemos trabajar con ellos.

Hemos comentado que en el SDK de Dart podrás encontrar muchas clases que retornan `Future` como resultado de su ejecución en `dart:html`, por ejemplo:

```
import 'dart:html';
main() {
  print('1. Lanzo una petición de manera asíncrona');
  var resultado = HttpRequest.getString('respuesta.json');
  resultado
   .then((future _ resultado) {
     print('2. Ya se ha completado el future y obtengo un resultado.');
     print(future _ resultado);
   }).catchError(( _ ) {
     print('3. Error ejecutando la petición al servidor.');
   });
  print('4. Continúo ejecutando más acciones.');
}
```

Para poder ejecutar este ejemplo deberás crear un fichero llamado `respuesta.json` con este contenido.

```
{'mensaje': 'Bienvenido a la programacion asincrona en Dart'}
```

Si ejecutas el bloque de código obtendrás este resultado.

```
1. Lanzo una petición de manera asíncrona
4. Continúo ejecutando más acciones.
2. Ya se ha completado el future y obtengo un resultado.

{'mensaje': 'Bienvenido a la programacion asincrona en Dart'}
```

Como puedes observar, independientemente del mensaje que mostramos, nuestra pequeña aplicación no se queda esperando la respuesta de `HttpRequest.getString`.

Continúa ejecutando más acciones y cuando el servidor responde con un valor de retorno correcto, o un error, se ejecutan los métodos `.then` o `.catchError` del objeto `Future` resultado.

La clase `Future` pone a tu disposición ciertos métodos interesantes para trabajar con este tipo de objetos que resultará imprescindible que domines.

Dispones del método estático `wait` para lanzar múltiples funciones asíncronas y esperar al resultado de todas ellas.

```
var futures = [HttpRequest.getString('respuesta1.json'),
               HttpRequest.getString('respuesta2.json'),
               HttpRequest.getString('respuesta3.json')];

Future.wait(futures).then((r) {
  r.forEach((e) => print(e));
});
```

El método ya visto `.then` te permite obtener el resultado devuelto cuando el objeto `Future` se ha completado devolviendo un valor.

```
var fut = HttpRequest.getString('http://www.google.es');
fut.then((respuesta) => manejaRespuesta(respuesta));
```

También hemos visto ya el método `.catchError` muy útil para detectar si se produce algún error durante la ejecución de la función asíncrona.

```
var fut = HttpRequest.getString('http://www.google.es');
fut.catchError((error) => manejaError(error));
```

El método `.whenComplete` te permite registrar una función que se ejecutará cuando el objeto `Future` haya completado su tarea, no importa si ha terminado correctamente o no.

```
var fut = HttpRequest.getString('http://www.google.es');
fut.whenComplete(() => manejaTareaCompletada());
```

El método `.asStream` permite generar un `Stream` a partir de este objeto `Future`.

```
var fut = HttpRequest.getString('respuesta.json');
fut.asStream().toList().then((v) => print(v));
```

Por último, dispones del método `.timeout` que permite registrar una función que se ejecutará después del tiempo especificado.

```
var fut = HttpRequest.getString('respuesta.json');
fut
 .then((v) => print(v))
 .timeout(new Duration(seconds:10), onTimeout: () => print('TimeOUT!'));
```

Stream

Un objeto `Stream` es muy similar a un objeto `Future`. La única diferencia que propone Dart es que está pensado para series repetitivas de datos, pero conceptualmente es igual que un objeto `Future`, devolver el resultado de una ejecución asíncrona en un momento concreto.

Por este motivo puedes convertir un `Stream` en un `Future` y viceversa.

En realidad `Stream` es la clase que utilizas cuando añades un gestor de eventos sobre un botón en un documento HTML en `dart:html`. Toda la gestión de eventos en HTML utiliza esta clase.

También se utiliza para gestión de ficheros o directorios en `dart:io`, abrir, crear, copiar ficheros o crear directorios y listar su contenido.

Trabajando con Stream

Veamos algunos ejemplos de gestión de eventos de `dart:html` y cómo trabajar con `Stream` cuando registramos un manejador de eventos.

Partiremos de un documento HTML donde añadiremos un botón y crearemos un manejador del evento click del botón. Nos quedaremos escuchando este tipo de evento en nuestro botón mediante la clase `Stream`.

Cada vez que el usuario pulse sobre este botón vamos a obtener como resultado de nuestro manejador de eventos un objeto `StreamSubscription` que nos permitirá, además de obtener el evento que se produce en el elemento HTML, poder cancelar la escucha de este evento o pausarlo temporalmente.

Veamos algunos ejemplos de utilización de `Stream` y `StreamSubscription` en `dart:html` sobre un `ButtonElement`.

```
var boton = document.querySelector('#boton');

boton.onClick.listen((event) {
  print('Botón pulsado !!!');
});
```

Con el método `.listen` de la clase `Stream` creamos una nueva suscripción y cada vez que el usuario haga clic en dicho botón se lanzará la función `onData` de `StreamSubscription` y mostraremos nuestro mensaje Botón pulsado !!!.

Con este método, además de la función `onData`, dispones de las funciones `onError`, `onDone` y `cancelOnError`, que como puedes imaginar se ocupan de registrar funciones que se ejecutarán en caso de error del `Stream`, cuando el `Stream` se cierre se ejecutará el método `onDone` y `cancelOnError` permite terminar la suscripción cuando se produce un error.

En la clase `Stream` verás muchos métodos que ya vimos en `dart:core` al trabajar con Iterable, es debido a que un `Stream` está pensado para series repetitivas de datos, es decir, un `Stream` permite trabajar con cada elemento que conforma el `Stream`, de ahí que dispongas de métodos como `where`, `map`, `fold`, `join`, `forEach`, `every`, `any`, `last`, `first`, `single`, etc.

```
void main() {

  print('1. Instrucciones');

  var serie = [1, 2, 3, 4, 5];
  var stream = new Stream.fromIterable(serie);

  // Nos suscribimos a los eventos del Stream
  stream.listen((valor) {
    print("3. Stream: $valor");
  });

  print('2. Instrucciones');
}
```

Tras ejecutar este código el resultado es el siguiente.

```
1. Instrucciones
2. Instrucciones
3. Stream: 1
3. Stream: 2
3. Stream: 3
3. Stream: 4
3. Stream: 5
```

Como puedes observar el resultado es similar a la ejecución de los ejemplos de `Future`. Nuestro programa `main` no se queda esperando al `Stream` sino que es totalmente asíncrono y según se van recibiendo los datos desde el `Stream` se va ejecutando la función `onData` registrada.

Crear funciones asíncronas

Vistas ya las clases `Future` y `Stream` llega el momento de crear y utilizar tus propias funciones asíncronas y utilizar todo su potencial.

Una clase que no hemos comentado hasta ahora y que está disponible en `dart:async` es `Completer`, que te permitirá crear objetos de tipo `Future` y completarlos más tarde con un valor determinado o con error, según proceda.

```
import 'dart:async';

Future getUser() {
  final c = new Completer();
  c.complete({'user': 'Moises'});
  return c.future;
}
```

```
void main() {
  print('1. Instrucciones');
  var u = getUser();
  u.then((v) {
    print('2. Resultado future.');
    print(v['user']);
  });
  print('3. Instrucciones');
}
```

Lo primero que hacemos es crear una función que retorne un objeto `Future`. Después desde nuestra aplicación llamamos a dicha función y ya disponemos de los métodos propios de los objetos `Future` para trabajar con ella.

Si ejecutas este fragmento puedes ver como la función se ejecuta en modo totalmente asíncrono no esperando el programa principal al retorno del valor.

```
1. Instrucciones
3. Instrucciones
2. Resultado future.
Moises
```

Resumen de sintaxis

EXCEPCIONES	Para lanzar excepciones	`throw`
	Para capturar excepciones	`try {} , on {}, catch {}`
	Permite ejecutar un fragmento de código, se produzca, o no, una excepción	`finally {}`
	Clase abstracta que representa un fallo del sistema y de la cual deberás heredar para crear tus propias excepciones	`Exception`
	Error de formato cuando una cadena o cualquier otro valor no representa el formato correcto	`FormatException`
	Excepción de división de un valor por cero	`IntegerDivisionByZeroException`
TIPOS DE ERRORES	Fallo en una sentencia assert	`AssertionError`
	Fallo en una sentencia assert de tipo	`TypeError`
	Fallo al convertir un valor de un tipo a otro	`CastError`
	Se realiza un throw de un valor null	`NullThrownError`
	Se produce algún error de argumentos en funciones	`ArgumentError`
	Se intenta acceder a un índice que está fuera del rango de índices del objeto	`RangeError`
	Se intenta acceder a un método inexistente en un objeto	`NoSuchMethodError`
	Operación no permitida por un objeto	`UnsupportedError`
	El método no está implementado en el objeto	`UnimplementedError`
	Sin memoria suficiente	`OutOfMemoryError`
	Desbordamiento de pila	`StackOverflowError`

CLASES	Crear una clase	`new ConstructorDeLa-Clase()`
	Acceder a su variable de instancia o método	`.MetodoDeInstancia()` `.VariableDeInstancia`
	Clase constante	`const NombreDeLaClase {}`
	Variable privada	`_ NombreDeLaVariable`
	En definición de clase, los operadores re-escribibles son:	

<	+	|	[]
>	/	^	[]=
<=	~/	&	~
>=	*	<<	==
-	%	>>	

Herencia	Crear subclases	`extends`
	Referir a la clase Padre	`super`
Variables y métodos estáticos	Crear métodos/variables estáticas	`static`
Interfaces	Definir una clase con las clases que implementa y la implementación de las interfaces	`NombreClase implements ClasesQueImplementa`
Clases abstractas	Definir clase abstracta	`abstract class NombreDeLaClase {}`
Generics	Indicar que el contenido de una lista o mapa es Generic	`<..>`
Typedefs	Permite indicar el tipo de resultado que debe devolver una función	`Typedef Tipo Funcion(); Ej. Typdef String NombreFuncion()`

Librerías	Definir Librería	`library nombreLibreria`
	Importar Librería	`import 'nombreLibreria'`
	Importar Librería nativa de Dart	`import 'dart:nombreLibreria'`
	Importar paquete	`import 'package:nombreLibreria'` `import 'package:ruta/nombreLibreria'`
	Importar librería con prefijo	`import 'dart:nombreLibreria' as Prefijo` `Ej. import 'dart:math' as Math`
	Importar parte de una librería	`import 'nombreLibreria' show parteAimportar`

	Crear librerías con múltiples ficheros	`library nombreLibreria;`
		`part 'fichero1.dart',`
		`part 'fichero2.dart'`

	Exportar librerías	`export nombrelibreria`
	Exportar parte de una librería	`export nombrelibreria show parteAexportar`
Future	Obtener resultado de la función asíncrona	`Future.then()`
	Obtener error de la función asíncrona	`Future.catchError()`
	Obtener cuando se ha completado la función asíncrona	`Future.whenComplete()`
	Detectar un exceso de tiempo consumido por la función asíncrona	`Future.timeout(Duration, onTiemout:)`
Stream	Suscribirse a los eventos del Stream	`Stream.listen(onData, onError, onDone, cancelOnError)`

6 DART, HTML Y CSS

A lo largo del libro hemos ido viendo ciertas similitudes y diferencias con JavaScript. También hemos dicho que Dart pretende solucionar algunos problemas del desarrollo web ya que permite desarrollar de manera más estructurada y uniforme así como depurar y mantener mucho mejor grandes aplicaciones.

Dart se relaciona perfectamente con HTML y con CSS. En la librería `dart:html` encontraremos todos los elementos, clases y métodos necesarios para manejar todo el DOM.

Además, a través de `dart:html` tendremos acceso a todas las APIs de HTML5.

> El DOM es el Modelo de Objetos del Documento ("Document Object Model"), su responsable es el W3C y se podria definir como una API que proporciona:
> - Un conjunto estándar de objetos para representar documentos HTML y XML.
> - Un modelo estándar sobre cómo pueden combinarse dichos objetos.
> - Una interfaz estándar para acceder a ellos y manipularlos.
>
> ¿qué es el DOM?

Como has visto en otros ejemplos a lo largo del libro para poder trabajar con la librería HTML debes importarla en tus aplicaciones de este modo.

```
import 'dart:html';
```

Comencemos con algunos ejemplos para ver toda la capacidad que nos brinda Dart para trabajar con documentos HTML.

Trabajando con HTML5

Partiendo de una aplicación web creada desde Dart Editor con contenido de ejemplo podemos comenzar rápidamente a trabajar y realizar pruebas con la librería `dart:html`.

```
import 'dart:html';
void main() {
```

```
  var text = document.querySelector('#sample_text_id');
  text.text = 'Bienvenido a Dart !';
  text.lang = 'ES';
  text.attributes.forEach((k, v) => print('$k: $v'));
  text.style.backgroundColor = '#f87c45';
}
```

Podemos utilizar los métodos `.querySelector` y `.querySelectorAll` indicando un selector para encontrar elementos dentro del documento.

```
var text = document.querySelector('#sample_text_id');
```

Si ya has trabajado con JavaScript verás que esto permite simplificar y unificar enormemente la forma de trabajo. En JavaScript disponemos de los métodos `getElementById`, `getElementByClassName`, `getElementByName` y `getElementByTagName`.

En cambio con estos nuevos métodos el trabajo es más parecido a cómo trabajas con el framework jQuery y su método query `$(selector)`.

Estos métodos comenzaron llamándose `document.query` y `document.queryAll` en sus primeras versiones, pero ahora reciben este nuevo nombre por compatibilidad con el nuevo estándar que propone HTML5.

Los métodos `document.querySelector` y `document.querySelectorAll` son métodos de la clase `Document`.

Cuando trabajas en un documento HTML, el DOM es un árbol donde se alojan todas las etiquetas o nodos que componen el documento. Cada nodo puede tener a su vez otros nodos y la raíz del árbol es el objeto `document` que a su vez desciende de un objeto `window`.

Además de estos dos métodos la clase `Document` también pone a tu disposición los mismos métodos que tiene JavaScript para localizar elementos en el documento.

Puedes seleccionar elementos por su atributo id.

```
var elemento = document.querySelector('#sample_text_id');
var elemento = document.getElementById('sample_text_id');
```

Puedes seleccionar elementos por el atributo class.

```
var elemento = document.querySelector('.clase');
var elemento = document.getElementByClassName('clase');
```

Puedes seleccionar elementos por su tipo.

```
var elemento = document.querySelector('p');
var elemento = document.getElementsByTagName('p');
```

Y también puedes seleccionar elementos por el atributo `name`.

```
var elemento = document.querySelector('[name="nombre"]');
var elemento = document.getElementsByName('nombre');
```

Los métodos `.querySelector` y `.querySelectorAll` como ves son siempre iguales y tan solo cambia el selector que debes utilizar en función de lo que desees encontrar.

El método `.querySelectorAll` devuelve una lista de elementos que coincidan con el selector especificado. Cada elemento encontrado es un objeto y por tanto dispone de todas las propiedades y métodos definidos en `dart:html`.

Además, otra gran ventaja es que al utilizar este método obtienes colecciones reales de datos nativas de Dart, es decir `.querySelectorAll` te devuelve una lista de objetos que coincidan con el selector especificado y por lo tanto tienes a tu disposición todos los métodos que ya has visto en `dart:core` para trabajar con el tipo de datos `List`.

```
var elementos = document.querySelectorAll('p').toList(growable: true);
var divs = document.querySelectorAll('div');
elementos.addAll(divs);
elementos.forEach((el) => print(el.tagName));
```

Sin lugar a dudas esta es una de las mayores ventajas que presenta Dart en esta librería ya que todo el DOM está perfectamente estructurado y todos los nodos representados son tratados como objetos nativos de Dart con sus propiedades y métodos.

Una vez que has localizado y recuperado un elemento concreto del DOM puedes acceder a todas sus propiedades, eventos o métodos de este modo.

Si tu documento contiene un elemento párrafo de HTML como el siguiente.

```
<p id="sample_text_id" class="clase" name="nombre"></p>
```

Desde Dart puedes recuperar el elemento, y modificar cualquiera de sus propiedades.

```
var text = document.querySelector('#sample_text_id');
text.text = 'Bienvenido a Dart !';
text.lang = 'ES';
```

Puedes visualizar todos los atributos del elemento con esta simple función, ya se hayan especificado directamente en el HTML o se hayan establecido sus valores a través de Dart.

```
text.attributes.forEach((k, v) => print('$k: $v'));
```

Cuando estás trabajando con Dart Editor y la librería `dart:html` podrás consultar todos los métodos, propiedades y eventos de un objeto de manera fácil y sencilla gracias al sistema de auto completado.

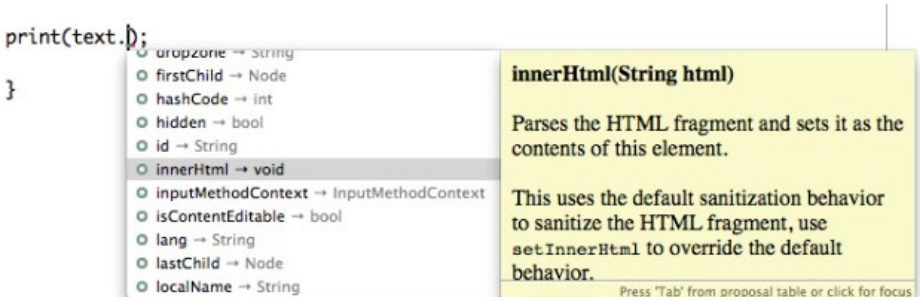


Figura 69. Sistema de auto completado para los objetos HTML.

Dentro de `dart:html` todo está perfectamente estructurado y jerarquizado en clases, todos los elementos heredan directamente de `HtmlElement` que a su vez hereda de `Element` la clase básica que define cualquier elemento HTML del documento y que contiene las propiedades y métodos comunes a todos los elementos.

Posteriormente las peculiaridades de cada elemento se definen dentro de su propia clase, por ejemplo, la propiedad `checked` de un `InputElement` o el método `.addRow()` de un `TableElement`, se definirán en sus respectivas clases.

Gracias a esta clasificación tan minuciosa la creación de elementos cuando trabajes con un documento HTML es sencilla y estructurada. Puedes crear un elemento párrafo de este modo y utilizar el operador de cascada para asignar valor a varias propiedades a la vez fácil y cómodamente.

```
var mip = new ParagraphElement();
mip..id = 'sample_text_2'
   ..text = 'Bienvenido a Dart !!'
   ..title = 'Bienvenido a Dart !!'
   ..lang = 'ES';
```

Ya has visto cómo encontrar elementos dentro de tu documento y crear nuevos elementos. Vamos a ver ahora cómo añadir elementos al DOM.

Como hemos comentado al comienzo, el documento HTML se estructura como un árbol donde cada elemento es un nodo y puede tener a su vez más nodos o ramas.

La clase `Element` de `dart:html` hereda de otra clase muy importante llamada `Node` y al mismo tiempo implementa otras dos llamadas `ParentNode` y `ChildNode`. Es decir, `Element` dispone de todas la funcionalidad para trabajar con los nodos del documento y, como todos los elementos HTML heredan directa o indirectamente de `Element`, también disponen de esta funcionalidad.

No importa que los elementos se creen en el documento HTML o los crees más tarde dinámicamente. Cualquier elemento puede ver qué nodos tiene como hijos y al mismo tiempo ver quién es su padre, por lo que a partir de un elemento puedes navegar por todo el documento subiendo por los padres o descendiendo por sus hijos.

Puedes recuperar un elemento y acceder a su padre.

```
print(p.parent.tagName); // DIV
print(p.parent.id); // sample_container_id
```

Puedes ir ascendiendo tanto como necesites.

```
print(p.parent.parent.tagName); // BODY
print(p.parent.parent.parent.tagName); // HTML
```

O recorrer sus hijos de cualquiera de estas dos formas.

```
var body = document.querySelector('body');
body.children.forEach((el) => print(el.tagName));
body.childNodes.forEach((el) => print(el));
```

La principal diferencia es que `.children` devuelve una lista de objetos Element y por lo tanto dispones de todas las propiedades y métodos de estos objetos, mientras que `.childNodes` devuelve una lista de objetos `Node` representando cada elemento del documento como un nodo.

Además de estos dos métodos, cada elemento del documento dispone del método `.nodes` que devuelve un `Iterable` modificable de todos los nodos del elemento.

Por lo tanto, utilizando este método puedes añadir elementos a tu documento así de fácil.

```
var mip = new ParagraphElement();
mip..id = 'sample_text_2'
   ..text = 'Bienvenido a Dart !!'
   ..title = 'Bienvenido a Dart !!'
   ..lang = 'ES';

document.body.nodes.add(mip);
```

Si quieres eliminar un elemento de tu documento HTML es tan sencillo como seleccionarlo y ejecutar el método `.remove()` presente en la clase `Node` y por lo tanto en todos los elementos HTML.

```
var p = document.querySelector('#sample_text_id');
p.remove();
```

Si lo prefieres también puedes reemplazar un nodo por otro con el método `.replaceWith`.

```
var p = document.querySelector('#sample_text_id');
var div = new DivElement()
  ..id='sample_div_id'
  ..text = 'Reemplazo elemento';
p.replaceWith(div);
```

Gestión de eventos

Ahora, vamos a ver como gestionar eventos que se producen en los elementos del documento cuando el usuario interactúa con ellos. Aprenderemos a añadir manejadores de eventos a un elemento de tu documento HTML.

Creamos un nuevo elemento `InputElement` y lo añadimos a nuestro contenedor principal. Le añadimos eventos a esta nueva caja de texto. Añadimos el evento `onClick` que se lanzará cada vez que el usuario haga clic con el ratón sobre la caja de texto y el evento `onKeyUp` que se lanzará cada vez que el usuario pulse una tecla dentro de la caja de texto.

```
var input = new TextInputElement()
  ..id = 'sample _ text _ input'
  ..value = 'Valor del input'
  ..placeholder = 'Introduzca valor';

// Evento onClick sobre el input.
input.onClick.listen((event) {
  if (input.value.trim() != '') {
    print(input.value);
  }
});
// Evento onKeyUp sobre el input.
input.onKeyUp.listen((KeyboardEvent e) {
  print(e.keyCode);
});

var div = document.querySelector('#sample _ container _ id');
div.nodes.add(input);
```

En nuestro ejemplo cada vez que el usuario pulse una tecla sobre la caja de texto mostraremos el código ASCII de la tecla pulsada.

Cada elemento HTML posee una serie de eventos con los que podemos trabajar. Aquí te mostramos todos los eventos disponibles para los elementos HTML.

```
onAbort
onBeforeCopy
onBeforeCut
onBeforePaste
onBlur
onChange
onClick
onContextMenu
onCopy
onCut
onDoubleClick
onDrag
onDragEnd
onDragEnter
onDragLeave
onDragOver
onDragStart
onDrop
onError
onFocus
onInput
onInvalid
onKeyDown
onKeyPress
onKeyUp
onLoad
onMouseDown
onMouseEnter
onMouseLeave
onMouseMove
onMouseOut
onMouseOver
onMouseUp
onMouseWheel
onPaste
```

```
onReset
onScroll
onSearch
onSelect
onSelectStart
onSubmit
onTouchCancel
onTouchEnd
onTouchEnter
onTouchLeave
onTouchMove
onTouchStart
onTransitionEnd
onFullscreenChange
onFullscreenError
```

El método `.listen()` nos devuelve las suscripción al evento (`StreamSubscription`) y por lo tanto podrás trabajar con el manejador a lo largo de tu aplicación, pudiendo pausar, reanudar o cancelar la suscripción.

```
var input = new TextInputElement()
  ..id = 'sample_text_input'
  ..value = 'Valor del input'
  ..placeholder = 'Introduzca valor';

var ss_keyup = input.onKeyUp.listen((KeyboardEvent e) {
  print(e.keyCode);
});

ss_keyup.pause();
ss_keyup.resume();
ss_keyup.cancel();
```

A continuación iremos viendo algunos ejemplos de trabajo con CSS y HTML y veremos en detalle como gestionar y manejar eventos.

Trabajando con CSS

Durante mucho tiempo el desarrollo web se limitó a crear documentos HTML donde se intercalaban elementos HTML, algunos scripts de código JavaScript y estilos, la mayor parte del tiempo dentro de los propios elementos HTML, para cambiar su apariencia, color, formato, etc.

Como mejora para estructurar mejor los documentos y para facilitar su mantenimiento, surgió CSS (*Cascading Style Sheets*) que te permite separar el estilo de la estructura de los elementos que componen un documento HTML.

Dart también se integra perfectamente con CSS y al proporcionarnos elementos HTML tan estructurados podemos acceder fácilmente a todas las propiedades relativas al estilo de los elementos ya hayan sido definidas directamente en el documento o a través de ficheros externos CSS.

Vamos a mostrarte a continuación algunos ejemplos de manipulación de todo lo relacionado con CSS y los estilos de elementos HTML y, cómo Dart vuelve a poner a tu disposición sencillos y bien definidos mecanismos para trabajar con este tipo de propiedades de manera consistente y uniforme.

Al igual que HTML, CSS ha ido evolucionando rápidamente para dar soporte a nuevas y mejoradas funciones todas ellas ya contempladas dentro de Dart.

Así puedes acceder a todas las propiedades relacionadas con estilos de los elementos HTML desde Dart.

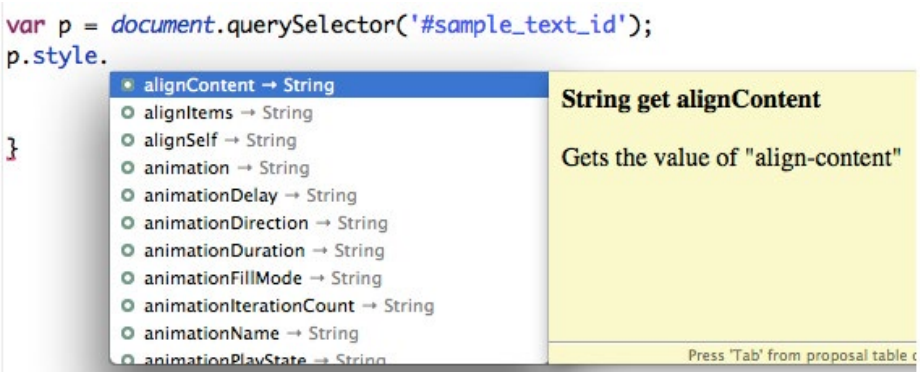


Figura 70. Accediendo a las propiedades de estilo de un elemento HTML.

Todo lo relacionado con CSS se ha estructurado en la clase `CssStyleDeclaration` de `dart:html`, y todos los elementos HTML disponen de la propiedad `.style` para acceder a las propiedades de estilo disponibles.

Cambiemos por ejemplo el color de fondo de nuestro párrafo `#sample_text_id`.

```
var p = document.querySelector('#sample_text_id');
p.text = 'PRUEBA';
p.style.backgroundColor = '#459fc2';
```

Si ejecutas este fragmento de código en Dartium obtendrás algo como lo que te mostramos a continuación.

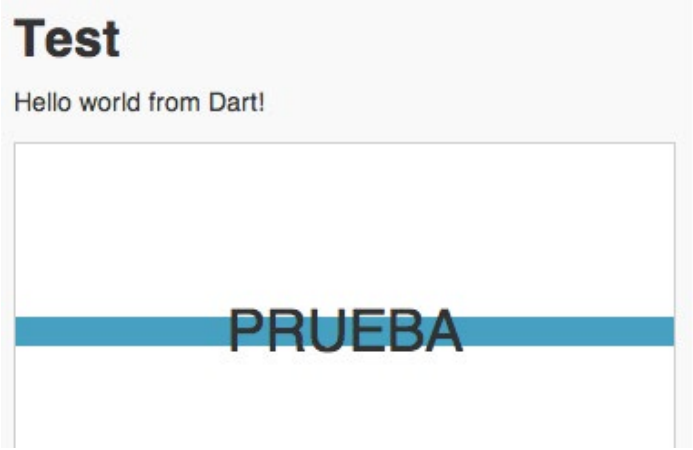


Figura 71. Dartium ejecutando la aplicación Dart que trabaja con los estilos de elementos.

Desde Dart puedes trabajar con cualquier propiedad relativa al estilo de los elementos HTML, pero si quieres tenerlo todo perfectamente estructurado y que sea fácilmente mantenible, te interesa separar HTML, Dart y CSS.

Partiendo del mismo ejemplo, vamos a editar el fichero CSS que genera Dart Editor cuando creamos una nueva aplicación web e incorporaremos unas clases CSS para aplicarlas posteriormente a nuestros elementos desde Dart.

Para nuestro párrafo vamos a crear diferentes clases CSS con diferentes estilos.

```
body {
  background-color: #F8F8F8;
  font-family: 'Open Sans', sans-serif;
  font-size: 14px;
  font-weight: normal;
  line-height: 1.2em;
  margin: 15px;
}


h1, p {
```

```
  color: #333;
}
#sample _ container _ id {
  width: 80%;
  height: 50px;
  position:relative;
}

#sample _ text _ id {
  font-size: 24pt;
  text-align: center;
  margin-top: 14px;
}

.correcto {
   background: #E6EFC2 5px 5px;
   padding: 1em;
   padding-left: 3.5em;
   border: 2px solid #C6D880;
   color: #529214;
}

.error {
  background: #FBE3E4 5px 5px;
  padding: 1em;
  padding-left: 3.5em;
  border: 2px solid #FBC2C4;
  color: #D12F19;
}

.alerta {
  background: #FFF6BF 5px 5px;
  padding: 1em;
  padding-left: 3.5em;
```

```
  border: 2px solid #FFD324;
  color: #817134;
}
```

El código HTML de esta aplicación de prueba tendrá esta apariencia.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Test</title>
    <link rel="stylesheet" href="test.css">
  </head>
  <body>
    <h1>Test</h1>
    <p>Hello world from Dart!</p>
    <div id="sample _ container _ id">
      <p id="sample _ text _ id">Pulsa los botones !</p>
    </div>
    <script type="application/dart" src="test.dart"></script>
    <script src="packages/browser/dart.js"></script>
  </body>
</html>
```

Ahora desde nuestra aplicación Dart vamos a añadir unos botones que cambiarán el estilo de nuestro documento HTML cuando los pulsemos.

```
import 'dart:html';

void main() {

var container = document.querySelector('#sample _ container _ id');
var p = document.querySelector('#sample _ text _ id');
```

```
    var btn1 = new ButtonElement()
      ..id = 'btn1'
      ..text = ' ERROR '
      ..onClick.listen((event){
        container.classes
          ..clear()
          ..add('error');
        p.text = 'Esto es un mensaje de error!';
      });

    var btn2 = new ButtonElement()
      ..id = 'btn2'
      ..text = 'ALERTA'
      ..onClick.listen((event) {
        container.classes
          ..clear()
          ..add('alerta');
        p.text = 'Esto es un mensaje de alerta !';
      });

    var btn3 = new ButtonElement()
      ..id = 'btn3'
      ..text = 'OK'
      ..onClick.listen((event) {
        container.classes
          ..clear()
          ..add('correcto');
        p.text = 'Esto es un mensaje de éxito';
      });
    document.body.nodes.addAll([btn1, btn2, btn3]);

}
```

Si ejecutas esta aplicación y pulsas sobre los botones, verás que el texto del párrafo cambia y aplicamos una clase, de las definidas en el CSS, al contenedor cambiando la apariencia.

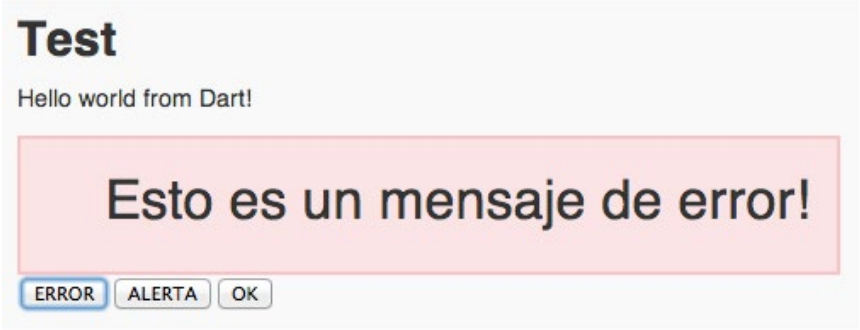


Figura 72. Pulsando el botón ERROR.



Figura 73. Pulsando el botón ALERTA.

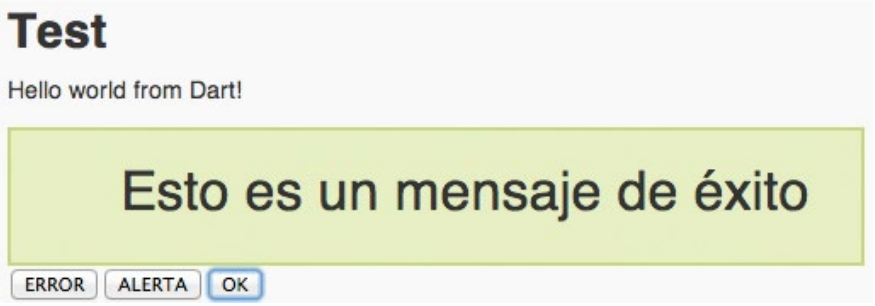


Figura 74. Pulsando el botón OK.

El código Dart controla el evento `onClick` sobre el botón pulsado y elimina cualquier clase que tuviese el contenedor, posteriormente añade la clase correspondiente.

```
var btn2 = new ButtonElement()
  ..id = 'btn2'
  ..text = 'ALERTA'
  ..onClick.listen((event) {
    container.classes
      ..clear()
      ..add('alerta');
    p.text = 'Esto es un mensaje de alerta !';
  });
```

Todos los elementos HTML disponen del método `.classes` para obtener las clases CSS aplicadas o para añadir otras nuevas.

Esta propiedad representa un conjunto de las clases CSS del elemento y permite operar con ellas a través de estos métodos.

- `.toggle` que añade una clase a un elemento si no la contiene, o la elimina si ya la tenía asignada.
- `.contains` permite comprobar si un elemento tiene asignada una determinada clase.
- `.add` añade una clases al elemento.
- `.addAll` añade múltiples clases al elemento.
- `.remove` elimina una clase de un elemento.
- `.removeAll` elimina todas las clases indicadas de un elemento.

Veamos como funciona el método `.toggle` añadiendo un nuevo botón a nuestra aplicación.

Añadimos la nueva clase `.toggle` al fichero CSS.

```
.toggle {
  background: #cbe0f4 5px 5px;
  padding: 1em;
  padding-left: 3.5em;
  border: 2px solid #96b6d3;
  color: #286eae;
}
```

Y en nuestra aplicación Dart un nuevo botón.

```
var btn4 = new ButtonElement()
  ..id = 'btn4'
  ..text = 'Toggle'
  ..onClick.listen((event) {
    container.classes.toggle('toggle');
    p.text = ' Toggle !';
  });
document.body.nodes.add(btn4);
```

Este fragmento de código crea un nuevo botón que muestra el texto Toggle ! y añade la clase `.toggle` al contenedor si no la tiene o la elimina si ya la tiene añadida.



Figura 75. Pulsando el nuevo botón Toggle.

Pulsamos este nuevo botón varias veces y lo combinamos con los otros que creamos antes para ver como funciona esta propiedad.

Efectos visuales

Cada vez es más común, sobre todo desde que se popularizó el uso de jQuery, ver páginas web con gran cantidad de información y que además disponen de sencillas y elegantes animaciones para hacerlas más atractivas al usuario.

Uniendo todo el potencial de HTML5, CSS3 y Dart podemos realizar efectos visuales muy llamativos de forma rápida y sencilla.

FadeIn y FadeOut

Vamos a añadir un par de botones a nuestro documento HTML que te permitan mostrar u ocultar un mensaje al usuario de un modo vistoso.

Creamos un fichero `test.css` con estas sencillas clases CSS.

```
body {
  background-color: #F8F8F8;
  font-family: 'Open Sans', sans-serif;
  font-size: 14px;
  font-weight: normal;
  line-height: 1.2em;
  margin: 15px;
}

.mensaje {
  background: #cbe0f4 5px 5px;
  padding: 1em;
  padding-left: 3.5em;
  border: 2px solid #96b6d3;
  color: #286eae;
  opacity: 0;
}
```

```
.fade-in {
  opacity: 1;
  -webkit-transition: opacity 1s ease-in;
  -moz-transition: opacity 1s ease-in;
  -o-transition: opacity 1s ease-in;
  -ms-transition: opacity 1s ease-in;
  transition: opacity 1s ease-in;
}

.fade-out {
  opacity: 0;
 -webkit-transition: opacity 1s ease-out;
  -moz-transition: opacity 1s ease-out;
  -o-transition: opacity 1s ease-out;
  -ms-transition: opacity 1s ease-out;
  transition: opacity 1s ease-out;
}
```

Tenemos un clase que define el estilo del `body` del documento, otra que da estilo al mensaje y otras dos para hacer el efecto `fade in` (aparecer) y `fade out` (desaparecer).

Creamos otro fichero llamado `test.html` con este contenido, que como veremos, es similar al ejemplo que crea Dart Editor al crear una nueva aplicación web.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Test</title>
<link rel="stylesheet" href="test.css">
</head>
<body>
  <div id="sample _ container _ id">
    <p id="sample _ text _ id"></p>
  </div>
```

```
  <script type="application/dart" src="test.dart"></script>
  <script src="packages/browser/dart.js"></script>
</body>
</html>
```

Por último, este será nuestro fichero `test.dart` con el código que nos permite dar funcionalidad a nuestro documento.

```
import 'dart:html';

void main() {

  var p = document.querySelector('#sample _ text _ id')
      ..classes.add('mensaje')
      ..text = 'Bienvenido a Dart !';

  var btn1 = new ButtonElement()
    ..id = 'mostrar'
    ..text = 'Mostrar'
    ..onClick.listen((e) {
      p.classes.retainWhere((c) => c=='mensaje');
      p.classes.add('fade-in');
    });

  var btn2 = new ButtonElement()
      ..id = 'ocultar'
      ..text = 'Ocultar'
      ..onClick.listen((e) {
        p.classes.retainWhere((c) => c=='mensaje');
        p.classes.add('fade-out');
      });

  document.body.nodes.addAll([btn1, btn2]);
}
```

En primer lugar, y como es habitual, los `imports` necesarios. Luego recuperamos el elemento párrafo y le añadimos un mensaje y la clase CSS `mensaje`. Por último, creamos dos botones, uno para mostrar y otro para ocultar el mensaje, y los añadimos a nuestro documento.

Si ejecutas este ejemplo podrás ver cómo el mensaje aparece, pulsando sobre Mostrar, y desaparece, pulsando sobre Ocultar, de un modo muy agradable realizando un efecto de desvanecimiento.



Figura 76. Efecto *fade in* pulsando en Mostrar.

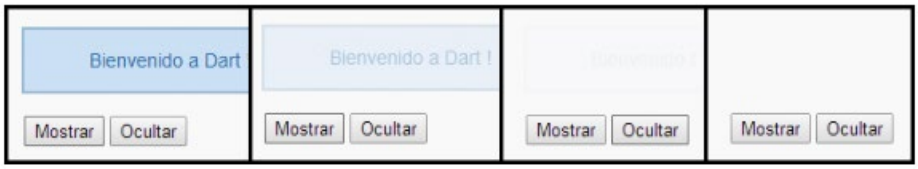


Figura 77. Efecto *fade out* pulsando en Ocultar.

Efecto parpadeo

Vamos a crear un efecto parpadeo de mensajes para el usuario. Crearemos un círculo de color rojo y dentro insertaremos el número de mensajes pendientes de leer. Haremos un pequeño efecto parpadeo del número de mensajes.

Nuestro fichero `test.css` tendrá estas clases que definen el estilo para el `body`. La clase `inbox` define el círculo rojo que contendrá el número de mensajes pendientes de leer. La clase `.msg` define el estilo del número contenido dentro de `.inbox` y, por último, la clase `.agranda` que hace que el número de mensajes pendientes cambie el tamaño de la fuente pasando de 20pt a 29pt.

```
body {
  background-color: #F8F8F8;
  font-family: 'Open Sans', sans-serif;
  font-size: 14px;
  font-weight: normal;
  line-height: 1.2em;
  margin: 15px;
}

.inbox {
  width: 50px;
  height: 50px;
  background-color: red;
  border-radius: 100%;
  text-align: center;
}

.msg {
  position: relative;
  top: 15px;
  color: #ffffff;
  font-family: Verdana;
  font-size: 20pt;
  font-weight: bold;

  -webkit-transition: font-size 1s ease-in-out;
  -moz-transition: font-size 1s ease-in-out;
  -o-transition: font-size 1s ease-in-out;
  transition: font-size 1s ease-in-out;
}

.agranda {
  font-size: 29pt;
}
```

El fichero `test.html` mostrará el `div` contenedor de los ejemplos anteriores, dentro de este tenemos el elemento párrafo que será el que muestre el número de mensajes.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Test</title>
<link rel="stylesheet" href="test.css">
</head>
<body>
  <div id="sample _ container _ id">
    <p id="sample _ text _ id" class="inbox">
      <span id="mensajes" class="msg">2</span>
    </p>
  </div>
  <script type="application/dart" src="test.dart"></script>
  <script src="packages/browser/dart.js"></script>
</body>
</html>
```

Y nuestro código Dart será así de sencillo:

```
import 'dart:html';
import 'dart:async';

void main() {
  var msg = document.querySelector('#mensajes');
  var timer = new Timer.periodic(new Duration(seconds: 1), ( _ ){
    msg.classes.toggle('agranda');
  });
}
```

Recuperamos el elemento que contiene el número de mensajes pendientes de leer y creamos un objeto `Timer` que se repite cada 2 segundos. Cada vez que se ejecute este proceso añade o quita la clase `.agranda` al elemento, haciendo que el número de mensajes agrande o encoja su tamaño simulando un efecto parpadeo.

El resultado final es que el número de mensajes tiene un efecto parpadeo muy suave para llamar la atención del usuario



Figura 78. Efecto parpadeo del número de mensajes pendientes de leer.

Expandir y contraer elementos

Otro interesante efecto, que ya habrás visto en muchas páginas web, es la posibilidad de expandir o contraer ciertos bloques de información. En documentos con gran cantidad de información es realmente útil brindar al usuario esta posibilidad para que pueda ampliar información sobre las partes que le interesen y dejar el resto ocultas.

Esto nos permite cargar gran cantidad de información dentro de nuestros documentos sin perder en usabilidad y navegabilidad.

Veamos cómo desarrollar con Dart, HTML5 y CSS3 este tipo de efectos. Creamos un fichero `test.html` con este contenido.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Test</title>
<link rel="stylesheet" href="test.css">
</head>
<body>
  <div id="demo">
    <h2><div class="arrow-right""></div>Frutas</h2>
    <ul>
      <li>Manzana</li>
      <li>Pera</li>
      <li>Naranja</li>
    </ul>
  </div>
  <script type="application/dart" src="test.dart"></script>
  <script src="packages/browser/dart.js"></script>
</body>
</html>
```

Creamos un `div` contenedor con el `id demo`, dentro de él creamos un elemento `h2` con el título `Frutas` y a continuación un listado de frutas. Dentro del elemento `h2` hemos añadido otro `div` con la clase `.arrow-right` que nos dibuja una flecha con la punta hacia la derecha. CSS3 nos permite hacer este tipo de cosas y evitar utilizar imágenes.

A continuación crea un fichero `test.css` con estas clases.

```
body {
  background-color: #F8F8F8;
  font-family: 'Open Sans', sans-serif;
  font-size: 14px;
  font-weight: normal;
  line-height: 1.2em;
  margin: 15px;
}
```

```
#demo h2 {
  font-size: 18px;
  display: block;
  padding-left: 30px;
  padding-left: 32px;
  margin: 0;
  color: #628066;
  text-decoration: none;
  font-weight: normal;
  border-bottom: 1px solid #628066;
}

#demo ul {
  color: #628066;
  border: 1 px solid #628066;
  background-color: #B0E5B8;
  margin-top: 0px;
  margin-left: 25px;
}

.hide {
  display:none;
}

.arrow-down {
  width: 0;
  height: 0;
  border-left: 7px solid transparent;
  border-right: 7px solid transparent;
  border-top: 7px solid #000000;
  float:left;
  margin-right: 7px;
  cursor: pointer;
}

.arrow-right {
  width: 0;
  height: 0;
```

```
  border-top: 7px solid transparent;
  border-bottom: 7px solid transparent;
  border-left: 7px solid #000000;
  float:left;
  margin-right: 7px;
  cursor: pointer;
}
```

Establecemos estilos para `body`, para los elementos `h2` y `ul` del contenedor `demo` y, por último, nos creamos 3 clases más una para esconder la lista de frutas, y las dos clases de flechas una apunta hacia abajo y otra hacia la derecha.

Creamos un fichero `test.dart` y añade este código.

```
import 'dart:html';

void main() {
  var demo = document.querySelector('#demo');
  var list = document.querySelector('#demo ul');
  list.classes.add('hide');
  demo.onClick.listen((e) {
    var arrow = document.querySelector('#demo h2 div');
    arrow.classes.toggleAll(['arrow-right', 'arrow-down']);
    list.classes.toggle('hide');
  });
}
```

Este sencillo bloque de código recupera el elemento contenedor principal `demo` y la lista de frutas. Añade la clase `.hide` a la lista de frutas para que no estén visibles y añade un evento `onClick` sobre el `div` contenedor.

Cuando pulses en el `div` cambiamos la flecha que apunta hacia la derecha por una flecha que apunte hacia abajo y mostramos la lista de frutas.

Así quedará este efecto en funcionamiento.



Figura 79. Elemento que muestra un listado de frutas plegado.

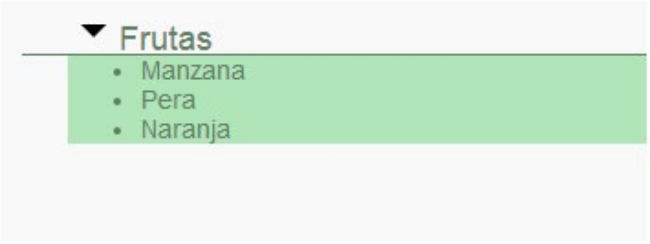


Figura 80. Elemento que muestra desplegado el listado de frutas.

7 SERVICIOS WEB

Hemos visto como Dart te permite manejar perfectamente los documentos HTML y todo lo relacionado con CSS. Actualmente las páginas webs comienzan a descargar gran parte de las funciones en el lado cliente aprovechando la potencia de los dispositivos y reduciendo el consumo de recursos en el lado servidor.

Una de las mayores ventajas para incrementar la potencia en el lado cliente es poder ir modificando el DOM a partir de nuevos datos que vayan llegando del servidor.

Dart nos aporta potencial para desarrollar el lado cliente y aquí queremos mostrarte como realizar peticiones HTTP asíncronas para ir obteniendo datos desde el servidor e ir modificando el DOM en función de las acciones que el usuario vaya realizando.

Además de las peticiones HTTP asíncronas vamos a utilizar JSON (JavaScript Object Notation) Notación de Objetos de JavaScript como modelo de intercambio ligero de datos entre nuestras aplicaciones cliente y servidor.

Dart trae de serie soporte para JSON en su paquete `dart:convert` y veremos como utilizarlo para simplificar el procesamiento del paso de mensajes entre cliente y servidor.

Peticiones HTTP asíncronas con Dart

Dart nos permite cargar o modificar dinámicamente ciertas partes de nuestros documentos HTML a partir de los datos obtenidos del servidor haciendo uso de la clase `HttpRequest`. En JavaScript las peticiones asíncronas se realizan utilizando el objeto `XMLHttpRequest`.

Gracias a `HttpRequest` desde Dart podrás comunicarte bi-direccionalmente con el servidor, es decir, obtener datos del servidor o hacer peticiones enviado datos desde el lado cliente.

La clase `HttpRequest` tiene una restricción de seguridad en el navegador que no permite realizar peticiones desde un cliente a un dominio en el cual no esté alojado el script que se carga en el cliente.

Evidentemente si el servidor tiene habilitada la opción CORS (*Cross-origin resource sharing*) si es posible que la aplicación Dart cargada en el cliente desde un dominio pueda realizar peticiones a otro dominio.

En nuestros ejemplos vamos a hacer peticiones contra nuestro propio cliente (simulando que actúa de servidor) para evitar estos problemas de acceso desde diferentes orígenes.

Vamos a ver a continuación ejemplos de peticiones al servidor y veremos el problema de orígenes que acabamos de comentar.

Vamos a crear una nueva aplicación web en Dart Editor para comenzar con nuestros ejemplos de `HttpRequest`.

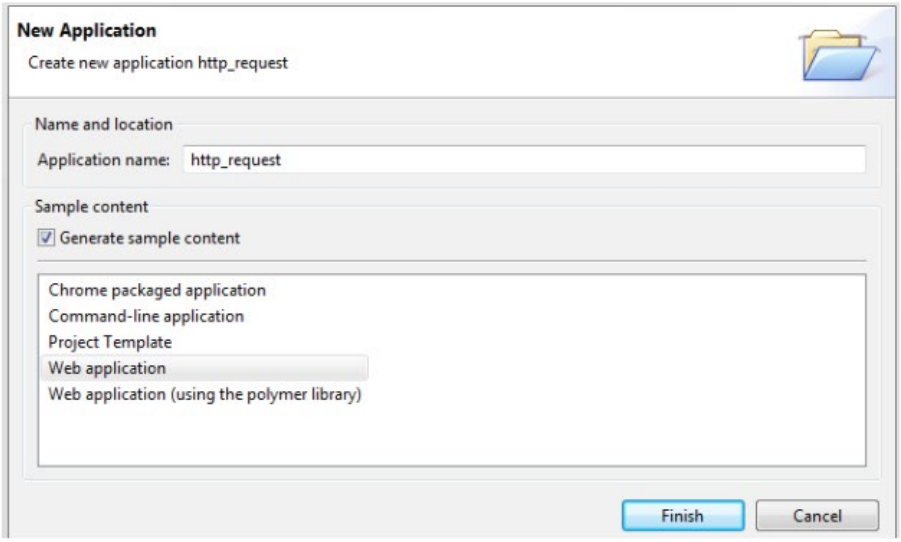


Figura 81. Nuevo proyecto web para HttpRequest.

La clase `HttpRequest` se encuentra dentro de `dart:html` y nos permite realizar las peticiones asíncronas. Esta misma clase pone a tu disposición ciertos métodos para simplificarte el trabajo con la recepción o envío de datos contra el servidor.

`HttpRequest.getString` permite realizar peticiones contra una URL concreta que devolverá la respuesta en formato texto.

`HttpRequest.postFormData` permite hacer una petición POST con los datos indicados codificados como un formulario, es decir, permite enviar datos al servidor.

`HttpRequest.request` este método permite realizar peticiones contra una URL indicada. Este método te permite crear y configurar todos los parámetros necesarios para la petición pudiendo establecer el método para hacer tanto peticiones GET, como POST, PUT, DELETE, etc. Y definir el formato de los datos que se van a enviar como FormData, ByteBuffer, String, Blob, Document.

También podrás definir el mime-type, el tipo de la respuesta del servidor y las cabeceras de la petición. `HttpRequest.requestCrossOrigin` permite realizar peticiones entre diferentes dominios si el dominio destino soporta CORS.

La clase `HttpRequest` además pone a tu disposición los métodos necesarios para abrir, enviar y cerrar los objetos HttpRequest y gestionar las respuestas desde el servidor. Saber el estado de la respuesta, el código devuelto o si se ha excedido el tiempo *(time out)*.

En nuestro nuevo proyecto vamos crear el fichero `http _ request.css` con el código que define los estilos de los elementos que vamos a utilizar.

```
body {
  background-color: #F8F8F8;
  font-family: 'Open Sans', sans-serif;
  font-size: 14px;
  font-weight: normal;
  line-height: 1.2em;
  margin: 15px;
}

h1, p {
  color: #333;
}
```

```
#respuesta {
  width: 100%;
  height: 400px;
  position: relative;
  border: 1px solid #ccc;
  background-color: #fff;
  text-align: left;
  padding-left: 5px;
  padding-top: 5px;
}
```

También crearemos el fichero `http _ request.html` con los elementos HTML que configuran el esqueleto base de nuestra aplicación.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Http request</title>
    <link rel="stylesheet" href="http _ request.css">
  </head>
  <body>
    <h1>Http request</h1>
    <div id="buttons"></div>
    <div id="respuesta"></div>
    <script type="application/dart" src="http _ request.dart"></script>
    <script src="packages/browser/dart.js"></script>
  </body>
</html>
```

Además deberás crear otro fichero llamado `datos _ usuario.txt` con un contenido similar a este que te mostramos en el ejemplo.

```
<b>Usuario:</b> Moises <br/>
<b>Email:</b> moisesbelchin@gmail.com <br/>
<b>Ultima conexion:</b> Ayer a las 21:32 <br/>
```

Este fichero se usa para simular la respuesta que nos debería devolver el servidor. Como hemos comentado anteriormente solo podemos hacer peticiones al mismo dominio que nos sirve nuestra aplicación.

Y, por último, nuestro fichero `http _ request.dart` tendrá este código encargado de crear nuestra aplicación que usa `HttpRequest` para pedir datos y modificar el contenido del documento sin necesidad de refrescar o navegar a otra URL.

```
import 'dart:html';

void request() {
  var respuesta = document.querySelector('#respuesta');
  var url = 'http://127.0.0.1:3030/http _ request/web/datos _ usuario.txt';
  HttpRequest.getString(url,
                onProgress:( _ ) => respuesta.text = 'Cargando ...')
    ..then((resp) {
      respuesta.setInnerHtml(resp);
    })
   ..catchError((error) => respuesta.text = 'ERROR !!!: '
                                            '${error.toString()}');
}

void main() {
  var buttons = document.querySelector('#buttons');
  var btn1 = new ButtonElement()
    ..id = 'btn1'
    ..text = 'Datos de usuaro'
    ..onClick.listen((e) => request());
  buttons.nodes.add(btn1);
}
```

El método `main` recupera el contenedor donde iremos añadiendo botones. Crea un nuevo botón pensado para recuperar los datos del usuario que ha accedido a nuestro sistema. Cuando pulses en este botón se lanzará el evento `onClick` asociado a la función `request` que hemos creado.

La función `request` recupera el contenedor donde irá la respuesta recibida del servidor. Define también la URL donde haremos la llamada.

Utiliza el objeto `HttpRequest` y el método `getString` para lanzar la llamada a nuestra URL. Además define qué hacer mientras dure el proceso de petición. En nuestro caso mostramos un texto que indica al usuario que estamos cargando la información solicitada.

`HttpRequest.getString` te devuelve un objeto `Future` y utilizando los métodos `.then` y `.catchError` podemos gestionar la respuesta o si se produce algún error.

En el primer caso cuando la respuesta es correcta la mostramos en el `div` respuesta. En caso de error muestra una alerta de error y el error devuelto por el sistema.

Ejecutando este ejemplo puedes ver cómo funciona esta pequeña aplicación que modifica dinámicamente el contenido de nuestro `div` respuesta mostrando el mensaje devuelto por el servidor cuando pulsas en el botón Datos de usuario.

Como habrás podido observar esta es la forma de ir cambiando la estructura del documento HTML sin necesidad de recargar nada.

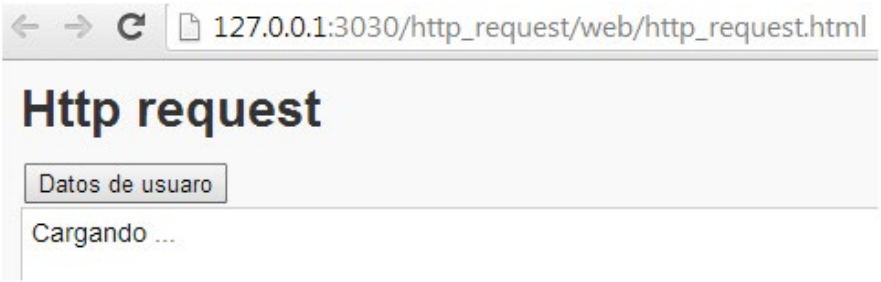


Figura 82. Ejecutando la petición contra el servidor.

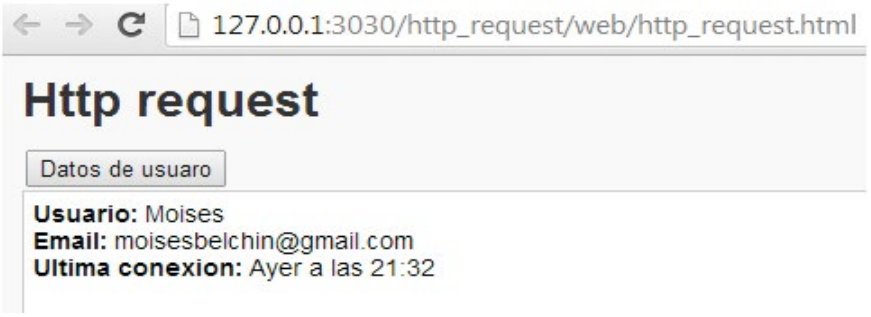


Figura 83. Recibida respuesta correcta desde el servidor.

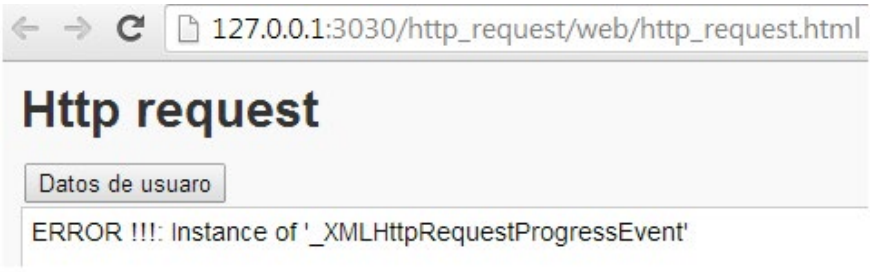


Figura 84. Respuesta de error recibida desde el servidor.

Vamos a ver ahora cómo enviar datos a un servidor a través del método `HttpRequest.postFormData`. Este método como ya hemos comentado permite enviar datos codificados como `Form Data`.

Vamos a crear un sencillo ejemplo que permita al usuario rellenar un formulario de contacto y enviar estos datos a nuestro servidor.

Para poder hacer que este ejemplo sea plenamente funcional vamos a requerir de un entorno servidor que maneje la petición `POST` de envío de datos que vamos a realizar desde nuestra aplicación Dart.

Para ello vamos a crear un sencillo servidor web con Python que nos responda con los parámetros `form data` que le enviemos cada vez que hagamos una petición `POST`.

Crea un fichero llamado `server.py` dentro del directorio web de nuestra aplicación `http _ request` y añade este código.

```
import SimpleHTTPServer
import cgi
from pprint import *

class CORSHTTPRequestHandler(SimpleHTTPServer.SimpleHTTPRequestHand-
ler):

  def do _ POST(self, *args, **kwargs):
    ctype, pdict = cgi.parse _ header(
                        self.headers.getheader('content-type'))
    postvars = {}
    try:
      if ctype == 'application/x-www-form-urlencoded':
        length = int(self.headers.getheader('content-length'))
        postvars = cgi.parse _ qs(self.rfile.read(length),
        keep _ blank _ values=1)

      body = pformat(postvars)

      # set headers
      self.send _ response(200)
      self.send _ header("Content-type", "text/html")
      self.send _ header("Content-length", str(len(body)))
      self.end _ headers()
      self.wfile.write(body)
      self.wfile.close()
    except Exception, e:
      print e

if _ _ name _ _ == " _ _ main _ _ ":
  import os
  import SocketServer
```

```
Handler = CORSHTTPRequestHandler
httpd = SocketServer.TCPServer(("127.0.0.1", 3031), Handler)
print "Servidor corriendo en http://127.0.0.1:3031/"
httpd.serve _ forever()
```

Luego ve al directorio web de nuestra aplicación y ejecuta este comando.

```
http _ request/web$ python server.py
```

Tras ejecutar el comando verás que el servidor responde este mensaje.

```
Servidor corriendo en http://127.0.0.1:3031/
```

Si abres tu navegador y te diriges a esta dirección podrás ver como muestra los ficheros de nuestro directorio web.



Figura 85. Accediendo a nuestro servidor web recién montado.

Veamos ahora cómo quedará nuestra aplicación Dart encargada de realizar las peticiones POST a este nuevo servidor.

Primero el fichero `http _ request.css` con los estilos necesarios.

```
body {
  background-color: #F8F8F8;
  font-family: 'Open Sans', sans-serif;
  font-size: 14px;
  font-weight: normal;
  line-height: 1.2em;
  margin: 15px;
}

h1, p {
  color: #333;
}

#respuesta {
  width: 280px;
  height: 200px;
  position: relative;
  border: 1px solid #ccc;
  background-color: #fff;
  text-align: left;
  padding-left: 5px;
  padding-top: 5px;
  float:right;
}

#form {
  width: 200px;
  float:left;
}
```

Hemos creado dos partes, en la parte izquierda `#form` se mostrará nuestro formulario de datos, en la derecha `#respuesta` la respuesta obtenida desde el servidor.



Figura 86. Formato de nuestra aplicación.

El código HTML esqueleto contenedor de nuestra aplicación tendrá este estilo.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Http request</title>
    <link rel="stylesheet" href="http_request.css">
  </head>
  <body>
    <h1>Http request</h1>
    <div id="buttons"></div>
    <div id="form"></div>
    <div id="respuesta"></div>
    <script type="application/dart" src="http_request.dart"></script>
    <script src="packages/browser/dart.js"></script>
  </body>
</html>
```

Puedes ver los 3 divs principales `#buttons`, `#form` y `#respuesta` con los que trabajaremos desde nuestra aplicación Dart.

Ahora veremos el código Dart que creará nuestra aplicación. Hemos creado diferentes funciones para cada una de las partes de la aplicación, estructurando mejor el código para que resulte más sencillo explicarlo.

Por un lado el método `main()` principal y el `import` de `dart:html` que utilizaremos en este ejemplo.

```
import 'dart:html';

void main() {
  var buttons = document.querySelector('#buttons');
  var btn1 = new ButtonElement()
    ..id = 'btn1'
    ..text = 'Contacto'
    ..onClick.listen((e) => form());
  buttons.nodes.add(btn1);
}
```

Recuperamos el `div #buttons` y creamos un nuevo botón con la etiqueta "Contacto". Cuando se pulse llamará a la función `form()`.

```
void form() {
  var resp = document.querySelector('#form');
  var nombre = new InputElement()
    ..id = 'nombre'
    ..size = 45
    ..placeholder = 'Introduzca su nombre';
  var email = new EmailInputElement()
    ..id = 'email'
    ..size = 45
    ..placeholder = 'Introduzca su email';
```

```
  var asunto = new TextInputElement()
    ..id = 'asunto'
    ..size = 45
    ..placeholder = '¿Cuál es el motivo de su consulta?';
  var mensaje = new TextAreaElement()
    ..id = 'mensaje'
    ..cols = 40
    ..rows = 10
    ..placeholder = '¿En qué podemos ayudarte?';
  var enviar = new ButtonElement()
    ..id = 'enviar'
    ..text = ' Enviar mensaje ! '
    ..onClick.listen((e) => enviarMensaje());

  resp.nodes.clear();
  resp.nodes.addAll([nombre, new BRElement(),
                     email, new BRElement(),
                     asunto, new BRElement(),
                     mensaje, new BRElement(),
                     enviar]);
  nombre.focus();
}
```

Esta función recupera el `div #form` y dentro de él añade una serie de campos `Input` y `TextArea HTML` que conformarán nuestro formulario de contacto.

Esta función además crea un botón con la etiqueta `"Enviar mensaje !"` que lanzará la función `enviarMensaje()` cuando se pulse.

```
void enviarMensaje() {
  var resp = document.querySelector('#respuesta');
  Map data = new Map();
  data['nombre'] = (document.querySelector('#nombre') as InputElement).
       value.trim();
  data['email'] = (document.querySelector('#email') as InputElement).
       value.trim();
  data['asunto'] = (document.querySelector('#asunto') as InputElement).
       value.trim();
```

```
    data['mensaje'] = (document.querySelector('#mensaje') as
         TextAreaElement).value.trim();

    var url = 'http://127.0.0.1:3031';
    var xhr = HttpRequest.postFormData(url, data,
        onProgress: ( _ ) => resp.text = 'Enviando ... ');
    xhr
      ..then((r) => respuesta(r))
      ..catchError((error) => resp.text = 'ERROR !: ${error.toString()}');
  }
```

La función `enviarMensaje()` recupera todos los valores de los campos del formulario compone un `Map` con los datos y utiliza el método `HttpRequest.postFormData` para enviarlos al servidor. Establece un texto temporal **"Enviando ..."** que se mostrará en el `div #respuesta` mientras la petición se esté procesando.

También establece los métodos para la gestión del `Future<HttpRequest>` que nos devuelve `HttpRequest.postFormData`. Si se produce algún error mostrará un texto en el `div #respuesta` con el error devuelto por el sistema. Cuando se recupere la respuesta correcta por parte del servidor ejecutará la función respuesta().

```
  void respuesta(r) {
    var div_resp = document.querySelector('#respuesta');
    var resp = r.responseText;
    resp = resp.replaceAll('{', '')
      .replaceAll('}', '')
      .replaceAll("['", "")
      .replaceAll("']", "")
      .replaceAll("'", "");
    resp = resp.split(",").join('<br/>');
    div_resp.setInnerHtml('<b>Status:</b> ${r.status} <br/>'
                          '<b>Respuesta:</b> <br/> ${resp}');
  }
```

Esta función, recibe como parámetro la respuesta por parte del servidor y, tras recuperar el `div #respuesta` y limpiar un poco la respuesta del servidor la muestra en pantalla.

Así es como quedará el código completo.

```
import 'dart:html';

void respuesta(r) {
  var div_resp = document.querySelector('#respuesta');
  var resp = r.responseText;
  resp = resp.replaceAll('{', '')
    .replaceAll('}', '')
    .replaceAll("['", "")
    .replaceAll("']", "")
    .replaceAll("'", "");
  resp = resp.split(",").join('<br/>');
  div_resp.setInnerHtml('<b>Status:</b> ${r.status} <br/>'
                    '<b>Respuesta:</b> <br/> ${resp}');
}

void enviarMensaje() {
  var resp = document.querySelector('#respuesta');
  Map data = new Map();
  data['nombre'] = (document.querySelector('#nombre') as InputElement).
value.trim();
   data['email'] = (document.querySelector('#email') as InputElement).
value.trim();
  data['asunto'] = (document.querySelector('#asunto') as InputElement).
value.trim();
  data['mensaje'] = (document.querySelector('#mensaje') as TextAreaEle-
ment).value.trim();

  var url = 'http://127.0.0.1:3031';
  var xhr = HttpRequest.postFormData(url, data,
      onProgress: (_) => resp.text = 'Enviando ... ');
```

```
 xhr
    ..then((r) => respuesta(r))
    ..catchError((error) => resp.text = 'ERROR !: ${error.toString()}');
}

void form() {
  var resp = document.querySelector('#form');
  var nombre = new InputElement()
    ..id = 'nombre'
    ..size = 45
    ..placeholder = 'Introduzca su nombre';
  var email = new EmailInputElement()
    ..id = 'email'
    ..size = 45

    ..placeholder = 'Introduzca su email';
  var asunto = new TextInputElement()
    ..id = 'asunto'
    ..size = 45
    ..placeholder = '¿Cuál es el motivo de su consulta?';
  var mensaje = new TextAreaElement()
    ..id = 'mensaje'
    ..cols = 40
    ..rows = 10
    ..placeholder = '¿En qué podemos ayudarte?';
  var enviar = new ButtonElement()
    ..id = 'enviar'
    ..text = ' Enviar mensaje ! '
    ..onClick.listen((e) => enviarMensaje());

  resp.nodes.clear();
  resp.nodes.addAll([nombre, new BRElement(),
                     email, new BRElement(),
                     asunto, new BRElement(),
                     mensaje, new BRElement(),
                     enviar]);
```

```
  nombre.focus();
}

void main() {
  var buttons = document.querySelector('#buttons');
  var btn1 = new ButtonElement()
    ..id = 'btn1'
    ..text = 'Contacto'
    ..onClick.listen((e) => form());
  buttons.nodes.add(btn1);
}
```

Si abres ahora Dartium y vas a la dirección `http://127.0.0.1:3031/http_request.html` podrás ver que carga nuestra aplicación para poder probar las peticiones POST contra el servidor Python que hemos creado.

Recuerda que el servidor deberá estar corriendo para que las peticiones POST funcionen.



Figura 87. Carga inicial de nuestra aplicación.

Pulsa en el botón `Contacto` y podrás ver cómo aparece el formulario que utilizaremos para rellenar y enviar al servidor.

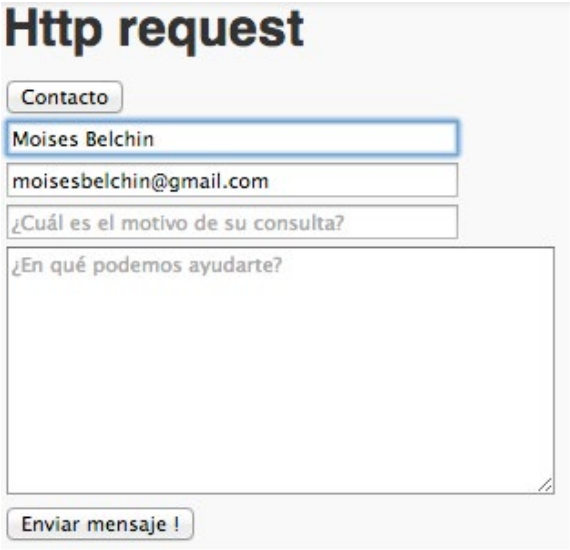


Figura 88. Rellenando el formulario de contacto para enviar al servidor.

Si rellenas los campos y pulsas en el botón `"Enviar mensaje !"` podrás ver como nos responde correctamente el servidor y vemos la respuesta en el `div #respuesta`.



Figura 89. Envío del formulario de contacto y respuesta por parte del servidor.

Uso de JSON en Dart

Hemos visto dos métodos que usarás con mucha frecuencia cuando quieras recibir datos desde un servidor o cuando necesites enviar datos codificados como `Form Data`.

Hablábamos al comienzo de este capítulo que Dart dispone del paquete `dart:convert` para poder trabajar con JSON y utilizarlo como formato ligero de intercambio de datos entre cliente y servidor.

Vamos ver como hacer uso de este paquete y en esta ocasión, para nuestros ejemplos, utilizaremos la clase `HttpRequest` para establecer las peticiones asíncronas.

`HttpRequest.request()`, al igual que los métodos vistos, `HttpRequest.getString()` y `HttpRequest.postFormData()` son métodos envoltura que se basan en la clase `HttpRequest`. Esta clase es la base para manejar las peticiones asíncronas de la que veremos un ejemplo completo de uso junto con JSON.

Vamos a crear una sencilla aplicación que realice una petición al servidor y obtenga unos datos en formato JSON para poder trabajar con ellos desde Dart.

Vamos a comenzar con lo que será nuestro fichero de estilos CSS `http _ request.css`.

```
body {
  background-color: #F8F8F8;
  font-family: 'Open Sans', sans-serif;
  font-size: 14px;
  font-weight: normal;
  line-height: 1.2em;
  margin: 15px;
}

h1, p {
  color: #333;
}
```

```
#respuesta {
  width: 600px;
  height: 200px;
  position: relative;
  border: 1px solid #ccc;
  background-color: #fff;
  text-align: left;
  padding-left: 5px;
  padding-top: 5px;
  float:right;
}

TH {
  background: #cccccc;
  text-align: center;
}
```

Definimos las clases para el `div #respuesta` que contendrá el resultado de nuestra petición y definimos los estilos de las cabeceras de las tablas para mostrar nuestros resultados de un modo más vistoso.

Vamos a ver que estructura HTML definimos para este ejemplo en nuestro fichero `http _ request.html`.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Http request</title>
    <link rel="stylesheet" href="http _ request.css">
  </head>
  <body>
    <h1>Http request</h1>
    <div id="buttons"></div>
```

```
    <div id="respuesta"></div>
    <script type="application/dart" src="http _ request.dart"></script>
    <script src="packages/browser/dart.js"></script>
  </body>
</html>
```

Hemos creado dos divs, uno donde almacenar los botones de nuestra aplicación y otro para la respuesta.

Vamos a crear un fichero llamado `clientes.json` que contendrá un listado con los clientes de nuestro sistema en formato JSON.

```
{"clientes": [
  {"id": "0001",
   "nombre": "Pedro",
   "apellidos": "Martinez Garcia",
   "direccion": {
      "calle": "Real",
      "numero": 2,
      "cp": 28001,
      "localidad": "Madrid",
      "provincia": "Madrid"}
  },

  {"id": "0002",
   "nombre": "Raquel",
   "apellidos": "Tomelloso Serrano",
   "direccion": {
      "calle": "Almadraba",
      "numero": 12,
      "cp": 46701,
      "localidad": "Gandia",
      "provincia": "Valencia"}
  },
```

```
    {"id": "0003",
     "nombre": "Maria del Carmen",
     "apellidos": "Jimenez Rodriguez",
     "direccion": {
         "calle": "Olmo",
         "numero": 23,
         "cp": 27003,
         "localidad": "Antas de Ulla",
         "provincia": "Lugo"}
    },

    {"id": "0004",
     "nombre": "Javier",
     "apellidos": "Domenec Perez",
     "direccion": {
         "calle": "Isabel Catolica",
         "numero": 8,
         "cp": 30022,
         "localidad": "Jumilla",
         "provincia": "Murcia"}
    }
]}
```

Como puedes ver definimos un objeto principal con la entrada clientes que será un listado de objetos. Los objetos de la lista definen cada cliente de nuestro sistema con los atributos id, nombre, apellidos y dirección. La dirección a su vez es otro objeto con los atributos calle, número, codigo postal, localidad y provincia.

Por último, veamos cómo hemos diseñado nuestro fichero `http _ request.dart` y como hemos estructurado nuestro código.

Para este ejemplo utilizaremos dos librerías para manejo de HTML y JSON. Y nuestro método `main()` crea un botón que al ser pulsado lanzará una petición al servidor.

```
import 'dart:html';
import 'dart:convert' show JSON;

void main() {
  var buttons = document.querySelector('#buttons');
  var btn1 = new ButtonElement()
    ..id = 'btn1'
    ..text = 'Ver clientes'
    ..onClick.listen((e) => getClientes());
  buttons.nodes.add(btn1);
}
```

La función `getClientes()` es la que crea el objeto `HttpRequest` y hace la petición para obtener el listado de clientes.

```
void getClientes() {
  var url = 'http://127.0.0.1:3030/http _ request/web/clientes.json';
  var xhr = new HttpRequest();
  xhr.onLoad.listen((e) {
    if ((xhr.status >= 200 && xhr.status < 300) ||
        xhr.status == 0 || xhr.status == 304) {
      listarClientes(xhr);
    } else {
      error(xhr.status);
    }
  });
  xhr.onError.listen((e) => error(e));
  xhr.onProgress.listen((_) => cargando());
  xhr.open('GET', url);
  xhr.send();
}
```

Crea un nuevo objeto `HttpRequest` con su constructor, vincula acciones a los diferentes métodos. Mientras la petición esté en progreso (`onProgress`) ejecutará el método cargando() que se ocupa de mostrar el mensaje `'Cargando...'` en el `div #respueta`. En caso de error lanzará el método `error()` encargado de mostrar el mensaje de error devuelto por el sistema en el `div #respuesta`.

El evento `onLoad` se invoca cuando la petición se ha completado. Hacemos una validación del estado de la petición y si ha finalizado correctamente ejecuta la función `listarClientes()`, en caso de que se haya completado, pero tenga un estado inadecuado volvemos a lanzar el método `error()` en esta ocasión con el estado devuelto por el sistema.

Estos son los métodos para los evento `onProgress` y `onError`.

```
void error(e) {
  var div_resp = document.querySelector('#respuesta');
  div_resp.setInnerHtml('Error realizando peticion: $e');
}

void cargando() {
  var div_resp = document.querySelector('#respuesta');
  div_resp.setInnerHtml('Cargando...');
}
```

Y esta es la función `listarClientes()` que, una vez obtenida la respuesta por parte del servidor, genera un elemento HTML tabla con los datos obtenidos.

Como el servidor nos devuelve los datos en formato JSON con el método `JSON.decode()` procesamos la respuesta y obtenemos un `Map` que podemos utilizar para generar nuestra tabla.

Adicionalmente hemos utilizado la clase `StringBuffer` para componer la plantilla HTML que finalmente mostraremos en nuestro `div #respuesta`.

```
void listarClientes(HttpRequest xhr) {
  var div_resp = document.querySelector('#respuesta');
  StringBuffer tpl = new StringBuffer('''<table border="1">
    <tr>
      <th colspan="3"></th>
      <th colspan="5"> Dirección </th>
    <tr>
      <th>ID</th>
      <th>Nombre</th>
      <th>Apellidos</th>
      <th>Calle</th>
      <th>Numero</th>
      <th>CP</th>
      <th>Localidad</th>
      <th>Provincia</th>
    </tr>''');
  Map datos = JSON.decode(xhr.responseText);
  datos['clientes'].forEach((Map c) {
    tpl.write('''
    <tr>
      <td>${c['id']}</td>
      <td>${c['nombre']}</td>
      <td>${c['apellidos']}</td>
      <td>${c['direccion']['calle']}</td>
      <td>${c['direccion']['numero']}</td>
      <td>${c['direccion']['cp']}</td>
      <td>${c['direccion']['localidad']}</td>
      <td>${c['direccion']['provincia']}</td>
    </tr>''');
  });
  tpl.write('</table>');
  div_resp.setInnerHtml(tpl.toString());
}
```

Este será nuestro fichero http_request.dart completo.

```
import 'dart:html';
import 'dart:convert' show JSON;

void listarClientes(HttpRequest xhr) {
  var div_resp = document.querySelector('#respuesta');
  StringBuffer tpl = new StringBuffer('''<table border="1">
    <tr>
      <th colspan="3"></th>
      <th colspan="5"> Dirección </th>
    <tr>
      <th>ID</th>
      <th>Nombre</th>
      <th>Apellidos</th>
      <th>Calle</th>
      <th>Numero</th>
      <th>CP</th>
      <th>Localidad</th>
      <th>Provincia</th>
    </tr>''');
  Map datos = JSON.decode(xhr.responseText);
  datos['clientes'].forEach((Map c) {
    tpl.write('''
    <tr>
      <td>${c['id']}</td>
      <td>${c['nombre']}</td>
      <td>${c['apellidos']}</td>
      <td>${c['direccion']['calle']}</td>
      <td>${c['direccion']['numero']}</td>
      <td>${c['direccion']['cp']}</td>
      <td>${c['direccion']['localidad']}</td>
      <td>${c['direccion']['provincia']}</td>
    </tr>''');
  });
  tpl.write('</table>');
  div_resp.setInnerHtml(tpl.toString());
}
```

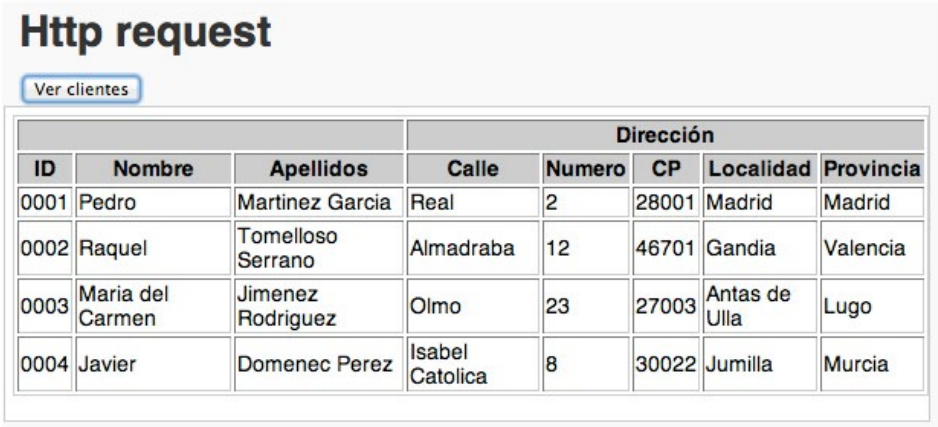
```
void error(e) {
  var div_resp = document.querySelector('#respuesta');
  div_resp.setInnerHtml('Error realizando peticion: $e');
}

void cargando() {
  var div_resp = document.querySelector('#respuesta');
  div_resp.setInnerHtml('Cargando...');
}

void getClientes() {
  var url = 'http://127.0.0.1:3030/http_request/web/clientes.json';
  var xhr = new HttpRequest();
  xhr.onLoad.listen((e) {
    if ((xhr.status >= 200 && xhr.status < 300) ||
        xhr.status == 0 || xhr.status == 304) {
      listarClientes(xhr);
    } else {
      error(e);
    }
  });
  xhr.onError.listen((e) => error(e));
  xhr.onProgress.listen((_) => cargando());
  xhr.open('GET', url);
  xhr.send();
}

void main() {
 var buttons = document.querySelector('#buttons');
 var btn1 = new ButtonElement()
    ..id = 'btn1'
    ..text = 'Ver clientes'
    ..onClick.listen((e) => getClientes());
  buttons.nodes.add(btn1);
}
```

Si ejecutas la aplicación podrás ver que, tras pulsar el botón `Ver clientes`, obtenemos los datos del servidor en formato JSON, los procesamos en el lado cliente y los mostramos correctamente formateados como una tabla en el `div#respuesta`.



			Dirección				
ID	Nombre	Apellidos	Calle	Numero	CP	Localidad	Provincia
0001	Pedro	Martinez Garcia	Real	2	28001	Madrid	Madrid
0002	Raquel	Tomelloso Serrano	Almadraba	12	46701	Gandia	Valencia
0003	Maria del Carmen	Jimenez Rodriguez	Olmo	23	27003	Antas de Ulla	Lugo
0004	Javier	Domenec Perez	Isabel Catolica	8	30022	Jumilla	Murcia

Figura 90. Respuesta obtenida del servidor en formato JSON y procesada como tabla HTML.

Como puedes imaginar la clase JSON de `dart:convert` también permite codificar objetos Dart como JSON para ser enviados y procesados por el servidor.

Imagina que disponemos de un objeto `Map` con los datos de acceso del usuario y su último acceso al sistema, podemos codificarlos como JSON y enviarlos al servidor de este modo.

```
void main() {

  Map datos = {
    'usuario': 'moisesbelchin@gmail.com',
    'nombre': 'Moises Belchin',
    'acceso': '2014-03-29 17:22'
  };

  var url = 'http://127.0.0.1:3030';
  var xhr = new HttpRequest();
```

```
    xhr.open('POST', url);
    xhr.setRequestHeader('Content-Type', 'application/json');
    xhr.send(JSON.encode(datos));
  }
```

Creamos un nuevo objeto `HttpRequest`, llamamos a su método `open()` indicando que vamos a realizar un `POST` y la URL de destino, añadimos la cabecera `Content-Type application/json` para indicar al servidor que lo que va a recibir es un mensaje JSON válido.

Por último, llamamos al método `send()` de `HttpRequest` enviando nuestro objeto `Map` con los datos codificados en formato JSON con `JSON.encode()`.

Como has podido ver `dart:convert` a través de JSON te permite gestionar perfectamente y de manera realmente sencilla el paso de mensajes entre cliente y servidor utilizando JSON con los método `decode()` y `encode()`.

8 DART EN EL LADO SERVIDOR

A lo largo del libro hemos visto cómo es Dart, las herramientas que pone a nuestra disposición para comenzar a trabajar y lo hemos comparado con lenguajes en el lado cliente como JavaScript y el framework jQuery.

Pero el desarrollo en el lado cliente no es lo único que te permite hacer este fantástico lenguaje. Dart también pone a tu disposición una librería completa para trabajar en el lado servidor.

La librería I/O de Dart está diseñada para desarrollar aplicaciones en el lado servidor que corren en una máquina virtual de Dart.

Esta librería no funciona en aplicaciones que corren en el navegador.

Nota:

Esta librería, llamada `dart:io` permite trabajar con ficheros, directorios, sockets, procesos y servidores HTTP entre otras opciones.

Si quieres comenzar a utilizar esta librería en tus proyectos tan solo debes realizar la importación del paquete correspondiente del SDK:

```
import 'dart:io';
```

Estos son todos los ficheros de código que encontrarás dentro del paquete `dart:io`

- `bytes_builder.dart`
- `common.dart`
- `crypto.dart`
- `data_transformer.dart`
- `directory.dart`
- `directory_impl.dart`
- `eventhandler.dart`
- `file.dart`

- file_impl.dart
- file_system_entity.dart
- http.dart
- http_date.dart
- http_headers.dart
- http_impl.dart
- http_parser.dart
- http_session.dart
- io_sink.dart
- io_service.dart
- link.dart
- platform.dart
- platform_impl.dart
- process.dart
- socket.dart
- stdio.dart
- string_transformer.dart
- timer_impl.dart
- secure_socket.dart
- secure_server_socket.dart
- websocket.dart
- websocket_impl.dart

El fichero `http.dart` pone a tu disposición las clases necesarias para crear servidores y clientes `HTTP`. `directory.dart` contiene la clase `Directory` para manejo y creación de directorios. `file.dart` contiene las clases necesarias para manejo de ficheros en el sistema.

`link.dart` contiene funcionalidad de manejo de enlaces simbólicos. Dentro de `platform.dart` encontrarás la clase `Platform` para acceder a todas las peculiaridades de la máquina y del sistema operativo.

`process.dart` contiene todas las clases y métodos para trabajar con procesos en el sistema. Si necesitas mecanismos para realizar comunicación en modo seguro a través de sockets TCP utilizando TLS y SSL dispones de todas las clases en los ficheros `secure _ server _ socket.dart` y `secure _ server.dart`.

También dispones del fichero `socket.dart` con todas clases necesarias para comunicación a través de TCP y manejo de sockets. `stdio.dart` te permite leer de la entrada estándar del sistema (`stdin`) y escribir tanto en la salida estándar (`stdout`) como en la salida estándar de errores (`stderr`).

Creando una aplicación servidor

A continuación vamos a ver cómo utilizar esta librería para crear aplicaciones en el lado servidor y utilizaremos algunos de los paquetes y clases que hemos comentado anteriormente para ver toda la funcionalidad que nos proporciona Dart.

Comenzaremos por un servidor web muy sencillo que te mostrará un mensaje de bienvenida cuando solicites un recurso.

Vamos a crear una nueva aplicación en Dart Editor: seleccionamos aplicación de línea de comandos (*Command-line application*) estableciendo un nombre para la aplicación.

```
import 'dart:io';
void main() {
  HttpServer.bind('127.0.0.1', 8080)
  .then((server) {
    server.listen((HttpRequest request) {
      // Manejamos las peticiones del servidor.
      request.response.write('Esto es Dart en el lado servidor !');
      request.response.close();
    });
  })
  .catchError((e) => print('Error :: $e'));
}
```

Si añadimos este código a nuestro proyecto y lo ejecutamos, podremos apuntar nuestro navegador web a la dirección `http://127.0.0.1:8080` y obtener un mensaje de respuesta por parte de tu nuevo servidor web escrito en Dart.

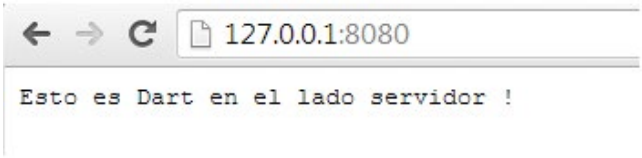


Figura 91. Primer mensaje de bienvenida de tu nuevo servidor web escrito en Dart.

No está nada mal, con tan solo 13 líneas ya tenemos totalmente operativo un servidor web con gestión de errores. Vamos a ir añadiendo más funcionalidades a nuestro servidor para hacerlo más interesante.

Creamos un directorio llamado web dentro de este nuevo proyecto y vamos a hacer que sea el directorio raíz de nuestro nuevo servidor.

Además, creamos el fichero principal de nuestro nuevo servidor llamado `index.html`. También podemos añadir algunos directorios adicionales para almacenar imágenes y hojas de estilo CSS.

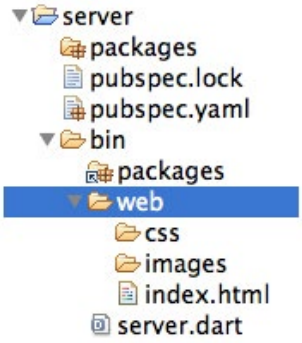


Figura 92. Nuevo directorio web con los ficheros y directorios de nuestro servidor.

```
import 'dart:io';

void main() {
  HttpServer.bind('127.0.0.1', 8080)
  .then((server) {
    server.listen((HttpRequest request) {
      // Manejamos las peticiones del servidor.
      switch (request.uri.path) {
        case '/':
        default:
          request.response.write('<h1>Indice de /web/</h1><ul>');
          new Directory('web/').listSync().forEach((FileSystemEntity
                                                    entity) {
            var df = entity.path.split('web/').last;
            request.response.write('<li><a href="web/${df}">${df}</a>
                                    </li>');
          });
          request.response.write('</ul>');
          request.response.close();
      }
    });
  })
  .catchError((e) => print('Error :: $e'));
}
```

Si dirigimos ahora nuestro navegador a la dirección `http://127.0.0.1:8080` veremos como aparecen listados los directorios y ficheros del directorio web que hemos definido como raíz de nuestro servidor web.



Figura 93. Listado del directorio raíz del servidor

Con esto, además has visto cómo abrir y listar el contenido de un directorio utilizando la clase `Directory` y el método `listSync()`.

Como habrás podido observar `dart:io` al igual que muchas partes del SDK está diseñado para funcionar de manera totalmente asíncrona y todos los métodos retornan objetos `Future` o `Stream` para implementar esta forma de trabajo.

No obstante muchas clases dentro de `dart:io` tienen métodos que realizan ciertas tareas en modo síncrono como por ejemplo el que hemos visto aquí `listSync()`.

Vamos a ir ahora un paso más adelante para que nuestro servidor web nos permita navegar dentro de los directorios y ver su contenido.

```
import 'dart:io';

void main() {
  HttpServer.bind('127.0.0.1', 8080)
  .then((server) {
    server.listen((HttpRequest request) {
      // Manejamos las peticiones del servidor.
      switch (request.uri.path) {
        default:
          request.response.write('<h1>Indice de /web${request.uri.path}
                                  </h1><ul>');

          if(FileStat.statSync('web${request.uri.path}').type ==
              FileSystemEntityType.DIRECTORY) {

            var dir = new Directory('web${request.uri.path}');

            if (request.uri.path != '/') {
              request.response.write('<li><a href="/">..</a></li>');
            }
```

```
            dir.listSync().forEach((FileSystemEntity entity) {
              var df = entity.path.split('web${request.uri.path}').last;
             request.response.write('<li><a href="${df}">${df}</a></li>');
            });
          }

          request.response.write('</ul>');
          request.response.close();
        }
      });
    })
    .catchError((e) => print('Error :: $e'));
}
```

Con este sencillo código ahora podemos visualizar el contenido de nuestro directorio raíz, y además navegar por los directorios del mismo listando el contenido de cada uno de ellos.

En esta ocasión hemos utilizado las clases `FileStat` y `FileSystemEntityType` para saber cuando estamos abriendo un directorio con el método `statSync()` y listar su contenido.



Figura 94. Listando el contenido del directorio raíz.

Hemos añadido un link de retorno al directorio padre para facilitar la navegación.



Figura 95. Navegando por los directorios de nuestro servidor web.

Nuestro servidor web va tomando cuerpo, vamos a permitir al usuario ahora que pueda visualizar el contenido de los ficheros hospedados dentro de nuestro servidor web Dart.

Añade contenido al fichero `main.css` y al fichero `index.html` para poder abrirlos desde nuestro servidor web y ver algo de contenido.

Para `main.css` hemos creado este sencillo contenido:

```
/**
 * CSS principal de la web
 */
BODY {
  background-color: #f4f4f4;
  color: #5a5656;
  font-family: 'Open Sans', Arial, Helvetica, sans-serif;
  font-size: 16px;
  line-height: 1.5em;
}
```

Para el fichero `index.html` podemos añadir algo así:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Index</title>
    <link rel="stylesheet" href="css/main.css">
  </head>
  <body>
    <h1>Bienvenido a Dart en el lado Servidor !!</h1>
  </body>
</html>
```

Nuestro servidor web ahora tendrá este código.

```
import 'dart:io';

void main() {
  HttpServer.bind('127.0.0.1', 8080)
  .then((server) {
    server.listen((HttpRequest request) {
     // Manejamos las peticiones del servidor.
     switch(FileStat.statSync('web${request.uri.path}').type) {
      case FileSystemEntityType.DIRECTORY:
        request.response.write('<h1>Indice de /web${request.uri.path}
                                </h1><ul>');
        var dir = new Directory('web${request.uri.path}');
        if (request.uri.path != '/') {
          request.response.write('<li><a href="/">..</a></li>');
        }
        dir.listSync().forEach((FileSystemEntity entity) {
         var df = entity.path.split('web${request.uri.path}').last;
         request.response.write('''<li><a href="${request.uri.path}${df}">
                                   ${df}</a></li>''');
        });
```

```
            request.response.write('</ul>');
            request.response.close();
            break;

          case FileSystemEntityType.FILE:
            var fich = new File('web${request.uri.path}');
            request.response.write(fich.readAsStringSync());
            request.response.close();
            break;
        }
      });
    })
    .catchError((e) => print('Error :: $e'));
}
```

Ahora no solo puedes navegar por los diferentes directorios del servidor web sino que también puedes seleccionar un fichero y ver su contenido en tu navegador.



Figura 96. Visualizando el contenido del fichero main.css



Figura 97. Visualizando el contenido del fichero index.html en nuestro servidor

Como puedes observar al visualizar el fichero index.html el navegador web ya lo interpreta y lo renderiza como un fichero HTML interpretando su contenido.

Trabajando con procesos

Ahora veremos cómo ejecutar procesos desde nuestras aplicaciones en el lado servidor.

Continuando con el ejemplo de nuestro servidor web vamos a crear un manejador específico para ejecutar cierta función del sistema ejecutando un proceso concreto utilizando la clase `Process`.

El manejador que vamos a crear es `/ps`, de este modo cuando el usuario ponga en la barra de direcciones del navegador esta dirección `http://127.0.0.1:8080/ps` vamos a mostrar los procesos que hay corriendo en nuestro sistema.

Así quedará nuestro nuevo servidor web:

```
import 'dart:io';

void main() {
  HttpServer.bind('127.0.0.1', 8080)
  .then((server) {
    server.listen((HttpRequest request) {
      // Manejamos las peticiones del servidor.
      switch(request.uri.path) {
        case '/ps':
          request.response.write('''
          <html>
            <head><title>Procesos de sistema</title></head>
            <body>
            <h2>Procesos del sistema</h2>
          ''');
          var proceso = Process.runSync('ps', []);
          request.response.write('<pre>');
```

```
      if(proceso.exitCode == 0) {
        request.response.write(proceso.stdout);
      } else {
        request.response.write(proceso.stderr);
      }
      request.response.write('</pre>');
      request.response.write('</body></hmtl>');
      request.response.close();

      break;

    default:
      switch(FileStat.statSync('web${request.uri.path}').type) {
        case FileSystemEntityType.DIRECTORY:
          request.response.write('<h1>Indice de /web$
                                    {request.uri.path}</h1><ul>');
          var dir = new Directory('web${request.uri.path}');
          if (request.uri.path != '/') {
            request.response.write('<li><a href="/">..</a></li>');
          }
          dir.listSync().forEach((FileSystemEntity entity) {
            var df = entity.path.split('web${request.uri.path}').last;
            request.response.write('''<li><a href="$
                          {request.uri.path}${df}"> ${df}</a></li>''');
          });
          request.response.write('</ul>');
          request.response.close();
          break;

        case FileSystemEntityType.FILE:
          var fich = new File('web${request.uri.path}');
          request.response.write(fich.readAsStringSync());
          request.response.close();
          break;
```

```
            case FileSystemEntityType.NOT_FOUND:
              request.response.write('No encontrado');
              request.response.close();
              break;
          }
        }
      });
    })
    .catchError((e) => print('Error :: $e'));
}
```

Hemos añadido un nuevo bloque para controlar manejadores específicos, de tal modo que si nos llega uno de los manejadores que controlamos ejecutamos la función concreta y en caso contrario seguimos trabajando con ficheros y directorios como hacía hasta ahora nuestro navegador web.

Dentro de este manejador concreto ejecutamos un proceso con el método `runSync()` de la clase `Process` y controlamos si el proceso termina o no correctamente para mostrar la salida estándar `(stdout)` o la salida de errores `(stderr)`.

```
var proceso = Process.runSync('ps', []);
if(proceso.exitCode == 0) {
  request.response.write(proceso.stdout);
} else {
  request.response.write(proceso.stderr);
}
```

Cuando quieras ejecutar procesos puedes también pasar parámetros al método `.run()` o `.runSync()` como una lista. En el ejemplo anterior no hemos añadido ningún parámetro para el proceso `ps`.



Figura 98. Ejecutando ps en nuestro servidor web.

Vamos a mejorar un poco nuestro servidor web para que nos permita pasar parámetros a nuestro manejador `/ps`.

Pasaremos los parámetros por queryString en la URL de este modo:

```
http://127.0.0.1:8080/ps?p=aW
```

En el servidor web recuperaremos los parámetros introducidos por el usuario y los pasaremos al comando `ps` para ejecutar el proceso con los parámetros seleccionados.

Añadiendo este bloque de código y cambiando la forma de llamar a `Process.runSync()`.

```
var parametros = [];
if (request.uri.queryParameters.keys.length > 0) {
  if (request.uri.queryParameters.containsKey('p')) {
    parametros.add('-${request.uri.queryParameters['p']}');
  }
}
var proceso = Process.runSync('ps', parametros);
```

Podemos ahora visualizar todos los procesos de nuestro sistema.

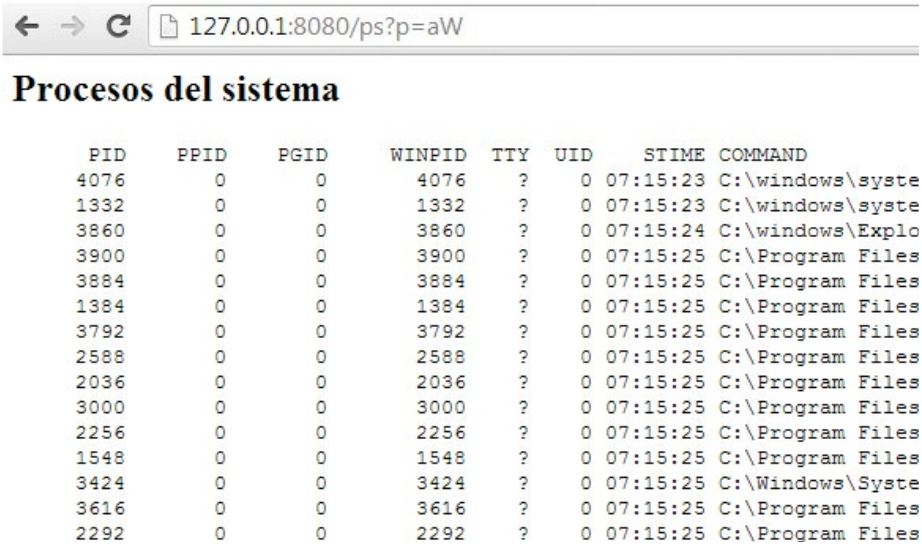


Figura 99. Ejecutando todos los procesos de nuestro sistema pasando parámetros.

Así quedará ahora nuestro servidor web con soporte de parámetros en el manejador /ps.

```
import 'dart:io';

void main() {
  HttpServer.bind('127.0.0.1', 8080)
  .then((server) {
    server.listen((HttpRequest request) {
      // Manejamos las peticiones del servidor.
      switch(request.uri.path) {
        case '/ps':
          request.response.write('''
          <html>
            <head><title>Procesos de sistema</title></head>
            <body>
            <h2>Procesos del sistema</h2>
          ''');
```

```
      var parametros = [];
      if (request.uri.queryParameters.keys.length > 0) {
        if (request.uri.queryParameters.containsKey('p')) {
          parametros.add('-${request.uri.queryParameters['p']}');
        }
      }
      var proceso = Process.runSync('ps', parametros);
      request.response.write('<pre>');
      if(proceso.exitCode == 0) {
        request.response.write(proceso.stdout);
      } else {
        request.response.write(proceso.stderr);
      }
      request.response.write('</pre>');
      request.response.write('</body></hmtl>');
      request.response.close();
      break;

    default:
      switch(FileStat.statSync('web${request.uri.path}').type) {
        case FileSystemEntityType.DIRECTORY:
          request.response.write('<h1>Indice de /web$
                                  {request.uri.path}</h1><ul>');
          var dir = new Directory('web${request.uri.path}');
          if (request.uri.path != '/') {
            request.response.write('<li><a href="/">..</a></li>');
          }
          dir.listSync().forEach((FileSystemEntity entity) {
            var df = entity.path.split('web${request.uri.path}').last;
            request.response.write('''<li><a href="$
                                      {request.uri.path}${df}">
              ${df}</a></li>''');
          });
          request.response.write('</ul>');
          request.response.close();
          break;
```

```
            case FileSystemEntityType.FILE:
              var fich = new File('web${request.uri.path}');
              request.response.write(fich.readAsStringSync());
              request.response.close();
              break;

            case FileSystemEntityType.NOT_FOUND:
              request.response.write('No encontrado');
              request.response.close();
              break;
          }
      }
    });
  })
  .catchError((e) => print('Error :: $e'));
}
```

Sockets

Además de todo lo visto hasta ahora, `dart:io` también te permite trabajar con `sockets` en aplicaciones de línea de comandos y `websockets` en aplicaciones navegador para establecer comunicación entre cliente y servidor via TCP.

Veamos un sencillo ejemplo de servidor y cliente utilizando `sockets` para comunicarse.

`server_socket.dart` será el encargado de crear el servidor `socket` y quedar a la escucha de los mensajes de los clientes.

```
import 'dart:io';
import 'dart:convert';

void main() {
  ServerSocket.bind('127.0.0.1', 4041)
  .then((serverSocket) {
    serverSocket.listen((socket) {
      socket.transform(UTF8.decoder).listen(print);
   });
  });
  stdout.writeln('Esperando mensajes desde los clientes...');
}
```

`cliente_socket.dart` es el cliente que establece conexión con el servidor en la dirección y puertos especificados y lanza un mensaje al servidor.

```
import 'dart:io';

void main() {
  Socket.connect('127.0.0.1', 4041).then((socket) {
    socket.write('saludos server socket !');
  });
}
```

Arranca en primer lugar `server_socket.dart` y posteriormente `cliente_socket.dart`. El cliente se conecta al socket del servidor y le envía un mensaje. El servidor recibe el mensaje y lo muestra en pantalla.

server_socket.dart client_socket.dart

```dart
import 'dart:io';

void main() {
  Socket.connect('127.0.0.1', 4041).then((socket) {
    socket.write('saludos server socket !');
  });
}
```

server_socket.dart

```
Esperando mensajes desde los clientes...
saludos server socket !
saludos server socket !
saludos server socket !
```

Figura 100. Comunicando cliente y servidor a través de Sockets.

9 POLYMER *WEB* COMPONENTS

Los *Web Components* son, sin duda alguna, la última moda y de lo que todos hablan. Es lógico, ya que permiten llevar el desarrollo web a un nuevo nivel.

Por un lado, permiten utilizar el patrón MVC (Modelo-Vista-Controlador) de un modo más natural y coherente y por otro lado permiten encapsular y reutilizar.

Si has trabajado con .NET o Java resulta realmente sencillo coger ciertas partes o componentes concretos de la interfaz de una aplicación y reutilizarlos para otra, incluso crear uno nuevo componente heredando de otro existente.

Con los componentes web es precisamente lo que se pretende conseguir en el desarrollo web. Todo esto es posible gracias al gran abanico de posibilidades que presenta HTML con las nuevas tecnologías que incorpora en su última versión.

- **DOM mutation observers**. Observadores que nos informan de cambios en los elementos del DOM.
- **Pointer events**. Para manejar el ratón y los dispositivos táctiles de la misma forma en todas las plataformas existentes.
- **Shadow DOM**. Para encapsular estructura y estilo dentro de los elementos.
- **Custom Elements**. Para definir tus propios elementos HTML5. Generalmente se utiliza un guión medio para los nombres de los elementos que definas lo que es una forma de hacer *namespacing* y diferenciarlos de los elementos estándar de HTML.
- **HTML Imports**. Una forma de empaquetar elementos propios. Estos paquetes pueden contener HTML, CSS y JavaScript.
- **Model-Driven Views (MDV)**. Permite vincular interfaz y lógica (data binding) directamente en HTML.
- **Web Animations**. Un API que unifica las diferentes técnicas de animación para web.

En este apartado vamos a mostrarte un ejemplo de creación y utilización de **Polymer.dart** el framework para poder trabajar con componentes web desde Dart.

Polymer.dart

Polymer.dart pone a tu disposición la librería Polymer (http://www.polymer-project.org/) para que desde Dart puedas utilizar todas estas últimas tecnologías web y crear tus propios componentes HTML.

Podrás crear componentes web que podrás utilizar en tus documentos web como si fuesen etiquetas HTML convencionales.

```
<login-usuario></login-usuario>
```

Si has trabajado en proyectos web de gran envergadura y has necesitado incluir algún framework o librería JavaScript u hoja de estilos, surge un gran problema: la colisión de nombres.

Tus CSS tienen definiciones con el mismo nombre que las CSS de alguna de las librerías que has incorporado a tu proyecto.

Para solucionar esto y para añadir otra serie de ventajas aparecen los componentes web y más concretamente Polymer.dart.

- Encapsular
- Reutilizar
- Vincular interfaz y lógica (Data Binding)

Podrás encapsular la estructura, el estilo y el comportamiento de un determinado elemento y reutilizar este elemento en otros documentos HTML. Además, puedes tener separada la representación gráfica del elemento de la lógica de negocio, pero con los datos perfectamente enlazados para que fluyan de un lado a otro, es decir, un cambio en la interfaz se refleja en tu modelo de datos y viceversa.

Polymer hace uso de muchas de las tecnologías web que hemos comentado al comienzo de la sección, *DOM mutation observers, HTML Imports, Model-Driven Views, Custom Elements*, pero especialmente de *Shadow DOM* para poder crear nuevos componentes web.

Esta tecnología permite aislarlos del resto del documento y de otros componentes para evitar colisiones de nombres, así como para encapsular su estructura y estilo.

¿Qué es Shadow DOM?

Shadow DOM hace referencia a la posibilidad que tiene el navegador web de crear e incluir un nuevo árbol de nodos DOM dentro del documento renderizado, pero no en el árbol DOM principal, sino que lo hace como una especie de sub-árbol.

Un nuevo elemento creado con Shadow DOM recibe el nombre de Shadow Host y el nodo principal del elemento del que colgarán el resto se conoce como *Shadow Root*.

Ya es suficiente teoría por el momento, vamos a ver esto en acción en Google Chrome por ejemplo. Si apuntas tu navegador web a http://jsfiddle.net/ podrás ver como tu pantalla se divide en 4 áreas donde puedes crear HTML, CSS, JavaScript y obtener el resultado de ejecutar todo el código.

Es una buena herramienta para realizar pequeñas pruebas de scripts o estilos CSS. Para este ejemplo nos sirve perfectamente.

Crea un elemento `input` de HTML5 con `type="date"` y a continuación pulsa en ***Run***.

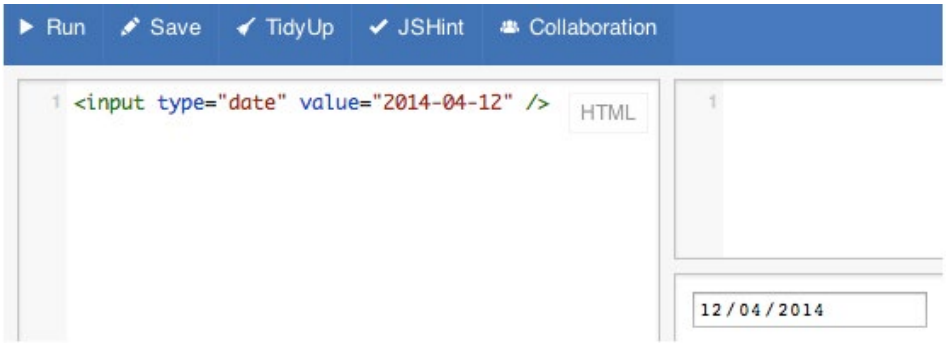


Figura 101. Elemento *input* date corriendo en *jsfiddle.net*

En el ejemplo hemos añadido una fecha como valor por defecto y puedes ver como en el recuadro de resultado aparece un *input* con la fecha establecida.

Si pulsas en el *input* que aparece en la ventana de resultado verás como para cambiar la fecha aparece un calendario que te permite navegar entre días, semanas, meses y años para que la selección de la fecha sea más sencillo para los usuarios.



Figura 102. Navegando por el calendario

Hemos creado un elemento *input* estándar de HTML, pero Google Chrome nos muestra ese magnífico calendario. ¿Cómo es posible? ¿Se trata de magia? Es Shadow DOW en acción. Los ingenieros de Google han hecho que el navegador detecte cuando creamos un elemento input con tipo fecha y automáticamente carga su propio elemento calendario.

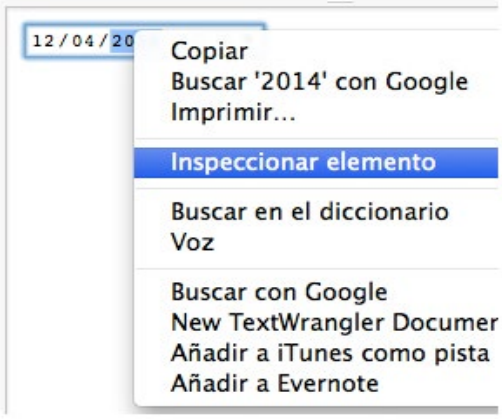


Figura 103. Inspecciona el elemento con DevTools de Google Chrome.

Si te posicionas sobre el elemento creado en `jsfiddle.net` y pulsas con el botón derecho del ratón, puedes seleccionar "inspeccionar elemento".

Aparecerán unos paneles especiales que muestran toda la información del documento renderizado y del elemento calendario. Selecciona la rueda dentada y marca la casilla *Elements > Show Shadow DOM*.

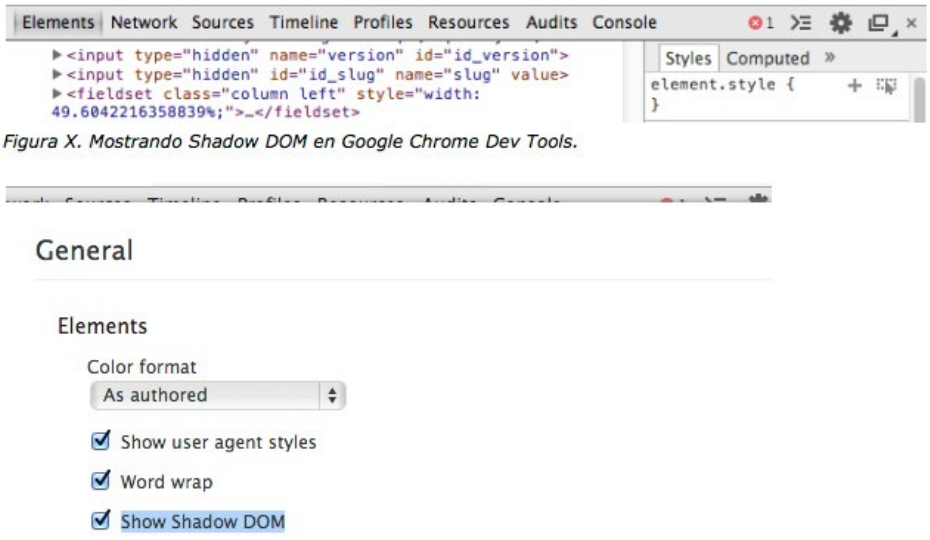


Figura X. Mostrando Shadow DOM en Google Chrome Dev Tools.

Figura 104. Mostrando Shadow DOM en Google Chrome Dev Tools.

Si activas esta opción en las herramientas de desarrollo de Google Chrome, podrás ver como aparecen opciones fantásticas que antes no veíamos al inspeccionar el elemento:

```
▼<input type="date" value="2014-04-12">
  ▼#shadow-root (user-agent)
    ▶<div pseudo="-webkit-datetime-edit" id="date-time-edit">…</div>
     <div pseudo="-webkit-clear-button" id="clear"></div>
     <div pseudo="-webkit-inner-spin-button" id="spin"></div>
     <div pseudo="-webkit-calendar-picker-indicator" id="picker"></div>
</input>
```

Figura 105. Inspeccionando nuestro input element con Shadow DOM habilitado.

Como puedes ver lo que antes era tan solo un elemento input de HTML ahora muestra todo su contenido. Una serie de elementos DIV que componen el fantástico calendario selector de fechas con su *shadow root*.

Trabajando con Polymer.dart

Ahora que ya hemos visto en detalle la magia de Shadow DOM y que hemos comentado lo que podemos hacer con las nuevas tecnologías que incorpora HTML5 llega el turno de poner esto en marcha y desarrollar tu primer elemento con Dart utilizando Polymer.

Hemos comentado que gracias a Polymer.dart podremos crear nuestros propios componentes web y utilizarlos en nuestros documentos HTML muy fácilmente.

Vamos a preparar un sencillo ejemplo que represente perfectamente esta idea y te permita ver como queda separado por un lado su estructura, qué etiquetas lo componen, por otro lado su estilo, todas las definiciones CSS que le darán formato, y por último, el código Dart que le proporcionará funcionalidad, gestionando eventos y con la vinculación de datos entre interfaz y lógica.

Vamos a crear un componente para gestionar todo el sistema de acceso de un usuario a nuestro sistema. Este componente estará compuesto por un lado de un fichero HTML que definirá su estructura. Por otro lado tendremos la definición CSS que conformará el estilo de nuestro sistema de login.

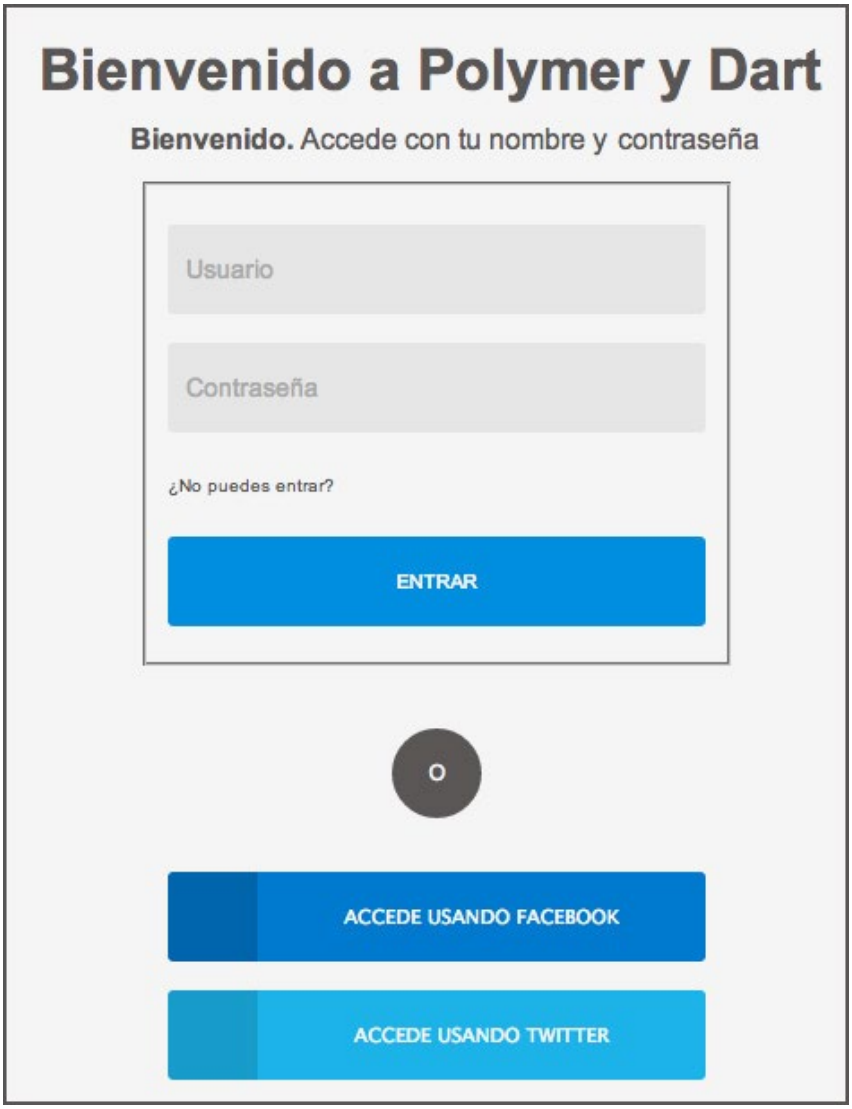


Figura 106. Ejemplo de cómo quedará nuestro polymer element para gestionar el acceso de usuarios.

Este será el código HTML de nuestro elemento.

```
<polymer-element name="login-usuario">
  <meta charset="utf-8">
  <template>
    <link rel="stylesheet" href="login.css">
    <div id="login">
      <h1><strong>Bienvenido.</strong> <span>Accede con tu nombre
                y contraseña</span></h1>

       <template if="{{error == true}}">
        <div id="login _ msg">{{login _ error}}</div>
       </template>

       <fieldset>
        <p><input type="text" required id="usuario" name="usuario"
            placeholder="Usuario"></p>
        <p><input type="password" required id="password"
            name="password" placeHolder="Contraseña"></p>
        <p><a href="#" on-click="{{recordarPassword}}">¿No puedes
            entrar?</a></p>
        <p><button on-click="{{entrar}}">Entrar</button></p>
       </fieldset>

      <p><span class="btn-round">o</span></p>
      <p>
       <a class="facebook-before"><span class="fontawesome-facebook">
          </span></a>
       <button class="facebook" on-click="{{facebookLogin}}">Accede
           Usando Facebook</button>
      </p>
```

```
        <p>
         <a class="twitter-before"><span class="fontawesome-twitter">
                                    </span></a>
<button class="twitter" on-click="{{twitterLogin}}">Accede Usando
 Twitter</button>
        </p>
       </div>
  </template>
  <script type="application/dart" src="login.dart"></script>
</polymer-element>
```

Como puedes observar todo comienza con unas etiquetas especiales que definen el elemento que estamos creando y el nombre del elemento, para posteriormente importarlo y utilizarlo.

Empezamos utilizando las etiquetas `<template>` para definir la plantilla base de nuestro elemento. Aquí dentro definimos todo lo que conformará nuestro elemento.

Al comienzo establecemos la referencia a la hoja de estilos que dará color y estilo a nuestro elemento. Al final de nuestra definición establecemos el enlace al fichero de código Dart que dotará de funcionalidad a nuestro sistema de acceso.

Dentro de nuestro elemento podemos referenciar variables de nuestra aplicación Dart con la notación `{{variable}}`. También podemos establecer acciones vinculando eventos en la interfaz con métodos de nuestro código Dart utilizando esta misma nomenclatura e indicando el evento que lo ejecutará.

```
<button on-click="{{entrar}}">Entrar</button>
```

Al hacer click en el botón Entrar ejecutamos el método entrar de nuestra aplicación Dart que veremos a continuación.

El sistema de plantillas de polymer nos permite iterar en nuestra estructura HTML y establecer condiciones como puedes ver en este bloque de código.

```
<template if="{{error == true}}">
  <div id="login _ msg">{{login _ error}}</div>
</template>
```

Si en el controlador Dart encontramos algún error definimos nuestra variable error con el valor `true` y con el sistema de plantillas podemos establecer una condición para que se muestre un mensaje concreto con la descripción del error en el sistema de acceso de usuarios.

Una vez definida la estructura de nuestro componente y diseñadas las acciones llega el turno de definir el estilo que acompañará a nuestro sistema de acceso. Este será el **código CSS de nuestro componente**:

```
/* CSS Document */
a { text-decoration: none; }

h1 { font-size: 1em; }

h1, p {
 margin-bottom: 10px;
}

span {
  color: #5a5656;
  font-weight: lighter;
}

strong {
 font-weight: bold;
}

.uppercase { text-transform: uppercase; }

/* ---------- LOGIN ---------- */

#login {
 margin: 50px auto;
 width: 300px;
}
```

```
#login _ msg {
  background: url('error.png') no-repeat #FBE3E4 5px 5px;
  padding: 1em;
  padding-left: 3.5em;
  border: 2px solid #FBC2C4;
  color: #D12F19;
}

fieldset input[type="text"], input[type="password"] {
 background-color: #e5e5e5;
 border: none;
 border-radius: 3px;
 -moz-border-radius: 3px;
 -webkit-border-radius: 3px;
 color: #5a5656;
 font-family: 'Open Sans', Arial, Helvetica, sans-serif;
 font-size: 14px;
 height: 50px;
 outline: none;
 padding: 0px 10px;
 width: 280px;
 -webkit-appearance:none;
}

fieldset button {
 background-color: #008dde;
 border: none;
 border-radius: 3px;
 -moz-border-radius: 3px;
 -webkit-border-radius: 3px;
 color: #f4f4f4;
 cursor: pointer;
 font-family: 'Open Sans', Arial, Helvetica, sans-serif;
 height: 50px;
 text-transform: uppercase;
 width: 300px;
 -webkit-appearance:none;
}
```

```
fieldset a {
 color: #5a5656;
 font-size: 10px;
}

fieldset a:hover { text-decoration: underline; }

.btn-round {
 background-color: #5a5656;
 border-radius: 50%;
 -moz-border-radius: 50%;
 -webkit-border-radius: 50%;
 color: #f4f4f4;
 display: block;
 font-size: 12px;
 height: 50px;
 line-height: 50px;
 margin: 30px 125px;
 text-align: center;
 text-transform: uppercase;
 width: 50px;
}

.facebook-before {
 background-color: #0064ab;
 border-radius: 3px 0px 0px 3px;
 -moz-border-radius: 3px 0px 0px 3px;
 -webkit-border-radius: 3px 0px 0px 3px;
 color: #f4f4f4;
 display: block;
 float: left;
 height: 50px;
 line-height: 50px;
 text-align: center;
 width: 50px;
}
```

```
.facebook {
 background-color: #0079ce;
 border: none;
 border-radius: 0px 3px 3px 0px;
 -moz-border-radius: 0px 3px 3px 0px;
 -webkit-border-radius: 0px 3px 3px 0px;
 color: #f4f4f4;
 cursor: pointer;
 height: 50px;


 text-transform: uppercase;
 width: 250px;
}

.twitter-before {
 background-color: #189bcb;
 border-radius: 3px 0px 0px 3px;
 -moz-border-radius: 3px 0px 0px 3px;
 -webkit-border-radius: 3px 0px 0px 3px;
 color: #f4f4f4;
 display: block;
 float: left;
 height: 50px;
 line-height: 50px;
 text-align: center;
 width: 50px;
}

.twitter {
 background-color: #1bb2e9;
 border: none;
 border-radius: 0px 3px 3px 0px;
 -moz-border-radius: 0px 3px 3px 0px;
 -webkit-border-radius: 0px 3px 3px 0px;
```

```
    color: #f4f4f4;
    cursor: pointer;
    height: 50px;
    text-transform: uppercase;
    width: 250px;
  }
```

Definida la estructura de nuestro elemento y el estilo que le dará formato le toca el turno al controlador que manejará los eventos y dotará de funcionalidad a nuestro componente polymer.

Este será el código Dart de nuestro fichero `login.dart`. No hemos entrado en la implementación de los métodos tan solo los hemos creado para que puedas evaluar cómo se realiza la vinculación entre las variables y métodos definidos en nuestro controlador y la interfaz:

```
import 'package:polymer/polymer.dart';

/**
 * Polymer Login System Element.
 */
@CustomTag('login-usuario')
class LoginUsuario extends PolymerElement {
  @observable String usuario;
  @observable String password;
  @observable bool error = false;
  @observable String login_error = '';

  LoginUsuario.created() : super.created();

  /// Valida nombre usuario y contraseña.
  void entrar() {
```

```
    if(usuario == null || usuario.trim() == '' ||
        password == null || password.trim() == '') {
      error = true;
      login_error = 'Introduce usuario y password.';
      return;
    }
  }

  /// Permite recordar la contraseña olvidada.
  void recordarPassword() {}

  /// Permite hacer login con Facebook.
  void facebookLogin() {}

  /// Permite hacer login con Twitter.
  void twitterLogin() {}
}
```

Lo primero que haremos, será importar el paquete polymer para crear nuestros propios componentes:

```
import 'package:polymer/polymer.dart';
```

Ahora debemos crear el nombre que recibirá el componente y crear una clase que herede de `PolymerElement`.

```
@CustomTag('login-usuario')
class LoginUsuario extends PolymerElement { … }
```

Todos los elementos que heredan de `PolymerElement` tienen el constructor `created()` que crea el elemento dentro de nuestro documento principal.

```
LoginUsuario.created() : super.created();
```

Ahora vamos a ver las variables de instancia y los métodos y cómo indicamos que los queremos vinculados con la interfaz:

```
@observable String usuario;
@observable String password;
@observable bool error = false;
@observable String login_error = '';

/// Valida nombre usuario y contraseña.
void entrar() {
  if(usuario == null || usuario.trim() == '' ||
      password == null || password.trim() == '') {
    error = true;
    login_error = 'Introduce usuario y password.';
    return;
  }
}

/// Permite recordar la contraseña olvidada.
void recordarPassword() {}


/// Permite hacer login con Facebook.
void facebookLogin() {}

/// Permite hacer login con Twitter.
void twitterLogin() {}
```

En el caso de los métodos no debemos hacer nada especial tan solo crear los diferentes métodos que tendrá nuestro elemento y que serán los que le proporcionen vida.

En el caso de las variables de instancia debemos utilizar la anotación `@observable` para indicar que esta variable es visible y referenciable desde nuestro elemento. Gracias a ello en la definición HTML de nuestro elemento podemos utilizar la notación `{{usuario}}`.

El verdadero poder de todo esto reside en el *data binding*, polymer registra cualquier cambio que se produzca en nuestras variables y lo refleja de manera bidireccional. Si el usuario escribe su nombre de usuario en la caja de texto establecida para ello nosotros tenemos ese dato de manera automática y en tiempo real en nuestra variable `@observable String usuario;`

Del mismo modo si cambiamos el valor de la variable `@observable String login _ error = '';` este se refleja automáticamente en la interfaz.

Ya hemos definido la estructura de nuestro primer elemento, lo hemos diseñado y le hemos dotado de vida gestionando acciones asociadas a ciertos eventos de la interfaz.

Ahora si quieres utilizar este componente que gestiona el acceso de los usuarios al sistema puedes realizar de una manera muy sencilla en cualquier documento HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Sample app</title>
    <link rel="stylesheet" href="aplicacion.css">


    <!-- importamos login-usuario -->
    <link rel="import" href="login.html">
    <script type="application/dart">export 'package:polymer/init.dart';
          </script>
    <script src="packages/browser/dart.js"></script>
  </head>
  <body>
    <h1>Bienvenido a Polymer y Dart</h1>
        <login-usuario></login-usuario>
```

```
  </body>
</html>
```

Tan solo debes importar el elemento polymer creado.

```
<link rel="import" href="login.html">
```

Ahora ya solo queda referenciarlo en la parte del documento HTML donde quieras que aparezca.

```
<login-usuario></login-usuario>
```

Tan solo necesitas dos cosas más las referencias a las librerías polymer para que toda esta magia comience a operar.

```
<script type="application/dart">export 'package:polymer/init.dart';
</script>
<script src="packages/browser/dart.js"></script>
```

Como has podido ver en este ejemplo Dart está preparado para el futuro de lo que será el desarrollo web gracias a Polymer.

REFERENCIAS

Página oficial del lenguaje Dart

https://www.dartlang.org/

Autores del proyecto Dartlang.

Página oficial con la documentación oficial del SDK

https://api.dartlang.org/

Autores del proyecto Dartlang.

Dart: Up and Running

Kathy Walrath, Seth Ladd

O'Reilly Media. Octubre 2012

Learning Dart

Dzenan Ridjanovic, Ivo Balbaert

PacktPub. Enero 2014

Dart in action

Chris Buckett

Manning Publications. Enero 2013

Wikipedia.org

http://en.wikipedia.org/wiki/Dart_(programming_language)

http://es.wikipedia.org/wiki/Dart

http://es.wikipedia.org/wiki/Programacion_Estructurada

http://es.wikipedia.org/wiki/CamelCase

http://en.wikipedia.org/wiki/Factory_method_pattern

http://es.wikipedia.org/wiki/Abstract_Factory

http://en.wikipedia.org/wiki/Interface

http://en.wikipedia.org/wiki/Factory_method_pattern

Autores colaborativos del proyecto Wikipedia.

O'Reilly Programming

http://programming.oreilly.com/2013/05/8-dart-features-those-fat-cats-dont-want-you-to-know.html

Seth Ladd. Mayo 2013.

Proyecto JSON

http://www.json.org/json-es.html

Autores del proyecto JSON.

ECMA 404. The JSON Data Interchange format.

http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf

Ecma International. 2013.

Proyecto Polymer

http://www.polymer-project.org/

Autores del proyecto Polymer. 2014.

Futures and Error Handling

https://www.dartlang.org/articles/futures-and-error-handling/

Shailen Tuli. March 2013 (actualizado Mayo 2013)

Improving the DOM

https://www.dartlang.org/articles/improving-the-dom/

Bob Nystrom. October 2011 (actualizado Marzo 2013)

JSON web service

https://www.dartlang.org/articles/json-web-service/

Chris Buckett. Abril 2012 (actualizado Febrero 2013)

Dart Get Started

https://www.dartlang.org/docs/tutorials/get-started/

Autores del proyecto Dartlang.